

Python 3 — Biblioteca Estándar

Tomo 1 de 6

Contenido de este tomo:

1. Introduction
2. Built-in Functions
3. Built-in Constants
4. Built-in Types
5. Built-in Exceptions
6. Thread Safety Guarantees
7. Text Processing Services
8. string— Common string operations
9. string.templatelib— Support for template string literals
10. re— Regular expression operations
11. difflib— Helpers for computing deltas
12. textwrap— Text wrapping and filling
13. unicodedata— Unicode Database
14. stringprep— Internet String Preparation
15. readline— GNU readline interface
16. rlcompleter— Completion function for GNU readline
17. Binary Data Services
18. struct— Interpret bytes as packed binary data
19. codecs— Codec registry and base classes
20. Data Types
21. datetime— Basic date and time types
22. zoneinfo— IANA time zone support
23. calendar— General calendar-related functions
24. collections— Container datatypes
25. collections.abc— Abstract Base Classes for Containers

- 26. `heapq`— Heap queue algorithm
- 27. `bisect`— Array bisection algorithm
- 28. `array`— Efficient arrays of numeric values
- 29. `weakref`— Weak references
- 30. `types`— Dynamic type creation and names for built-in types
- 31. `copy`— Shallow and deep copy operations
- 32. `pprint`— Data pretty printer
- 33. `reprlib`— Alternate `repr()` implementation
- 34. `enum`— Support for enumerations
- 35. `graphlib`— Functionality to operate with graph-like structures
- 36. Numeric and Mathematical Modules
- 37. `numbers`— Numeric abstract base classes
- 38. `math`— Mathematical functions
- 39. `cmath`— Mathematical functions for complex numbers
- 40. `decimal`— Decimal fixed-point and floating-point arithmetic
- 41. `fractions`— Rational numbers
- 42. `random`— Generate pseudo-random numbers
- 43. `statistics`— Mathematical statistics functions
- 44. Functional Programming Modules
- 45. `itertools`— Functions creating iterators for efficient looping

Introduction

The “Python library” contains several different kinds of components.

It contains data types that would normally be considered part of the “core” of a language, such as numbers and lists. For these types, the Python language core defines the form of literals and places some constraints on their semantics, but does not fully define the semantics. (On the other hand, the language core does define syntactic properties like the spelling and priorities of operators.)

The library also contains built-in functions and exceptions — objects that can be used by all Python code without the need of an import statement. Some of these are defined by the core language, but many are not essential for the core semantics and are only described here.

The bulk of the library, however, consists of a collection of modules. There are many ways to dissect this collection. Some modules are written in C and built in to the Python interpreter; others are written in Python and imported in source form. Some modules provide interfaces that are highly specific to Python, like printing a stack trace; some provide interfaces that are specific to particular operating systems, such as access to specific hardware; others provide interfaces that are specific to a particular application domain, like the World Wide Web. Some modules are available in all versions and ports of Python; others are only available when the underlying system supports or requires them; yet others are available only when a particular configuration option was chosen at the time when Python was compiled and installed.

This manual is organized “from the inside out:” it first describes the built-in functions, data types and exceptions, and finally the modules, grouped in chapters of related modules.

This means that if you start reading this manual from the start, and skip to the next chapter when you get bored, you will get a reasonable overview of the available modules and application areas that are supported by the Python library. Of course, you don’t have to read it like a novel — you can also browse the table of contents (in front of the manual), or look for a specific function, module or term in the index (in the back). And finally, if you enjoy learning about random subjects, you choose a random page number (see `module.random`) and read a section or two. Regardless of the order in which you read the sections of this manual, it helps to start with chapter Built-in Functions, as the remainder of the manual assumes familiarity with this material.

Let the show begin!

Notes on availability

- An “Availability: Unix” note means that this function is commonly found on Unix systems. It does not make any claims about its existence on a specific operating system.
- If not separately noted, all functions that claim “Availability: Unix” are supported on macOS, iOS and Android, all of which build on a Unix core.
- If an availability note contains both a minimum Kernel version and a minimum libc version, then both conditions must hold. For example a feature with note `Availability: Linux >= 3.17 with glibc >= 2.27` requires both Linux 3.17 or newer and glibc 2.27 or newer.

WebAssembly platforms

The WebAssembly platforms `wasmb32-emscripten` (Emscripten) and `wasmb32-wasi` (WASI) provide a subset of POSIX APIs. WebAssembly runtimes and browsers are sandboxed and have limited access to the host and external resources. Any Python standard library module that uses processes, threading, networking, signals, or other forms of inter-process communication (IPC), is either not available or may not work as on other Unix-like systems. File I/O, file system, and Unix permission-related functions are restricted, too. Emscripten does not

permit blocking I/O. Other blocking operations like `sleep()` block the browser event loop.

The properties and behavior of Python on WebAssembly platforms depend on the Emscripten-SDK or WASI-SDK version, WASM runtimes (browser, NodeJS, wasmtime), and Python build time flags.

WebAssembly, Emscripten, and WASI are evolving standards; some features like networking may be supported in the future.

For Python in the browser, users should consider Pyodide or PyScript. PyScript is built on top of Pyodide, which itself is built on top of CPython and Emscripten. Pyodide provides access to browsers' JavaScript and DOM APIs as well as limited networking capabilities with JavaScript's XMLHttpRequest and Fetch APIs.

- Process-related APIs are not available or always fail with an error. That includes APIs that spawn new processes (`fork()`, `execve()`), wait for processes (`waitpid()`), send signals (`kill()`), or otherwise interact with processes. The `subprocess` module is importable but does not work.
- The `socket` module is available, but is limited and behaves differently from other platforms. On Emscripten, sockets are always non-blocking and require additional JavaScript code and helpers on the server to proxy TCP through WebSockets; see Emscripten Networking for more information. WASI snapshot preview 1 only permits sockets from an existing file descriptor.
- Some functions are stubs that either don't do anything and always return hardcoded values.
- Functions related to file descriptors, file permissions, file ownership, and links are limited and don't support some operations. For example, WASI does not permit symlinks with absolute file names.

Mobile platforms

Android and iOS are, in most respects, POSIX operating systems. File I/O, socket handling, and threading all behave as they would on any POSIX operating system. However, there are several major differences:

- Mobile platforms can only use Python in “embedded” mode. There is no Python REPL, and no ability to use separate executables such as `python` or `pip`. To add Python code to your mobile app, you must use the Python embedding API. For more details, see [Using Python on Android](#) and [Using Python on iOS](#).
- Subprocesses: On Android, creating subprocesses is possible but officially unsupported. In particular, Android does not support any part of the System V IPC API, so multiprocessing is not available. An iOS app cannot use any form of subprocess, multiprocessing, or inter-process communication. If an iOS app attempts to create a subprocess, the process creating the subprocess will either lock up, or crash.
- Mobile apps have limited access to modify system resources (such as the system clock). These resources will often be read-only, but attempts to modify those resources will usually fail.
- Console input and output: On Android, the native `stdout` and `stderr` are not connected to anything, so Python installs its own streams which redirect messages to the system log. These can be seen under the tags `python.stdout` and `python.stderr` respectively. iOS apps have a limited concept of console output. `stdout` and `stderr` exist, and content written to `stdout` and `stderr` will be visible in logs when running in Xcode,

Built-in Functions

The Python interpreter has a number of functions and types built into it that are always available. They are listed here in alphabetical order.

```
Built-in Functions | A | abs() | aiter() | all() | anext() | any() | ascii() | B | bin() |
bool() | breakpoint() | bytearray() | bytes() | C | callable() | chr() | classmethod() |
compile() | complex() | D | delattr() | dict() | dir() | divmod() | E | enumerate() | eval() |
exec() | F | filter() | float() | format() | frozenset() | G | getattr() | globals() | H |
hasattr() | hash() | help() | hex() | I | id() | input() | int() | isinstance() | issubclass() |
iter() | L | len() | list() | locals() | M | map() | max() | memoryview() | min() | N | next() |
O | object() | oct() | open() | ord() | P | pow() | print() | property() | R | range() | repr()
| reversed() | round() | S | set() | setattr() | slice() | sorted() | staticmethod() | str() |
sum() | super() | T | tuple() | type() | V | vars
```

abs(number, /)

Return the absolute value of a number. The argument may be an integer, a floating-point number, or an object implementing `__abs__()`. If the argument is a complex number, its magnitude is returned.

aiter(async_iterable, /)

Return an asynchronous iterator for an asynchronous iterable. Equivalent to calling `x.__aiter__()`. Note: Unlike `iter()`, `aiter()` has no 2-argument variant. Added in version 3.10.

all(iterable, /)

Return `True` if all elements of the iterable are `True` (or if the iterable is empty). Equivalent to: `def all(iterable): for element in iterable: if not element: return False return True`

awaitable_anext(async_iterator, /)

awaitable_anext(async_iterator, default, /)

When awaited, return the next item from the given asynchronous iterator, or `default` if given and the iterator is exhausted. This is the `async` variant of the `next()` builtin, and behaves similarly. This calls the `__anext__()` method of `async_iterator`, returning an `awaitable`. Awaiting this returns the next value of the iterator. If `default` is given, it is returned if the iterator is exhausted, otherwise `StopAsyncIteration` is raised. Added in version 3.10.

any(iterable, /)

Return `True` if any element of the iterable is `True`. If the iterable is empty, return `False`. Equivalent to: `def any(iterable): for element in iterable: if element: return True return False`

ascii(object, /)

As `repr()`, return a string containing a printable representation of an object, but escape the non-ASCII characters in the string returned by `repr()` using `\x`, `\u`, or `\U` escapes. This generates a string similar to that returned by `repr()` in Python 2.

bin(integer, /)

Convert an integer number to a binary string prefixed with `"0b"`. The result is a valid Python expression. If `integer` is not a Python `int` object, it has to define an `__index__()` method that returns an integer. Some examples: `>>> bin(3) '0b11' >>> bin(-10) '-0b1010'` If the prefix `"0b"` is desired or not, you can use either of the following ways: `>>> format(14, '#b'), format(14, 'b') ('0b1110', '1110') >>> f'{14:#b}', f'{14:b}' ('0b1110', '1110')` See also `enum.bin()` to represent negative values as two's complement. See also `for`

class bool(object=False, /)

Return a Boolean value, i.e. one of `True` or `False`. The argument is converted using the standard truth testing procedure. If the argument is `false` or omitted, this returns `False`; otherwise, it returns `True`. The `bool` class is a

subclass of `int` (see [Numeric Types — int, float, complex](#)). It cannot be subclassed further. Its only instances are `False` and `True` (see [Boolean Type - bool](#)). Changed in version 3.7: The parameter is now positional-only.

`breakpoint(*args,**kws)`

This function drops you into the debugger at the call site. Specifically, it calls `sys.breakpointhook()`, passing `args` and `kws` straight through. By default, `sys.breakpointhook()` calls `pdb.set_trace()` expecting no arguments. In this case, it is purely a convenience function so you don't have to explicitly import `pdb` or type as much code to enter the debugger. However, `sys.breakpointhook()` can be set to some other function and `breakpoint()` will automatically call that, allowing you to drop into the debugger o

`class bytearray(source=b'')`

`class bytearray(source, encoding, errors='strict')`

Return a new array of bytes. The `bytearray` class is a mutable sequence of integers in the range $0 \leq x < 256$. It has most of the usual methods of mutable sequences, described in [Mutable Sequence Types](#), as well as most methods that the `bytes` type has, see [Bytes and bytearray Operations](#). The optional `source` parameter can be used to initialize the array in a few different ways: If it is a string, you must also give the `encoding` (and optionally, `errors`) parameters; `bytearray()` then converts the string to bytes us

`class bytes(source=b'')`

`class bytes(source, encoding, errors='strict')`

Return a new “bytes” object which is an immutable sequence of integers in the range $0 \leq x < 256$. `bytes` is an immutable version of `bytearray`— it has the same non-mutating methods and the same indexing and slicing behavior. Accordingly, constructor arguments are interpreted as for `bytearray()`. Bytes objects can also be created with literals, see [String and Bytes literals](#). See also [Binary Sequence Types — bytes, bytearray, memoryview, Bytes Objects, and Bytes and bytearray Operations](#).

`callable(object, /)`

Return `True` if the `object` argument appears callable, `False` if not. If this returns `True`, it is still possible that a call fails, but if it is `False`, calling `object` will never succeed. Note that classes are callable (calling a class returns a new instance); instances are callable if their class has a `__call__()` method. Added in version 3.2: This function was first removed in Python 3.0 and then brought back in Python 3.2.

`chr(codepoint, /)`

Return the string representing a character with the specified Unicode code point. For example, `chr(97)` returns the string `'a'`, while `chr(8364)` returns the string `'€'`. This is the inverse of `ord()`. The valid range for the argument is from 0 through 1,114,111 (0x10FFFF in base 16). `ValueError` will be raised if it is outside that range.

`@classmethod`

Transform a method into a class method. A class method receives the class as an implicit first argument, just like an instance method receives the instance. To declare a class method, use this idiom: `class C: @classmethod def f(cls, arg1, arg2): ...`. The `@classmethod` form is a function decorator— see [Function definitions](#) for details. A class method can be called either on the class (such as `C.f()`) or on an instance (such as `c.f()`). The instance is ignored except for its class. If a class method is called for a

`compile(source, filename, mode, flags=0, dont_inherit=False, optimize=-1)`

Compile the `source` into a code or AST object. Code objects can be executed by `exec()` or `eval()`. `source` can either be a normal string, a byte string, or an AST object. Refer to the `ast` module documentation for information on how to work with AST objects. The `filename` argument should give the file from which the code was read; pass some recognizable value if it wasn't read from a file ('<string>' is commonly used). The `mode` argument specifies what kind of code must be compiled; it can be `'exec'` if `source` consists o

Raises:

Parameters:source(str|code object) – A Python expression.globals(dict|None) – The global namespace (default:None).locals(mapping|None) – The local namespace (default:None).Returns:The result of the evaluated expression.Raises:Syntax errors are reported as exceptions.WarningThis function executes arbitrary code. Calling it with untrusted user-supplied input will lead to security vulnerabilities.The source argument is parsed and evaluated as a Python expression (technically speaking, a condition is

source(str|code object) – A Python expression.globals(dict|None) – The global namespace (default:None).locals(mapping|None) – The local namespace (default:None).

The result of the evaluated expression.

Syntax errors are reported as exceptions.

exec(source,/,globals=None,locals=None,*,closure=None)

WarningThis function executes arbitrary code. Calling it with untrusted user-supplied input will lead to security vulnerabilities.This function supports dynamic execution of Python code.source must be either a string or a code object. If it is a string, the string is parsed as a suite of Python statements which is then executed (unless a syntax error occurs).[1]If it is a code object, it is simply executed. In all cases, the code that's executed is expected to be valid as file input (see the se

filter(function,iterable,/)

Construct an iterator from those elements of iterable for which function is true.iterable may be either a sequence, a container which supports iteration, or an iterator. If function is None, the identity function is assumed, that is, all elements of iterable that are false are removed.Note that filter(function,iterable) is equivalent to the generator expression (item for item in iterable if function(item)) if function is not None and (item for item in iterable if item) if function is None. See itertools.filterfalse() for the co

class float(number=0.0,/)

class float(string,/)

Return a floating-point number constructed from a number or a string.Examples:>>>float('+1.23')1.23>>>float('-12345\n')-12345.0>>>float('1e-003')0.001>>>float('+1E6')1000000.0>>>float('-Infinity')-infIf the argument is a string, it should contain a decimal number, optionally preceded by a sign, and optionally embedded in whitespace. The optional sign may be '+' or '-'; a '+' sign has no effect on the value produced. The argument may also be a string representing a NaN (not-a-number), or positive

format(value,format_spec="",/)

Convert a value to a “formatted” representation, as controlled by format_spec. The interpretation of format_spec will depend on the type of the value argument; however, there is a standard formatting syntax that is used by most built-in types:Format specification mini-language.The default format_spec is an empty string which usually gives the same effect as calling str(value).A call to format(value,format_spec) is translated to type(value).__format__(value,format_spec) which bypasses the instance dictionary

class frozenset(iterable=(),/)

Return a new frozenset object, optionally with elements taken from iterable.frozenset is a built-in class. See frozenset and Set Types — set, frozenset for documentation about this class.For other containers see the built-in set, list, tuple, and dict classes, as well as the collections module.

getattr(object,name,/)

getattr(object,name,default,/)

Return the value of the named attribute of object.name must be a string. If the string is the name of one of the object's attributes, the result is the value of that attribute. For example, getattr(x,'foobar') is equivalent to x.foobar. If the named attribute does not exist, default is returned if provided, otherwise AttributeError is raised.name need

not be a Python identifier (see `setattr()`). Note Since private name mangling happens at compilation time, one must manually mangle a private attribute's (attri

globals()

Return the dictionary implementing the current module namespace. For code within functions, this is set when the function is defined and remains the same regardless of where the function is called.

hasattr(object,name,/)

The arguments are an object and a string. The result is `True` if the string is the name of one of the object's attributes, `False` if not. (This is implemented by calling `getattr(object,name)` and seeing whether it raises an `AttributeError` or not.)

hash(object,/)

Return the hash value of the object (if it has one). Hash values are integers. They are used to quickly compare dictionary keys during a dictionary lookup. Numeric values that compare equal have the same hash value (even if they are of different types, as is the case for 1 and 1.0). Note For objects with custom `__hash__()` methods, note that `hash()` truncates the return value based on the bit width of the host machine.

help()

help(request)

Invoke the built-in help system. (This function is intended for interactive use.) If no argument is given, the interactive help system starts on the interpreter console. If the argument is a string, then the string is looked up as the name of a module, function, class, method, keyword, or documentation topic, and a help page is printed on the console. If the argument is any other kind of object, a help page on the object is generated. Note that if a slash (/) appears in the parameter list of a

hex(integer,/)

Convert an integer number to a lowercase hexadecimal string prefixed with "0x". If `integer` is not a Python int object, it has to define an `__index__()` method that returns an integer. Some examples: `>>> hex(255) '0xff' >>> hex(-42) '-0x2a'` If you want to convert an integer number to an uppercase or lower hexadecimal string with prefix or not, you can use either of the following ways: `>>> '%#x'%255, '%x'%255, '%X'%255 ('0xff', 'ff', 'FF') >>> format(255, '#x'), format(255, 'x'), format(255, 'X') ('0xff', 'ff', 'FF') >>> f'{2`

id(object,/)

Return the "identity" of an object. This is an integer which is guaranteed to be unique and constant for this object during its lifetime. Two objects with non-overlapping lifetimes may have the same `id()` value. CPython implementation detail: This is the address of the object in memory. Raises an `auditing event` builtins.`id` with argument `id`.

input()

input(prompt,/)

If the `prompt` argument is present, it is written to standard output without a trailing newline. The function then reads a line from input, converts it to a string (stripping a trailing newline), and returns that. When EOF is read, `EOFError` is raised. Example: `>>> s=input('--> ') --> Monty Python's Flying Circus >>> s` "Monty Python's Flying Circus" If the `readline` module was loaded, then `input()` will use it to provide elaborate line editing and history features. Raises an `auditing event` builtins.`input` with argum

classint(number=0,/)

classint(string,/,base=10)

Return an integer object constructed from a number or a string, or return 0 if no arguments are given. Examples: `>>>int(123.45)123>>>int('123')123>>>int('-12_345\n')-12345>>>int('FACE',16)64206>>>int('0xface',0)64206>>>int('01110011',base=2)115` If the argument defines `__int__()`, `int(x)` returns `x.__int__()`. If the argument defines `__index__()`, it returns `x.__index__()`. For floating-point numbers, this truncates towards zero. If the argument is not a number or if `base` is given, then it must be a string, byte

isinstance(object, classinfo, /)

Return `True` if the `object` argument is an instance of the `classinfo` argument, or of a (direct, indirect, or virtual) subclass thereof. If `object` is not an object of the given type, the function always returns `False`. If `classinfo` is a tuple of type objects (or recursively, other such tuples) or a Union Type of multiple types, return `True` if `object` is an instance of any of the types. If `classinfo` is not a type or tuple of types and such tuples, a `TypeError` exception is raised. `TypeError` may not be raised for an invalid `t`

issubclass(class, classinfo, /)

Return `True` if `class` is a subclass (direct, indirect, or virtual) of `classinfo`. A class is considered a subclass of itself. `classinfo` may be a tuple of class objects (or recursively, other such tuples) or a Union Type, in which case return `True` if `class` is a subclass of any entry in `classinfo`. In any other case, a `TypeError` exception is raised. Changed in version 3.10: `classinfo` can be a Union Type.

iter(iterable, /)

iter(callable, sentinel, /)

Return an iterator object. The first argument is interpreted very differently depending on the presence of the second argument. Without a second argument, the single argument must be a collection object which supports the `iterable` protocol (the `__iter__()` method), or it must support the sequence protocol (the `__getitem__()` method with integer arguments starting at 0). If it does not support either of those protocols, `TypeError` is raised. If the second argument, `sentinel`, is given, then the first argument

len(object, /)

Return the length (the number of items) of an object. The argument may be a sequence (such as a string, bytes, tuple, list, or range) or a collection (such as a dictionary, set, or frozen set). CPython implementation detail: `len` raises `OverflowError` on lengths larger than `sys.maxsize`, such as `range(2**100)`.

classlist(iterable=(), /)

Rather than being a function, `list` is actually a mutable sequence type, as documented in `Lists and Sequence Types` — list, tuple, range.

locals()

Return a mapping object representing the current local symbol table, with variable names as the keys, and their currently bound references as the values. At module scope, as well as when using `exec()` or `eval()` with a single namespace, this function returns the same namespace as `globals()`. At class scope, it returns the namespace that will be passed to the metaclass constructor. When using `exec()` or `eval()` with separate local and global arguments, it returns the local namespace passed in to the function call

map(function, iterable, /, *iterables, strict=False)

Return an iterator that applies `function` to every item of `iterable`, yielding the results. If additional `iterables` arguments are passed, `function` must take that many arguments and is applied to the items from all iterables in parallel. With multiple iterables, the iterator stops when the shortest iterable is exhausted. If `strict` is `True` and one of the iterables is exhausted before the others, a `ValueError` is raised. For cases where the function inputs are already arranged into argument tuples, see `itertools`

max(iterable, /, *, key=None)

max(iterable,/,*,default,key=None)

max(arg1,arg2,/,*,args,key=None)

Return the largest item in an iterable or the largest of two or more arguments. If one positional argument is provided, it should be an iterable. The largest item in the iterable is returned. If two or more positional arguments are provided, the largest of the positional arguments is returned. There are two optional keyword-only arguments. The `key` argument specifies a one-argument ordering function like that used for `list.sort()`. The `default` argument specifies an object to return if the provided iterab

class memoryview(object)

Return a “memory view” object created from the given argument. See [Memory Views](#) for more information.

min(iterable,/,*,key=None)

min(iterable,/,*,default,key=None)

min(arg1,arg2,/,*,args,key=None)

Return the smallest item in an iterable or the smallest of two or more arguments. If one positional argument is provided, it should be an iterable. The smallest item in the iterable is returned. If two or more positional arguments are provided, the smallest of the positional arguments is returned. There are two optional keyword-only arguments. The `key` argument specifies a one-argument ordering function like that used for `list.sort()`. The `default` argument specifies an object to return if the provided it

next(iterator,/)

next(iterator,default,/)

Retrieve the next item from the iterator by calling its `__next__()` method. If `default` is given, it is returned if the iterator is exhausted, otherwise `StopIteration` is raised.

class object

This is the ultimate base class of all other classes. It has methods that are common to all instances of Python classes. When the constructor is called, it returns a new featureless object. The constructor does not accept any arguments. Note object instances do not have `__dict__` attributes, so you can't assign arbitrary attributes to an instance of `object`.

oct(integer,/)

Convert an integer number to an octal string prefixed with “0o”. The result is a valid Python expression. If `integer` is not a Python int object, it has to define an `__index__()` method that returns an integer. For example: `>>> oct(8) '0o10' >>> oct(-56) '-0o70'` If you want to convert an integer number to an octal string either with the prefix “0o” or not, you can use either of the following ways. `>>> '%#o'%10, '%o'%10 ('0o12', '12') >>> format(10, '#o'), format(10, 'o') ('0o12', '12') >>> f'{10:#o}', f'{10:o}' ('0o12', '12')`

open(file,mode='r',buffering=-1,encoding=None,errors=None,newline=None,closefd=True,opener=None)

Open file and return a corresponding file object. If the file cannot be opened, an `OSError` is raised. See [Reading and Writing Files](#) for more examples of how to use this function. `file` is a path-like object giving the pathname (absolute or relative to the current working directory) of the file to be opened or an integer file descriptor of the file to be wrapped. (If a file descriptor is given, it is closed when the returned I/O object is closed unless `closefd` is set to `False`.) `mode` is an optional string that s

ord(character,/)

Return the ordinal value of a character. If the argument is a one-character string, return the Unicode code point of that character. For example, `ord('a')` returns the integer 97 and `ord('€')` (Euro sign) returns 8364. This is the inverse of `chr()`. If the argument is a bytes or bytearray object of length 1, return its single byte value. For example, `ord(b'a')` returns the integer 97.

pow(base,exp,mod=None)

Return `base` to the power `exp`; if `mod` is present, return `base` to the power `exp`, modulo `mod` (computed more efficiently than `pow(base,exp)%mod`). The two-argument form `pow(base,exp)` is equivalent to using the power operator: `base**exp`. When arguments are builtin numeric types with mixed operand types, the coercion rules for binary arithmetic operators apply. For int operands, the result has the same type as the operands (after coercion) unless the second argument is negative; in that case, all arguments are converted

print(*objects,sep=" ",end="\n",file=None,flush=False)

Print `objects` to the text stream `file`, separated by `sep` and followed by `end`. `sep`, `end`, `file`, and `flush`, if present, must be given as keyword arguments. All non-keyword arguments are converted to strings like `str()` does and written to the stream, separated by `sep` and followed by `end`. Both `sep` and `end` must be strings; they can also be `None`, which means to use the default values. If no `objects` are given, `print()` will just write `end`. The `file` argument must be an object with a `write(string)` method; if it is not present or `None`, sys

class property(fget=None, fset=None, fdel=None, doc=None)

@getter

@setter

@deleter

__name__

Return a property attribute. `fget` is a function for getting an attribute value. `fset` is a function for setting an attribute value. `fdel` is a function for deleting an attribute value. `and doc` creates a docstring for the attribute. A typical use is to define a managed attribute: `class C: def __init__(self): self._x = None def getx(self): return self._x def setx(self, value): self._x = value def delx(self): del self._x x = property(getx, setx, delx, "I'm the 'x' property.")` If `c` is an instance of `C`, `c.x` will invoke the getter, `c.x = value` will

A property object has `getter`, `setter`, and `deleter` methods usable as decorators that create a copy of the property with the corresponding accessor function set to the decorated function. This is best explained with an example: `class C: def __init__(self): self._x = None @property def x(self): """I'm the 'x' property.""" return self._x @x.setter def x(self, value): self._x = value @x.deleter def x(self): del self._x` This code is exactly equivalent to the first example. Be sure to give the additional functions the same name as

Attribute holding the name of the property. The name of the property can be changed at runtime. Added in version 3.13.

class range(stop, /)

class range(start, stop, step=1, /)

Rather than being a function, `range` is actually an immutable sequence type, as documented in [Ranges and Sequence Types — list, tuple, range](#).

repr(object, /)

Return a string containing a printable representation of an object. For many types, this function makes an attempt to return a string that would yield an object with the same value when passed to `eval()`; otherwise, the representation is a string enclosed in angle brackets that contains the name of the type of the object together with additional information often including the name and address of the object. A class can control what this function returns for its instances by defining a `__repr__()`

reversed(object, /)

Return a reverse iterator. The argument must be an object which has a `__reversed__()` method or supports the sequence protocol (the `__len__()` method and the `__getitem__()` method with integer arguments starting at 0).

round(number, ndigits=None)

Return number rounded to `ndigits` precision after the decimal point. If `ndigits` is omitted or is `None`, it returns the nearest integer to its input. For the built-in types supporting `round()`, values are rounded to the closest multiple of 10 to the power minus `ndigits`; if two multiples are equally close, rounding is done toward the even choice (so, for example, both `round(0.5)` and `round(-0.5)` are 0, and `round(1.5)` is 2). Any integer value is valid for `ndigits` (positive, zero, or negative). The return value is an in

class`set(iterable=(),/)`

Return a new `set` object, optionally with elements taken from `iterable`. `set` is a built-in class. See `set` and `Set` Types — `set`, `frozenset` for documentation about this class. For other containers see the built-in `frozenset`, `list`, `tuple`, and `dict` classes, as well as the `collections` module.

class`setattr(object,name,value,/)`

This is the counterpart of `getattr()`. The arguments are an object, a string, and an arbitrary value. The string may name an existing attribute or a new attribute. The function assigns the value to the attribute, provided the object allows it. For example, `setattr(x,'foobar',123)` is equivalent to `x.foobar=123`. `name` need not be a Python identifier as defined in `Names` (identifiers and keywords) unless the object chooses to enforce that, for example in a custom `__getattr__()` or via `__slots__`. An attri

class`slice(stop,/)`

class`slice(start,stop,step=None,/)`

start

stop

step

Return a `slice` object representing the set of indices specified by `range(start,stop,step)`. The `start` and `step` arguments default to `None`. `Slice` objects are also generated when slicing syntax is used. For example: `a[start:stop:step]` or `a[start:stop,i]`. See `itertools.islice()` for an alternate version that returns an iterator. `start`, `stop`, `step` These read-only attributes are set to the argument values (or their default). They have no other explicit functionality; however, they are used by NumPy and other third-party pac

These read-only attributes are set to the argument values (or their default). They have no other explicit functionality; however, they are used by NumPy and other third-party packages.

sorted(iterable,/,*,key=None,reverse=False)

Return a new sorted list from the items in `iterable`. Has two optional arguments which must be specified as keyword arguments. `key` specifies a function of one argument that is used to extract a comparison key from each element in `iterable` (for example, `key=str.lower`). The default value is `None` (compare the elements directly). `reverse` is a boolean value. If set to `True`, then the list elements are sorted as if each comparison were reversed. Use `functools.cmp_to_key()` to convert an old-style `cmp` function to a `key` fu

@staticmethod

Transform a method into a static method. A static method does not receive an implicit first argument. To declare a static method, use this idiom: `class C: @staticmethod def f(arg1, arg2, argN): ...`. The `@staticmethod` form is a function decorator— see `Function` definitions for details. A static method can be called either on the class (such as `C.f()`) or on an instance (such as `C().f()`). Moreover, the static method descriptor is also callable, so it can be used in the class definition (such as `f`). Static methods in Pyt

class`str(*,encoding='utf-8',errors='strict')`

class`str(object)`

class`str(object,encoding,errors='strict')`

class`str(object,*,errors)`

Return a string version of object. See `str()` for details. `str` is the built-in string class. For general information about strings, see [Text Sequence Type — str](#).

`sum(iterable, /, start=0)`

Sums `start` and the items of an iterable from left to right and returns the total. The iterable's items are normally numbers, and the start value is not allowed to be a string. For some use cases, there are good alternatives to `sum()`. The preferred, fast way to concatenate a sequence of strings is by calling `".join(sequence)"`. To add floating-point values with extended precision, see `math.fsum()`. To concatenate a series of iterables, consider using `itertools.chain()`. Changed in version 3.8: The `start` parameter

`classsuper`

`classsuper(type, object_or_type=None, /)`

Return a proxy object that delegates method calls to a parent or sibling class of `type`. This is useful for accessing inherited methods that have been overridden in a class. The `object_or_type` determines the method resolution order to be searched. The search starts from the class right after the `type`. For example, if `__mro__` of `object_or_type` is `D->B->C->A->object` and the value of `type` is `B`, then `super()` searches `C->A->object`. The `__mro__` attribute of the class corresponding to `object_or_type` lists the method resolution

`tuple(iterable=(), /)`

Rather than being a function, `tuple` is actually an immutable sequence type, as documented in [Tuples and Sequence Types — list, tuple, range](#).

`class type(object, /)`

`class type(name, bases, dict, /, **kwargs)`

With one argument, return the type of an object. The return value is a type object and generally the same object as returned by `object.__class__`. The `isinstance()` built-in function is recommended for testing the type of an object, because it takes subclasses into account. With three arguments, return a new type object. This is essentially a dynamic form of the `class` statement. The `name` string is the class name and becomes the `__name__` attribute. The `bases` tuple contains the base classes and becomes the `__bases__`

`vars()`

`vars(object, /)`

Return the `__dict__` attribute for a module, class, instance, or any other object with a `__dict__` attribute. Objects such as modules and instances have an updateable `__dict__` attribute; however, other objects may have write restrictions on their `__dict__` attributes (for example, classes use `types.MappingProxyType` to prevent direct dictionary updates). Without an argument, `vars()` acts like `locals()`. A `TypeError` exception is raised if an object is specified but it doesn't have a `__dict__` attribute (for example, if `i`

`zip(*iterables, strict=False)`

Iterate over several iterables in parallel, producing tuples with an item from each one. Example: `>>> for item in zip([1, 2, 3], ['sugar', 'spice', 'everything nice']): ... print(item) ... (1, 'sugar') (2, 'spice') (3, 'everything nice')` More formally: `zip()` returns an iterator of tuples, where the *i*-th tuple contains the *i*-th element from each of the argument iterables. Another way to think of `zip()` is that it turns rows into columns, and columns into rows. This is similar to transposing a matrix. `zip()` is lazy: The elem

`__import__(name, globals=None, locals=None, fromlist=(), level=0)`

Note This is an advanced function that is not needed in everyday Python programming, unlike `importlib.import_module()`. This function is invoked by the `import` statement. It can be replaced (by importing the `builtins` module and assigning to `builtins.__import__`) in order to change semantics of the `import` statement, but

doing so is strongly discouraged as it is usually simpler to use import hooks (see PEP 302) to attain the same goals and does not cause issues with code which assumes the default import implemen

Footnotes

Note that the parser only accepts the Unix-style end of line convention. If you are reading the code from a file, make sure to use newline conversion mode to convert Windows or Mac-style newlines.

Built-in Constants

A small number of constants live in the built-in namespace. They are:

False

The false value of the `bool` type. Assignments to `False` are illegal and raise a `SyntaxError`.

True

The true value of the `bool` type. Assignments to `True` are illegal and raise a `SyntaxError`.

None

An object frequently used to represent the absence of a value, as when default arguments are not passed to a function. Assignments to `None` are illegal and raise a `SyntaxError`. `None` is the sole instance of the `NoneType` type.

NotImplemented

A special value which should be returned by the binary special methods (e.g. `__eq__()`, `__lt__()`, `__add__()`, `__rsub__()`, etc.) to indicate that the operation is not implemented with respect to the other type; may be returned by the in-place binary special methods (e.g. `__imul__()`, `__iand__()`, etc.) for the same purpose. It should not be evaluated in a boolean context. `NotImplemented` is the sole instance of the `NotImplementedType` type. Note: When a binary (or in-place) method returns `NotImplemented` the

Ellipsis

The same as the ellipsis literal "...", an object frequently used to indicate that something is omitted. Assignment to `Ellipsis` is possible, but assignment to ... raises a `SyntaxError`. `Ellipsis` is the sole instance of the `EllipsisType` type.

__debug__

This constant is true if Python was not started with an `-O` option. See also the `assert` statement.

■ Nota: Note that the names `None`, `False`, `True` and `__debug__` cannot be reassigned (assignments to them, even as an attribute name, raise `SyntaxError`), so they can be considered "true" constants.

Constants added by the `site` module

The `site` module (which is imported automatically during startup, except if the `-S` command-line option is given) adds several constants to the built-in namespace. They are useful for the interactive interpreter shell and should not be used in programs.

`quit(code=None)`

`exit(code=None)`

Objects that when printed, print a message like "Use quit() or Ctrl-D (i.e. EOF) to exit", and when accessed directly in the interactive interpreter or called as functions, raise `SystemExit` with the specified exit code.

`help`

Object that when printed, prints the message "Type help() for interactive help, or help(object) for help about object.", and when accessed directly in the interactive interpreter, invokes the built-in help system (see `help()`).

`copyright`

`credits`

Objects that when printed or called, print the text of copyright or credits, respectively.

license

Object that when printed, prints the message "Type license() to see the full license text", and when called, displays the full license text in a pager-like fashion (one screen at a time).

Built-in Types

The following sections describe the standard types that are built into the interpreter.

The principal built-in types are numerics, sequences, mappings, classes, instances and exceptions.

Some collection classes are mutable. The methods that add, subtract, or rearrange their members in place, and don't return a specific item, never return the collection instance itself but `None`.

Some operations are supported by several object types; in particular, practically all objects can be compared for equality, tested for truth value, and converted to a string (with `repr()` function or the slightly different `str()` function). The latter function is implicitly used when an object is written by the `print()` function.

Truth Value Testing

Any object can be tested for truth value, for use in an `if` or `while` condition or as operand of the Boolean operations below.

By default, an object is considered true unless its class defines either a `__bool__()` method that returns `False` or a `__len__()` method that returns zero, when called with the object.^[1] If one of the methods raises an exception when called, the exception is propagated and the object does not have a truth value (for example, `NotImplemented`). Here are most of the built-in objects considered false:

- constants defined to be false: `None` and `False`
- zero of any numeric type: `0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0,1)`
- empty sequences and collections: `''`, `()`, `[]`, `{}`, `set()`, `range(0)`

Operations and built-in functions that have a Boolean result always return `0` or `False` for false and `1` or `True` for true, unless otherwise stated. (Important exception: the Boolean operations `or` and `and` always return one of their operands.)

Boolean Operations —and,or,not

These are the Boolean operations, ordered by ascending priority:

Operation	Result	Notes
<code>x or y</code>	<code>x</code> if <code>x</code> is true, else <code>y</code>	(1) <code>x or y</code> if <code>x</code> is false, then <code>y</code>
<code>x and y</code>	<code>x</code> if <code>x</code> is true, else <code>y</code>	(2) <code>not x</code> if <code>x</code> is false, then <code>True</code> , else <code>False</code>
<code>not x</code>	<code>True</code> if <code>x</code> is false, else <code>False</code>	(3)

Notes:

- This is a short-circuit operator, so it only evaluates the second argument if the first one is false.
- This is a short-circuit operator, so it only evaluates the second argument if the first one is true.
- `not` has a lower priority than non-Boolean operators, so `not a == b` is interpreted as `not (a == b)`, and `a == not b` is a syntax error.

Comparisons

There are eight comparison operations in Python. They all have the same priority (which is higher than that of the Boolean operations). Comparisons can be chained arbitrarily; for example, `x < y <= z` is equivalent to `x < y` and `y <= z`, except that `y` is evaluated only once (but in both cases `z` is not evaluated at all when `x < y` is found to be false).

This table summarizes the comparison operations:

Operation	Meaning	<		strictly less than	<=		less than or equal	>		strictly greater than	>=		greater than or equal	==		equal	!=		not equal	is		object identity	is not		negated object identity
-----------	---------	---	--	--------------------	----	--	--------------------	---	--	-----------------------	----	--	-----------------------	----	--	-------	----	--	-----------	----	--	-----------------	--------	--	-------------------------

Unless stated otherwise, objects of different types never compare equal. The `==` operator is always defined but for some object types (for example, class objects) is equivalent to `is`. The `<`, `<=`, `>` and `>=` operators are only defined where they make sense; for example, they raise a `TypeError` exception when one of the arguments is a complex number.

Non-identical instances of a class normally compare as non-equal unless the class defines the `__eq__()` method.

Instances of a class cannot be ordered with respect to other instances of the same class, or other types of object, unless the class defines enough of the methods `__lt__()`, `__le__()`, `__gt__()`, and `__ge__()` (in general, `__lt__()` and `__eq__()` are sufficient, if you want the conventional meanings of the comparison operators).

The behavior of the `is` and `is not` operators cannot be customized; also they can be applied to any two objects and never raise an exception.

Two more operations with the same syntactic priority, `in` and `not in`, are supported by types that are iterable or implement the `__contains__()` method.

Numeric Types —int,float,complex

There are three distinct numeric types: integers, floating-point numbers, and complex numbers. In addition, Booleans are a subtype of integers. Integers have unlimited precision. Floating-point numbers are usually implemented using `double` in C; information about the precision and internal representation of floating-point numbers for the machine on which your program is running is available in `sys.float_info`. Complex numbers have a real and imaginary part, which are each a floating-point number. To extract these parts from a complex number `z`, use `z.real` and `z.imag`. (The standard library includes the additional numeric types `Fraction`, for rationals, and `decimal.Decimal`, for floating-point numbers with user-definable precision.)

Numbers are created by numeric literals or as the result of built-in functions and operators. Unadorned integer literals (including hex, octal and binary numbers) yield integers. Numeric literals containing a decimal point or an exponent sign yield floating-point numbers. Appending `'j'` or `'J'` to a numeric literal yields an imaginary number (a complex number with a zero real part) which you can add to an integer or float to get a complex number with real and imaginary parts.

The constructors `int()`, `float()`, and `complex()` can be used to produce numbers of a specific type.

Python fully supports mixed arithmetic: when a binary arithmetic operator has operands of different built-in numeric types, the operand with the “narrower” type is widened to that of the other:

- If both arguments are complex numbers, no conversion is performed;
- if either argument is a complex or a floating-point number, the other is converted to a floating-point number;
- otherwise, both must be integers and no conversion is necessary.

Arithmetic with complex and real operands is defined by the usual mathematical formula, for example:

```
x + complex(u, v) = complex(x + u, v)
x * complex(u, v) = complex(x * u, x * v)
```

A comparison between numbers of different types behaves as though the exact values of those numbers were being compared.^[2]

All numeric types (except complex) support the following operations (for priorities of the operations, see `Operator precedence`):

Operation	Result	Notes	Full documentation
<code>x + y</code>	sum of <code>x</code> and <code>y</code>	<code>x</code> <code>+</code> <code>y</code> sum of <code>x</code> and <code>y</code>	<code>x + y</code>
<code>x - y</code>	difference of <code>x</code> and <code>y</code>	<code>x</code> <code>-</code> <code>y</code> difference of <code>x</code> and <code>y</code>	<code>x - y</code>
<code>x * y</code>	product of <code>x</code> and <code>y</code>	<code>x</code> <code>*</code> <code>y</code> product of <code>x</code> and <code>y</code>	<code>x * y</code>
<code>x / y</code>	quotient of <code>x</code> and <code>y</code>	<code>x</code> <code>/</code> <code>y</code> quotient of <code>x</code> and <code>y</code>	<code>x / y</code>
<code>x // y</code>	floored quotient of <code>x</code> and <code>y</code>	<code>x</code> <code>//</code> <code>y</code> floored quotient of <code>x</code> and <code>y</code>	<code>x // y</code>
<code>x % y</code>	remainder of <code>x</code> and <code>y</code>	<code>x</code> <code>%</code> <code>y</code> remainder of <code>x</code> and <code>y</code>	<code>x % y</code>
<code>-x</code>	negated <code>x</code>	<code>-x</code> <code>x</code> negated	<code>-x</code>
<code>abs(x)</code>	absolute value or magnitude of <code>x</code>	<code>abs(x)</code> absolute value or magnitude of <code>x</code>	<code>abs(x)</code>
<code>int(x)</code>	<code>x</code> converted to integer	<code>(3)(6)</code> <code>int()</code> <code>float(x)</code> <code>x</code> converted to floating point	<code>int(x)</code>
<code>float(x)</code>	<code>x</code> converted to floating point	<code>(4)(6)</code> <code>float()</code> <code>complex(re, im)</code> a complex number with real part <code>re</code> and imaginary part <code>im</code>	<code>float(x)</code>
<code>complex(re, im)</code>	a complex number with real part <code>re</code> and imaginary part <code>im</code>	<code>(6)</code> <code>complex()</code> <code>c.conjugate()</code> conjugate of the complex number <code>c</code>	<code>complex(re, im)</code>
<code>divmod(x, y)</code>	the pair <code>(x // y, x % y)</code>	<code>(2)</code> <code>divmod()</code> <code>pow(x, y)</code> <code>x</code> to the power of <code>y</code>	<code>divmod(x, y)</code>

Notes:

- Also referred to as integer division. For operands of type `int`, the result has type `int`. For operands of type `float`, the result has type `float`. In general, the result is a whole integer, though the result's type is not necessarily `int`. The result is always rounded towards minus infinity: `1//2` is `0`, `(-1)//2` is `-1`, `1//(-2)` is `-1`, and `(-1)//(-2)` is `0`.
- Not for complex numbers. Instead convert to floats using `abs()` if appropriate.
- Conversion from `float` to `int` truncates, discarding the fractional part. See functions `math.floor()` and `math.ceil()` for alternative conversions.
- `float` also accepts the strings "nan" and "inf" with an optional prefix "+" or "-" for Not a Number (NaN) and positive or negative infinity.
- Python defines `pow(0,0)` and `0**0` to be `1`, as is common for programming languages.
- The numeric literals accepted include the digits `0` to `9` or any Unicode equivalent (code points with the `Nd` property). See the Unicode Standard for a complete list of code points with the `Nd` property.

All numbers. Real types (`int` and `float`) also include the following operations:

Operation	Result	Notes
<code>math.trunc(x)</code>	<code>x</code> truncated to an integer	<code>round(x[, n])</code> <code>x</code> rounded to <code>n</code> digits, rounding half to even. If <code>n</code> is omitted, it defaults to <code>0</code> .
<code>math.floor(x)</code>	the greatest integer less than or equal to <code>x</code>	<code>math.ceil(x)</code> the least integer greater than or equal to <code>x</code>

For additional numeric operations see the `math` and `cmath` modules.

Bitwise Operations on Integer Types

Bitwise operations only make sense for integers. The result of bitwise operations is calculated as though carried out in two's complement with an infinite number of sign bits.

The priorities of the binary bitwise operations are all lower than the numeric operations and higher than the comparisons; the unary operation `~` has the same priority as the other unary numeric operations (`+` and `-`).

This table lists the bitwise operations sorted in ascending priority:

Operation	Result	Notes
<code>x y</code>	bitwise or of <code>x</code> and <code>y</code>	<code>(4)</code> <code>x ^ y</code> bitwise exclusive or of <code>x</code> and <code>y</code>
<code>x & y</code>	bitwise and of <code>x</code> and <code>y</code>	<code>(4)</code> <code>x & y</code> bitwise and of <code>x</code> and <code>y</code>
<code>x << n</code>	<code>x</code> shifted left by <code>n</code> bits	<code>(1)(2)</code> <code>x >> n</code> <code>x</code> shifted right by <code>n</code> bits
<code>~x</code>	the bits of <code>x</code> inverted	<code>(1)(3)</code> <code>~x</code> the bits of <code>x</code> inverted

Notes:

- Negative shift counts are illegal and cause a `ValueError` to be raised.
- A left shift by `n` bits is equivalent to multiplication by `pow(2, n)`.
- A right shift by `n` bits is equivalent to floor division by `pow(2, n)`.
- Performing these calculations with at least one extra sign extension bit in a finite two's complement representation (a working bit-width of `1 + max(x.bit_length(), y.bit_length())` or more) is sufficient to get the same result as if there were an infinite number of sign bits.

Additional Methods on Integer Types

The `int` type implements the `numbers.Integral` abstract base class. In addition, it provides a few more methods:

`int.bit_length()`

Return the number of bits necessary to represent an integer in binary, excluding the sign and leading zeros: `>>>n=-37>>>bin(n)'-0b100101'>>>n.bit_length()6` More precisely, if `x` is nonzero, then `x.bit_length()` is the unique positive integer `k` such that $2^{k-1} \leq \text{abs}(x) < 2^k$. Equivalently, when `abs(x)` is small enough to have a correctly rounded logarithm, then `k = 1 + int(log(abs(x), 2))`. If `x` is zero, then `x.bit_length()` returns 0. Equivalent to: `def bit_length(self): s = bin(self) # binary representation: bin(-37) --> '-0b`

`int.bit_count()`

Return the number of ones in the binary representation of the absolute value of the integer. This is also known as the population count. Example: `>>>n=19>>>bin(n)'0b10011'>>>n.bit_count()3>>>(-n).bit_count()3` Equivalent to: `def bit_count(self): return bin(self).count("1")` Added in version 3.10.

`int.to_bytes(length=1, byteorder='big', *, signed=False)`

Return an array of bytes representing an integer. `>>>(1024).to_bytes(2, byteorder='big')b'\x04\x00'>>>(1024).to_bytes(10, byteorder='big')b'\x00\x00\x00\x00\x00\x00\x00\x00\x04\x00'>>>(-1024).to_bytes(10, byteorder='big', signed=True)b'\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xfc\x00'>>>x=1000>>>x.to_bytes((x.bit_length()+7)//8, byteorder='little')b'\xe8\x03'` The integer is represented using `length` bytes, and defaults to 1. An `OverflowError` is raised if the integer is not representable with the given number of bytes.

`classmethod int.from_bytes(bytes, byteorder='big', *, signed=False)`

Return the integer represented by the given array of bytes. `>>>int.from_bytes(b'\x00\x10', byteorder='big')16>>>int.from_bytes(b'\x00\x10', byteorder='little')4096>>>int.from_bytes(b'\xfc\x00', byteorder='big', signed=True)-1024>>>int.from_bytes(b'\xfc\x00', byteorder='big', signed=False)64512>>>int.from_bytes([255, 0], byteorder='big')16711680` The argument `bytes` must either be a bytes-like object or an iterable producing bytes. The `byteorder` argument determines the byte order used to represent the integer, an

`int.as_integer_ratio()`

Return a pair of integers whose ratio is equal to the original integer and has a positive denominator. The integer ratio of integers (whole numbers) is always the integer as the numerator and 1 as the denominator. Added in version 3.8.

`int.is_integer()`

Returns `True`. Exists for duck type compatibility with `float.is_integer()`. Added in version 3.12.

Additional Methods on Float

The `float` type implements the `numbers.Real` abstract base class. `float` also has the following additional methods.

`classmethod float.from_number(x)`

Class method to return a floating-point number constructed from a number `x`. If the argument is an integer or a floating-point number, a floating-point number with the same value (within Python's floating-point precision) is returned. If the argument is outside the range of a Python float, an `OverflowError` will be raised. For a general Python object `x`, `float.from_number(x)` delegates to `x.__float__()`. If `__float__()` is not defined then it falls back to `__index__()`. Added in version 3.14.

`float.as_integer_ratio()`

Return a pair of integers whose ratio is exactly equal to the original float. The ratio is in lowest terms and has a positive denominator. Raises `OverflowError` on infinities and a `ValueError` on NaNs.

`float.is_integer()`

Return True if the float instance is finite with integral value, and False otherwise:

```
>>> (-2.0).is_integer()
True
>>> (3.2).is_integer()
False
```

Two methods support conversion to and from hexadecimal strings. Since Python's floats are stored internally as binary numbers, converting a float to or from a decimal string usually involves a small rounding error. In contrast, hexadecimal strings allow exact representation and specification of floating-point numbers. This can be useful when debugging, and in numerical work.

float.hex()

Return a representation of a floating-point number as a hexadecimal string. For finite floating-point numbers, this representation will always include a leading 0x and a trailing p and exponent.

classmethod float.fromhex(s)

Class method to return the float represented by a hexadecimal string. The string may have leading and trailing whitespace.

Note that `float.hex()` is an instance method, while `float.fromhex()` is a class method.

A hexadecimal string takes the form:

```
[sign] ['0x'] integer ['.' fraction] ['p' exponent]
```

where the optional sign may be either + or -, integer and fraction are strings of hexadecimal digits, and exponent is a decimal integer with an optional leading sign. Case is not significant, and there must be at least one hexadecimal digit in either the integer or the fraction. This syntax is similar to the syntax specified in section 6.4.4.2 of the C99 standard, and also to the syntax used in Java 1.5 onwards. In particular, the output of `float.hex()` is usable as a hexadecimal floating-point literal in C or Java code, and hexadecimal strings produced by C's %a format character or Java's `Double.toHexString()` are accepted by `float.fromhex()`.

Note that the exponent is written in decimal rather than hexadecimal, and that it gives the power of 2 by which to multiply the coefficient. For example, the hexadecimal string `0x3.a7p10` represents the floating-point number $(3 + 10./16 + 7./16**2) * 2.0**10$, or 3740.0:

```
>>> float.fromhex('0x3.a7p10')
3740.0
```

Applying the reverse conversion to 3740.0 gives a different hexadecimal string representing the same number:

```
>>> float.hex(3740.0)
'0x1.d380000000000p+11'
```

Additional Methods on Complex

The `complex` type implements the `numbers.Complex` abstract base class. `complex` also has the following additional methods.

classmethod complex.from_number(x)

Class method to convert a number to a complex number. For a general Python object `x`, `complex.from_number(x)` delegates to `x.__complex__()`. If `__complex__()` is not defined then it falls back to `__float__()`. If `__float__()` is not defined then it falls back to `__index__()`. Added in version 3.14.

Hashing of numeric types

For numbers `x` and `y`, possibly of different types, it's a requirement that `hash(x) == hash(y)` whenever `x == y` (see the `__hash__()` method documentation for more details). For ease of implementation and efficiency across a variety of numeric types (including `int`, `float`, `decimal.Decimal` and `fractions.Fraction`) Python's hash for numeric types is based on a single mathematical function that's defined for any rational number, and hence applies to all instances of `int` and `fractions.Fraction`, and all finite instances of `float` and `decimal.Decimal`. Essentially, this function is given by reduction modulo `P` for a fixed prime `P`. The value of `P` is made available to Python as

the modulus attribute of `sys.hash_info`.

CPython implementation detail: Currently, the prime used is $P=2^{31}-1$ on machines with 32-bit C longs and $P=2^{61}-1$ on machines with 64-bit C longs.

Here are the rules in detail:

- If $x=m/n$ is a nonnegative rational number and n is not divisible by P , define `hash(x)` as $m \cdot \text{invmod}(n, P) \% P$, where `invmod(n, P)` gives the inverse of n modulo P .
- If $x=m/n$ is a nonnegative rational number and n is divisible by P (but m is not) then n has no inverse modulo P and the rule above doesn't apply; in this case define `hash(x)` to be the constant value `sys.hash_info.inf`.
- If $x=m/n$ is a negative rational number define `hash(x)` as `-hash(-x)`. If the resulting hash is `-1`, replace it with `-2`.
- The particular values `sys.hash_info.inf` and `-sys.hash_info.inf` are used as hash values for positive infinity or negative infinity (respectively).
- For a complex number z , the hash values of the real and imaginary parts are combined by computing `hash(z.real) + sys.hash_info.imag * hash(z.imag)`, reduced modulo $2^{\text{sys.hash_info.width}}$ so that it lies in range $(-2^{(\text{sys.hash_info.width}-1)}, 2^{(\text{sys.hash_info.width}-1)})$. Again, if the result is `-1`, it's replaced with `-2`.

To clarify the above rules, here's some example Python code, equivalent to the built-in hash, for computing the hash of a rational number, float, or complex:

```
import sys, math

def hash_fraction(m, n):
    """Compute the hash of a rational number m / n.

    Assumes m and n are integers, with n positive.
    Equivalent to hash(fractions.Fraction(m, n)).

    """
    P = sys.hash_info.modulus
    # Remove common factors of P. (Unnecessary if m and n already coprime.)
    while m % P == n % P == 0:
        m, n = m // P, n // P

    if n % P == 0:
        hash_value = sys.hash_info.inf
    else:
        # Fermat's Little Theorem: pow(n, P-1, P) is 1, so
        # pow(n, P-2, P) gives the inverse of n modulo P.
        hash_value = (abs(m) % P) * pow(n, P - 2, P) % P
    if m < 0:
        hash_value = -hash_value
    if hash_value == -1:
        hash_value = -2
    return hash_value

def hash_float(x):
    """Compute the hash of a float x."""

    if math.isnan(x):
        return object.__hash__(x)
    elif math.isinf(x):
        return sys.hash_info.inf if x > 0 else -sys.hash_info.inf
    else:
        return hash_fraction(*x.as_integer_ratio())

def hash_complex(z):
    """Compute the hash of a complex number z."""

    hash_value = hash_float(z.real) + sys.hash_info.imag * hash_float(z.imag)
```

```
# do a signed reduction modulo 2**sys.hash_info.width
M = 2**(sys.hash_info.width - 1)
hash_value = (hash_value & (M - 1)) - (hash_value & M)
if hash_value == -1:
    hash_value = -2
return hash_value
```

Boolean Type -bool

Booleans represent truth values. The `bool` type has exactly two constant instances: `True` and `False`.

The built-in function `bool()` converts any value to a boolean, if the value can be interpreted as a truth value (see section [Truth Value Testing](#) above).

For logical operations, use the boolean operators `and`, `or` and `not`. When applying the bitwise operators `&`, `|`, `^` to two booleans, they return a `bool` equivalent to the logical operations “and”, “or”, “xor”. However, the logical operators `and`, `or` and `!=` should be preferred over `&`, `|` and `^`.

Deprecated since version 3.12: The use of the bitwise inversion operator `~` is deprecated and will raise an error in Python 3.16.

`bool` is a subclass of `int` (see [Numeric Types — int, float, complex](#)). In many numeric contexts, `False` and `True` behave like the integers 0 and 1, respectively. However, relying on this is discouraged; explicitly convert using `int()` instead.

Iterator Types

Python supports a concept of iteration over containers. This is implemented using two distinct methods; these are used to allow user-defined classes to support iteration. Sequences, described below in more detail, always support the iteration methods.

One method needs to be defined for container objects to provide `iterable` support:

container.__iter__()

Return an `iterator` object. The object is required to support the iterator protocol described below. If a container supports different types of iteration, additional methods can be provided to specifically request iterators for those iteration types. (An example of an object supporting multiple forms of iteration would be a tree structure which supports both breadth-first and depth-first traversal.) This method corresponds to the `tp_iterslot` of the type structure for Python objects in the Python

The iterator objects themselves are required to support the following two methods, which together form the `iterator` protocol:

iterator.__iter__()

Return the `iterator` object itself. This is required to allow both containers and iterators to be used with the `for` and `in` statements. This method corresponds to the `tp_iterslot` of the type structure for Python objects in the Python/C API.

iterator.__next__()

Return the next item from the `iterator`. If there are no further items, raise the `StopIteration` exception. This method corresponds to the `tp_iternextslot` of the type structure for Python objects in the Python/C API.

Python defines several iterator objects to support iteration over general and specific sequence types, dictionaries, and other more specialized forms. The specific types are not important beyond their implementation of the iterator protocol.

Once an iterator's `__next__()` method raises `StopIteration`, it must continue to do so on subsequent calls. Implementations that do not obey this property are deemed broken.

Generator Types

Python's generators provide a convenient way to implement the iterator protocol. If a container object's `__iter__()` method is implemented as a generator, it will automatically return an iterator object (technically, a generator object) supplying the `__iter__()` and `__next__()` methods. More information about generators can be found in the documentation for the `yield` expression.

Sequence Types —list,tuple,range

There are three basic sequence types: lists, tuples, and range objects. Additional sequence types tailored for processing of binary data and text strings are described in dedicated sections.

Common Sequence Operations

The operations in the following table are supported by most sequence types, both mutable and immutable. The `collections.abc.SequenceABC` is provided to make it easier to correctly implement these operations on custom sequence types.

This table lists the sequence operations sorted in ascending priority. In the table, `s` and `t` are sequences of the same type, `n`, `i`, `j`, and `k` are integers and `x` is an arbitrary object that meets any type and value restrictions imposed by `s`.

The `in` and `not in` operations have the same priorities as the comparison operations. The `+` (concatenation) and `*` (repetition) operations have the same priority as the corresponding numeric operations. [3]

Operation	Result	Notes
<code>x in s</code>	<code>True</code> if an item of <code>s</code> is equal to <code>x</code> , else <code>False</code>	(1) <code>x</code> not in <code>s</code> <code>False</code> if an item of <code>s</code> is equal to <code>x</code> , else <code>True</code>
<code>s + t</code>	the concatenation of <code>s</code> and <code>t</code>	(6)(7) <code>s * n</code> or <code>n * s</code> equivalent to adding <code>s</code> to itself <code>n</code> times
<code>s[i]</code>	<code>i</code> th item of <code>s</code> , origin 0	(3)(8) <code>s[i:j]</code> slice of <code>s</code> from <code>i</code> to <code>j</code>
<code>s[i:j:k]</code>	slice of <code>s</code> from <code>i</code> to <code>j</code> with step <code>k</code>	(3)(5) <code>len(s)</code> length of <code>s</code>
<code>min(s)</code>	smallest item of <code>s</code>	
<code>max(s)</code>	largest item of <code>s</code>	

Sequences of the same type also support comparisons. In particular, tuples and lists are compared lexicographically by comparing corresponding elements. This means that to compare equal, every element must compare equal and the two sequences must be of the same type and have the same length. (For full details see `Comparisons` in the language reference.)

Forward and reversed iterators over mutable sequences access values using an index. That index will continue to march forward (or backward) even if the underlying sequence is mutated. The iterator terminates only when an `IndexError` or `StopIteration` is encountered (or when the index drops below zero).

Notes:

- While the `in` and `not in` operations are used only for simple containment testing in the general case, some specialised sequences (such as `str`, `bytes` and `bytearray`) also use them for subsequence testing: `>>> "gg" in "eggs" True`
- Values of `len` less than 0 are treated as 0 (which yields an empty sequence of the same type as `s`). Note that items in the sequences are not copied; they are referenced multiple times. This often haunts new Python programmers; consider: `>>> lists = [] * 3 >>> lists[0], [], [] >>> lists[0].append(3) >>> lists [[3], [3], [3]]` What has happened is that `[]` is a one-element list containing an empty list, so all three elements
- If `i` or `j` is negative, the index is relative to the end of sequences: `len(s) + i` or `len(s) + j` is substituted. But note that `-0` is still 0.

- The slice `s[from:to]` is defined as the sequence of items with index `x` such that $i \leq x < j$. If `i` is omitted or `None`, use `0`. If `j` is omitted or `None`, use `len(s)`. If `i` is less than `-len(s)`, use `0`. If `j` is greater than `len(s)`, use `len(s)`. If `i` is greater than or equal to `j`, the slice is empty.
- The slice `s[from:to:step]` with `step` is defined as the sequence of items with index $x = i + n * k$ such that $0 \leq n < (j - i) / k$. In other words, the indices are `i`, `i+k`, `i+2*k`, `i+3*k` and so on, stopping when `j` is reached (but never including `j`). When `k` is positive, `i` and `j` are reduced to `len(s)` if they are greater. When `k` is negative, `i` and `j` are reduced to `len(s) - 1` if they are greater. If `i` or `j` are omitted or `None`, they become “end” values (where `end` is `0` if `step` is positive and `len(s) - 1` if `step` is negative).
- Concatenating immutable sequences always results in a new object. This means that building up a sequence by repeated concatenation will have a quadratic runtime cost in the total sequence length. To get a linear runtime cost, you must switch to one of the alternatives below: if concatenating `str` objects, you can build a list and use `str.join()` at the end or else write to `io.StringIO` instance and return its value.
- Some sequence types (such as `range`) only support item sequences that follow specific patterns, and hence don't support sequence concatenation or repetition.
- An `IndexError` is raised if `i` is outside the sequence range.

Sequence Methods

Sequence types also support the following methods:

`sequence.count(value, /)`

Return the total number of occurrences of `value` in `sequence`.

`sequence.index(value[, start[, stop]])`

Return the index of the first occurrence of `value` in `sequence`. Raises `ValueError` if `value` is not found in `sequence`. The `start` or `stop` arguments allow for efficient searching of subsections of the sequence, beginning at `start` and ending at `stop`. This is roughly equivalent to `start + sequence[start:stop].index(value)`, only without copying any data. Caution: Not all sequence types support passing the `start` and `stop` arguments.

Immutable Sequence Types

The only operation that immutable sequence types generally implement that is not also implemented by mutable sequence types is support for the `hash()` built-in.

This support allows immutable sequences, such as `tuple` instances, to be used as dict keys and stored in set and frozen set instances.

Attempting to hash an immutable sequence that contains unhashable values will result in `TypeError`.

Mutable Sequence Types

The operations in the following table are defined on mutable sequence types.

The `collections.abc.MutableSequenceABC` is provided to make it easier to correctly implement these operations on custom sequence types.

In the table `s` is an instance of a mutable sequence type, `t` is any iterable object and `x` is an arbitrary object that meets any type and value restrictions imposed by `s` (for example, `bytearray` only accepts integers that meet the value restriction $0 \leq x \leq 255$).

Operation	Result	Notes
<code>s[i]</code>	<code>x</code>	item <code>i</code> of <code>s</code> is replaced by <code>x</code>
<code>del s[i]</code>		removes item <code>i</code> of <code>s</code>
<code>s[i:j]</code>	<code>t</code>	slice of <code>s</code> from <code>i</code> to <code>j</code> is replaced by the contents of the iterable <code>t</code>
<code>del s[i:j]</code>		removes the elements of <code>s[i:j]</code> from the list (same as <code>s[i:j] = []</code>)
<code>s[i:j:k]</code>	<code>t</code>	the elements of <code>s[i:j:k]</code> are replaced by those of <code>t</code> (1)
<code>del s[i:j:k]</code>		removes the elements of <code>s[i:j:k]</code> from the list
<code>s += t</code>		<code>s</code> extends <code>s</code> with the contents of <code>t</code> (for the most part the same as <code>s[len(s):len(s)] = t</code>)
<code>s *= n</code>		updates <code>s</code> with its contents repeated <code>n</code> times (2)

Notes:

- If `k` is not equal to 1, `t` must have the same length as the slice it is replacing.
- The `value` is an integer, or an object implementing `__index__()`. Zero and negative values of `n` clear the sequence. Items in the sequence are not copied; they are referenced multiple times, as explained for `*n` under Common Sequence Operations.

Mutable Sequence Methods

Mutable sequence types also support the following methods:

sequence.append(value, /)

Append `value` to the end of the sequence. This is equivalent to writing `seq[len(seq):len(seq)]=[value]`.

sequence.clear()

Added in version 3.3. Remove all items from `sequence`. This is equivalent to writing `del sequence[:]`.

sequence.copy()

Added in version 3.3. Create a shallow copy of `sequence`. This is equivalent to writing `sequence[:]`. Hint: The `copy()` method is not part of the `MutableSequenceABC`, but most concrete mutable sequence types provide it.

sequence.extend(iterable, /)

Extend `sequence` with the contents of `iterable`. For the most part, this is the same as writing `seq[len(seq):len(seq)]=iterable`.

sequence.insert(index, value, /)

Insert `value` into `sequence` at the given `index`. This is equivalent to writing `sequence[index:index]=[value]`.

sequence.pop(index=-1, /)

Retrieve the item at `index` and also removes it from `sequence`. By default, the last item in `sequence` is removed and returned.

sequence.remove(value, /)

Remove the first item from `sequence` where `sequence[i]==value`. Raises `ValueError` if `value` is not found in `sequence`.

sequence.reverse()

Reverse the items of `sequence` in place. This method maintains economy of space when reversing a large sequence. To remind users that it operates by side-effect, it returns `None`.

Lists

Lists are mutable sequences, typically used to store collections of homogeneous items (where the precise degree of similarity will vary by application).

class list(iterable=(), /)

sort(*, key=None, reverse=False)

Lists may be constructed in several ways: Using a pair of square brackets to denote the empty list: `[]` Using square brackets, separating items with commas: `[a], [a,b,c]` Using a list comprehension: `[x for x in iterable]` Using the type constructor: `list()` or `list(iterable)` The constructor builds a list whose items are the same and in the same order as `iterable`'s items. `iterable` may be either a sequence, a container that supports iteration, or an iterator object. If `iterable` is already a list, a copy is made and returned.

This method sorts the list in place, using only comparisons between items. Exceptions are not suppressed - if any comparison operations fail, the entire sort operation will fail (and the list will likely be left in a partially modified state). `sort()` accepts two arguments that can only be passed by keyword (keyword-only arguments): `key` specifies a function of one argument that is used to extract a comparison key from each list element (for example, `key=str.lower`). The key corresponding to each item `i`

See also

For detailed information on thread-safety guarantees for list objects, see Thread safety for list objects.

Tuples

Tuples are immutable sequences, typically used to store collections of heterogeneous data (such as the 2-tuples produced by `enumerate()` built-in). Tuples are also used for cases where an immutable sequence of homogeneous data is needed (such as allowing storage in a set or dict instance).

`tuple(iterable=(),/)`

Tuples may be constructed in a number of ways: Using a pair of parentheses to denote the empty tuple: `()` Using a trailing comma for a singleton tuple: `a,or(a,)` Separating items with commas: `a,b,cor(a,b,c)` Using `tuple()` built-in: `tuple()` or `tuple(iterable)` The constructor builds a tuple whose items are the same and in the same order as `iterable`'s items. `iterable` may be either a sequence, a container that supports iteration, or an iterator object. If `iterable` is already a tuple, it is returned unchanged. For `e`

For heterogeneous collections of data where access by name is clearer than access by index, `collections.namedtuple()` may be a more appropriate choice than a simple tuple object.

Ranges

The `range` type represents an immutable sequence of numbers and is commonly used for looping a specific number of times in for loops.

`class range(stop,/)`

`class range(start,stop,step=1,/)`

`start`

`stop`

`step`

The arguments to the range constructor must be integers (either built-in int or any object that implements the `__index__()` special method). If the `step` argument is omitted, it defaults to 1. If the `start` argument is omitted, it defaults to 0. If `step` is zero, `ValueError` is raised. For a positive `step`, the contents of a range are determined by the formula `r[i]=start+step*i` where `i ≥ 0` and `r[i] < stop`. For a negative `step`, the contents of the range are still determined by the formula `r[i]=start+step*i`, but the constraints

The value of the `start` parameter (or 0 if the parameter was not supplied)

The value of the `stop` parameter

The value of the `step` parameter (or 1 if the parameter was not supplied)

The advantage of the `range` type over a regular list or tuple is that a range object will always take the same (small) amount of memory, no matter the size of the range it represents (as it only stores the `start`, `stop` and `step` values, calculating individual items and subranges as needed).

Range objects implement the `collections.abc.Sequence` ABC, and provide features such as containment tests, element index lookup, slicing and support for negative indices (see Sequence Types — list, tuple, range):

```
>>> r = range(0, 20, 2)
>>> r
```

```
range(0, 20, 2)
>>> 11 in r
False
>>> 10 in r
True
>>> r.index(10)
5
>>> r[5]
10
>>> r[:5]
range(0, 10, 2)
>>> r[-1]
18
```

Testing range objects for equality with `==` and `!=` compares them as sequences. That is, two range objects are considered equal if they represent the same sequence of values. (Note that two range objects that compare equal might have different `start`, `stop` and `step` attributes, for example `range(0) == range(2, 1, 3)` or `range(0, 3, 2) == range(0, 4, 2)`.)

Changed in version 3.2: Implement the Sequence ABC. Support slicing and negative indices. Testintobjects for membership in constant time instead of iterating through all items.

Changed in version 3.3: Define '==' and '!=' to compare range objects based on the sequence of values they define (instead of comparing based on object identity).

Added thestart,stopandstepattributes.

See also

- The `linspace` recipe shows how to implement a lazy version of `range` suitable for floating-point applications.

Text and Binary Sequence Type Methods Summary

The following table summarizes the text and binary sequence types methods by category.

Category | str | methods | bytes | and | bytearray | methods | Formatting | str.format() | str.format_map() | f-strings | printf-style String Formatting | printf-style Bytes Formatting | Searching and Replacing | str.find() | str.rfind() | bytes.find() | bytes.rfind() | str.index() | str.rindex() | bytes.index() | bytes.rindex() | str.startswith() | bytes.startswith() | str.endswith() | bytes.endswith() | str.count() | bytes.count() | str.replace() | bytes.replace() | Splitting and Joining | str.split() | str.rsplit() | bytes.split() | bytes.rsplit() | str.splitlines() | bytes.splitlines() | str.partition() | bytes.partition() | str.rpartition() | bytes.rpartition() | str.join() | bytes.join() | String Classification | str.isalpha() | bytes.isalpha() | str.isdecimal() | str.isdigit() | byt

Text Sequence Type —str

Textual data in Python is handled with `str` objects, or `strings`. Strings are immutable sequences of Unicode code points. String literals are written in a variety of ways:

- Single quotes: 'allowsemdouble"quotes'
- Double quotes: "allowsemdouble'single'quotes"
- Triple quoted: """Threesinglequotes""", """Threedoublequotes"""

Triple quoted strings may span multiple lines - all associated whitespace will be included in the string literal.

String literals that are part of a single expression and have only whitespace between them will be implicitly converted to a single string literal. That is, `("spam""eggs")=="spameggs"`.

See `String` and `Bytes` literals for more about the various forms of string literal, including supported escape sequences, and the `"raw"` prefix that disables most escape sequence processing.

Strings may also be created from other objects using the `str` constructor.

Since there is no separate "character" type, indexing a string produces strings of length 1. That is, for a non-empty string `s`, `s[0]==s[0:1]`.

There is also no mutable string type, but `str.join()` or `io.StringIO` can be used to efficiently construct strings from multiple fragments.

Changed in version 3.3: For backwards compatibility with the Python 2 series, the `r` prefix is once again permitted on string literals. It has no effect on the meaning of string literals and cannot be combined with the `r` prefix.

`class str(*, encoding='utf-8', errors='strict')`

`class str(object)`

`class str(object, encoding, errors='strict')`

`class str(object, *, errors)`

Return a string version of `object`. If `object` is not provided, returns the empty string. Otherwise, the behavior of `str()` depends on whether `encoding` or `errors` is given, as follows. If neither `encoding` nor `errors` is given, `str(object)` returns `type(object).__str__(object)`, which is the "informal" or nicely printable string representation of `object`. For string objects, this is the string itself. If `object` does not have a `__str__()` method, then `str()` falls back to returning `repr(object)`. If at least one of `encoding` or `errors` is

String Methods

Strings implement all of the common sequence operations, along with the additional methods described below.

Strings also support two styles of string formatting, one providing a large degree of flexibility and customization (see `str.format()`, `Format string syntax` and `Custom string formatting`) and the other based on `Cprintf` style formatting that handles a narrower range of types and is slightly harder to use correctly, but is often faster for the cases it can handle (`printf`-style `String Formatting`).

The `Text Processing Services` section of the standard library covers a number of other modules that provide various text related utilities (including regular expression support in `re` module).

`str.capitalize()`

Return a copy of the string with its first character capitalized and the rest lowercased. Changed in version 3.8: The first character is now put into titlecase rather than uppercase. This means that characters like digraphs will only have their first letter capitalized, instead of the full character.

`str.casefold()`

Return a casefolded copy of the string. Casefolded strings may be used for caseless matching. Casefolding is similar to lowercasing but more aggressive because it is intended to remove all case distinctions in a string. For example, the German lowercase letter 'ß' is equivalent to "ss". Since it is already lowercase, `lower()` would do nothing to 'ß'; `casefold()` converts it to "ss". For

example: `>>>'straße'.lower()'straße'>>>'straße'.casefold()'strasse'` The casefolding algorithm is described in section 3.13 'D'

`str.center(width, fillchar=' ', /)`

Return centered in a string of length `width`. Padding is done using the specified `fillchar` (default is an ASCII space). The original string is returned if `width` is less than or equal to `len(s)`. For example: `>>>'Python'.center(10)'Python ' >>>'Python'.center(10, '-')--Python--' >>>'Python'.center(4)'Python'`

`str.count(sub[, start[, end]])`

Return the number of non-overlapping occurrences of substringsubin the range [start,end]. Optional argumentsstartandendare interpreted as in slice notation.Ifsubis empty, returns the number of empty strings between characters which is the length of the string plus one. For example:>>>'spam, spam, spam'.count('spam')3>>>'spam, spam, spam'.count('spam',5)2>>>'spam, spam, spam'.count('spam',5,10)1>>>'spam, spam, spam'.count('eggs')0>>>'spam, spam, spam'.count("")17

str.encode(encoding='utf-8',errors='strict')

Return the string encoded tobytes.encodingdefaults to'utf-8'; seeStandard Encodingsfor possible values.errorscontrols how encoding errors are handled. If'strict'(the default), aUnicodeErrorexception is raised. Other possible values are'ignore','replace','xmlcharrefreplace','backslashreplace'and any other name registered viacodecs.register_error(). SeeError Handlersfor details.For performance reasons, the value oferrorsis not checked for validity unless an encoding error actually occurs,Python De

str.endswith(suffix[,start[,end]])

ReturnTrueif the string ends with the specifiedsuffix, otherwise returnFalse.suffixcan also be a tuple of suffixes to look for. With optionalstart, test beginning at that position. With optionalend, stop comparing at that position. Usingstartandendis equivalent tostr[start:end].endswith(suffix). For example:>>>'Python'.endswith('on')True>>>'a tuple of suffixes'.endswith(('at','in'))False>>>'a tuple of suffixes'.endswith(('at','es'))True>>>'Python is amazing'.endswith('is',0,9)TrueSee alsostart

str.expandtabs(tabsize=8)

Return a copy of the string where all tab characters are replaced by one or more spaces, depending on the current column and the given tab size. Tab positions occur everytabsizecharacters (default is 8, giving tab positions at columns 0, 8, 16 and so on). To expand the string, the current column is set to zero and the string is examined character by character. If the character is a tab (t), one or more space characters are inserted in the result until the current column is equal to the next

str.find(sub[,start[,end]])

Return the lowest index in the string where substringsubis found within the slices[start:end]. Optional argumentsstartandendare interpreted as in slice notation. Return-1ifsubis not found. For example:>>>'spam, spam, spam'.find('sp')0>>>'spam, spam, spam'.find('sp',5)6See alsorfind()andindex().NoteThefind()method should be used only if you need to know the position ofsub. To check ifsubis a substring or not, use theinoperator:>>>'Py'in'Python'True

str.format(*args,kwargs)**

Perform a string formatting operation. The string on which this method is called can contain literal text or replacement fields delimited by braces{}. Each replacement field contains either the numeric index of a positional argument, or the name of a keyword argument. Returns a copy of the string where each replacement field is replaced with the string value of the corresponding argument. For example:>>>"The sum of 1 + 2 is{0}".format(1+2)'The sum of 1 + 2 is 3'>>>"The sum of{a}+{b}is{answer}"

str.format_map(mapping,/)

Similar tostr.format(**mapping), except thatmappingis used directly and not copied to adict. This is useful if for examplemappingis a dict

```
subclass:>>>classDefault(dict):...def__missing__(self,key):...returnkey...>>>'{name}was born in{country}'.format_map(Default(name='Guido'))'Guido was born in country'Added in version 3.2.
```

str.index(sub[,start[,end]])

Likefind(), but raiseValueErrorwhen the substring is not found. For example:>>>'spam, spam, spam'.index('spam')0>>>'spam, spam, spam'.index('eggs')Traceback (most recent call last):File"<python-input-0>", line1, in<module>'spam, spam, spam'.index('eggs')~~~~~ValueError:substring not foundSee alsorindex().

str.isalnum()

Return True if all characters in the string are alphanumeric and there is at least one character, False otherwise. A character is alphanumeric if one of the following returns True: `c.isalpha()`, `c.isdecimal()`, `c.isdigit()`, or `c.isnumeric()`. For example: `>>>'abc123'.isalnum()True>>>'abc123!@#'.isalnum()False>>>''.isalnum()False>>>' '.isalnum()False`

str.isalpha()

Return True if all characters in the string are alphabetic and there is at least one character, False otherwise. Alphabetic characters are those characters defined in the Unicode character database as “Letter”, i.e., those with general category property being one of “Lm”, “Lt”, “Lu”, “Ll”, or “Lo”. Note that this is different from the `isAlphabetic` property defined in the section 4.10 ‘Letters, Alphabetic, and Ideographic’ of the Unicode Standard. For example: `>>>'Letters and spaces'.isalpha()False>>>'`

str.isascii()

Return True if the string is empty or all characters in the string are ASCII, False otherwise. ASCII characters have code points in the range U+0000-U+007F. For example: `>>>'ASCII characters'.isascii()True>>>'μ'.isascii()False` Added in version 3.7.

str.isdecimal()

Return True if all characters in the string are decimal characters and there is at least one character, False otherwise. Decimal characters are those that can be used to form numbers in base 10, such as U+0660, ARABIC-INDIC DIGIT ZERO. Formally a decimal character is a character in the Unicode General Category “Nd”. For example: `>>>'0123456789'.isdecimal()True>>>'■■■■■■■■■■'.isdecimal()False` # Arabic-Indic digits zero to nine True `>>>'alphabetic'.isdecimal()False`

str.isdigit()

Return True if all characters in the string are digits and there is at least one character, False otherwise. Digits include decimal characters and digits that need special handling, such as the compatibility superscript digits. This covers digits which cannot be used to form numbers in base 10, like the Kharosthi numbers. Formally, a digit is a character that has the property value `Numeric_Type=Digit` or `Numeric_Type=Decimal`.

str.isidentifier()

Return True if the string is a valid identifier according to the language definition, section Names (identifiers and keywords). `keyword.iskeyword()` can be used to test whether a string is a reserved identifier, such as `def` and `class`. Example: `>>>from keyword import iskeyword>>>'hello'.isidentifier(), iskeyword('hello')(True, False)>>>'def'.isidentifier(), iskeyword('def')(True, True)`

str.islower()

Return True if all cased characters in the string are lowercase and there is at least one cased character, False otherwise.

str.isnumeric()

Return True if all characters in the string are numeric characters, and there is at least one character, False otherwise. Numeric characters include digit characters, and all characters that have the Unicode numeric value property, e.g. U+2155, VULGAR FRACTION ONE FIFTH. Formally, numeric characters are those with the property value `Numeric_Type=Digit`, `Numeric_Type=Decimal` or `Numeric_Type=Numeric`. For example: `>>>'0123456789'.isnumeric()True>>>'■■■■■■■■■■'.isnumeric()False` # Arabic-indic digit zero to nine

str.isprintable()

Return True if all characters in the string are printable, False if it contains at least one non-printable character. Here “printable” means the character is suitable for representation to use in its output; “non-printable” means

that repr() on built-in types will hex-escape the character. It has no bearing on the handling of strings written to sys.stdout or sys.stderr. The printable characters are those which in the Unicode character database (see unicodedata) have a general category in group Letter, Mark, Number,

str.isspace()

Return True if there are only whitespace characters in the string and there is at least one character, False otherwise. For example: >>> ''.isspace() False >>> ' '.isspace() True >>> '\t\n'.isspace() # TAB and BREAK LINE True >>> '\u3000'.isspace() # IDEOGRAPHIC SPACE True A character is whitespace if in the Unicode character database (see unicodedata), either its general category is Zs (“Separator, space”), or its bidirectional class is one of WS, B, or S. See also isprintable().

str.istitle()

Return True if the string is a titlecased string and there is at least one character, for example uppercase characters may only follow uncased characters and lowercase characters only cased ones. Return False otherwise. For example: >>> 'Spam, Spam, Spam'.istitle() True >>> 'spam, spam, spam'.istitle() False >>> 'SPAM, SPAM, SPAM'.istitle() False See also title().

str.isupper()

Return True if all cased characters [4] in the string are uppercase and there is at least one cased character, False otherwise. >>> 'BANANA'.isupper() True >>> 'banana'.isupper() False >>> 'baNana'.isupper() False >>> '.isupper() False

str.join(iterable, /)

Return a string which is the concatenation of the strings iterable. A TypeError will be raised if there are any non-string values in iterable, including bytes objects. The separator between elements is the string providing this method. For example: >>> ', '.join(['spam', 'spam', 'spam']) 'spam, spam, spam' >>> '-'.join('Python') 'P-y-t-h-o-n' See also split().

str.ljust(width, fillchar=" ", /)

Return the string left justified in a string of length width. Padding is done using the specified fillchar (default is an ASCII space). The original string is returned if width is less than or equal to len(s). For example: >>> 'Python'.ljust(10) 'Python ' >>> 'Python'.ljust(10, '.') 'Python....' >>> 'Monty Python'.ljust(10, '.') 'Monty Python' See also rjust().

str.lower()

Return a copy of the string with all the cased characters [4] converted to lowercase. For example: >>> 'Lower Method Example'.lower() 'lower method example' The lowercasing algorithm used is described in section 3.13 ‘Default Case Folding’ of the Unicode Standard.

str.lstrip(chars=None, /)

Return a copy of the string with leading characters removed. The chars argument is a string specifying the set of characters to be removed. If omitted or None, the chars argument defaults to removing whitespace. The chars argument is not a prefix; rather, all combinations of its values are stripped: >>> 'spacious'.lstrip() 'spacious' >>> 'www.example.com'.lstrip('cmowz.') 'example.com' See str.removeprefix() for a method that will remove a single prefix string rather than all of a set of characters.

static str.maketrans(dict, /)

static str.maketrans(from, to, remove=" ", /)

This static method returns a translation table usable for str.translate(). If there is only one argument, it must be a dictionary mapping Unicode ordinals (integers) or characters (strings of length 1) to Unicode ordinals, strings (of arbitrary lengths) or None. Character keys will then be converted to ordinals. If there are two arguments, they

must be strings of equal length, and in the resulting dictionary, each character infromwill be mapped to the character at the same position into. If there

str.partition(sep, /)

Split the string at the first occurrence ofsep, and return a 3-tuple containing the part before the separator, the separator itself, and the part after the separator. If the separator is not found, return a 3-tuple containing the string itself, followed by two empty strings. For example:>>>'Monty Python'.partition(' ')('Monty', ' ', 'Python')>>>"Monty Python's Flying Circus".partition(' ')('Monty', ' ', "Python's Flying Circus")>>>'Monty Python'.partition('-')('Monty Python', '-', '')See also rpartition()

str.removeprefix(prefix, /)

If the string starts with the prefixstring, returnstring[len(prefix):]. Otherwise, return a copy of the original string:>>>'TestHook'.removeprefix('Test')'Hook'>>>'BaseTestCase'.removeprefix('Test')'BaseTestCase'Added in version 3.9. See also removesuffix() and startswith().

str.removesuffix(suffix, /)

If the string ends with the suffixstring and that suffixis not empty, returnstring[:-len(suffix)]. Otherwise, return a copy of the original string:>>>'MiscTests'.removesuffix('Tests')'Misc'>>>'TmpDirMixin'.removesuffix('Tests')'TmpDirMixin'Added in version 3.9. See also removeprefix() and endswith().

str.replace(old, new, /, count=-1)

Return a copy of the string with all occurrences of substringgoldreplaced bynew. Ifcountis given, only the firstcountoccurrences are replaced. Ifcountis not specified or-1, then all occurrences are replaced. For example:>>>'spam, spam, spam'.replace('spam', 'eggs')'eggs, eggs, eggs'>>>'spam, spam, spam'.replace('spam', 'eggs', 1)'eggs, spam, spam'Changed in version 3.13: countis now supported as a keyword argument.

str.rfind(sub[, start[, end]])

Return the highest index in the string where substringsubis found, such thatsubis contained within s[start:end]. Optional argumentsstartandendare interpreted as in slice notation. Return-1on failure. For example:>>>'spam, spam, spam'.rfind('sp')12>>>'spam, spam, spam'.rfind('sp', 0, 10)6See also find() and rindex().

str.rindex(sub[, start[, end]])

Like rfind()but raises ValueErrorwhen the substringsubis not found. For example:>>>'spam, spam, spam'.rindex('spam')12>>>'spam, spam, spam'.rindex('eggs')Traceback (most recent call last):File "<stdin>-0>", line 1, in <module>'spam, spam, spam, spam'.rindex('eggs')~~~~~ValueError: substring not foundSee also index() and find().

str.rjust(width, fillchar=" ", /)

Return the string right justified in a string of lengthwidth. Padding is done using the specified fillchar(default is an ASCII space). The original string is returned ifwidthis less than or equal to len(s). For example:>>>'Python'.rjust(10)' Python'>>>'Python'.rjust(10, '.')'.....Python'>>>'Monty Python'.rjust(10, '.')'Monty Python'See also ljust() and zfill().

str.rpartition(sep, /)

Split the string at the last occurrence ofsep, and return a 3-tuple containing the part before the separator, the separator itself, and the part after the separator. If the separator is not found, return a 3-tuple containing two empty strings, followed by the string itself. For example:>>>'Monty Python'.rpartition(' ')('Monty', ' ', 'Python')>>>"Monty Python's Flying Circus".rpartition(' ')("Monty Python's Flying", ' ', 'Circus')>>>'Monty Python'.rpartition('-')(' ', '-', 'Monty Python')See also partition()

str.rsplit(sep=None,maxsplit=-1)

Return a list of the words in the string, using `sep` as the delimiter string. If `maxsplit` is given, at most `maxsplit` splits are done, therightmostones. If `sep` is not specified or `None`, any whitespace string is a separator. Except for splitting from the right, `rsplit()` behaves like `split()` which is described in detail below.

str.rstrip(chars=None,/)

Return a copy of the string with trailing characters removed. The `chars` argument is a string specifying the set of characters to be removed. If omitted or `None`, the `chars` argument defaults to removing whitespace. The `chars` argument is not a suffix; rather, all combinations of its values are stripped. For example: >>> 'spacious'.rstrip() 'spacious' >>> 'mississippi'.rstrip('ipz') 'mississ' See `removeprefix()` for a method that will remove a single suffix string rather than all of a set of characters.

str.split(sep=None,maxsplit=-1)

Return a list of the words in the string, using `sep` as the delimiter string. If `maxsplit` is given, at most `maxsplit` splits are done (thus, the list will have at most `maxsplit+1` elements). If `maxsplit` is not specified or `-1`, then there is no limit on the number of splits (all possible splits are made). If `sep` is given, consecutive delimiters are not grouped together and are deemed to delimit empty strings (for example, '1,,2'.split(',') returns ['1','', '2']). The `sep` argument may consist of multiple characters a

str.splitlines(keepends=False)

Return a list of the lines in the string, breaking at line boundaries. Line breaks are not included in the resulting list unless `keepends` is given and true. This method splits on the following line boundaries. In particular, the boundaries are a superset of universal newlines. Representation Description \n Line Feed \r Carriage Return \r\n Carriage Return + Line Feed \vor \x0b Line Tabulation \for \x0c Form Feed \x1c File Separator \x1d Group Separator \x1e Record Separator \x85 Next Line (C1 Control Code) \u2028 Line Se

str.startswith(prefix[,start[,end]])

Return `True` if string starts with the `prefix`, otherwise return `False`. `prefix` can also be a tuple of prefixes to look for. With optional `start`, test string beginning at that position. With optional `end`, stop comparing string at that position. For example: >>> 'Python'.startswith('Py') `True` >>> 'a tuple of prefixes'.startswith(('at','a')) `True` >>> 'Python is amazing'.startswith('is', 7) `True` See also `endswith()` and `removeprefix()`.

str.strip(chars=None,/)

Return a copy of the string with the leading and trailing characters removed. The `chars` argument is a string specifying the set of characters to be removed. If omitted or `None`, the `chars` argument defaults to removing whitespace. The `chars` argument is not a prefix or suffix; rather, all combinations of its values are stripped. For example: >>> 'spacious'.strip() 'spacious' >>> 'www.example.com'.strip('cmowz.') 'example' The outermost leading and trailing `chars` argument values are stripped from the string. C

str.swapcase()

Return a copy of the string with uppercase characters converted to lowercase and vice versa. Note that it is not necessarily true that `s.swapcase().swapcase()==s`.

str.title()

Return a titlecased version of the string where words start with an uppercase character and the remaining characters are lowercase. For example: >>> 'Hello world'.title() 'Hello World' The algorithm uses a simple language-independent definition of a word as groups of consecutive letters. The definition works in many contexts but it means that apostrophes in contractions and possessives form word boundaries, which may not be the desired result: >>> "they're bill's friends from the UK".title() "They'Re B

str.translate(table,/)

Return a copy of the string in which each character has been mapped through the given translation table. The table must be an object that implements indexing via `__getitem__()`, typically a mapping or sequence. When indexed by a Unicode ordinal (an integer), the table object can do any of the following: return a Unicode ordinal or a string, to map the character to one or more other characters; return `None`, to delete the character from the return string; or raise a `LookupError` exception, to map the cha

str.upper()

Return a copy of the string with all the cased characters[4] converted to uppercase. Note that `str.upper().isupper()` might be `False` if `s` contains uncased characters or if the Unicode category of the resulting character(s) is not “Lu” (Letter, uppercase), but e.g. “Lt” (Letter, titlecase). The uppercasing algorithm used is described in section 3.13 ‘Default Case Folding’ of the Unicode Standard.

str.zfill(width, /)

Return a copy of the string left filled with ASCII ‘0’ digits to make a string of length `width`. A leading sign prefix (‘+’/‘-’) is handled by inserting the padding after the sign character rather than before. The original string is returned if `width` is less than or equal to `len(s)`. For example: `>>> "42".zfill(5)` ‘00042’ `>>> "-42".zfill(5)` ‘-0042’ See also `just()`.

Formatted String Literals (f-strings)

Added in version 3.6.

Changed in version 3.7: The `await` and `async for` can be used in expressions within f-strings.

Changed in version 3.8: Added the debug specifier (`=`)

Changed in version 3.12: Many restrictions on expressions within f-strings have been removed. Notably, nested strings, comments, and backslashes are now permitted.

An f-string (formally a formatted string literal) is a string literal that is prefixed with `f` or `F`. This type of string literal allows embedding the results of arbitrary Python expressions within replacement fields, which are delimited by curly brackets (`{}`). Each replacement field must contain an expression, optionally followed by:

- a debug specifier – an equal sign (`=`);
- a conversion specifier – `!s`, `!r` or `!a`; and/or
- a format specifier prefixed with a colon (`:`).

See the [Lexical Analysis](#) section on f-strings for details on the syntax of these fields.

Debug specifier

Added in version 3.8.

If a debug specifier – an equal sign (`=`) – appears after the replacement field expression, the resulting f-string will contain the expression’s source, the equal sign, and the value of the expression. This is often useful for debugging:

```
>>> number = 14.3
>>> f'{number=}'
'number=14.3'
```

Whitespace before, inside and after the expression, as well as whitespace after the equal sign, is significant — it is retained in the result:

```
>>> f'{ number - 4 = }'
' number - 4 = 10.3'
```

Conversion specifier

By default, the value of a replacement field expression is converted to a string using `str()`:

```
>>> from fractions import Fraction
>>> one_third = Fraction(1, 3)
>>> f'{one_third}'
'1/3'
```

When a debug specifier but no format specifier is used, the default conversion instead uses `repr()`:

```
>>> f'{one_third = }'
'one_third = Fraction(1, 3)'
```

The conversion can be specified explicitly using one of these specifiers:

- `!sforstr()`
- `!rforrepr()`
- `!aforascii()`

For example:

```
>>> str(one_third)
'1/3'
>>> repr(one_third)
'Fraction(1, 3)'
```



```
>>> f'{one_third!s} is {one_third!r}'
'1/3 is Fraction(1, 3)'
```



```
>>> string = "jko█ka █!"
>>> ascii(string)
"'\xa1ko\u010dka \U0001f638!'"
```



```
>>> f'{string = !a}'
"string = '\xa1ko\u010dka \U0001f638!'"
```

Format specifier

After the expression has been evaluated, and possibly converted using an explicit conversion specifier, it is formatted using the `format()` function. If the replacement field includes a format specifier introduced by a colon (:), the specifier is passed to `format()` as the second argument. The result of `format()` is then used as the final value for the replacement field. For example:

```
>>> from fractions import Fraction
>>> one_third = Fraction(1, 3)
>>> f'{one_third:.6f}'
'0.333333'
>>> f'{one_third:_^+10}'
'__+1/3__'
>>> f'{one_third!r:_^20}'
'__Fraction(1, 3)__'
>>> f'{one_third = :~>10}~'
'one_third = ~~~~~~1/3~'
```

Template String Literals (t-strings)

Ant-string (formally a template string literal) is a string literal that is prefixed with `t` or `T`.

These strings follow the same syntax and evaluation rules as formatted string literals, with the following differences:

- Rather than evaluating to an object, template string literals evaluate to a `string.Template` object.
- The `format()` protocol is not used. Instead, the format specifier and conversions (if any) are passed to a new `Interpolation` object that is created for each evaluated expression. It is up to code that processes the

resulting `Template` object to decide how to handle format specifiers and conversions.

- Format specifiers containing nested replacement fields are evaluated eagerly, prior to being passed to the `Interpolation` object. For instance, an interpolation of the form `{amount:.{precision}f}` will evaluate the inner expression `{precision}` to determine the value of the `format_spec` attribute. If `precision` were to be `2`, the resulting format specifier would be `%.2f`.
- When the equals sign `=` is provided in an interpolation expression, the text of the expression is appended to the literal string that precedes the relevant interpolation. This includes the equals sign and any surrounding whitespace. The `Interpolation` instance for the expression will be created as normal, except that `conversion` will be set to `'r'` (`repr()`) by default. If an explicit conversion or format

printf-style String Formatting

■ **Note:** The formatting operations described here exhibit a variety of quirks that lead to a number of common errors (such as failing to display tuples and dictionaries correctly). Using formatted string literals, the `str.format()` interface, or `string.Template` may help avoid these errors. Each of these alternatives provides their own trade-offs and benefits of simplicity, flexibility, and/or extensibility.

String objects have one unique built-in operation: the `%` operator (modulo). This is also known as the string formatting or interpolation operator. Given `format%values` (where `format` is a string), `%` conversion specifications in `format` are replaced with zero or more elements of `values`. The effect is similar to using the `printf()` function in the C language. For example:

```
>>> print('%s has %d quote types.' % ('Python', 2))
Python has 2 quote types.
```

If `format` requires a single argument, `values` may be a single non-tuple object.^[5] Otherwise, `values` must be a tuple with exactly the number of items specified by the format string, or a single mapping object (for example, a dictionary).

A conversion specifier contains two or more characters and has the following components, which must occur in this order:

- The `%` character, which marks the start of the specifier.
- Mapping key (optional), consisting of a parenthesised sequence of characters (for example, `(somename)`).
- Conversion flags (optional), which affect the result of some conversion types.
- Minimum field width (optional). If specified as an `*` (asterisk), the actual width is read from the next element of the tuple in `values`, and the object to convert comes after the minimum field width and optional precision.
- Precision (optional), given as a `.` (dot) followed by the precision. If specified as `*` (an asterisk), the actual precision is read from the next element of the tuple in `values`, and the value to convert comes after the precision.
- Length modifier (optional).
- Conversion type.

When the right argument is a dictionary (or other mapping type), then the formats in the string must include a parenthesised mapping key into that dictionary inserted immediately after the `%` character. The mapping key selects the value to be formatted from the mapping. For example:

```
>>> print('%(language)s has %(number)03d quote types.' %
...       {'language': "Python", "number": 2})
Python has 002 quote types.
```

In this case `no*` specifiers may occur in a format (since they require a sequential parameter list).

The conversion flag characters are:

Flag	Meaning
<code>'#'</code>	The value conversion will use the “alternate form” (where defined below).
<code>'0'</code>	The conversion will be zero padded for numeric values.
<code>'-'</code>	The converted value is

left adjusted (overrides the | '0' | conversion if both are given). | ' ' | (a space) A blank should be left before a positive number (or empty string) produced by a signed conversion. | '+' | A sign character (| '+' | or | '-' |) will precede the conversion (overrides a "space" flag).

A length modifier (h, l, or L) may be present, but is ignored as it is not necessary for Python – so e.g. %ld is identical to %d.

The conversion types are:

Conversion	Meaning	Notes
'd'	Signed integer decimal.	
'i'	Signed integer decimal.	
'o'	Signed octal value.	(1) 'u' Obsolete type – it is identical to 'd' . (6) 'x' Signed hexadecimal (lowercase).
'X'	Signed hexadecimal (uppercase).	(2) 'e' Floating-point exponential format (lowercase).
'E'	Floating-point exponential format (uppercase).	(3) 'f' Floating-point decimal format.
'F'	Floating-point decimal format.	(3) 'g' Floating-point format. Uses lowercase exponential format if exponent is less than -4 or not less than precision, decimal format otherwise.
'G'	Floating-point format. Uses uppercase exponential format if exponent is less than -4 or not less than precision, decimal format otherwise.	(4) 'c'

For floating-point formats, the result should be correctly rounded to a given precision of digits after the decimal point. The rounding mode matches that of the round() builtin.

Notes:

- The alternate form causes a leading octal specifier ('0o') to be inserted before the first digit.
- The alternate form causes a leading '0x' or '0X' (depending on whether the 'x' or 'X' format was used) to be inserted before the first digit.
- The alternate form causes the result to always contain a decimal point, even if no digits follow it. The precision determines the number of digits after the decimal point and defaults to 6.
- The alternate form causes the result to always contain a decimal point, and trailing zeroes are not removed as they would otherwise be. The precision determines the number of significant digits before and after the decimal point and defaults to 6.
- If precision is N, the output is truncated to N characters.
- See PEP 237.

Since Python strings have an explicit length, %s conversions do not assume that '\0' is the end of the string.

Changed in version 3.1: %f conversions for numbers whose absolute value is over 1e50 are no longer replaced by %g conversions.

Binary Sequence Types — bytes, bytearray, memoryview

The core built-in types for manipulating binary data are bytes and bytearray. They are supported by memoryview which uses the buffer protocol to access the memory of other binary objects without needing to make a copy.

The array module supports efficient storage of basic data types like 32-bit integers and IEEE754 double-precision floating values.

Bytes Objects

Bytes objects are immutable sequences of single bytes. Since many major binary protocols are based on the ASCII text encoding, bytes objects offer several methods that are only valid when working with ASCII compatible data and are closely related to string objects in a variety of other ways.

class bytes(source=b")

```
class bytes(source, encoding, errors='strict')
```

```
classmethod fromhex(string, /)
```

```
hex(*, bytes_per_sep=1)
```

```
hex(sep, bytes_per_sep=1)
```

Firstly, the syntax for bytes literals is largely the same as that for string literals, except that the prefix is added: Single quotes: `b'still allow embedded "double" quotes'` Double quotes: `b"still allow embedded 'single' quotes"` Triple quoted: `b"""3 single quotes""", b"""3 double quotes"""` Only ASCII characters are permitted in bytes literals (regardless of the declared source code encoding). Any binary values over 127 must be entered into bytes literals using the appropriate escape sequence. As with string literals

This `bytes` class method returns a bytes object, decoding the given string object. The string must contain two hexadecimal digits per byte, with ASCII whitespace being ignored. `>>> bytes.fromhex('2Ef0 F1f2')b'\xf0\xf1\xf2'` Changed in version 3.7: `bytes.fromhex()` now skips all ASCII whitespace in the string, not just spaces. Changed in version 3.14: `bytes.fromhex()` now accepts ASCII bytes and bytes-like objects as input.

Return a string object containing two hexadecimal digits for each byte in the instance. `>>> b'\xf0\xf1\xf2'.hex() 'f0f1f2'` If you want to make the hex string easier to read, you can specify a single character separator `sep` parameter to include in the output. By default, this separator will be included between each byte. A second optional `bytes_per_sep` parameter controls the spacing. Positive values calculate the separator position from the right, negative values from the left. `>>> value = b'\xf0\xf1\xf2' >>>`

Since bytes objects are sequences of integers (akin to a tuple), for a bytes object `b`, `b[0]` will be an integer, while `b[0:1]` will be a bytes object of length 1. (This contrasts with text strings, where both indexing and slicing will produce a string of length 1)

The representation of bytes objects uses the literal format (`b'...'`) since it is often more useful than e.g. `bytes([46, 46, 46])`. You can always convert a bytes object into a list of integers using `list(b)`.

Bytearray Objects

`bytearray` objects are a mutable counterpart to `bytes` objects.

```
class bytearray(source=b'')
```

```
class bytearray(source, encoding, errors='strict')
```

```
classmethod fromhex(string, /)
```

```
hex(*, bytes_per_sep=1)
```

```
hex(sep, bytes_per_sep=1)
```

```
resize(size, /)
```

There is no dedicated literal syntax for `bytearray` objects, instead they are always created by calling the constructor: Creating an empty instance: `bytearray()` Creating a zero-filled instance with a given length: `bytearray(10)` From an iterable of integers: `bytearray(range(20))` Copying existing binary data via the buffer protocol: `bytearray(b'Hi!')` As `bytearray` objects are mutable, they support the mutable sequence operations in addition to the common bytes and bytearray operations described in `Bytes` and `Byte`

This `bytearray` class method returns bytearray object, decoding the given string object. The string must contain two hexadecimal digits per byte, with ASCII whitespace being ignored. `>>> bytearray.fromhex('2Ef0 F1f2')bytearray(b'\xf0\xf1\xf2')` Changed in version 3.7: `bytearray.fromhex()` now skips all ASCII whitespace in the string, not just spaces. Changed in version 3.14: `bytearray.fromhex()` now accepts ASCII bytes and bytes-like objects as input.

Return a string object containing two hexadecimal digits for each byte in the instance. >>> bytearray(b'\xf0\xf1\xf2').hex() 'f0f1f2' Added in version 3.5. Changed in version 3.8: Similar to bytes.hex(), bytearray.hex() now supports optional sep and bytes_per_sep parameters to insert separators between bytes in the hex output.

Resize the bytearray to contain size bytes. size must be greater than or equal to 0. If the bytearray needs to shrink, bytes beyond size are truncated. If the bytearray needs to grow, all new bytes, those beyond size, will be set to null bytes. This is equivalent to: >>> def resize(ba, size): ... if len(ba) > size: ... del ba[size:] ... else: ... ba += b'\0' * (size - len(ba)) Examples: >>> shrink = bytearray(b'abc') >>> shrink.resize(1) >>> (shrink, len(shrink)) (bytearray(b'a'), 1) >>> grow = bytearray(b'abc') >>> grow.resize(5) >>> (grow, len(grow))

Since bytearray objects are sequences of integers (akin to a list), for a bytearray object b, b[0] will be an integer, while b[0:1] will be a bytearray object of length 1. (This contrasts with text strings, where both indexing and slicing will produce a string of length 1)

The representation of bytearray objects uses the bytes literal format (bytearray(b'...')) since it is often more useful than e.g. bytearray([46, 46, 46]). You can always convert a bytearray object into a list of integers using list(b).

See also

For detailed information on thread-safety guarantees for bytearray objects, see Thread safety for bytearray objects.

Bytes and Bytearray Operations

Both bytes and bytearray objects support the common sequence operations. They interoperate not just with operands of the same type, but with any bytes-like object. Due to this flexibility, they can be freely mixed in operations without causing errors. However, the return type of the result may depend on the order of operands.

■ **Note:** The methods on bytes and bytearray objects don't accept strings as their arguments, just as the methods on strings don't accept bytes as their arguments. For example, you have to write: a = "abc" b = a.replace("a", "f") and: a = b"abc" b = a.replace(b"a", b"f")

Some bytes and bytearray operations assume the use of ASCII compatible binary formats, and hence should be avoided when working with arbitrary binary data. These restrictions are covered below.

■ **Note:** Using these ASCII based operations to manipulate binary data that is not stored in an ASCII based format may lead to data corruption.

The following methods on bytes and bytearray objects can be used with arbitrary binary data.

bytes.count(sub[, start[, end]])

bytearray.count(sub[, start[, end]])

Return the number of non-overlapping occurrences of subsequence sub in the range [start, end]. Optional arguments start and end are interpreted as in slice notation. The subsequence to search for may be any bytes-like object or an integer in the range 0 to 255. If sub is empty, returns the number of empty slices between characters which is the length of the bytes object plus one. Changed in version 3.3: Also accept an integer in the range 0 to 255 as the subsequence.

bytes.removeprefix(prefix, /)

bytearray.removeprefix(prefix, /)

If the binary data starts with the prefix string, return bytes[len(prefix):]. Otherwise, return a copy of the original binary data: >>> b'TestHook'.removeprefix(b'Test') b'Hook' >>> b'BaseTestCase'.removeprefix(b'Test') b'BaseTestCase' The prefix may be any bytes-like object. **Note:** The bytearray version of this method does not operate in place - it always produces a new object, even if no changes were made. Added in version 3.9.

bytes.removesuffix(suffix, /)

bytearray.removesuffix(suffix, /)

If the binary data ends with the suffix string and that suffix is not empty, return `bytes[:-len(suffix)]`. Otherwise, return a copy of the original binary data: `>>> b'MiscTests'.removesuffix(b'Tests') b'Misc'>>> b'TmpDirMixin'.removesuffix(b'Tests') b'TmpDirMixin'` The suffix may be any bytes-like object. Note The bytearray version of this method does not operate in place - it always produces a new object, even if no changes were made. Added in version 3.9.

bytes.decode(encoding='utf-8', errors='strict')

bytearray.decode(encoding='utf-8', errors='strict')

Return the bytes decoded to a str. encoding defaults to 'utf-8'; see Standard Encodings for possible values. errors controls how decoding errors are handled. If 'strict' (the default), a `UnicodeError` exception is raised. Other possible values are 'ignore', 'replace', and any other name registered via `codecs.register_error()`. See Error Handlers for details. For performance reasons, the value of errors is not checked for validity unless a decoding error actually occurs, Python Development Mode is enabled or a debug build

bytes.endswith(suffix[, start[, end]])

bytearray.endswith(suffix[, start[, end]])

Return `True` if the binary data ends with the specified suffix, otherwise return `False`. suffix can also be a tuple of suffixes to look for. With optional start, test beginning at that position. With optional end, stop comparing at that position. The suffix(es) to search for may be any bytes-like object.

bytes.find(sub[, start[, end]])

bytearray.find(sub[, start[, end]])

Return the lowest index in the data where the subsequence sub is found, such that sub is contained in the slices `[start:end]`. Optional arguments start and end are interpreted as in slice notation. Return -1 if sub is not found. The subsequence to search for may be any bytes-like object or an integer in the range 0 to 255. Note The `find()` method should be used only if you need to know the position of sub. To check if sub is a substring or not, use the in operator: `>>> b'Py' in b'Python' True` Changed in version 3.3: Also accept

bytes.index(sub[, start[, end]])

bytearray.index(sub[, start[, end]])

Like `find()`, but raise `ValueError` when the subsequence is not found. The subsequence to search for may be any bytes-like object or an integer in the range 0 to 255. Changed in version 3.3: Also accept an integer in the range 0 to 255 as the subsequence.

bytes.join(iterable, /)

bytearray.join(iterable, /)

Return a bytes or bytearray object which is the concatenation of the binary data sequences in iterable. A `TypeError` will be raised if there are any values in iterable that are not bytes-like objects, including `str` objects. The separator between elements is the contents of the bytes or bytearray object providing this method.

static bytes.maketrans(from, to, /)

static bytearray.maketrans(from, to, /)

This static method returns a translation table usable for `bytes.translate()` that will map each character in from into the character at the same position in to; from and to must both be bytes-like objects and have the same length. Added in version 3.1.

bytes.partition(sep, /)

bytes.partition(sep, /)

Split the sequence at the first occurrence of `sep`, and return a 3-tuple containing the part before the separator, the separator itself or its bytearray copy, and the part after the separator. If the separator is not found, return a 3-tuple containing a copy of the original sequence, followed by two empty bytes or bytearray objects. The separator to search for may be any bytes-like object.

bytes.replace(old, new, count=-1, /)**bytearray.replace(old, new, count=-1, /)**

Return a copy of the sequence with all occurrences of subsequence `old` replaced by `new`. If the optional argument `count` is given, only the first `count` occurrences are replaced. The subsequence to search for and its replacement may be any bytes-like object. Note: The bytearray version of this method does not operate in place - it always produces a new object, even if no changes were made.

bytes.rfind(sub[, start[, end]])**bytearray.rfind(sub[, start[, end]])**

Return the highest index in the sequence where the subsequence `sub` is found, such that `sub` is contained within `s[start:end]`. Optional arguments `start` and `end` are interpreted as in slice notation. Return -1 on failure. The subsequence to search for may be any bytes-like object or an integer in the range 0 to 255. Changed in version 3.3: Also accept an integer in the range 0 to 255 as the subsequence.

bytes.rindex(sub[, start[, end]])**bytearray.rindex(sub[, start[, end]])**

Like `find()` but raises `ValueError` when the subsequence `sub` is not found. The subsequence to search for may be any bytes-like object or an integer in the range 0 to 255. Changed in version 3.3: Also accept an integer in the range 0 to 255 as the subsequence.

bytes.rpartition(sep, /)**bytearray.rpartition(sep, /)**

Split the sequence at the last occurrence of `sep`, and return a 3-tuple containing the part before the separator, the separator itself or its bytearray copy, and the part after the separator. If the separator is not found, return a 3-tuple containing two empty bytes or bytearray objects, followed by a copy of the original sequence. The separator to search for may be any bytes-like object.

bytes.startswith(prefix[, start[, end]])**bytearray.startswith(prefix[, start[, end]])**

Return `True` if the binary data starts with the specified `prefix`, otherwise return `False`. `prefix` can also be a tuple of prefixes to look for. With optional `start`, test beginning at that position. With optional `end`, stop comparing at that position. The prefix(es) to search for may be any bytes-like object.

bytes.translate(table, /, delete=b'')**bytearray.translate(table, /, delete=b'')**

Return a copy of the bytes or bytearray object where all bytes occurring in the optional argument `delete` are removed, and the remaining bytes have been mapped through the given translation table, which must be a bytes object of length 256. You can use the `bytes.maketrans()` method to create a translation table. Set the `table` argument to `None` for translations that only delete characters: `>>> b'read this short text'.translate(None, b'aeiou') b'rd ths shrt txt'` Changed in version 3.6: `delete` is now supported as a key

The following methods on bytes and bytearray objects have default behaviours that assume the use of ASCII compatible binary formats, but can still be used with arbitrary binary data by passing appropriate arguments.

Note that all of the bytearray methods in this section do not operate in place, and instead produce new objects.

bytes.center(width, fillbyte=b" ", /)

bytearray.center(width, fillbyte=b" ", /)

Return a copy of the object centered in a sequence of length `width`. Padding is done using the specified `fillbyte` (default is an ASCII space). For bytes objects, the original sequence is returned if `width` is less than or equal to `len(s)`. Note: The bytearray version of this method does not operate in place - it always produces a new object, even if no changes were made.

bytes.ljust(width, fillbyte=b" ", /)

bytearray.ljust(width, fillbyte=b" ", /)

Return a copy of the object left justified in a sequence of length `width`. Padding is done using the specified `fillbyte` (default is an ASCII space). For bytes objects, the original sequence is returned if `width` is less than or equal to `len(s)`. Note: The bytearray version of this method does not operate in place - it always produces a new object, even if no changes were made.

bytes.lstrip(bytes=None, /)

bytearray.lstrip(bytes=None, /)

Return a copy of the sequence with specified leading bytes removed. The `bytes` argument is a binary sequence specifying the set of byte values to be removed. If omitted or `None`, the `bytes` argument defaults to removing ASCII whitespace. The `bytes` argument is not a prefix; rather, all combinations of its values are stripped: `>>>b' spacious '.lstrip()b' spacious '>>>b'www.example.com'.lstrip(b'cmowz.')b'example.com'` The binary sequence of byte values to remove may be any bytes-like object. See `removeprefix`

bytes.rjust(width, fillbyte=b" ", /)

bytearray.rjust(width, fillbyte=b" ", /)

Return a copy of the object right justified in a sequence of length `width`. Padding is done using the specified `fillbyte` (default is an ASCII space). For bytes objects, the original sequence is returned if `width` is less than or equal to `len(s)`. Note: The bytearray version of this method does not operate in place - it always produces a new object, even if no changes were made.

bytes.rsplit(sep=None, maxsplit=-1)

bytearray.rsplit(sep=None, maxsplit=-1)

Split the binary sequence into subsequences of the same type, using `sep` as the delimiter string. If `maxsplit` is given, at most `maxsplit` splits are done, the rightmost ones. If `sep` is not specified or `None`, any subsequence consisting solely of ASCII whitespace is a separator. Except for splitting from the right, `rsplit()` behaves like `split()` which is described in detail below.

bytes.rstrip(bytes=None, /)

bytearray.rstrip(bytes=None, /)

Return a copy of the sequence with specified trailing bytes removed. The `bytes` argument is a binary sequence specifying the set of byte values to be removed. If omitted or `None`, the `bytes` argument defaults to removing ASCII whitespace. The `bytes` argument is not a suffix; rather, all combinations of its values are stripped: `>>>b' spacious '.rstrip()b' spacious '>>>b'mississippi'.rstrip(b'ipz')b'mississ'` The binary sequence of byte values to remove may be any bytes-like object. See `removesuffix()` for

bytes.split(sep=None, maxsplit=-1)

bytearray.split(sep=None, maxsplit=-1)

Split the binary sequence into subsequences of the same type, using `sep` as the delimiter string. If `maxsplit` is given and non-negative, at most `maxsplit` splits are done (thus, the list will have at most `maxsplit+1` elements). If `maxsplit` is not specified or is -1, then there is no limit on the number of splits (all possible splits are made). If `sep` is given, consecutive delimiters are not grouped together and are deemed to delimit empty subsequences (for example, `b'1,,2'.split(b',')` returns `[b'1', b'', b'2']`). The

`bytes.strip(bytes=None,)`

`bytearray.strip(bytes=None,)`

Return a copy of the sequence with specified leading and trailing bytes removed. The `bytes` argument is a binary sequence specifying the set of byte values to be removed. If omitted or `None`, the `bytes` argument defaults to removing ASCII whitespace. The `bytes` argument is not a prefix or suffix; rather, all combinations of its values are stripped: `>>> b' spacious '.strip(b' spacious')>>> b'www.example.com'.strip(b'cmowz.')` `b'example'` The binary sequence of byte values to remove may be any `bytes`-like object.

The following methods on `bytes` and `bytearray` objects assume the use of ASCII compatible binary formats and should not be applied to arbitrary binary data. Note that all of the `bytearray` methods in this section do not operate in place, and instead produce new objects.

`bytes.capitalize()`

`bytearray.capitalize()`

Return a copy of the sequence with each byte interpreted as an ASCII character, and the first byte capitalized and the rest lowercased. Non-ASCII byte values are passed through unchanged. Note the `bytearray` version of this method does not operate in place - it always produces a new object, even if no changes were made.

`bytes.expandtabs(tabsize=8)`

`bytearray.expandtabs(tabsize=8)`

Return a copy of the sequence where all ASCII tab characters are replaced by one or more ASCII spaces, depending on the current column and the given tab size. Tab positions occur every `tabsize` bytes (default is 8, giving tab positions at columns 0, 8, 16 and so on). To expand the sequence, the current column is set to zero and the sequence is examined byte by byte. If the byte is an ASCII tab character (`b'\t'`), one or more space characters are inserted in the result until the current column is

`bytes.isalnum()`

`bytearray.isalnum()`

Return `True` if all bytes in the sequence are alphabetical ASCII characters or ASCII decimal digits and the sequence is not empty, `False` otherwise. Alphabetical ASCII characters are those byte values in the sequence `b'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'`. ASCII decimal digits are those byte values in the sequence `b'0123456789'`. For example: `>>> b'ABCabc1'.isalnum()` `True` `>>> b'ABCabc1'.isalnum()` `False`

`bytes.isalpha()`

`bytearray.isalpha()`

Return `True` if all bytes in the sequence are alphabetic ASCII characters and the sequence is not empty, `False` otherwise. Alphabetic ASCII characters are those byte values in the sequence `b'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'`. For example: `>>> b'ABCabc'.isalpha()` `True` `>>> b'ABCabc1'.isalpha()` `False`

`bytes.isascii()`

`bytearray.isascii()`

Return True if the sequence is empty or all bytes in the sequence are ASCII, False otherwise. ASCII bytes are in the range 0-0x7F. Added in version 3.7.

bytes.isdigit()

bytearray.isdigit()

Return True if all bytes in the sequence are ASCII decimal digits and the sequence is not empty, False otherwise. ASCII decimal digits are those byte values in the sequence b'0123456789'. For example: >>> b'1234'.isdigit() True >>> b'1.23'.isdigit() False

bytes.islower()

bytearray.islower()

Return True if there is at least one lowercase ASCII character in the sequence and no uppercase ASCII characters, False otherwise. For example: >>> b'hello world'.islower() True >>> b'Hello world'.islower() False Lowercase ASCII characters are those byte values in the sequence b'abcdefghijklmnopqrstuvwxyz'. Uppercase ASCII characters are those byte values in the sequence b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'.

bytes.isspace()

bytearray.isspace()

Return True if all bytes in the sequence are ASCII whitespace and the sequence is not empty, False otherwise. ASCII whitespace characters are those byte values in the sequence b'\t\n\r\x0b\f' (space, tab, newline, carriage return, vertical tab, form feed).

bytes.istitle()

bytearray.istitle()

Return True if the sequence is ASCII titlecase and the sequence is not empty, False otherwise. See bytes.title() for more details on the definition of "titlecase". For example: >>> b'Hello World'.istitle() True >>> b'Hello world'.istitle() False

bytes.isupper()

bytearray.isupper()

Return True if there is at least one uppercase alphabetic ASCII character in the sequence and no lowercase ASCII characters, False otherwise. For example: >>> b'HELLO WORLD'.isupper() True >>> b'Hello world'.isupper() False Lowercase ASCII characters are those byte values in the sequence b'abcdefghijklmnopqrstuvwxyz'. Uppercase ASCII characters are those byte values in the sequence b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'.

bytes.lower()

bytearray.lower()

Return a copy of the sequence with all the uppercase ASCII characters converted to their corresponding lowercase counterpart. For example: >>> b'Hello World'.lower() b'hello world' Lowercase ASCII characters are those byte values in the sequence b'abcdefghijklmnopqrstuvwxyz'. Uppercase ASCII characters are those byte values in the sequence b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'. Note The bytearray version of this method does not operate in place - it always produces a new object, even if no changes were made.

bytes.splitlines(keepends=False)

bytearray.splitlines(keepends=False)

Return a list of the lines in the binary sequence, breaking at ASCII line boundaries. This method uses the universal newlines approach to splitting lines. Line breaks are not included in the resulting list

unless `keepends` is given and `true`. For example: `>>>b'ab c\n\nde fg\rkl\r\n'.splitlines()[b'ab c', b'', b'de fg', b'kl']>>>b'ab c\n\nde fg\rkl\r\n'.splitlines(keepends=True)[b'ab c\n', b'\n', b'de fg\r', b'kl\r\n']` Unlike `split()` when a delimiter string `sep` is given, this method returns an empty list for

`bytes.swapcase()`

`bytearray.swapcase()`

Return a copy of the sequence with all the lowercase ASCII characters converted to their corresponding uppercase counterpart and vice-versa. For example: `>>>b'Hello World'.swapcase()b'hELLO wORLD'` Lowercase ASCII characters are those byte values in the sequence `b'abcdefghijklmnopqrstuvwxyz'`. Uppercase ASCII characters are those byte values in the sequence `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`. Unlike `str.swapcase()`, it is always the case that `bin.swapcase().swapcase() == bin` for the binary versions. Case conversio

`bytes.title()`

`bytearray.title()`

Return a titlecased version of the binary sequence where words start with an uppercase ASCII character and the remaining characters are lowercase. Uncased byte values are left unmodified. For example: `>>>b'Hello world'.title()b'Hello World'` Lowercase ASCII characters are those byte values in the sequence `b'abcdefghijklmnopqrstuvwxyz'`. Uppercase ASCII characters are those byte values in the sequence `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`. All other byte values are uncased. The algorithm uses a simple language-i

`bytes.upper()`

`bytearray.upper()`

Return a copy of the sequence with all the lowercase ASCII characters converted to their corresponding uppercase counterpart. For example: `>>>b'Hello World'.upper()b'HELLO WORLD'` Lowercase ASCII characters are those byte values in the sequence `b'abcdefghijklmnopqrstuvwxyz'`. Uppercase ASCII characters are those byte values in the sequence `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`. Note The bytearray version of this method does not operate in place - it always produces a new object, even if no changes were made.

`bytes.zfill(width, /)`

`bytearray.zfill(width, /)`

Return a copy of the sequence left filled with ASCII `b'0'` digits to make a sequence of length `width`. A leading sign prefix (`b'+'`/`b'-'`) is handled by inserting the padding after the sign character rather than before. For bytes objects, the original sequence is returned if `width` is less than or equal to `len(seq)`. For example: `>>>b"42".zfill(5)b'00042'>>>b"-42".zfill(5)b'-0042'` Note The bytearray version of this method does not operate in place - it always produces a new object, even if no changes were made.

printf-style Bytes Formatting

■ Nota: Note The formatting operations described here exhibit a variety of quirks that lead to a number of common errors (such as failing to display tuples and dictionaries correctly). If the value being printed may be a tuple or dictionary, wrap it in a tuple.

Bytes objects (`bytes`/`bytearray`) have one unique built-in operation: the `%` operator (modulo). This is also known as the bytes formatting or interpolation operator. Given `format%values` (where `format` is a bytes object), `%` conversion specifications in `format` are replaced with zero or more elements of `values`. The effect is similar to using `printf()` in the C language.

If `format` requires a single argument, `values` may be a single non-tuple object. [5] Otherwise, `values` must be a tuple with exactly the number of items specified by the format bytes object, or a single mapping object (for example, a dictionary).

A conversion specifier contains two or more characters and has the following components, which must occur in this order:

- The '%' character, which marks the start of the specifier.
- Mapping key (optional), consisting of a parenthesised sequence of characters (for example, (somename)).
- Conversion flags (optional), which affect the result of some conversion types.
- Minimum field width (optional). If specified as an '*' (asterisk), the actual width is read from the next element of the tuple in values, and the object to convert comes after the minimum field width and optional precision.
- Precision (optional), given as a '.' (dot) followed by the precision. If specified as '*' (an asterisk), the actual precision is read from the next element of the tuple in values, and the value to convert comes after the precision.
- Length modifier (optional).
- Conversion type.

When the right argument is a dictionary (or other mapping type), then the formats in the bytes object must include a parenthesised mapping key into that dictionary inserted immediately after the '%' character. The mapping key selects the value to be formatted from the mapping. For example:

```
>>> print(b'%(language)s has %(number)03d quote types.' %
...       {b'language': b'Python', b'number': 2})
b'Python has 002 quote types.'
```

In this case no * specifiers may occur in a format (since they require a sequential parameter list).

The conversion flag characters are:

Flag	Meaning
'#'	The value conversion will use the "alternate form" (where defined below).
'0'	The conversion will be zero padded for numeric values.
'-'	The converted value is left adjusted (overrides the '0' conversion if both are given).
' '	A blank should be left before a positive number (or empty string) produced by a signed conversion.
'+'	A sign character ('+' or '-') will precede the conversion (overrides a "space" flag).

A length modifier (h, l, or L) may be present, but is ignored as it is not necessary for Python – so e.g. %ld is identical to %d.

The conversion types are:

Conversion	Meaning	Notes
'd'	Signed integer decimal.	
'i'	Signed integer decimal.	
'o'	Signed octal value.	(1) 'u' Obsolete type – it is identical to 'd' (8) 'x' Signed hexadecimal (lowercase).
'X'	Signed hexadecimal (uppercase).	(2) 'e' Floating-point exponential format (lowercase).
'E'	Floating-point exponential format (uppercase).	(3) 'f' Floating-point decimal format.
'F'	Floating-point decimal format.	(3) 'g' Floating-point format. Uses lowercase exponential format if exponent is less than -4 or not less than precision, decimal format otherwise.
'G'	Floating-point format. Uses uppercase exponential format if exponent is less than -4 or not less than precision, decimal format otherwise.	(4) 'c'

Notes:

- The alternate form causes a leading octal specifier ('0o') to be inserted before the first digit.
- The alternate form causes a leading '0x' or '0X' (depending on whether the 'x' or 'X' format was used) to be inserted before the first digit.
- The alternate form causes the result to always contain a decimal point, even if no digits follow it. The precision determines the number of digits after the decimal point and defaults to 6.
- The alternate form causes the result to always contain a decimal point, and trailing zeroes are not removed as they would otherwise be. The precision determines the number of significant digits before and after the decimal point and defaults to 6.

- If precision is N, the output is truncated to N characters.
- b'%s' is deprecated, but will not be removed during the 3.x series.
- b'%r' is deprecated, but will not be removed during the 3.x series.
- See PEP 237.

■ Nota: Note The bytearray version of this method does not operate in place - it always produces a new object, even if no changes were made.

See also

PEP 461- Adding % formatting to bytes and bytearray

Added in version 3.5.

Memory Views

memoryview objects allow Python code to access the internal data of an object that supports the buffer protocol without copying.

class memoryview(object)

__eq__(exporter)

tobytes(order='C')

hex(*, bytes_per_sep=1)

hex(sep, bytes_per_sep=1)

tolist()

toreadonly()

release()

cast(format, /)

cast(format, shape, /)

count(value, /)

index(value, start=0, stop=sys.maxsize, /)

obj

nbytes

readonly

format

itemsize

ndim

shape

strides

suboffsets

c_contiguous

f_contiguous

contiguous

Create a `memoryview` that references object. object must support the buffer protocol. Built-in objects that support the buffer protocol include `bytes` and `bytearray`. A `memoryview` has the notion of an element, which is the atomic memory unit handled by the originating object. For many simple types such as `bytes` and `bytearray`, an element is a single byte, but other types such as `array.array` may have bigger elements. `len(view)` is equal to the length of `tolist()`, which is the nested list representation of the view. If `view`

A `memoryview` and a `PEP 3118` exporter are equal if their shapes are equivalent and if all corresponding values are equal when the operands' respective format codes are interpreted using `struct` syntax. For the subset of `struct` format strings currently supported by `tolist()`, `v.tobytes()` and `w.tobytes()` are equal if `v.tobytes() == w.tobytes()`.>>>import array>>>a=array.array('l',[1,2,3,4,5])>>>b=array.array('d',[1.0,2.0,3.0,4.0,5.0])>>>c=array.array('b',[5,3,1])>>>x=memoryview(a)>>>y=memoryview(b)>>>x==a==y==b True>>>x.tolist()==a.tolist()

Return the data in the buffer as a `bytes` string. This is equivalent to calling the `bytes` constructor on the `memoryview`.>>>m=memoryview(b"abc")>>>m.tobytes() b'abc'>>>bytes(m) b'abc' For non-contiguous arrays the result is equal to the flattened list representation with all elements converted to bytes. `tobytes()` supports all format strings, including those that are not `struct` module syntax. Added in version 3.8: order can be {'C', 'F', 'A'}. When order is 'C' or 'F', the data of the original array is converted

Return a string object containing two hexadecimal digits for each byte in the buffer.>>>m=memoryview(b"abc")>>>m.hex() '616263' Added in version 3.5. Changed in version 3.8: Similar to `bytes.hex()`, `memoryview.hex()` now supports optional `sep` and `bytes_per_sep` parameters to insert separators between bytes in the hex output.

Return the data in the buffer as a list of elements.>>>memoryview(b'abc').tolist()[97, 98, 99]>>>import array>>>a=array.array('d',[1.1,2.2,3.3])>>>m=memoryview(a)>>>m.tolist()[1.1, 2.2, 3.3] Changed in version 3.3: `tolist()` now supports all single character native formats in `struct` module syntax as well as multi-dimensional representations.

Return a read-only version of the `memoryview` object. The original `memoryview` object is unchanged.>>>m=memoryview(bytearray(b'abc'))>>>mm=m.toreadonly()>>>mm.tolist()[97, 98, 99]>>>mm[0]=42 Traceback (most recent call last): File "<stdin>", line 1, in <module> TypeError: cannot modify read-only memory>>>m[0]=43>>>mm.tolist()[43, 98, 99] Added in version 3.8.

Release the underlying buffer exposed by the `memoryview` object. Many objects take special actions when a view is held on them (for example, a `bytearray` would temporarily forbid resizing); therefore, calling `release()` is handy to remove these restrictions (and free any dangling resources) as soon as possible. After this method has been called, any further operation on the view raises a `ValueError` (except `release()` itself which can be called multiple times).>>>m=memoryview(b'abc')>>>m.release()>>>m[0] Tr

Cast a `memoryview` to a new format or shape. `shape` defaults to `[byte_length//new_itemsize]`, which means that the result view will be one-dimensional. The return value is a new `memoryview`, but the buffer itself is not copied. Supported casts are 1D -> C-contiguous and C-contiguous -> 1D. The destination format is restricted to a single element native format in `struct` syntax. One of the formats must be a byte format ('B', 'b' or 'c'). The byte length of the result must be the same as the original length.

Count the number of occurrences of `value`. Added in version 3.14.

Return the index of the first occurrence of `value` (at or after `indexstart` and before `indexstop`). Raises a `ValueError` if `value` cannot be found. Added in version 3.14. There are also several read-only attributes available: `obj` The underlying object of the `memoryview`:>>>b=bytearray(b'xyz')>>>m=memoryview(b)>>>m.obj is b True Added in version 3.3. `nbytes` `nbytes == product(shape)*itemsize == len(m.tobytes())`. This is the amount of space in bytes that the array would use in a contiguous representation. It is not necessarily

The underlying object of the `memoryview`:>>>b=bytearray(b'xyz')>>>m=memoryview(b)>>>m.obj is b True Added in version 3.3.

`nbytes==product(shape)*itemsize==len(m.tobytes())`. This is the amount of space in bytes that the array would use in a contiguous representation. It is not necessarily equal to `len(m)`:

```
>>>importarray>>>a=array.array('i',[1,2,3,4,5])>>>m=memoryview(a)>>>len(m)5>>>m.nbytes20>>>y=m[:2]>>>len(y)3>>>y.nbytes12>>>len(y.tobytes())12
```

Multi-dimensional arrays:

```
>>>importstruct>>>buf=struct.pack("d"*12,[1.5*xforxinrange(12)])>>>x=memoryview(buf)>>>y=x.cast('d',shape=[3,4])>>>y.tolist()[[0.0, 1.5, 3.0, 4.5], [
```

A bool indicating whether the memory is read only.

A string containing the format (struct module style) for each element in the view. A memoryview can be created from exporters with arbitrary format strings, but some methods (e.g. `tolist()`) are restricted to native single element formats. Changed in version 3.3: format 'B' is now handled according to the struct module syntax. This means that `memoryview(b'abc')[0]==b'abc'[0]==97`.

The size in bytes of each element of the memoryview:

```
>>>importarray,struct>>>m=memoryview(array.array('H',[32000,32001,32002]))>>>m.itemsize2>>>m[0]32000>>>struct.calcsize('H')==m.itemsizeTrue
```

An integer indicating how many dimensions of a multi-dimensional array the memory represents.

A tuple of integers the length of `ndim` giving the shape of the memory as an N-dimensional array. Changed in version 3.3: An empty tuple instead of `None` when `ndim = 0`.

A tuple of integers the length of `ndim` giving the size in bytes to access each element for each dimension of the array. Changed in version 3.3: An empty tuple instead of `None` when `ndim = 0`.

Used internally for PIL-style arrays. The value is informational only.

A bool indicating whether the memory is C-contiguous. Added in version 3.3.

A bool indicating whether the memory is Fortran contiguous. Added in version 3.3.

A bool indicating whether the memory is contiguous. Added in version 3.3.

For information on the thread safety of memoryview objects in the free-threaded build, see Thread safety for memoryview objects.

Set Types — `set`, `frozenset`

A set object is an unordered collection of distinct hashable objects. Common uses include membership testing, removing duplicates from a sequence, and computing mathematical operations such as intersection, union, difference, and symmetric difference. (For other containers see the built-in `dict`, `list`, and `tuple` classes, and the `collections` module.)

Like other collections, sets support `in`, `len(set)`, and `for` in `set`. Being an unordered collection, sets do not record element position or order of insertion. Accordingly, sets do not support indexing, slicing, or other sequence-like behavior.

There are currently two built-in set types, `set` and `frozenset`. The `set` type is mutable — the contents can be changed using methods like `add()` and `remove()`. Since it is mutable, it has no hash value and cannot be used as either a dictionary key or as an element of another set. The `frozenset` type is immutable and hashable — its contents cannot be altered after it is created; it can therefore be used as a dictionary key or as an element of another set.

Non-empty sets (not `frozensets`) can be created by placing a comma-separated list of elements within braces, for example: `{'jack','sjoerd'}`, in addition to the `set` constructor.

The constructors for both classes work the same:

```
class set(iterable=(),/)
```

```
class frozenset(iterable=(),/)
```

Return a new set or frozenset object whose elements are taken from iterable. The elements of a set must be hashable. To represent sets of sets, the inner sets must be frozenset objects. If iterable is not specified, a new empty set is returned.

Sets can be created by several means:

- Use a comma-separated list of elements within braces: {'jack', 'sjoerd'}
- Use a set comprehension: {c for c in 'abracadabra' if c not in 'abc'}
- Use the type constructor: set(), set('foobar'), set(['a', 'b', 'foo'])

Instances of set and frozenset provide the following operations:

len(s)

Return the number of elements in sets (cardinality of s).

x in s

Test x for membership in s.

x not in s

Test x for non-membership in s.

frozenset.isdisjoint(other, /)

set.isdisjoint(other, /)

Return True if the set has no elements in common with other. Sets are disjoint if and only if their intersection is the empty set.

frozenset.issubset(other, /)

set.issubset(other, /)

set <= other

Test whether every element in the set is in other.

set < other

Test whether the set is a proper subset of other, that is, set <= other and set != other.

frozenset.issuperset(other, /)

set.issuperset(other, /)

set >= other

Test whether every element in other is in the set.

set > other

Test whether the set is a proper superset of other, that is, set >= other and set != other.

frozenset.union(*others)

set.union(*others)

set | other | ...

Return a new set with elements from the set and all others.

frozenset.intersection(*others)

set.intersection(*others)

set & other & ...

Return a new set with elements common to the set and all others.

frozenset.difference(*others)

set.difference(*others)

set-other-...

Return a new set with elements in the set that are not in the others.

frozenset.symmetric_difference(other, /)

set.symmetric_difference(other, /)

set^other

Return a new set with elements in either the set or other but not both.

frozenset.copy()

set.copy()

Return a shallow copy of the set.

Note, the non-operator versions of `union()`, `intersection()`, `difference()`, `symmetric_difference()`, `issubset()`, and `issuperset()` methods will accept any iterable as an argument. In contrast, their operator based counterparts require their arguments to be sets. This precludes error-prone constructions like `set('abc') & 'cbs'` in favor of the more readable `set('abc').intersection('cbs')`.

Both `set` and `frozenset` support set to set comparisons. Two sets are equal if and only if every element of each set is contained in the other (each is a subset of the other). A set is less than another set if and only if the first set is a proper subset of the second set (is a subset, but is not equal). A set is greater than another set if and only if the first set is a proper superset of the second set (is a superset, but is not equal).

Instances of `set` are compared to instances of `frozenset` based on their members. For example, `set('abc') == frozenset('abc')` returns `True` and so does `set('abc') in set([frozenset('abc')])`.

The subset and equality comparisons do not generalize to a total ordering function. For example, any two nonempty disjoint sets are not equal and are not subsets of each other, so all of the following return `False`: `a < b`, `a == b`, or `a > b`.

Since sets only define partial ordering (subset relationships), the output of the `list.sort()` method is undefined for lists of sets.

Set elements, like dictionary keys, must be hashable.

Binary operations that mix `set` instances with `frozenset` return the type of the first operand. For example: `frozenset('ab') | set('bc')` returns an instance of `frozenset`.

The following table lists operations available for `set` that do not apply to immutable instances of `frozenset`:

set.update(*others)

set |= other | ...

Update the set, adding elements from all others.

set.intersection_update(*others)

set &= other & ...

Update the set, keeping only elements found in it and all others.

set.difference_update(*others)

set -= other | ...

Update the set, removing elements found in others.

set.symmetric_difference_update(other, /)

set ^= other

Update the set, keeping only elements found in either set, but not in both.

set.add(elem, /)

Add element *elem* to the set.

set.remove(elem, /)

Remove element *elem* from the set. Raises `KeyError` if *elem* is not contained in the set.

set.discard(elem, /)

Remove element *elem* from the set if it is present.

set.pop()

Remove and return an arbitrary element from the set. Raises `KeyError` if the set is empty.

set.clear()

Remove all elements from the set.

Note, the non-operator versions of the `update()`, `intersection_update()`, `difference_update()`, and `symmetric_difference_update()` methods will accept any iterable as an argument.

Note, the *elem* argument to the `__contains__()`, `remove()`, and `discard()` methods may be a set. To support searching for an equivalent frozen set, a temporary one is created from *elem*.

See also

For detailed information on thread-safety guarantees for set objects, see [Thread safety for set objects](#).

Mapping Types —dict

A mapping object maps hashable values to arbitrary objects. Mappings are mutable objects. There is currently only one standard mapping type, the dictionary. (For other containers see the built-in `list`, `set`, and `tuple` classes, and the `collections` module.)

A dictionary's keys are almost arbitrary values. Values that are not hashable, that is, values containing lists, dictionaries or other mutable types (that are compared by value rather than by object identity) may not be used as keys. Values that compare equal (such as `1`, `1.0`, and `True`) can be used interchangeably to index the same dictionary entry.

class dict(kwargs)**

class dict(mapping, /, **kwargs)

class dict(iterable, /, **kwargs)

list(d)

len(d)

d[key]

d[key]=value

del d[key]

key in d

keynotind

iter(d)

clear()

copy()

classmethodfromkeys(iterable,value=None,/)

get(key,default=None,/)

items()

keys()

pop(key,/)

pop(key,default,/)

popitem()

reversed(d)

setdefault(key,default=None,/)

update(kwargs)**

update(mapping,/,kwargs)**

update(iterable,/,kwargs)**

values()

d|other

d|=other

Return a new dictionary initialized from an optional positional argument and a possibly empty set of keyword arguments. Dictionaries can be created by several means: Use a comma-separated list of key:value pairs within braces: {'jack':4098,'sjoerd':4127} or {4098:'jack',4127:'sjoerd'} Use a dict comprehension: {}, {x:x**2 for x in range(10)} Use the type constructor: dict(), dict([('foo',100),('bar',200)]), dict(foo=100,bar=200) If no positional argument is given, an empty dictionary is created. If a positional ar

Return a list of all the keys used in the dictionaryd.

Return the number of items in the dictionaryd.

Return the item of d with keykey. Raises a KeyError if key is not in the map. If a subclass of dict defines a method __missing__() and key is not present, the d[key] operation calls that method with the key key as argument. The d[key] operation then returns or raises whatever is returned or raised by the __missing__(key) call. No other operations or methods invoke __missing__(). If __missing__() is not defined, KeyError is raised. __missing__() must be a method; it cannot be an instance variable:>>> class Counter(dict):.

Set d[key] to value.

Removed[key] from d. Raises a KeyError if key is not in the map.

Return True if d has a key key, else False.

Equivalent to not key in d.

Return an iterator over the keys of the dictionary. This is a shortcut for iter(d.keys()).

Remove all items from the dictionary.

Return a shallow copy of the dictionary.

Create a new dictionary with keys from `iterable` and values set to `value`. `fromkeys()` is a class method that returns a new dictionary. `value` defaults to `None`. All of the values refer to just a single instance, so it generally doesn't make sense for `value` to be a mutable object such as an empty list. To get distinct values, use `adict` comprehension instead.

Return the value for `key` if `key` is in the dictionary, else `default`. If `default` is not given, it defaults to `None`, so that this method never raises a `KeyError`.

Return a new view of the dictionary's items ((`key,value`)pairs). See the documentation of view objects.

Return a new view of the dictionary's keys. See the documentation of view objects.

If `key` is in the dictionary, remove it and return its value, else return `default`. If `default` is not given and `key` is not in the dictionary, a `KeyError` is raised.

Remove and return a (`key,value`)pair from the dictionary. Pairs are returned in LIFO order. `popitem()` is useful to destructively iterate over a dictionary, as often used in set algorithms. If the dictionary is empty, calling `popitem()` raises a `KeyError`. Changed in version 3.7: LIFO order is now guaranteed. In prior versions, `popitem()` would return an arbitrary key/value pair.

Return a reverse iterator over the keys of the dictionary. This is a shortcut for `reversed(d.keys())`. Added in version 3.8.

If `key` is in the dictionary, return its value. If not, insert `key` with a value of `default` and return `default`. `default` defaults to `None`.

Update the dictionary with the key/value pairs from `mapping` or `iterable` and `kwargs`, overwriting existing keys. Return `None`. `update()` accepts either another object with a `keys()` method (in which case `__getitem__()` is called with every key returned from the method) or an iterable of key/value pairs (as tuples or other iterables of length two). If keyword arguments are specified, the dictionary is then updated with those key/value pairs: `d.update(red=1,blue=2)`.

Return a new view of the dictionary's values. See the documentation of view objects. An equality comparison between one `dict.values()` view and another will always return `False`. This also applies when comparing `dict.values()` to itself: `>>>d={'a':1}>>>d.values()==d.values()False`

Create a new dictionary with the merged keys and values of `d` and `other`, which must both be dictionaries. The values of `other` take priority when `d` and `other` share keys. Added in version 3.9.

Update the dictionary `d` with keys and values from `other`, which may be either a `mapping` or an `iterable` of key/value pairs. The values of `other` take priority when `d` and `other` share keys. Added in version 3.9.

See also

`types.MappingProxyType` can be used to create a read-only view of `adict`.

See also

For detailed information on thread-safety guarantees for dict objects, see Thread safety for dict objects.

Dictionary view objects

The objects returned by `dict.keys()`, `dict.values()` and `dict.items()` are view objects. They provide a dynamic view on the dictionary's entries, which means that when the dictionary changes, the view reflects these changes.

Dictionary views can be iterated over to yield their respective data, and support membership tests:

len(dictview)

Return the number of entries in the dictionary.

iter(dictview)

Return an iterator over the keys, values or items (represented as tuples of (key,value)) in the dictionary. Keys and values are iterated over in insertion order. This allows the creation of (value,key) pairs using `zip()`: `pairs=zip(d.values(),d.keys())`. Another way to create the same list is `pairs=[(v,k)for(k,v)ind.items()]`. Iterating views while adding or deleting entries in the dictionary may raise a `RuntimeError` fail to iterate over all entries. Changed in version 3.7: Dictionary order is guaranteed

xindictview

Return `True` if `x` is in the underlying dictionary's keys, values or items (in the latter case, `x` should be a (key,value) tuple).

reversed(dictview)

Return a reverse iterator over the keys, values or items of the dictionary. The view will be iterated in reverse order of the insertion. Changed in version 3.8: Dictionary views are now reversible.

dictview.mapping

Return a `types.MappingProxyType` that wraps the original dictionary to which the view refers. Added in version 3.10.

Keys views are set-like since their entries are unique and hashable. Items views also have set-like operations since the (key, value) pairs are unique and the keys are hashable. If all values in an items view are hashable as well, then the items view can interoperate with other sets. (Values views are not treated as set-like since the entries are generally not unique.) For set-like views, all of the operations defined for the abstract base class `collections.abc.Set` are available (for example, `==`, `<`, or `^`). While using set operators, set-like views accept any iterable as the other operand, unlike sets which only accept sets as the input.

An example of dictionary view usage:

```
>>> dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
>>> keys = dishes.keys()
>>> values = dishes.values()

>>> # iteration
>>> n = 0
>>> for val in values:
...     n += val
...
>>> print(n)
504

>>> # keys and values are iterated over in the same order (insertion order)
>>> list(keys)
['eggs', 'sausage', 'bacon', 'spam']
>>> list(values)
[2, 1, 1, 500]

>>> # view objects are dynamic and reflect dict changes
>>> del dishes['eggs']
>>> del dishes['sausage']
>>> list(keys)
['bacon', 'spam']

>>> # set operations
>>> keys & {'eggs', 'bacon', 'salad'}
{'bacon'}
>>> keys ^ {'sausage', 'juice'} == {'juice', 'sausage', 'bacon', 'spam'}
True
>>> keys | ['juice', 'juice', 'juice'] == {'bacon', 'spam', 'juice'}
True
```

```
>>> # get back a read-only proxy for the original dictionary
>>> values.mapping
mappingproxy({'bacon': 1, 'spam': 500})
>>> values.mapping['spam']
500
```

Context Manager Types

Python's `with` statement supports the concept of a runtime context defined by a context manager. This is implemented using a pair of methods that allow user-defined classes to define a runtime context that is entered before the statement body is executed and exited when the statement ends:

contextmanager.__enter__()

Enter the runtime context and return either this object or another object related to the runtime context. The value returned by this method is bound to the identifier in the `as` clause of `with` statements using this context manager. An example of a context manager that returns itself is a file object. File objects return themselves from `__enter__()` to allow `open()` to be used as the context expression in a `with` statement. An example of a context manager that returns a related object is the one returned by `decimal`.

contextmanager.__exit__(exc_type, exc_val, exc_tb)

Exit the runtime context and return a Boolean flag indicating if any exception that occurred should be suppressed. If an exception occurred while executing the body of the `with` statement, the arguments contain the exception type, value and traceback information. Otherwise, all three arguments are `None`. Returning a true value from this method will cause the `with` statement to suppress the exception and continue execution with the statement immediately following the `with` statement. Otherwise the exception

Python defines several context managers to support easy thread synchronisation, prompt closure of files or other objects, and simpler manipulation of the active decimal arithmetic context. The specific types are not treated specially beyond their implementation of the context management protocol. See the `contextlib` module for some examples.

Python's generators and the `contextlib.contextmanager` decorator provide a convenient way to implement these protocols. If a generator function is decorated with the `contextlib.contextmanager` decorator, it will return a context manager implementing the necessary `__enter__()` and `__exit__()` methods, rather than the iterator produced by an undecorated generator function.

Note that there is no specific slot for any of these methods in the type structure for Python objects in the Python/C API. Extension types wanting to define these methods must provide them as a normal Python accessible method. Compared to the overhead of setting up the runtime context, the overhead of a single class dictionary lookup is negligible.

Type Annotation Types —Generic Alias, Union

The core built-in types for type annotations are `GenericAlias` and `Union`.

Generic Alias Type

`GenericAlias` objects are generally created by subscripting a class. They are most often used with container classes, such as `list` or `dict`. For example, `list[int]` is a `GenericAlias` object created by subscripting the `list` class with the argument `int`. `GenericAlias` objects are intended primarily for use with type annotations.

■ **Nota:** Note it is generally only possible to subscript a class if the class implements the special method `__class_getitem__()`.

A `GenericAlias` object acts as a proxy for a generic type, implementing parameterized generics.

For a container class, the argument(s) supplied to a subscription of the class may indicate the type(s) of the elements an object contains. For example, `set[bytes]` can be used in type annotations to signify a set in which all the elements are of type `bytes`.

For a class which defines `__class_getitem__()` but is not a container, the argument(s) supplied to a subscription of the class will often indicate the return type(s) of one or more methods defined on an object. For example, `re.Pattern` can be used on both the `str` data type and the `bytes` data type:

- If `x=re.search('foo','foo')`, `x` will be a `re.Match` object where the return values of `x.group(0)` and `x[0]` will both be of type `str`. We can represent this kind of object in type annotations with the `GenericAlias` `re.Match[str]`.
- If `y=re.search(b'bar',b'bar')`, (note the `b` for `bytes`), `y` will also be an instance of `re.Match`, but the return values of `y.group(0)` and `y[0]` will both be of type `bytes`. In type annotations, we would represent this variety of `re.Match` objects with `re.Match[bytes]`.

`GenericAlias` objects are instances of the class `types.GenericAlias`, which can also be used to create `GenericAlias` objects directly.

T[X,Y,...]

Creates a `GenericAlias` representing a type `T` parameterized by types `X`, `Y`, and more depending on the `T` used.

For example, a function expecting

a list containing float elements: `def average(values: list[float]) -> float: return sum(values)/len(values)` Another example for mapping objects, using `adict`, which is a generic type expecting two type parameters representing the key type and the value type. In this example, the function expects `adict` with keys of type `str` and values of type `int`: `def send_post_request(url: str, body: dict`

The builtin functions `isinstance()` and `issubclass()` do not accept `GenericAlias` types for their second argument:

```
>>> isinstance([1, 2], list[str])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: isinstance() argument 2 cannot be a parameterized generic
```

The Python runtime does not enforce type annotations. This extends to generic types and their type parameters. When creating a container object from a `GenericAlias`, the elements in the container are not checked against their type. For example, the following code is discouraged, but will run without errors:

```
>>> t = list[str]
>>> t([1, 2, 3])
[1, 2, 3]
```

Furthermore, parameterized generics erase type parameters during object creation:

```
>>> t = list[str]
>>> type(t)
<class 'types.GenericAlias'>

>>> l = t()
>>> type(l)
<class 'list'>
```

Calling `repr()` or `str()` on a generic shows the parameterized type:

```
>>> repr(list[int])
'list[int]'

>>> str(list[int])
'list[int]'
```

The `__getitem__()` method of generic containers will raise an exception to disallow mistakes like `dict[str][str]`:

```
>>> dict[str][str]
Traceback (most recent call last):
```

```
...
TypeError: dict[str] is not a generic class
```

However, such expressions are valid when type variables are used. The index must have as many elements as there are type variable items in the GenericAlias object's `__args__`.

```
>>> from typing import TypeVar
>>> Y = TypeVar('Y')
>>> dict[str, Y][int]
dict[str, int]
```

Standard Generic Classes

The following standard library classes support parameterized generics. This list is non-exhaustive.

- tuple
- list
- dict
- set
- frozenset
- type
- asyncio.Future
- asyncio.Task
- collections.deque
- collections.defaultdict
- collections.OrderedDict
- collections.Counter
- collections.ChainMap
- collections.abc.Awaitable
- collections.abc.Coroutine
- collections.abc.AsyncIterable
- collections.abc.AsyncIterator
- collections.abc.AsyncGenerator
- collections.abc.Iterable
- collections.abc.Iterator
- collections.abc.Generator
- collections.abc.Reversible
- collections.abc.Container
- collections.abc.Collection
- collections.abc.Callable
- collections.abc.Set
- collections.abc.MutableSet
- collections.abc.Mapping

- collections.abc.MutableMapping
- collections.abc.Sequence
- collections.abc.MutableSequence
- collections.abc.ByteString
- collections.abc.MappingView
- collections.abc.KeysView
- collections.abc.ItemsView
- collections.abc.ValuesView
- contextlib.AbstractContextManager
- contextlib.AbstractAsyncContextManager
- dataclasses.Field
- functools.cached_property
- functools.partialmethod
- os.PathLike
- queue.LifoQueue
- queue.Queue
- queue.PriorityQueue
- queue.SimpleQueue
- re.Pattern
- re.Match
- shelve.BsdDbShelf
- shelve.DbfilenameShelf
- shelve.Shelf
- types.MappingProxyType
- weakref.WeakKeyDictionary
- weakref.WeakMethod
- weakref.WeakSet
- weakref.WeakValueDictionary

Special Attributes of Generic Alias objects

All parameterized generics implement special read-only attributes.

genericalias.__origin__

This attribute points at the non-parameterized generic class: `>>>list[int].__origin__<class 'list'>`

genericalias.__args__

This attribute is a tuple (possibly of length 1) of generic types passed to the original `__class_getitem__()` of the generic class: `>>>dict[str,list[int]].__args__(<class 'str'>, list[int])`

genericalias.__parameters__

This attribute is a lazily computed tuple (possibly empty) of unique type variables found in `__args__`:>>>from typing import TypeVar>>>T=TypeVar('T')>>>list[T].__parameters__(~T,)Note A GenericAlias object with typing.ParamSpec parameters may not have correct `__parameters__` after substitution because typing.ParamSpec is intended primarily for static type checking.

genericalias.__unpacked__

A boolean that is true if the alias has been unpacked using the `*` operator (see `TypeVarTuple`). Added in version 3.11.

See also

PEP 484- Type Hints

PEP 585- Type Hinting Generics In Standard Collections

Generics, user-defined generics and typing.Generic

Introducing Python's framework for type annotations.

Introducing the ability to natively parameterize standard-library classes, provided they implement the special class method `__class_getitem__()`.

Documentation on how to implement generic classes that can be parameterized at runtime and understood by static type-checkers.

Added in version 3.9.

Union Type

A union object holds the value of the `|` (bitwise or) operation on multiple type objects. These types are intended primarily for type annotations. The union type expression enables cleaner type hinting syntax compared to subscripting `typing.Union`.

X|Y|...

Defines a union object which holds types `X`, `Y`, and so forth. `X|Y` means either `X` or `Y`. It is equivalent to `typing.Union[X, Y]`. For example, the following function expects an argument of type `int` or `float`:
`def square(number: int|float) -> int|float: return number**2`
Note The `|` operand cannot be used at runtime to define unions where one or more members is a forward reference. For example, `int|"Foo"`, where `"Foo"` is a reference to a class not yet defined, will fail at runtime. For unions which include forward references,

union_object==other

Union objects can be tested for equality with other union objects. Details: Unions of unions are flattened: `(int|str)|float==int|str|float` Redundant types are removed: `int|str|int==int|str` When comparing unions, the order is ignored: `int|str==str|int` It creates instances of `typing.Union`: `int|str==typing.Union[int, str]` type(`int|str`) is `typing.Union` Optional types can be spelled as a union with `None`: `str|None==typing.Optional[str]`

isinstance(obj, union_object)

issubclass(obj, union_object)

Calls `toinstance()` and `issubclass()` are also supported with a union object:>>>instance("", int|str) True However, parameterized generics in union objects cannot be checked:>>>instance(1, int|list[int]) # short-circuit evaluation True>>>instance([1], int|list[int]) Traceback (most recent call last): ...TypeError: isinstance() argument 2 cannot be a parameterized generic

The user-exposed type for the union object can be accessed from `typing.Union` and used for `isinstance()` checks:

```
>>> import typing
>>> isinstance(int | str, typing.Union)
```

```
True
>>> typing.Union()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot create 'typing.Union' instances
```

■ Nota: NoteThe `__or__()` method for type objects was added to support the syntax `X|Y`. If a metaclass implements `__or__()`, the Union may override it: `>>> class M(type): ... def __or__(self, other): ... return "Hello" ... >>> class C(metaclass=M): ... pass ... >>> C | int 'Hello' >>> int | C`

See also

PEP 604— PEP proposing the `X|Y` syntax and the Union type.

Added in version 3.10.

Changed in version 3.14: Union objects are now instances of `typing.Union`. Previously, they were instances of `types.UnionType`, which remains an alias for `typing.Union`.

Other Built-in Types

The interpreter supports several other kinds of objects. Most of these support only one or two operations.

Modules

The only special operation on a module is attribute access: `m.name`, where `m` is a module and `name` accesses a name defined in `m`'s symbol table. Module attributes can be assigned to. (Note that the `import` statement is not, strictly speaking, an operation on a module object; `import foo` does not require a module object named `foo` to exist, rather it requires an (external) definition for a module named `foo` somewhere.)

A special attribute of every module is `__dict__`. This is the dictionary containing the module's symbol table. Modifying this dictionary will actually change the module's symbol table, but direct assignment to the `__dict__` attribute is not possible (you can write `__dict__['a']=1`, which defines `m.a` to be 1, but you can't write `__dict__={}`). Modifying `__dict__` directly is not recommended.

Modules built into the interpreter are written like this: `<module 'sys' (built-in)>`. If loaded from a file, they are written as `<module 'os' from '/usr/local/lib/pythonX.Y/os.pyc'>`.

Classes and Class Instances

See [Objects](#), [values and types](#) and [Class definitions](#) for these.

Functions

Function objects are created by function definitions. The only operation on a function object is to call it: `func(argument-list)`.

There are really two flavors of function objects: built-in functions and user-defined functions. Both support the same operation (to call the function), but the implementation is different, hence the different object types.

See [Function definitions](#) for more information.

Methods

Methods are functions that are called using the attribute notation. There are two flavors: built-in methods (such as `append()` on lists) and class instance method. Built-in methods are described with the types that support them.

If you access a method (a function defined in a class namespace) through an instance, you get a special object: a bound method (also called instance method) object. When called, it will add the `self` argument to the argument list. Bound methods have two special read-only attributes: `m.__self__` is the object on which the method operates, and `m.__func__` is the function implementing the method. Calling `m(arg-1, arg-2, ..., arg-n)` is completely

equivalent to calling `m.__func__(m.__self__,arg-1,arg-2,...,arg-n)`.

Like function objects, bound method objects support getting arbitrary attributes. However, since method attributes are actually stored on the underlying function object (`method.__func__`), setting method attributes on bound methods is disallowed. Attempting to set an attribute on a method results in an `AttributeError` being raised. In order to set a method attribute, you need to explicitly set it on the underlying function object:

```
>>> class C:
...     def method(self):
...         pass
...
>>> c = C()
>>> c.method.whoami = 'my name is method' # can't set on the method
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'method' object has no attribute 'whoami'
>>> c.method.__func__.whoami = 'my name is method'
>>> c.method.whoami
'my name is method'
```

See `Instance methods` for more information.

Code Objects

Code objects are used by the implementation to represent “pseudo-compiled” executable Python code such as a function body. They differ from function objects because they don’t contain a reference to their global execution environment. Code objects are returned by the built-in `compile()` function and can be extracted from function objects through their `__code__` attribute. See also `the code module`.

Accessing `__code__` raises an `auditing event` object. `__getattr__` with arguments `obj` and `"__code__"`.

A code object can be executed or evaluated by passing it (instead of a source string) to the `exec()` or `eval()` built-in functions.

See `The standard type hierarchy` for more information.

Type Objects

Type objects represent the various object types. An object’s type is accessed by the built-in function `type()`. There are no special operations on types. The standard module `types` defines names for all standard built-in types.

Types are written like this: `<class'int'>`.

The Null Object

This object is returned by functions that don’t explicitly return a value. It supports no special operations. There is exactly one null object, named `None` (a built-in name). `type(None)` produces the same singleton.

It is written as `None`.

The Ellipsis Object

This object is commonly used to indicate that something is omitted. It supports no special operations. There is exactly one ellipsis object, named `Ellipsis` (a built-in name). `type(Ellipsis)` produces the `Ellipsis` singleton.

It is written as `Ellipsis` or `...`.

In typical use, `...` as the `Ellipsis` object appears in a few different places, for instance:

- In type annotations, such as callable arguments or tuple elements.
- As the body of a function instead of a `pass` statement.

- In third-party libraries, such as Numpy's slicing and striding.

Python also uses three dots in ways that are not Ellipsis objects, for instance:

- Doctest's ELLIPSIS, as a pattern for missing content.
- The default Python prompt of the interactive shell when partial input is incomplete.

Lastly, the Python documentation often uses three dots in conventional English usage to mean omitted content, even in code examples that also use them as the Ellipsis.

The NotImplemented Object

This object is returned from comparisons and binary operations when they are asked to operate on types they don't support. See Comparisons for more information. There is exactly one `NotImplementedObject.type(NotImplemented())` produces the singleton instance.

It is written as `NotImplemented`.

Internal Objects

See The standard type hierarchy for this information. It describes stack frame objects, traceback objects, and slice objects.

Special Attributes

The implementation adds a few special read-only attributes to several object types, where they are relevant. Some of these are not reported by the `dir()` built-in function.

definition. `__name__`

The name of the class, function, method, descriptor, or generator instance.

definition. `__qualname__`

The qualified name of the class, function, method, descriptor, or generator instance. Added in version 3.3.

definition. `__module__`

The name of the module in which a class or function was defined.

definition. `__doc__`

The documentation string of a class or function, or `None` if undefined.

definition. `__type_params__`

The type parameters of generic classes, functions, and type aliases. For classes and functions that are not generic, this will be an empty tuple. Added in version 3.12.

Integer string conversion length limitation

CPython has a global limit for converting between `int` and `str` to mitigate denial of service attacks. This limit only applies to decimal or other non-power-of-two number bases. Hexadecimal, octal, and binary conversions are unlimited. The limit can be configured.

The `int` type in CPython is an arbitrary length number stored in binary form (commonly known as a “bignum”). There exists no algorithm that can convert a string to a binary integer or a binary integer to a string in linear time, unless the base is a power of 2. Even the best known algorithms for base 10 have sub-quadratic complexity. Converting a large value such as `int('1'*500_000)` can take over a second on a fast CPU.

- `int(string,base)` with base 2, 4, 8, 16, or 32.
- `int.from_bytes()` and `int.to_bytes()`.
- `hex()`, `oct()`, `bin()`.
- Format specification mini-language for hex, octal, and binary numbers.
- `strtofloat`.
- `strtodecimal`, `Decimal`.

Configuring the limit

Before Python starts up you can use an environment variable or an interpreter command line flag to configure the limit:

- `PYTHONINTMAXSTRDIGITS`, e.g. `PYTHONINTMAXSTRDIGITS=640python3` to set the limit to 640 or `PYTHONINTMAXSTRDIGITS=0python3` to disable the limitation.
- `-Xint_max_str_digits`, e.g. `python3-Xint_max_str_digits=640`
- `sys.flags.int_max_str_digits` contains the value of `PYTHONINTMAXSTRDIGITS` or `-Xint_max_str_digits`. If both the env var and the `-X` option are set, the `-X` option takes precedence. A value of `-1` indicates that both were unset, thus a value of `sys.int_info.default_max_str_digits` was used during initialization.

From code, you can inspect the current limit and set a new one using these `sys` APIs:

- `sys.get_int_max_str_digits()` and `sys.set_int_max_str_digits()` are a getter and setter for the interpreter-wide limit. Subinterpreters have their own limit.

Information about the default and minimum can be found in `sys.int_info`:

- `sys.int_info.default_max_str_digits` is the compiled-in default limit.
- `sys.int_info.str_digits_check_threshold` is the lowest accepted value for the limit (other than 0 which disables it).

Added in version 3.11.

Caution

Setting a low limit can lead to problems. While rare, code exists that contains integer constants in decimal in their source that exceed the minimum threshold. A consequence of setting the limit is that Python source code containing decimal integer literals longer than the limit will encounter an error during parsing, usually at startup time or import time or even at installation time - anytime an up to date `pyc` does not already exist for the code. A workaround for source that contains such large constants is to convert them to 0x hexadecimal form as it has no limit.

Test your application thoroughly if you use a low limit. Ensure your tests run with the limit set early via the environment or flag so that it applies during startup and even during any installation step that may invoke Python to precompile `.py` sources to `.pyc` files.

Recommended configuration

The default `sys.int_info.default_max_str_digits` is expected to be reasonable for most applications. If your application requires a different limit, set it from your main entry point using Python version agnostic code as these APIs were added in security patch releases in versions before 3.12.

Example:

```
>>> import sys
>>> if hasattr(sys, "set_int_max_str_digits"):
...     upper_bound = 68000
...     lower_bound = 4004
...     current_limit = sys.get_int_max_str_digits()
...     if current_limit == 0 or current_limit > upper_bound:
...         sys.set_int_max_str_digits(upper_bound)
...     elif current_limit < lower_bound:
...         sys.set_int_max_str_digits(lower_bound)
```

If you need to disable it entirely, set it to 0.

Footnotes

Additional information on these special methods may be found in the Python Reference Manual (Basic customization).

As a consequence, the list[1,2] is considered equal to [1.0,2.0], and similarly for tuples.

They must have since the parser can't tell the type of the operands.

Cased characters are those with general category property being one of "Lu" (Letter, uppercase), "Ll" (Letter, lowercase), or "Lt" (Letter, titlecase).

To format only a tuple you should therefore provide a singleton tuple whose only element is the tuple to be formatted.

Built-in Exceptions

In Python, all exceptions must be instances of a class that derives from `BaseException`. In a statement with an `except` clause that mentions a particular class, that clause also handles any exception classes derived from that class (but not exception classes from which it is derived). Two exception classes that are not related via subclassing are never equivalent, even if they have the same name.

The built-in exceptions listed in this chapter can be generated by the interpreter or built-in functions. Except where mentioned, they have an “associated value” indicating the detailed cause of the error. This may be a string or a tuple of several items of information (e.g., an error code and a string explaining the code). The associated value is usually passed as arguments to the exception class’s constructor.

User code can raise built-in exceptions. This can be used to test an exception handler or to report an error condition “just like” the situation in which the interpreter raises the same exception; but beware that there is nothing to prevent user code from raising an inappropriate error.

The built-in exception classes can be subclassed to define new exceptions; programmers are encouraged to derive new exceptions from the `Exception` class or one of its subclasses, and not from `BaseException`. More information on defining exceptions is available in the Python Tutorial under `User-defined Exceptions`.

Exception context

Three attributes on exception objects provide information about the context in which the exception was raised:

`BaseException.__context__`

`BaseException.__cause__`

`BaseException.__suppress_context__`

When raising a new exception while another exception is already being handled, the new exception’s `__context__` attribute is automatically set to the handled exception. An exception may be handled when an `except` or `finally` clause, or a `with` statement, is used. This implicit exception context can be supplemented with an explicit cause by using `from` with `raise`: `raise new_exc from original_exc`. The expression following `from` must be an exception or `None`. It will be set as `__cause__` on the raised exception. Setting `__cause__`

Inheriting from built-in exceptions

User code can create subclasses that inherit from an exception type. It’s recommended to only subclass one exception type at a time to avoid any possible conflicts between how the bases handle the `args` attribute, as well as due to possible memory layout incompatibilities.

CPython implementation detail: Most built-in exceptions are implemented in C for efficiency, see: `Objects/exceptions.c`. Some have custom memory layouts which makes it impossible to create a subclass that inherits from multiple exception types. The memory layout of a type is an implementation detail and might change between Python versions, leading to new conflicts in the future. Therefore, it’s recommended to avoid subclassing multiple exception types altogether.

Base classes

The following exceptions are used mostly as base classes for other exceptions.

`exception BaseException`

args

with_traceback(tb)

__traceback__

add_note(note)

__notes__

The base class for all built-in exceptions. It is not meant to be directly inherited by user-defined classes (for that, use `Exception`). If `str()` is called on an instance of this class, the representation of the argument(s) to the instance are returned, or the empty string when there were no arguments. `args` The tuple of arguments given to the exception constructor. Some built-in exceptions (like `OSError`) expect a certain number of arguments and assign a special meaning to the elements of this tuple,

The tuple of arguments given to the exception constructor. Some built-in exceptions (like `OSError`) expect a certain number of arguments and assign a special meaning to the elements of this tuple, while others are usually called only with a single string giving an error message.

This method sets `tb` as the new traceback for the exception and returns the exception object. It was more commonly used before the exception chaining features of PEP 3134 became available. The following example shows how we can convert an instance of `SomeException` into an instance of `OtherException` while preserving the traceback. Once raised, the current frame is pushed onto the traceback of the `OtherException`, as would have happened to the traceback of the original `SomeException` had we allowed it to pro

A writable field that holds the `traceback` object associated with this exception. See also: `The raise statement`.

Add the string `note` to the exception's notes which appear in the standard traceback after the exception string. `A TypeError` is raised if `note` is not a string. Added in version 3.11.

A list of the notes of this exception, which were added with `add_note()`. This attribute is created when `add_note()` is called. Added in version 3.11.

exceptionException

All built-in, non-system-exiting exceptions are derived from this class. All user-defined exceptions should also be derived from this class.

exceptionArithmeticError

The base class for those built-in exceptions that are raised for various arithmetic errors: `OverflowError`, `ZeroDivisionError`, `FloatingPointError`.

exceptionBufferError

Raised when a buffer-related operation cannot be performed.

exceptionLookupError

The base class for the exceptions that are raised when a key or index used on a mapping or sequence is invalid: `IndexError`, `KeyError`. This can be raised directly by `codecs.lookup()`.

Concrete exceptions

The following exceptions are the exceptions that are usually raised.

exceptionAssertionError

Raised when an `assert` statement fails.

exceptionAttributeError

name

obj

Raised when an attribute reference (see `Attribute references`) or assignment fails. (When an object does not support attribute references or attribute assignments at all, `TypeError` is raised.) The optional `name` and `obj` keyword-only arguments set the corresponding attributes: `name` The name of the attribute that was attempted to be accessed. `obj` The object that was accessed for the named attribute. Changed in version 3.10: Added the `name` and `obj` attributes.

The name of the attribute that was attempted to be accessed.

The object that was accessed for the named attribute.

exception EOFError

Raised when the `input()` function hits an end-of-file condition (EOF) without reading any data. (Note: the `io.TextIOBase.read()` and `io.IOBase.readline()` methods return an empty string when they hit EOF.)

exception FloatingPointError

Not currently used.

exception GeneratorExit

Raised when a generator or coroutine is closed; see `generator.close()` and `coroutine.close()`. It directly inherits from `BaseException` instead of `Exception` since it is technically not an error.

exception ImportError

name

path

Raised when the `import` statement has troubles trying to load a module. Also raised when the “from list” in `from...import` has a name that cannot be found. The optional `name` and `path` keyword-only arguments set the corresponding attributes: `name` The name of the module that was attempted to be imported. `path` The path to any file which triggered the exception. Changed in version 3.3: Added the `name` and `path` attributes.

The name of the module that was attempted to be imported.

The path to any file which triggered the exception.

exception ModuleNotFoundError

A subclass of `ImportError` which is raised by `import` when a module could not be located. It is also raised when `None` is found in `sys.modules`. Added in version 3.6.

exception IndexError

Raised when a sequence subscript is out of range. (Slice indices are silently truncated to fall in the allowed range; if an index is not an integer, `TypeError` is raised.)

exception KeyError

Raised when a mapping (dictionary) key is not found in the set of existing keys.

exception KeyboardInterrupt

Raised when the user hits the interrupt key (normally `Control-C` or `Delete`). During execution, a check for interrupts is made regularly. The exception inherits from `BaseException` so as to not be accidentally caught by code that catches `Exception` and thus prevent the interpreter from exiting. Note: Catching a `KeyboardInterrupt` requires special consideration. Because it can be raised at unpredictable points, it may, in some circumstances, leave the running program in an inconsistent state. It is generally best

exceptionMemoryError

Raised when an operation runs out of memory but the situation may still be rescued (by deleting some objects). The associated value is a string indicating what kind of (internal) operation ran out of memory. Note that because of the underlying memory management architecture (C's `malloc()` function), the interpreter may not always be able to completely recover from this situation; it nevertheless raises an exception so that a stack traceback can be printed, in case a run-away program was the cause.

exceptionNameError

name

Raised when a local or global name is not found. This applies only to unqualified names. The associated value is an error message that includes the name that could not be found. The optional `namekeyword-only` argument sets the attribute: `name` The name of the variable that was attempted to be accessed. Changed in version 3.10: Added the `name` attribute.

The name of the variable that was attempted to be accessed.

exceptionNotImplementedError

This exception is derived from `RuntimeError`. In user defined base classes, abstract methods should raise this exception when they require derived classes to override the method, or while the class is being developed to indicate that the real implementation still needs to be added. Note it should not be used to indicate that an operator or method is not meant to be supported at all – in that case either leave the operator / method undefined or, if a subclass, set it to `None`. Caution: `NotImplementedError`

exceptionOSError([arg])

exceptionOSError(errno, strerror[, filename[, winerror[, filename2]]])

errno

winerror

strerror

filename

filename2

This exception is raised when a system function returns a system-related error, including I/O failures such as “file not found” or “disk full” (not for illegal argument types or other incidental errors). The second form of the constructor sets the corresponding attributes, described below. The attributes default to `None` if not specified. For backwards compatibility, if three arguments are passed, the `args` attribute contains only a 2-tuple of the first two constructor arguments. The constructor often

A numeric error code from the C variable `errno`.

Under Windows, this gives you the native Windows error code. The `errno` attribute is then an approximate translation, in POSIX terms, of that native error code. Under Windows, if the `winerror` constructor argument is an integer, the `errno` attribute is determined from the Windows error code, and the `errno` argument is ignored. On other platforms, the `winerror` argument is ignored, and the `winerror` attribute does not exist.

The corresponding error message, as provided by the operating system. It is formatted by the C functions `strerror()` under POSIX, and `FormatMessage()` under Windows.

For exceptions that involve a file system path (such as `open()` or `os.unlink()`), `filename` is the file name passed to the function. For functions that involve two file system paths (such as `os.rename()`), `filename2` corresponds to the second file name passed to the function.

exceptionOverflowError

Raised when the result of an arithmetic operation is too large to be represented. This cannot occur for integers (which would rather raise `MemoryError` than give up). However, for historical reasons, `OverflowError` is sometimes raised for integers that are outside a required range. Because of the lack of standardization of floating-point exception handling in C, most floating-point operations are not checked.

exception `PythonFinalizationError`

This exception is derived from `RuntimeError`. It is raised when an operation is blocked during interpreter shutdown also known as Python finalization. Examples of operations which can be blocked with a `PythonFinalizationError` during the Python finalization: Creating a new Python thread. Joining a running daemon thread. `os.fork()`. See also the `sys.is_finalizing()` function. Added in version 3.13: Previously, a plain `RuntimeError` was raised. Changed in version 3.14: `threading.Thread.join()` can now raise this exception.

exception `RecursionError`

This exception is derived from `RuntimeError`. It is raised when the interpreter detects that the maximum recursion depth (see `sys.getrecursionlimit()`) is exceeded. Added in version 3.5: Previously, a plain `RuntimeError` was raised.

exception `ReferenceError`

This exception is raised when a weak reference proxy, created by the `weakref.proxy()` function, is used to access an attribute of the referent after it has been garbage collected. For more information on weak references, see the `weakref` module.

exception `RuntimeError`

Raised when an error is detected that doesn't fall in any of the other categories. The associated value is a string indicating what precisely went wrong.

exception `StopIteration`

value

Raised by built-in function `next()` and an iterator's `__next__()` method to signal that there are no further items produced by the iterator. The exception object has a single attribute `value`, which is given as an argument when constructing the exception, and defaults to `None`. When a generator or coroutine function returns, a new `StopIteration` instance is raised, and the value returned by the function is used as the `value` parameter to the constructor of the exception. If a generator code directly or indirectly

The exception object has a single attribute `value`, which is given as an argument when constructing the exception, and defaults to `None`.

exception `StopAsyncIteration`

Must be raised by `__anext__()` method of an asynchronous iterator object to stop the iteration. Added in version 3.5.

exception `SyntaxError(message, details)`

filename

lineno

offset

text

end_lineno

end_offset

Raised when the parser encounters a syntax error. This may occur in an `import` statement, in a call to the built-in functions `compile()`, `exec()`, `eval()`, or when reading the initial script or standard input (also interactively). The `str()` of the exception instance returns only the error message. `Details` is a tuple whose members are also available as separate attributes: `filename` The name of the file the syntax error occurred in. `lineno` Which line number in the file the error occurred in. This is 1-indexed

The name of the file the syntax error occurred in.

Which line number in the file the error occurred in. This is 1-indexed: the first line in the file has a `lineno` of 1.

The column in the line where the error occurred. This is 1-indexed: the first character in the line has an `offset` of 1.

The source code text involved in the error.

Which line number in the file the error occurred ends in. This is 1-indexed: the first line in the file has a `lineno` of 1.

The column in the end line where the error occurred finishes. This is 1-indexed: the first character in the line has an `offset` of 1.

exceptionIndentationError

Base class for syntax errors related to incorrect indentation. This is a subclass of `SyntaxError`.

exceptionTabError

Raised when indentation contains an inconsistent use of tabs and spaces. This is a subclass of `IndentationError`.

exceptionSystemError

Raised when the interpreter finds an internal error, but the situation does not look so serious to cause it to abandon all hope. The associated value is a string indicating what went wrong (in low-level terms). In CPython, this could be raised by incorrectly using Python's C API, such as returning a `NULL` value without an exception set. If you're confident that this exception wasn't your fault, or the fault of a package you're using, you should report this to the author or maintainer of your Python i

exceptionSystemExit

code

This exception is raised by the `sys.exit()` function. It inherits from `BaseException` instead of `Exception` so that it is not accidentally caught by code that catches `Exception`. This allows the exception to properly propagate up and cause the interpreter to exit. When it is not handled, the Python interpreter exits; no stack traceback is printed. The constructor accepts the same optional argument passed to `sys.exit()`. If the value is an integer, it specifies the system exit status (passed to C's `exit()`)

The exit status or error message that is passed to the constructor. (Defaults to `None`.)

exceptionTypeError

Raised when an operation or function is applied to an object of inappropriate type. The associated value is a string giving details about the type mismatch. This exception may be raised by user code to indicate that an attempted operation on an object is not supported, and is not meant to be. If an object is meant to support a given operation but has not yet provided an implementation, `NotImplementedError` is the proper exception to raise. Passing arguments of the wrong type (e.g. passing a list when

exceptionUnboundLocalError

Raised when a reference is made to a local variable in a function or method, but no value has been bound to that variable. This is a subclass of `NameError`.

exceptionUnicodeError

encoding

reason

object

start

end

Raised when a Unicode-related encoding or decoding error occurs. It is a subclass of `ValueError`. `UnicodeError` has attributes that describe the encoding or decoding error. For example, `err.object[err.start:err.end]` gives the particular invalid input that the codec failed on. `encoding` The name of the encoding that raised the error. `reason` A string describing the specific codec error. `object` The object the codec was attempting to encode or decode. `start` The first index of invalid data in `object`. This value should

The name of the encoding that raised the error.

A string describing the specific codec error.

The object the codec was attempting to encode or decode.

The first index of invalid data in `object`. This value should not be negative as it is interpreted as an absolute offset but this constraint is not enforced at runtime.

The index after the last invalid data in `object`. This value should not be negative as it is interpreted as an absolute offset but this constraint is not enforced at runtime.

exceptionUnicodeEncodeError

Raised when a Unicode-related error occurs during encoding. It is a subclass of `UnicodeError`.

exceptionUnicodeDecodeError

Raised when a Unicode-related error occurs during decoding. It is a subclass of `UnicodeError`.

exceptionUnicodeTranslateError

Raised when a Unicode-related error occurs during translating. It is a subclass of `UnicodeError`.

exceptionValueError

Raised when an operation or function receives an argument that has the right type but an inappropriate value, and the situation is not described by a more precise exception such as `IndexError`.

exceptionZeroDivisionError

Raised when the second argument of a division or modulo operation is zero. The associated value is a string indicating the type of the operands and the operation.

The following exceptions are kept for compatibility with previous versions; starting from Python 3.3, they are aliases of `OSError`.

exceptionEnvironmentError

exceptionIOError

exceptionWindowsError

Only available on Windows.

OS exceptions

The following exceptions are subclasses of `OSError`, they get raised depending on the system error code.

exceptionBlockingIOError

characters_written

Raised when an operation would block on an object (e.g. socket) set for non-blocking operation. Corresponds to `errno.EAGAIN`, `errno.EALREADY`, `errno.EWOULDBLOCK` and `errno.EINPROGRESS`. In addition to those of `OSError`, `BlockingIOError` can have one more attribute: `characters_written`. An integer containing the number of bytes written to the stream before it blocked. This attribute is available when using the buffered I/O classes from the `io` module.

An integer containing the number of bytes written to the stream before it blocked. This attribute is available when using the buffered I/O classes from the `io` module.

exceptionChildProcessError

Raised when an operation on a child process failed. Corresponds to `errno.ECHILD`.

exceptionConnectionError

A base class for connection-related issues. Subclasses are `BrokenPipeError`, `ConnectionAbortedError`, `ConnectionRefusedError` and `ConnectionResetError`.

exceptionBrokenPipeError

A subclass of `ConnectionError`, raised when trying to write on a pipe while the other end has been closed, or trying to write on a socket which has been shutdown for writing. Corresponds to `errno.EPIPE` and `errno.ESHUTDOWN`.

exceptionConnectionAbortedError

A subclass of `ConnectionError`, raised when a connection attempt is aborted by the peer. Corresponds to `errno.ECONNABORTED`.

exceptionConnectionRefusedError

A subclass of `ConnectionError`, raised when a connection attempt is refused by the peer. Corresponds to `errno.ECONNREFUSED`.

exceptionConnectionResetError

A subclass of `ConnectionError`, raised when a connection is reset by the peer. Corresponds to `errno.ECONNRESET`.

exceptionFileExistsError

Raised when trying to create a file or directory which already exists. Corresponds to `errno.EEXIST`.

exceptionFileNotFoundError

Raised when a file or directory is requested but doesn't exist. Corresponds to `errno.ENOENT`.

exceptionInterruptedError

Raised when a system call is interrupted by an incoming signal. Corresponds to `errno.EINTR`. Changed in version 3.5: Python now retries system calls when a syscall is interrupted by a signal, except if the signal handler raises an exception (see PEP 475 for the rationale), instead of raising `InterruptedError`.

exceptionIsADirectoryError

Raised when a file operation (such as `os.remove()`) is requested on a directory. Corresponds to `errno.EISDIR`.

exceptionNotADirectoryError

Raised when a directory operation (such as `os.listdir()`) is requested on something which is not a directory. On most POSIX platforms, it may also be raised if an operation attempts to open or traverse a non-directory file as if it were a directory. Corresponds to `errno.ENOTDIR`.

exceptionPermissionError

Raised when trying to run an operation without the adequate access rights - for example filesystem permissions. Corresponds to `errno.EACCES`, `EPERM`, and `ENOTCAPABLE`. Changed in version 3.11.1: `WASI`'s `ENOTCAPABLE` is now mapped to `PermissionError`.

exceptionProcessLookupError

Raised when a given process doesn't exist. Corresponds to `errno.ESRCH`.

exceptionTimeoutError

Raised when a system function timed out at the system level. Corresponds to `errno.ETIMEDOUT`.

Added in version 3.3: All the above `OSError` subclasses were added.

See also

PEP 3151- Reworking the OS and IO exception hierarchy

Warnings

The following exceptions are used as warning categories; see the [Warning Categories](#) documentation for more details.

exceptionWarning

Base class for warning categories.

exceptionUserWarning

Base class for warnings generated by user code.

exceptionDeprecationWarning

Base class for warnings about deprecated features when those warnings are intended for other Python developers. Ignored by the default warning filters, except in the `__main__` module (PEP 565). Enabling the `Python Development Mode` shows this warning. The deprecation policy is described in PEP 387.

exceptionPendingDeprecationWarning

Base class for warnings about features which are obsolete and expected to be deprecated in the future, but are not deprecated at the moment. This class is rarely used as emitting a warning about a possible upcoming deprecation is unusual, and `DeprecationWarning` is preferred for already active deprecations. Ignored by the default warning filters. Enabling the `Python Development Mode` shows this warning. The deprecation policy is described in PEP 387.

exceptionSyntaxWarning

Base class for warnings about dubious syntax. This warning is typically emitted when compiling Python source code, and usually won't be reported when running already compiled code.

exceptionRuntimeWarning

Base class for warnings about dubious runtime behavior.

exceptionFutureWarning

Base class for warnings about deprecated features when those warnings are intended for end users of applications that are written in Python.

exceptionImportWarning

Base class for warnings about probable mistakes in module imports. Ignored by the default warning filters. Enabling the Python Development Mode shows this warning.

exceptionUnicodeWarning

Base class for warnings related to Unicode.

exceptionEncodingWarning

Base class for warnings related to encodings. See `Opt-in EncodingWarning` for details. Added in version 3.10.

exceptionBytesWarning

Base class for warnings related to `bytes` and `bytearray`.

exceptionResourceWarning

Base class for warnings related to resource usage. Ignored by the default warning filters. Enabling the Python Development Mode shows this warning. Added in version 3.2.

Exception groups

The following are used when it is necessary to raise multiple unrelated exceptions. They are part of the exception hierarchy so they can be handled with `except` like all other exceptions. In addition, they are recognised by `except*`, which matches their subgroups based on the types of the contained exceptions.

exceptionExceptionGroup(msg, excs)

exceptionBaseExceptionGroup(msg, excs)

message

exceptions

subgroup(condition)

split(condition)

derive(excs)

Both of these exception types wrap the exceptions in the sequence `excs`. The `msg` parameter must be a string. The difference between the two classes is that `BaseExceptionGroup` extends `BaseException` and it can wrap any exception, while `ExceptionGroup` extends `Exception` and it can only wrap subclasses of `Exception`. This design is so that `except Exception` catches an `ExceptionGroup` but not `BaseExceptionGroup`. The `BaseExceptionGroup` constructor returns an `ExceptionGroup` rather than a `BaseExceptionGroup` if all contained exceptions

The `msg` argument to the constructor. This is a read-only attribute.

A tuple of the exceptions in the `excs` sequence given to the constructor. This is a read-only attribute.

Returns an exception group that contains only the exceptions from the current group that match `condition`, or `None` if the result is empty. The condition can be an exception type or tuple of exception types, in which case each exception is checked for a match using the same check that is used in an `except` clause. The condition can also be a callable (other than a type object) that accepts an exception as its single argument and returns `true` for the exceptions that should be in the subgroup. The nesting

Like `subgroup()`, but returns the pair `(match, rest)` where `match` is `subgroup(condition)` and `rest` is the remaining non-matching part.

Returns an exception group with the same `message`, but which wraps the exceptions in `excs`. This method is used by `subgroup()` and `split()`, which are used in various contexts to break up an exception group. A subclass

needs to override it in order to make `subgroup()` and `split()` return instances of the subclass rather than `ExceptionGroup.subgroup()` and `split()` copy the `__traceback__`, `__cause__`, `__context__` and `__notes__` fields from the original exception group to the one returned by `derive()`, so these fields do not need

Exception hierarchy

The class hierarchy for built-in exceptions is:

```
BaseException
├── BaseExceptionGroup
├── GeneratorExit
├── KeyboardInterrupt
├── SystemExit
├── Exception
│   ├── ArithmeticError
│   │   ├── FloatingPointError
│   │   ├── OverflowError
│   │   └── ZeroDivisionError
│   ├── AssertionError
│   ├── AttributeError
│   ├── BufferError
│   ├── EOFError
│   ├── ExceptionGroup [BaseExceptionGroup]
│   ├── ImportError
│   │   └── ModuleNotFoundError
│   ├── LookupError
│   │   ├── IndexError
│   │   └── KeyError
│   ├── MemoryError
│   ├── NameError
│   │   └── UnboundLocalError
│   ├── OSError
│   │   ├── BlockingIOError
│   │   ├── ChildProcessError
│   │   ├── ConnectionError
│   │   │   ├── BrokenPipeError
│   │   │   ├── ConnectionAbortedError
│   │   │   ├── ConnectionRefusedError
│   │   │   └── ConnectionResetError
│   │   ├── FileExistsError
│   │   ├── FileNotFoundError
│   │   ├── InterruptedError
│   │   ├── IsADirectoryError
│   │   ├── NotADirectoryError
│   │   ├── PermissionError
│   │   ├── ProcessLookupError
│   │   └── TimeoutError
│   ├── ReferenceError
│   ├── RuntimeError
│   │   ├── NotImplementedError
│   │   ├── PythonFinalizationError
│   │   └── RecursionError
│   ├── StopAsyncIteration
│   ├── StopIteration
│   ├── SyntaxError
│   │   ├── IndentationError
│   │   └── TabError
│   ├── SystemError
│   ├── TypeError
│   ├── ValueError
│   │   ├── UnicodeError
│   │   └── UnicodeDecodeError
```

- ■■■ UnicodeEncodeError
- ■■■ UnicodeTranslateError
- Warning
 - BytesWarning
 - DeprecationWarning
 - EncodingWarning
 - FutureWarning
 - ImportWarning
 - PendingDeprecationWarning
 - ResourceWarning
 - RuntimeWarning
 - SyntaxWarning
 - UnicodeWarning
 - UserWarning

Thread Safety Guarantees

This page documents thread-safety guarantees for built-in types in Python's free-threaded build. The guarantees described here apply when using Python with the GIL disabled (free-threaded mode). When the GIL is enabled, most operations are implicitly serialized.

For general guidance on writing thread-safe code in free-threaded Python, see [Python support for free threading](#).

Thread safety levels

The C API documentation uses the following levels to describe the thread safety guarantees of each function. The levels are listed from least to most safe.

Incompatible

A function or operation that cannot be made safe for concurrent use even with external synchronization. Incompatible code typically accesses global state in an unsynchronized way and must only be called from a single thread throughout the program's lifetime.

Example: a function that modifies process-wide state such as signal handlers or environment variables, where concurrent calls from any threads, even with external locking, can conflict with the runtime or other libraries.

Compatible

A function or operation that is safe to call from multiple threads provided the caller supplies appropriate external synchronization, for example by holding a lock for the duration of each call. Without such synchronization, concurrent calls may produce race conditions or data races.

Example: a function that reads from or writes to an object whose internal state is not protected by a lock. Callers must ensure that no two threads access the same object at the same time.

Safe on distinct objects

A function or operation that is safe to call from multiple threads without external synchronization, as long as each thread operates on a different object. Two threads may call the function at the same time, but they must not pass the same object (or objects that share underlying state) as arguments.

Example: a function that modifies fields of a struct using non-atomic writes. Two threads can each call the function on their own struct instance safely, but concurrent calls on the same instance require external synchronization.

Safe on shared objects

A function or operation that is safe for concurrent use on the same object. The implementation uses internal synchronization (such as per-object locks or critical sections) to protect shared mutable state, so callers do not need to supply their own locking.

Example: `PyList_GetItemRef()` can be called from multiple threads on the same `PyListObject` - it uses internal synchronization to serialize access.

Atomic

A function or operation that appears atomic with respect to other threads - it executes instantaneously from the perspective of other threads. This is the strongest form of thread safety.

Example: `PyMutex_IsLocked()` performs an atomic read of the mutex state and can be called from any thread at any time.

Thread safety for list objects

Reading a single element from a list is atomic:

```
lst[i] # list.__getitem__
```

The following methods traverse the list and use atomic reads of each item to perform their function. That means that they may return results affected by concurrent modifications:

```
item in lst
lst.index(item)
lst.count(item)
```

All of the above operations avoid acquiring per-object locks. They do not block concurrent modifications. Other operations that hold a lock will not block these from observing intermediate states.

All other operations from here on block using the per-object lock.

Writing a single item via `lst[i]=x` is safe to call from multiple threads and will not corrupt the list.

The following operations return new objects and appear atomic to other threads:

```
lst1 + lst2 # concatenates two lists into a new list
x * lst     # repeats lst x times into a new list
lst.copy()  # returns a shallow copy of the list
```

The following methods that only operate on a single element with no shifting required are atomic:

```
lst.append(x) # append to the end of the list, no shifting required
lst.pop()     # pop element from the end of the list, no shifting required
```

The `clear()` method is also atomic. Other threads cannot observe elements being removed.

The `sort()` method is not atomic. Other threads cannot observe intermediate states during sorting, but the list appears empty for the duration of the sort.

The following operations may allow lock-free operations to observe intermediate states since they modify multiple elements in place:

```
lst.insert(idx, item) # shifts elements
lst.pop(idx)          # idx not at the end of the list, shifts elements
lst *= x              # copies elements in place
```

The `remove()` method may allow concurrent modifications since element comparison may execute arbitrary Python code (via `__eq__()`).

`extend()` is safe to call from multiple threads. However, its guarantees depend on the iterable passed to it. If it is a list, a tuple, a set, a frozen set, a dict or a dictionary view object (but not their subclasses), the `extend` operation is safe from concurrent modifications to the iterable. Otherwise, an iterator is created which can be concurrently modified by another thread. The same applies to in-place concatenation of a list with other iterables when using `lst+=iterable`.

Similarly, assigning to a list slice with `lst[i:j]=iterable` is safe to call from multiple threads, but the iterable is only locked when it is also a list (but not its subclasses).

Operations that involve multiple accesses, as well as iteration, are never atomic. For example:

```
# NOT atomic: read-modify-write
lst[i] = lst[i] + 1

# NOT atomic: check-then-act
if lst:
    item = lst.pop()
```

```
# NOT thread-safe: iteration while modifying
for item in lst:
    process(item) # another thread may modify lst
```

Consider external synchronization when sharing list instances across threads.

Thread safety for dict objects

Creating a dictionary with the dict constructor is atomic when the argument to it is a dict or a tuple. When using the dict.fromkeys() method, dictionary creation is atomic when the argument is a dict, tuple, set, or frozenset.

The following operations and functions are lock-free and atomic.

```
d[key]          # dict.__getitem__
d.get(key)      # dict.get
key in d        # dict.__contains__
len(d)          # dict.__len__
```

All other operations from here on hold the per-object lock.

Writing or removing a single item is safe to call from multiple threads and will not corrupt the dictionary:

```
d[key] = value    # write
del d[key]        # delete
d.pop(key)        # remove and return
d.popitem()       # remove and return last item
d.setdefault(key, v) # insert if missing
```

These operations may compare keys using `__eq__()`, which can execute arbitrary Python code. During such comparisons, the dictionary may be modified by another thread. For built-in types like `str`, `int`, and `float`, that implement `__eq__()` in C, the underlying lock is not released during comparisons and this is not a concern.

The following operations return new objects and hold the per-object lock for the duration of the operation:

```
d.copy()         # returns a shallow copy of the dictionary
d | other        # merges two dicts into a new dict
d.keys()         # returns a new dict_keys view object
d.values()       # returns a new dict_values view object
d.items()        # returns a new dict_items view object
```

The `clear()` method holds the lock for its duration. Other threads cannot observe elements being removed.

The following operations lock both dictionaries. For `update()` and `|=`, this applies only when the other operand is a dict that uses the standard dict iterator (but not subclasses that override iteration). For equality comparison, this applies to dict and its subclasses:

```
d.update(other_dict) # both locked when other_dict is a dict
d |= other_dict      # both locked when other_dict is a dict
d == other_dict      # both locked for dict and subclasses
```

All comparison operations also compare values using `__eq__()`, so for non-built-in types the lock may be released during comparison.

`fromkeys()` locks both the new dictionary and the iterable when the iterable is exactly a dict, set, or frozenset (not subclasses):

```
dict.fromkeys(a_dict)    # locks both
dict.fromkeys(a_set)     # locks both
dict.fromkeys(a_frozenset) # locks both
```

When updating from a non-dict iterable, only the target dictionary is locked. The iterable may be concurrently modified by another thread:

```
d.update(iterable)      # iterable is not a dict: only d locked
```

```
d |= iterable          # iterable is not a dict: only d locked
dict.fromkeys(iterable) # iterable is not a dict/set/frozenset: only result locked
```

Operations that involve multiple accesses, as well as iteration, are never atomic:

```
# NOT atomic: read-modify-write
d[key] = d[key] + 1

# NOT atomic: check-then-act (TOCTOU)
if key in d:
    del d[key]

# NOT thread-safe: iteration while modifying
for key, value in d.items():
    process(key) # another thread may modify d
```

To avoid time-of-check to time-of-use (TOCTOU) issues, use atomic operations or handle exceptions:

```
# Use pop() with default instead of check-then-delete
d.pop(key, None)

# Or handle the exception
try:
    del d[key]
except KeyError:
    pass
```

To safely iterate over a dictionary that may be modified by another thread, iterate over a copy:

```
# Make a copy to iterate safely
for key, value in d.copy().items():
    process(key)
```

Consider external synchronization when sharing dict instances across threads.

Thread safety for set objects

The `len()` function is lock-free and atomic.

The following read operation is lock-free. It does not block concurrent modifications and may observe intermediate states from operations that hold the per-object lock:

```
elem in s    # set.__contains__
```

This operation may compare elements using `__eq__()`, which can execute arbitrary Python code. During such comparisons, the set may be modified by another thread. For built-in types like `str`, `int`, and `float`, `__eq__()` does not release the underlying lock during comparisons and this is not a concern.

All other operations from here on hold the per-object lock.

Adding or removing a single element is safe to call from multiple threads and will not corrupt the set:

```
s.add(elem)        # add element
s.remove(elem)     # remove element, raise if missing
s.discard(elem)    # remove element if present
s.pop()            # remove and return arbitrary element
```

These operations also compare elements, so the same `__eq__()` considerations as above apply.

The `copy()` method returns a new object and holds the per-object lock for the duration so that it is always atomic.

The `clear()` method holds the lock for its duration. Other threads cannot observe elements being removed.

The following operations only accept `set` or `frozenset` as operands and always lock both objects:

```
s |= other          # other must be set/frozenset
```

```

s &= other          # other must be set/frozenset
s -= other          # other must be set/frozenset
s ^= other          # other must be set/frozenset
s & other           # other must be set/frozenset
s | other           # other must be set/frozenset
s - other           # other must be set/frozenset
s ^ other           # other must be set/frozenset

```

`set.update()`, `set.union()`, `set.intersection()` and `set.difference()` can take multiple iterables as arguments. They all iterate through all the passed iterables and do the following:

- `set.update()` and `set.union()` lock both objects only when the other operand is a `set`, `frozenset`, or `dict`.
- `set.intersection()` and `set.difference()` always try to lock all objects.

`set.symmetric_difference()` tries to lock both objects.

The update variants of the above methods also have some differences between them:

- `set.difference_update()` and `set.intersection_update()` try to lock all objects one-by-one.
- `set.symmetric_difference_update()` only locks the arguments if it is of type `set`, `frozenset`, or `dict`.

The following methods always try to lock both objects:

```

s.isdisjoint(other)    # both locked
s.issubset(other)      # both locked
s.issuperset(other)    # both locked

```

Operations that involve multiple accesses, as well as iteration, are never atomic:

```

# NOT atomic: check-then-act
if elem in s:
    s.remove(elem)

# NOT thread-safe: iteration while modifying
for elem in s:
    process(elem)  # another thread may modify s

```

Consider external synchronization when sharing `set` instances across threads. See [Python support for free threading](#) for more information.

Thread safety for bytearray objects

The `len()` function is lock-free and atomic.

Concatenation and comparisons use the buffer protocol, which prevents resizing but does not hold the per-object lock. These operations may observe intermediate states from concurrent modifications:

```

ba + other          # may observe concurrent writes
ba == other          # may observe concurrent writes
ba < other           # may observe concurrent writes

```

All other operations from here on hold the per-object lock.

Reading a single element or slice is safe to call from multiple threads:

```

ba[i]               # bytearray.__getitem__
ba[i:j]             # slice

```

The following operations are safe to call from multiple threads and will not corrupt the `bytearray`:

```

ba[i] = x           # write single byte
ba[i:j] = values     # write slice
ba.append(x)         # append single byte
ba.extend(other)     # extend with iterable

```

```

ba.insert(i, x)    # insert single byte
ba.pop()          # remove and return last byte
ba.pop(i)         # remove and return byte at index
ba.remove(x)      # remove first occurrence
ba.reverse()      # reverse in place
ba.clear()        # remove all bytes

```

Slice assignment locks both objects when `value` is a bytearray:

```
ba[i:j] = other_bytearray # both locked
```

The following operations return new objects and hold the per-object lock for the duration:

```

ba.copy()         # returns a shallow copy
ba * n            # repeat into new bytearray

```

The membership test holds the lock for its duration:

```
x in ba           # bytearray.__contains__
```

All other bytearray methods (such as `find()`, `replace()`, `split()`, `decode()`, etc.) hold the per-object lock for their duration.

Operations that involve multiple accesses, as well as iteration, are never atomic:

```

# NOT atomic: check-then-act
if x in ba:
    ba.remove(x)

# NOT thread-safe: iteration while modifying
for byte in ba:
    process(byte) # another thread may modify ba

```

To safely iterate over a bytearray that may be modified by another thread, iterate over a copy:

```

# Make a copy to iterate safely
for byte in ba.copy():
    process(byte)

```

Consider external synchronization when sharing bytearray instances across threads. See [Python support for free threading](#) for more information.

Thread safety for memoryview objects

memoryview objects provide access to the internal data of an underlying object without copying. Thread safety depends on both the memoryview itself and the underlying buffer exporter.

The memoryview implementation uses atomic operations to track its own exports in the free-threaded build. Creating and releasing a memoryview are thread-safe. Attribute access (e.g., `shape`, `format`) reads fields that are immutable for the lifetime of the memoryview, so concurrent reads are safe as long as the memoryview has not been released.

However, the actual data accessed through the memoryview is owned by the underlying object. Concurrent access to this data is only safe if the underlying object supports it:

- For immutable objects like `bytes`, concurrent reads through multiple memoryviews are safe.
- For mutable objects like `bytearray`, reading and writing the same memory region from multiple threads without external synchronization is not safe and may result in data corruption. Note that even read-only memoryviews of mutable objects do not prevent data races if the underlying object is modified from another thread.

```

# NOT safe: concurrent writes to the same buffer
data = bytearray(1000)
view = memoryview(data)

```

```
# Thread 1: view[0:500] = b'x' * 500
# Thread 2: view[0:500] = b'y' * 500

# Safe: use a lock for concurrent access
import threading
lock = threading.Lock()
data = bytearray(1000)
view = memoryview(data)

with lock:
    view[0:500] = b'x' * 500
```

Resizing or reallocating the underlying object (such as calling `bytearray.resize()`) while a `memoryview` is exported raises `BufferError`. This is enforced regardless of threading.

Text Processing Services

The modules described in this chapter provide a wide range of string manipulation operations and other text processing services.

The `codecs` module described under [Binary Data Services](#) is also highly relevant to text processing. In addition, see the documentation for Python's built-in string type in [Text Sequence Type — str](#).

- [string](#)— Common string operationsString constantsCustom string formattingFormat string syntaxFormat specification mini-languageFormat examplesTemplate strings (\$-strings)Helper functions
- [string.template](#)lib— Support for template string literalsTemplate stringsTypesHelper functions
- [re](#)— Regular expression operationsRegular Expression SyntaxModule
ContentsFlagsFunctionsExceptionsRegular Expression ObjectsMatch ObjectsRegular Expression ExamplesChecking for a PairSimulating `scanf()``search()` vs. `match()`Making a PhonebookText MungingFinding all AdverbsFinding all Adverbs and their PositionsRaw String NotationWriting a Tokenizer
- [difflib](#)— Helpers for computing deltasSequenceMatcher objectsSequenceMatcher examplesDiffer objectsDiffer exampleA command-line interface to `difflib``diff` example
- [textwrap](#)— Text wrapping and filling
- [unicodedata](#)— Unicode Database
- [stringprep](#)— Internet String Preparation
- [readline](#)— GNU readline interfaceInit fileLine bufferHistory fileHistory listStartup hooksCompletionExample
- [rlcompleter](#)— Completion function for GNU readline

string— Common string operations

Source code: `Lib/string/__init__.py`

See also

[Text Sequence Type — str](#)

[String Methods](#)

String constants

The constants defined in this module are:

string.ascii_letters

The concatenation of the `ascii_lowercase` and `ascii_uppercase` constants described below. This value is not locale-dependent.

string.ascii_lowercase

The lowercase letters `'abcdefghijklmnopqrstuvwxyz'`. This value is not locale-dependent and will not change.

string.ascii_uppercase

The uppercase letters `'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`. This value is not locale-dependent and will not change.

string.digits

The string `'0123456789'`.

string.hexdigits

The string `'0123456789abcdefABCDEF'`.

string.octdigits

The string `'01234567'`.

string.punctuation

String of ASCII characters which are considered punctuation characters in the C locale: `!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~`.

string.printable

String of ASCII characters which are considered printable by Python. This is a combination of `digits`, `ascii_letters`, `punctuation`, and `whitespace`. Note By design, `string.printable.isprintable()` returns `False`. In particular, `string.printable` is not printable in the POSIX sense (see `LC_CTYPE`).

string.whitespace

A string containing all ASCII characters that are considered whitespace. This includes the characters space, tab, linefeed, return, formfeed, and vertical tab.

Custom string formatting

The built-in string class provides the ability to do complex variable substitutions and value formatting via the `format()` method described in [PEP 3101](#). The `Formatter` class in the `string` module allows you to create and

customize your own string formatting behaviors using the same implementation as the built-in `format()` method.

`class string.Formatter`

`format(format_string,/,*args,**kwargs)`

`vformat(format_string,args,kwargs)`

`parse(format_string)`

`get_field(field_name,args,kwargs)`

`get_value(key,args,kwargs)`

`check_unused_args(used_args,args,kwargs)`

`format_field(value,format_spec)`

`convert_field(value,conversion)`

The `Formatter` class has the following public methods:

- `format(format_string,/,*args,**kwargs)` The primary API method. It takes a format string and an arbitrary set of positional and keyword arguments. It is just a wrapper that calls `vformat()`. Changed in version 3.7: A format string argument is now positional-only.
- `vformat(format_string,args,kwargs)` This function does the actual work of formatting. It is exposed as a separate function for cases where you want to pass in a predefined dictionary of arguments.

The primary API method. It takes a format string and an arbitrary set of positional and keyword arguments. It is just a wrapper that calls `vformat()`. Changed in version 3.7: A format string argument is now positional-only.

This function does the actual work of formatting. It is exposed as a separate function for cases where you want to pass in a predefined dictionary of arguments, rather than unpacking and repacking the dictionary as individual arguments using the `*args` and `**kwargs` syntax. `vformat()` does the work of breaking up the format string into character data and replacement fields. It calls the various methods described below.

Loop over the `format_string` and return an iterable of tuples (`literal_text`, `field_name`, `format_spec`, `conversion`). This is used by `vformat()` to break the string into either literal text, or replacement fields. The values in the tuple conceptually represent a span of literal text followed by a single replacement field. If there is no literal text (which can happen if two replacement fields occur consecutively), then `literal_text` will be a zero-length string. If there is no replacement field, then the `v`

Given `field_name`, convert it to an object to be formatted. Auto-numbering off `field_name` returned from `parse()` is done by `vformat()` before calling this method. Returns a tuple (`obj`, `used_key`). The default version takes strings of the form defined in PEP 3101, such as `"0[name]"` or `"label.title"`. `args` and `kwargs` are as passed in to `vformat()`. The return value `used_key` has the same meaning as the `key` parameter to `get_value()`.

Retrieve a given field value. The `key` argument will be either an integer or a string. If it is an integer, it represents the index of the positional argument in `args`; if it is a string, then it represents a named argument in `kwargs`. The `args` parameter is set to the list of positional arguments to `vformat()`, and the `kwargs` parameter is set to the dictionary of keyword arguments. For compound field names, these functions are only called for the first component of the field name; subsequent components are handled by `format_field()`.

Implement checking for unused arguments if desired. The arguments to this function is the set of all argument keys that were actually referred to in the format string (integers for positional arguments, and strings for named arguments), and a reference to the `args` and `kwargs` that was passed to `vformat`. The set of unused args can be calculated from these parameters. `check_unused_args()` is assumed to raise an exception if the check fails.

`format_field()` simply calls the global `format()` built-in. The method is provided so that subclasses can override it.

Converts the value (returned by `get_field()`) given a conversion type (as in the tuple returned by the `parse()` method). The default version understands `'s'` (str), `'r'` (repr) and `'a'` (ascii) conversion types.

Format string syntax

The `str.format()` method and the `Formatter` class share the same syntax for format strings (although in the case of `Formatter`, subclasses can define their own format string syntax). The syntax is related to that of formatted string literals and template string literals, but it is less sophisticated and, in particular, does not support arbitrary expressions in interpolations.

Format strings contain “replacement fields” surrounded by curly braces `{}`. Anything that is not contained in braces is considered literal text, which is copied unchanged to the output. If you need to include a brace character in the literal text, it can be escaped by doubling: `{{and}}`.

The grammar for a replacement field is as follows:

```
replacement_field: "{" [field_name] ["!" conversion] [":" format_spec] "}"
field_name:       arg_name ("." attribute_name | "[" element_index "]")*
arg_name:         [identifier | digit+]
attribute_name:   identifier
element_index:    digit+ | index_string
index_string:     <any source character except "]"> +
conversion:      "r" | "s" | "a"
format_spec:      format-spec:format_spec
```

In less formal terms, the replacement field can start with a `field_name` that specifies the object whose value is to be formatted and inserted into the output instead of the replacement field. The `field_name` is optionally followed by a `conversion` field, which is preceded by an exclamation point `!`, and a `format_spec`, which is preceded by a colon `:`. These specify a non-default format for the replacement value.

See also the `Format` specification mini-language section.

The `field_name` itself begins with an `arg_name` that is either a number or a keyword. If it's a number, it refers to a positional argument, and if it's a keyword, it refers to a named keyword argument. An `arg_name` is treated as a number if a call to `str.isdecimal()` on the string would return `true`. If the numerical `arg_names` in a format string are 0, 1, 2, ... in sequence, they can all be omitted (not just some) and the numbers 0, 1, 2, ... will be automatically inserted in that order. Because `arg_name` is not quote-delimited, it is not possible to specify arbitrary dictionary keys (e.g., the strings `'10'` or `':'`) within a format string. The `arg_name` can be followed by any number of index or attribute expressions. An expression of the form `'name'` selects the named attribute using `getattr()`, while an expression of the form `'[index]'` does an index lookup using `__getitem__()`.

Changed in version 3.1: The positional argument specifiers can be omitted for `str.format()`, so `'{}'{}.format(a,b)` is equivalent to `'{0}{1}'.format(a,b)`.

Changed in version 3.4: The positional argument specifiers can be omitted for `Formatter`.

Some simple format string examples:

```
"First, thou shalt count to {0}" # References first positional argument
"Bring me a {}"                 # Implicitly references the first positional argument
"From {} to {}".format(0,1)     # Same as "From {0} to {1}"
"My quest is {name}"            # References keyword argument 'name'
"Weight in tons {0.weight}"      # 'weight' attribute of first positional arg
"Units destroyed: {players[0]}"  # First element of keyword argument 'players'.
```

The `conversion` field causes a type coercion before formatting. Normally, the job of formatting a value is done by the `__format__()` method of the value itself. However, in some cases it is desirable to force a type to be formatted as a string, overriding its own definition of formatting. By converting the value to a string before calling `__format__()`, the normal formatting logic is bypassed.

Three conversion flags are currently supported: `!s` which calls `str()` on the value, `!r` which calls `repr()` and `!a` which calls `ascii()`.

Some examples:

```
"Harold's a clever {0!s}"      # Calls str() on the argument first
"Bring out the holy {name!r}"  # Calls repr() on the argument first
"More {!a}"                   # Calls ascii() on the argument first
```

The `format_specfield` contains a specification of how the value should be presented, including such details as field width, alignment, padding, decimal precision and so on. Each value type can define its own “formatting mini-language” or interpretation of the `format_spec`.

Most built-in types support a common formatting mini-language, which is described in the next section.

A `format_specfield` can also include nested replacement fields within it. These nested replacement fields may contain a field name, conversion flag and format specification, but deeper nesting is not allowed. The replacement fields within the `format_spec` are substituted before the `format_specstring` is interpreted. This allows the formatting of a value to be dynamically specified.

See the `Format` examples section for some examples.

Format specification mini-language

“Format specifications” are used within replacement fields contained within a format string to define how individual values are presented (see `Format` string syntax, `f-strings`, and `t-strings`). They can also be passed directly to the built-in `format()` function. Each formattable type may define how the format specification is to be interpreted.

Most built-in types implement the following options for format specifications, although some of the formatting options are only supported by the numeric types.

A general convention is that an empty format specification produces the same result as if you had called `str()` on the value. A non-empty format specification typically modifies the result.

The general form of a standard format specifier is:

```
format_spec:      [options][width_and_precision][type]
options:          [[fill]align][sign]["z"]["#"]["0"]
fill:             <any character>
align:            "<" | ">" | "=" | "^"
sign:             "+" | "-" | " "
width_and_precision: [width_with_grouping][precision_with_grouping]
width_with_grouping: [width][grouping]
precision_with_grouping: "." [precision][grouping] | "." grouping
width:            digit+
precision:         digit+
grouping:          ",", "_" | ""
type:             "b" | "c" | "d" | "e" | "E" | "f" | "F" | "g"
                  | "G" | "n" | "o" | "s" | "x" | "X" | "%"
```

If a valid `align` value is specified, it can be preceded by a `fill` character that can be any character and defaults to a space if omitted. It is not possible to use a literal curly brace (“{” or “}”) as the `fill` character in a formatted string literal or when using the `str.format()` method. However, it is possible to insert a curly brace with a nested replacement field. This limitation doesn’t affect the `format()` function.

The meaning of the various alignment options is as follows:

Option | Meaning | '<' | Forces the field to be left-aligned within the available space (this is the default for most objects). | '>' | Forces the field to be right-aligned within the available space (this is the default for numbers). | '=' | Forces the padding to be placed after the sign (if any) but before the digits. This is used for printing fields in the form ‘+000000120’. This alignment option is only valid for numeric types, excluding | complex | . It becomes the default for numbers when ‘0’ immediately precedes the field width. | '^' | Forces the field to be centered within the available space.

Note that unless a minimum field width is defined, the field width will always be the same size as the data to fill it, so that the alignment option has no meaning in this case.

The `signoption` is only valid for number types, and can be one of the following:

Option	Meaning
'+'	Indicates that a sign should be used for both positive as well as negative numbers.
'-'	Indicates that a sign should be used only for negative numbers (this is the default behavior).
space	Indicates that a leading space should be used on positive numbers, and a minus sign on negative numbers.

The `z` option coerces negative zero floating-point values to positive zero after rounding to the format precision. This option is only valid for floating-point presentation types.

Changed in version 3.11: Added the `z` option (see also PEP 682).

The `#` option causes the “alternate form” to be used for the conversion. The alternate form is defined differently for different types. This option is only valid for integer, float and complex types. For integers, when binary, octal, or hexadecimal output is used, this option adds the respective prefix `0b`, `0o`, `0x`, or `0X` to the output value. For float and complex the alternate form causes the result of the conversion to always contain a decimal-point character, even if no digits follow it. Normally, a decimal-point character appears in the result of these conversions only if a digit follows it. In addition, for `g` and `G` conversions, trailing zeros are not removed from the result.

The `width` is a decimal integer defining the minimum total field width, including any prefixes, separators, and other formatting characters. If not specified, then the field width will be determined by the content.

When no explicit alignment is given, preceding the `width` field by a zero (`'0'`) character enables sign-aware zero-padding for numeric types, excluding complex. This is equivalent to a fill character of `'0'` with an alignment type of `'>'`.

Changed in version 3.10: Preceding the `width` field by `'0'` no longer affects the default alignment for strings.

The `precision` is a decimal integer indicating how many digits should be displayed after the decimal point for presentation types `'f'` and `'F'`, or before and after the decimal point for presentation types `'g'` or `'G'`. For string presentation types the field indicates the maximum field size - in other words, how many characters will be used from the field content. The `precision` is not allowed for integer presentation types.

The `grouping` option after `width` and `precision` fields specifies a digit group separator for the integral and fractional parts of a number respectively. It can be one of the following:

Option	Meaning
','	Inserts a comma every 3 digits for integer presentation type <code>'d'</code> and floating-point presentation types, excluding <code>'n'</code> . For other presentation types, this option is not supported.
'_'	Inserts an underscore every 3 digits for integer presentation type <code>'d'</code> and floating-point presentation types, excluding <code>'n'</code> . For integer presentation types <code>'b'</code> , <code>'o'</code> , <code>'x'</code> , and <code>'X'</code> , underscores are inserted every 4 digits. For other presentation types, this option is not supported.

For a locale aware separator, use the `'n'` presentation type instead.

Changed in version 3.1: Added the `'_'` option (see also PEP 378).

Changed in version 3.6: Added the `'_'` option (see also PEP 515).

Changed in version 3.14: Support the `grouping` option for the fractional part.

Finally, the `type` determines how the data should be presented.

The available string presentation types are:

Type	Meaning
's'	String format. This is the default type for strings and may be omitted.
None	The same as <code>'s'</code> .

The available integer presentation types are:

Type	Meaning
'b'	Binary format. Outputs the number in base 2.
'c'	Character. Converts the integer to the corresponding unicode character before printing.
'd'	Decimal Integer. Outputs the number in base 10.
'o'	Octal format. Outputs the number in base 8.
'x'	Hex format. Outputs the number in base 16, using lower-case letters for the digits above 9.
'X'	Hex format. Outputs the number in base 16, using upper-case letters for the digits above 9.

Hex format. Outputs the number in base 16, using upper-case letters for the digits above 9. In case | '#' | is specified, the prefix | '0x' | will be upper-cased to | '0X' | as well. | 'n' | Number. This is the same as | 'd' | , except that it uses the current locale setting to insert the appropriate digit group separators. | None | The same as | 'd' | .

In addition to the above presentation types, integers can be formatted with the floating-point presentation types listed below (except 'n' and None). When doing so, float() is used to convert the integer to a floating-point number before formatting.

The available presentation types for float and Decimal values are:

Type | Meaning | 'e' | Scientific notation. For a given precision | p | , formats the number in scientific notation with the letter 'e' separating the coefficient from the exponent. The coefficient has one digit before and | p | digits after the decimal point, for a total of | p | + | 1 | significant digits. With no precision given, uses a precision of | 6 | digits after the decimal point for | float | , and shows all coefficient digits for | Decimal | . If | p=0 | , the decimal point is omitted unless the | # | option is used. | For | float | , the exponent always contains at least two digits, and is zero if the value is zero. | 'E' | Scientific notation. Same as | 'e' | except it uses an upper case 'E' as the separator character. | 'f' | Fixed-point notation. For a given precision | p |

The result should be correctly rounded to a given precision of digits after the decimal point. The rounding mode for float matches that of the round() builtin. For Decimal, the rounding mode of the current context will be used.

The available presentation types for complex are the same as those for float ('%' is not allowed). Both the real and imaginary components of a complex number are formatted as floating-point numbers, according to the specified presentation type. They are separated by the mandatory sign of the imaginary part, the latter being terminated by a j suffix. If the presentation type is missing, the result will match the output of str() (complex numbers with a non-zero real part are also surrounded by parentheses), possibly altered by other format modifiers.

Format examples

This section contains examples of the str.format() syntax and comparison with the old %-formatting.

In most of the cases the syntax is similar to the old %-formatting, with the addition of the {} and with: used instead of %. For example, '%03.2f' can be translated to '{:03.2f}'.

The new format syntax also supports new and different options, shown in the following examples.

Accessing arguments by position:

```
>>> '{0}, {1}, {2}'.format('a', 'b', 'c')
'a, b, c'
>>> '{}, {}, {}'.format('a', 'b', 'c') # 3.1+ only
'a, b, c'
>>> '{2}, {1}, {0}'.format('a', 'b', 'c')
'c, b, a'
>>> '{2}, {1}, {0}'.format(*'abc')      # unpacking argument sequence
'c, b, a'
>>> '{0}{1}{0}'.format('abra', 'cad')  # arguments' indices can be repeated
'abracadabra'
```

Accessing arguments by name:

```
>>> 'Coordinates: {latitude}, {longitude}'.format(latitude='37.24N', longitude='-115.81W')
'Coordinates: 37.24N, -115.81W'
>>> coord = {'latitude': '37.24N', 'longitude': '-115.81W'}
>>> 'Coordinates: {latitude}, {longitude}'.format(**coord)
'Coordinates: 37.24N, -115.81W'
```

Accessing arguments' attributes:

```
>>> c = 3-5j
>>> ('The complex number {0} is formed from the real part {0.real} '
... 'and the imaginary part {0.imag}.').format(c)
```

```
'The complex number (3-5j) is formed from the real part 3.0 and the imaginary part -5.0.'
>>> class Point:
...     def __init__(self, x, y):
...         self.x, self.y = x, y
...     def __str__(self):
...         return 'Point({self.x}, {self.y})'.format(self=self)
...
>>> str(Point(4, 2))
'Point(4, 2)'
```

Accessing arguments' items:

```
>>> coord = (3, 5)
>>> 'X: {0[0]}; Y: {0[1]}'.format(coord)
'X: 3; Y: 5'
```

Replacing%sand%r:

```
>>> "repr() shows quotes: {!r}; str() doesn't: {!s}".format('test1', 'test2')
"repr() shows quotes: 'test1'; str() doesn't: test2"
```

Aligning the text and specifying a width:

```
>>> '{:<30}'.format('left aligned')
'left aligned'
>>> '{:>30}'.format('right aligned')
'right aligned'
>>> '{:^30}'.format('centered')
'centered'
>>> '{:*^30}'.format('centered') # use '*' as a fill char
'*****centered*****'
```

Replacing%+f,%-f, and%fand specifying a sign:

```
>>> '{:+f}; {:+f}'.format(3.14, -3.14) # show it always
'+3.140000; -3.140000'
>>> '{: f}; {: f}'.format(3.14, -3.14) # show a space for positive numbers
' 3.140000; -3.140000'
>>> '{:-f}; {: -f}'.format(3.14, -3.14) # show only the minus -- same as '{:f}; {:f}'
'3.140000; -3.140000'
```

Replacing%xand%oand converting the value to different bases:

```
>>> # format also supports binary numbers
>>> "int: {0:d}; hex: {0:x}; oct: {0:o}; bin: {0:b}".format(42)
'int: 42; hex: 2a; oct: 52; bin: 101010'
>>> # with 0x, 0o, or 0b as prefix:
>>> "int: {0:d}; hex: {0:#x}; oct: {0:#o}; bin: {0:#b}".format(42)
'int: 42; hex: 0x2a; oct: 0o52; bin: 0b101010'
```

Using the comma or the underscore as a digit group separator:

```
>>> '{:,}'.format(1234567890)
'1,234,567,890'
>>> '{:_}'.format(1234567890)
'1_234_567_890'
>>> '{:_b}'.format(1234567890)
'100_1001_1001_0110_0000_0010_1101_0010'
>>> '{:_x}'.format(1234567890)
'4996_02d2'
>>> '{:_}'.format(123456789.123456789)
'123_456_789.12345679'
>>> '{:.,}'.format(123456789.123456789)
'123456789.123,456,79'
>>> '{:,. _}'.format(123456789.123456789)
'123,456,789.123_456_79'
```

Expressing a percentage:

```
>>> points = 19
>>> total = 22
>>> 'Correct answers: {:.2%}'.format(points/total)
'Correct answers: 86.36%'
```

Using type-specific formatting:

```
>>> import datetime as dt
>>> d = dt.datetime(2010, 7, 4, 12, 15, 58)
>>> '{:%Y-%m-%d %H:%M:%S}'.format(d)
'2010-07-04 12:15:58'
```

Nesting arguments and more complex examples:

```
>>> for align, text in zip('<^>', ['left', 'center', 'right']):
...     '{0:{fill}{align}16}'.format(text, fill=align, align=align)
...
'left<<<<<<<<<<<<'
'^^^^^center^^^^^'
'>>>>>>>>>>>>right'
>>>
>>> octets = [192, 168, 0, 1]
>>> '{:02X}{:02X}{:02X}{:02X}'.format(*octets)
'C0A80001'
>>> int(_, 16)
3232235521
>>>
>>> width = 5
>>> for num in range(5,12):
...     for base in 'dXob':
...         print('{0:{width}{base}}'.format(num, base=base, width=width), end=' ')
...         print()
...
5      5      5    101
6      6      6    110
7      7      7    111
8      8     10   1000
9      9     11   1001
10     A     12   1010
11     B     13   1011
```

Template strings (\$-strings)

■ **Nota:** NoteThe feature described here was introduced in Python 2.4; a simple templating method based upon regular expressions. It predates`str.format()`,formatted string literals, andtemplate string literals.It is unrelated to template string literals (t-strings), which were introduced in Python 3.14. These evaluate to`string.Template`objects, found in the`string` module.

Template strings provide simpler string substitutions as described inPEP 292. A primary use case for template strings is for internationalization (i18n) since in that context, the simpler syntax and functionality makes it easier to translate than other built-in string formatting facilities in Python. As an example of a library built on template strings for i18n, see the`fluff.i18n`package.

Template strings support\$-based substitutions, using the following rules:

- `$$` is an escape; it is replaced with a single\$.
- `$identifier` names a substitution placeholder matching a mapping key of "identifier". By default, "identifier" is restricted to any case-insensitive ASCII alphanumeric string (including underscores) that starts with an underscore or ASCII letter. The first non-identifier character after the\$character terminates this placeholder specification.

- `{identifier}` is equivalent to `$identifier`. It is required when valid identifier characters follow the placeholder but are not part of the placeholder, such as `{noun}ification`.

Any other appearance of `$` in the string will result in a `ValueError` being raised.

The `string` module provides a `Template` class that implements these rules. The methods of `Template` are:

`string.Template(template)`

`substitute(mapping={}, **kwds)`

`safe_substitute(mapping={}, **kwds)`

`is_valid()`

`get_identifiers()`

`template`

The constructor takes a single argument which is the template string. `substitute(mapping={}, **kwds)` Performs the template substitution, returning a new string. `mapping` is any dictionary-like object with keys that match the placeholders in the template. Alternatively, you can provide keyword arguments, where the keywords are the placeholders. When both `mapping` and `kwds` are given and there are duplicates, the placeholders from `kwds` take precedence. `safe_substitute(mapping={}, **kwds)` Likes `substitute()`, except

Performs the template substitution, returning a new string. `mapping` is any dictionary-like object with keys that match the placeholders in the template. Alternatively, you can provide keyword arguments, where the keywords are the placeholders. When both `mapping` and `kwds` are given and there are duplicates, the placeholders from `kwds` take precedence.

Likes `substitute()`, except that if placeholders are missing from `mapping` and `kwds`, instead of raising a `KeyError` exception, the original placeholder will appear in the resulting string intact. Also, unlike with `substitute()`, any other appearances of the `$` will simply return `$` instead of raising `ValueError`. While other exceptions may still occur, this method is called “safe” because it always tries to return a usable string instead of raising an exception. In another sense, `safe_substitute()` may be anything other

Returns `False` if the template has invalid placeholders that will cause `substitute()` to raise `ValueError`. Added in version 3.11.

Returns a list of the valid identifiers in the template, in the order they first appear, ignoring any invalid identifiers. Added in version 3.11.

This is the object passed to the constructor's `template` argument. In general, you shouldn't change it, but read-only access is not enforced.

Here is an example of how to use a `Template`:

```
>>> from string import Template
>>> s = Template('$who likes $what')
>>> s.substitute(who='tim', what='kung pao')
'tim likes kung pao'
>>> d = dict(who='tim')
>>> Template('Give $who $100').substitute(d)
Traceback (most recent call last):
...
ValueError: Invalid placeholder in string: line 1, col 11
>>> Template('$who likes $what').substitute(d)
Traceback (most recent call last):
...
KeyError: 'what'
>>> Template('$who likes $what').safe_substitute(d)
'tim likes $what'
```

Advanced usage: you can derive subclasses of `Template` to customize the placeholder syntax, delimiter character, or the entire regular expression used to parse template strings. To do this, you can override these class attributes:

- `delimiter`– This is the literal string describing a placeholder introducing delimiter. The default value is `$`. Note that this should not be a regular expression, as the implementation will call `re.escape()` on this string as needed. Note further that you cannot change the delimiter after class creation (i.e. a different delimiter must be set in the subclass's class namespace).
- `idpattern`– This is the regular expression describing the pattern for non-braced placeholders. The default value is the regular expression `(?a:[_a-z][_a-z0-9]*)`. If this is given and `braceidpattern` is `None` this pattern will also apply to braced placeholders. Note Since default `flags` is `re.IGNORECASE`, `pattern[a-z]` can match with some non-ASCII characters. That's why we use the `localaflag` here. Changed in version 3.7.
- `braceidpattern`– This is like `idpattern` but describes the pattern for braced placeholders. Defaults to `None` which means to fall back to `idpattern` (i.e. the same pattern is used both inside and outside braces). If given, this allows you to define different patterns for braced and unbraced placeholders. Added in version 3.7.
- `flags`– The regular expression flags that will be applied when compiling the regular expression used for recognizing substitutions. The default value is `re.IGNORECASE`. Note that `re.VERBOSE` will always be added to the flags, so custom `idpatterns` must follow conventions for verbose regular expressions. Added in version 3.2.

Alternatively, you can provide the entire regular expression pattern by overriding the class attribute `pattern`. If you do this, the value must be a regular expression object with four named capturing groups. The capturing groups correspond to the rules given above, along with the invalid placeholder rule:

- `escaped`– This group matches the escape sequence, e.g. `$$`, in the default pattern.
- `named`– This group matches the unbraced placeholder name; it should not include the delimiter in capturing group.
- `braced`– This group matches the brace enclosed placeholder name; it should not include either the delimiter or braces in the capturing group.
- `invalid`– This group matches any other delimiter pattern (usually a single delimiter), and it should appear last in the regular expression.

The methods on this class will raise `ValueError` if the pattern matches the template without one of these named groups matching.

Helper functions

`string.capwords(s, sep=None)`

Split the argument into words using `str.split()`, capitalize each word using `str.capitalize()`, and join the capitalized words using `str.join()`. If the optional second argument `sep` is absent or `None`, runs of whitespace characters are replaced by a single space and leading and trailing whitespace are removed, otherwise `sep` is used to split and join the words.

string.templatelib— Support for template string literals

Source code: `Lib/string/templatelib.py`

See also

- Format strings
- Template string literal (t-string) syntax
- PEP 750

Template strings

Added in version 3.14.

Template strings are a mechanism for custom string processing. They have the full flexibility of Python's f-strings, but return a `Template` instance that gives access to the static and interpolated (in curly brackets) parts of a string before they are combined.

To write a t-string, use a `t` prefix instead of an `f`, like so:

```
>>> pi = 3.14
>>> t't-strings are new in Python {pi!s}!'
Template(
  strings=('t-strings are new in Python ', '!'),
  interpolations=(Interpolation(3.14, 'pi', 's', '')),
)
```

Types

class `string.templatelib.Template`

strings: `tuple[str,...]`

interpolations: `tuple[Interpolation,...]`

values: `tuple[object,...]`

__new__ `(*args:str|Interpolation)`

iter `(template)`

template+other

template+=other

The `Template` class describes the contents of a template string. It is immutable, meaning that attributes of a template cannot be reassigned. The most common way to create a `Template` instance is to use the template string literal syntax. This syntax is identical to that of f-strings, except that it uses a `t` prefix in place of an `f`:
`>>> cheese = 'Red Leicester' >>> template = t'We're fresh out of {cheese}, sir.' >>> type(template) <class 'string.templatelib.Template'>`
Templates are stored as sequences of literal strings.

A tuple of the static strings in the template.
`>>> cheese = 'Camembert' >>> template = t'Ah! We do have {cheese}.' >>> template.strings('Ah! We do have ', '.')`
Empty strings are included in the tuple.
`>>> response = 'We do have' >>> cheese = 'Camembert' >>> template = t'Ah! {response} {cheese}.' >>> template.strings('Ah! ', ', ', '.')`
The string tuple is never empty, and always contains one more string than the interpolations and value tuples.
`>>> t".strings(",)>>> t".values()>>> t'{cheese}'.strings(",`

```
"")>>>t['cheese'].values('
```

A tuple of the interpolations in the template.>>>cheese='Camembert'>>>template=t'Ah! We do have{cheese}.'>>>template.interpolations(Interpolation('Camembert', 'cheese', None, ""))The interpolation tuple may be empty and always contains one fewer values than the string tuple:>>>t'Red Leicester'.interpolations()

A tuple of all interpolated values in the template.>>>cheese='Camembert'>>>template=t'Ah! We do have{cheese}.'>>>template.values('Camembert')The value tuple always has the same length as the interpolation tuple. It is always equivalent to tuple(i.value for i in template.interpolations).

While literal syntax is the most common way to create a Template, it is also possible to create them directly using the constructor:>>>from string import Interpolation, Template>>>cheese='Camembert'>>>template=Template(...'Ah! We do have ', Interpolation(cheese, 'cheese'), '...')>>>list(template)['Ah! We do have ', Interpolation('Camembert', 'cheese', None, ""), '.']If multiple strings are passed consecutively, they will be concatenated into a single value in the strings attribute. For example

Iterate over the template, yielding each non-empty string and Interpolation in the correct order:>>>cheese='Camembert'>>>list(t'Ah! We do have{cheese}.')['Ah! We do have ', Interpolation('Camembert', 'cheese', None, ""), '.']Caution Empty strings are not included in the iteration:>>>response='We do have '>>>cheese='Camembert'>>>list(t'Ah!{response}{cheese}.')['Ah! ', Interpolation('We do have ', 'response', None, ""), Interpolation('Camembert', 'cheese', None, ""), '.']

Concatenate this template with another, returning a new Template instance:>>>cheese='Camembert'>>>list(t'Ah! '+t'We do have{cheese}.')['Ah! We do have ', Interpolation('Camembert', 'cheese', None, ""), '.']Concatenating a Template and a string is not supported. This is because it is unclear whether the string should be treated as a static string or an interpolation. If you want to concatenate a Template with a string, you should either wrap the string directly in a Template (to treat it as a static string) or

class string.templatelib.Interpolation

value: object

expression: str

conversion: Literal['a', 'r', 's'] | None

format_spec: str

__new__ (value: object, expression: str, conversion: Literal['a', 'r', 's'] | None = None, format_spec: str = "")

Parameters:

The Interpolation type represents an expression inside a template string. It is immutable, meaning that attributes of an interpolation cannot be reassigned. Interpolations support pattern matching, allowing you to match against their attributes with the match statement:>>>from string import Interpolation>>>interpolation=t'{1.+2...2f}'.interpolations[0]>>>interpolationInterpolation(3.0, '1. + 2.', None, '.2f')>>>match interpolation:...case Interpolation(value, expression, conversion, format_spec)

The evaluated value of the interpolation.>>>t'{1+2}'.interpolations[0].value3

For interpolations created by t-string literals, expression is the expression text found inside the curly brackets ({&}), including any whitespace, excluding the curly brackets themselves, and ending before the first !, :, or = if any is present. For manually created interpolations, expression is the arbitrary string provided when constructing the interpolation instance. We recommend using valid Python expressions or the empty string for the expression field of manually created Interpolation instances, although

The conversion to apply to the value, or None. The conversion is the optional conversion to apply to the value:>>>t'{1+2!a}'.interpolations[0].conversion'a'Note Unlike f-strings, where conversions are applied automatically, the expected behavior with t-strings is that code that processes the Template will decide how to

interpret and whether to apply the conversion. For convenience, the `convert()` function can be used to mimic f-string conversion semantics.

The format specification to apply to the value. The `format_spec` is an optional, arbitrary string used as the format specification to present the value: `>>> t'{1+2:.2f}'.interpolations[0].format_spec'.2f'` Note Unlike f-strings, where format specifications are applied automatically via the `format()` protocol, the expected behavior with t-strings is that code that processes the interpolation will decide how to interpret and whether to apply the format specification. As a result, `format_spec` values in interpolation

Create a new `Interpolation` object from component parts. Parameters: `value`— The evaluated, in-scope result of the interpolation. `expression`— The text of a valid Python expression, or an empty string. `conversion`— The conversion to be used, one of `None`, `'a'`, `'r'`, or `'s'`. `format_spec`— An optional, arbitrary string used as the format specification to present the value.

`value`— The evaluated, in-scope result of the interpolation. `expression`— The text of a valid Python expression, or an empty string. `conversion`— The conversion to be used, one of `None`, `'a'`, `'r'`, or `'s'`. `format_spec`— An optional, arbitrary string used as the format specification to present the value.

Helper functions

`string.TemplateLib.convert(obj, l, conversion)`

Applies formatted string literal conversion semantics to the given object `obj`. This is frequently useful for custom template string processing logic. Three conversion flags are currently supported: `'s'` which calls `str()` on the value (like `!s`), `'r'` which calls `repr()` (like `!r`), and `'a'` which calls `ascii()` (like `!a`). If the conversion flag is `None`, `obj` is returned unchanged.

re— Regular expression operations

Source code: [Lib/re/](#)

This module provides regular expression matching operations similar to those found in Perl.

Both patterns and strings to be searched can be Unicode strings (str) as well as 8-bit strings (bytes). However, Unicode strings and 8-bit strings cannot be mixed: that is, you cannot match a Unicode string with a bytes pattern or vice-versa; similarly, when asking for a substitution, the replacement string must be of the same type as both the pattern and the search string.

Regular expressions use the backslash character ('\') to indicate special forms or to allow special characters to be used without invoking their special meaning. This collides with Python's usage of the same character for the same purpose in string literals; for example, to match a literal backslash, one might have to write '\\\\' as the pattern string, because the regular expression must be \\, and each backslash must be expressed as \\ inside a regular Python string literal. Also, please note that any invalid escape sequences in Python's usage of the backslash in string literals now generate a `SyntaxWarning` and in the future this will become a `SyntaxError`. This behaviour will happen even if it is a valid escape sequence for a regular expression.

The solution is to use Python's raw string notation for regular expression patterns; backslashes are not handled in any special way in a string literal prefixed with 'r'. So `"\\n"` is a two-character string containing backslash and 'n', while `"\n"` is a one-character string containing a newline. Usually patterns will be expressed in Python code using this raw string notation.

It is important to note that most regular expression operations are available as module-level functions and methods on compiled regular expressions. The functions are shortcuts that don't require you to compile a regex object first, but miss some fine-tuning parameters.

See also

The third-party `regex` module, which has an API compatible with the standard library `re` module, but offers additional functionality and a more thorough Unicode support.

Regular Expression Syntax

A regular expression (or RE) specifies a set of strings that matches it; the functions in this module let you check if a particular string matches a given regular expression (or if a given regular expression matches a particular string, which comes down to the same thing).

Regular expressions can be concatenated to form new regular expressions; if `A` and `B` are both regular expressions, then `AB` is also a regular expression. In general, if a string `p` matches `A` and another string `q` matches `B`, the string `pq` will match `AB`. This holds unless `A` or `B` contain low precedence operations; boundary conditions between `A` and `B`; or have numbered group references. Thus, complex expressions can easily be constructed from simpler primitive expressions like the ones described here. For details of the theory and implementation of regular expressions, consult the Friedl book [Frie09], or almost any textbook about compiler construction.

A brief explanation of the format of regular expressions follows. For further information and a gentler presentation, consult the [Regular Expression HOWTO](#).

Regular expressions can contain both special and ordinary characters. Most ordinary characters, like `'A'`, `'a'`, or `'0'`, are the simplest regular expressions; they simply match themselves. You can concatenate ordinary characters, so `last` matches the string `'last'`. (In the rest of this section, we'll write RE's in this special style, usually without quotes, and strings to be matched in single quotes.)

Some characters, like '|' or '(', are special. Special characters either stand for classes of ordinary characters, or affect how the regular expressions around them are interpreted.

Repetition operators or quantifiers (*, +, ?, {m,n}, etc) cannot be directly nested. This avoids ambiguity with the non-greedy modifier suffix ?, and with other modifiers in other implementations. To apply a second repetition to an inner repetition, parentheses may be used. For example, the expression (?:a{6})* matches any multiple of six 'a' characters.

The special characters are:

.

(Dot.) In the default mode, this matches any character except a newline. If the DOTALL flag has been specified, this matches any character including a newline. (?s:) matches any character regardless of flags.

^

(Caret.) Matches the start of the string, and in MULTILINE mode also matches immediately after each newline.

\$

Matches the end of the string or just before the newline at the end of the string, and in MULTILINE mode also matches before a newline. foo matches both 'foo' and 'foobar', while the regular expression foo\$ matches only 'foo'. More interestingly, searching for foo.\$ in 'foo1\nfoo2\n' matches 'foo2' normally, but 'foo1' in MULTILINE mode; searching for a single \$ in 'foo\n' will find two (empty) matches: one just before the newline, and one at the end of the string.

*

Causes the resulting RE to match 0 or more repetitions of the preceding RE, as many repetitions as are possible. ab* will match 'a', 'ab', or 'a' followed by any number of 'b's.

+

Causes the resulting RE to match 1 or more repetitions of the preceding RE. ab+ will match 'a' followed by any non-zero number of 'b's; it will not match just 'a'.

?

Causes the resulting RE to match 0 or 1 repetitions of the preceding RE. ab? will match either 'a' or 'ab'.

*?, +?, ??

The '*', '+', and '?' quantifiers are all greedy; they match as much text as possible. Sometimes this behaviour isn't desired; if the RE <.*> is matched against '<a>b<c>', it will match the entire string, and not just '<a>'. Adding ? after the quantifier makes it perform the match in non-greedy or minimal fashion; as few characters as possible will be matched. Using the RE <.*?> will match only '<a>'.

*+, ++, ?+

Like the '*', '+', and '?' quantifiers, those where '+' is appended also match as many times as possible. However, unlike the true greedy quantifiers, these do not allow back-tracking when the expression following it fails to match. These are known as possessive quantifiers. For example, a*+ will match 'aaaa' because the a* will match all 4 'a's, but, when the final 'a' is encountered, the expression is backtracked so that in the end the a* ends up matching 3 'a's total, and the fourth 'a' is matched by the final 'a'.

{m}

{m,n}

{m,n}?

{m,n}+

Specifies that exactly *n* copies of the previous RE should be matched; fewer matches cause the entire RE not to match. For example, `a{6}` will match exactly six 'a' characters, but not five.

Causes the resulting RE to match from *m* to *n* repetitions of the preceding RE, attempting to match as many repetitions as possible. For example, `a{3,5}` will match from 3 to 5 'a' characters. Omitting *m* specifies a lower bound of zero, and omitting *n* specifies an infinite upper bound. As an example, `a{4,}` will match 'aaaab' or a thousand 'a' characters followed by a 'b', but not 'aaab'. The comma may not be omitted or the modifier would be confused with the previously described form.

Causes the resulting RE to match from *m* to *n* repetitions of the preceding RE, attempting to match as few repetitions as possible. This is the non-greedy version of the previous quantifier. For example, on the 6-character string 'aaaaaa', `a{3,5}` will match 5 'a' characters, while `a{3,5}?` will only match 3 characters.

Causes the resulting RE to match from *m* to *n* repetitions of the preceding RE, attempting to match as many repetitions as possible without establishing any backtracking points. This is the possessive version of the quantifier above. For example, on the 6-character string 'aaaaaa', `a{3,5}+aa` attempt to match 5 'a' characters, then, requiring 2 more 'a's, will need more characters than available and thus fail, while `a{3,5}a` will match with `a{3,5}` capturing 5, then 4 'a's by backtracking and then the final 2 'a's are

Either escapes special characters (permitting you to match characters like '*', '?', and so forth), or signals a special sequence; special sequences are discussed below. If you're not using a raw string to express the pattern, remember that Python also uses the backslash as an escape sequence in string literals; if the escape sequence isn't recognized by Python's parser, the backslash and subsequent character are included in the resulting string. However, if Python would recognize the resulting sequence

[]

Used to indicate a set of characters. In a set, characters can be listed individually, e.g. `[amk]` will match 'a', 'm', or 'k'. Ranges of characters can be indicated by giving two characters and separating them by a hyphen, for example `[a-z]` will match any lowercase ASCII letter, `[0-5][0-9]` will match all the two-digit numbers from 00 to 59, and `[0-9A-Fa-f]` will match any hexadecimal digit. If a hyphen is escaped (e.g. `[a\ -z]`) or if it's placed as the first or last character (e.g. `[-a]` or `[a-]`), it will match a literal hyphen. Special

|

`A|B`, where *A* and *B* can be arbitrary REs, creates a regular expression that will match either *A* or *B*. An arbitrary number of REs can be separated by the pipe in this way. This can be used inside groups (see below) as well. As the target string is scanned, REs separated by pipes are tried from left to right. When one pattern completely matches, that branch is accepted. This means that once *A* matches, *B* will not be tested further, even if it would produce a longer overall match. In other words, the pipe operator is

(...)

Matches whatever regular expression is inside the parentheses, and indicates the start and end of a group; the contents of a group can be retrieved after a match has been performed, and can be matched later in the string with the backreference special sequence, described below. To match the literals '(' or ')', use `\(` or `\)`, or enclose them inside a character class: `[(,)]`.

(?...)

(?aiLmsux)

This is an extension notation (`a'?'` following `a'` is not meaningful otherwise). The first character after the `'?'` determines what the meaning and further syntax of the construct is. Extensions usually do not create a new group; `(?P<name>...)` is the only exception to this rule. Following are the currently supported extensions.

(One or more letters from the set 'a', 'i', 'l', 'm', 's', 'u', 'x'.) The group matches the empty string; the letters set the corresponding flags for the entire regular expression: `re.A` (ASCII-only matching) `re.I` (ignore case) `re.L` (locale dependent) `re.M` (multi-line) `re.S` (dot matches all) `re.U` (Unicode matching) `re.X` (verbose) (The flags are described in `Module Contents`.) This is useful if you wish to include the flags as part of the regular expression, instead of passing a flag argument to `re.compile()` function. `re.I`

(?:...)

(?aiLmsux-imsx:...)

(?>...)

A non-capturing version of regular parentheses. Matches whatever regular expression is inside the parentheses, but the substring matched by the group cannot be retrieved after performing a match or referenced later in the pattern.

(Zero or more letters from the set 'a', 'i', 'l', 'm', 's', 'u', 'x', optionally followed by '-' followed by one or more letters from the set 'a', 'i', 'l', 'm', 's', 'u', 'x'.) The letters set or remove the corresponding flags for the part of the expression: `re.A` (ASCII-only matching) `re.I` (ignore case) `re.L` (locale dependent) `re.M` (multi-line) `re.S` (dot matches all) `re.U` (Unicode matching) `re.X` (verbose) (The flags are described in `Module Contents`.) The letters 'a', 'l' and 'u' are mutually exclusive when used as inline flags, so they can't be co

Attempts to match... as if it was a separate regular expression, and if successful, continues to match the rest of the pattern following it. If the subsequent pattern fails to match, the stack can only be unwound to a point before the `(?>...)` because once exited, the expression, known as an atomic group, has thrown away all stack points within itself. Thus, `(?>.*).` would never match anything because first the `*` would match all characters possible, then, having nothing left to match, the final `.` would fail

(?P<name>...)

Similar to regular parentheses, but the substring matched by the group is accessible via the symbolic group name `name`. Group names must be valid Python identifiers, and in bytes patterns they can only contain bytes in the ASCII range. Each group name must be defined only once within a regular expression. A symbolic group is also a numbered group, just as if the group were not named. Named groups can be referenced in three contexts. If the pattern is `(?P<quote>["])*?(?P=quote)` (i.e. matching a st

(?P=name)

A backreference to a named group; it matches whatever text was matched by the earlier group named `name`.

(?#...)

A comment; the contents of the parentheses are simply ignored.

(?=...)

Matches if... matches next, but doesn't consume any of the string. This is called a lookahead assertion. For example, `Isaac(?=Asimov)` will match 'Isaac' only if it's followed by 'Asimov'.

(?!...)

Matches if... doesn't match next. This is a negative lookahead assertion. For example, `Isaac(?!Asimov)` will match 'Isaac' only if it's not followed by 'Asimov'.

(?<=...)

Matches if the current position in the string is preceded by a match for... that ends at the current position. This is called a positive lookbehind assertion. `(?<=abc)def` will find a match in 'abcdef', since the lookbehind will back up 3 characters and check if the contained pattern matches. The contained pattern must only match strings of some fixed length, meaning that `abc|ora|b` are allowed, but `a*` and `a{3,4}` are not. Note that patterns which start with positive lookbehind assertions will not match at t

(?<!...)

Matches if the current position in the string is not preceded by a match for.... This is called a negative lookbehind assertion. Similar to positive lookbehind assertions, the contained pattern must only match strings of some fixed length. Patterns which start with negative lookbehind assertions may match at the beginning of the string being searched.

(?(id/name)yes-pattern|no-pattern)

Will try to match with yes-pattern if the group with given id or name exists, and with no-pattern if it doesn't. no-pattern is optional and can be omitted. For example, `(<)?(\w+@\w+(?:\.\w+)+)(?(1)>|$)` is a poor email matching pattern, which will match with `<user@host.com>` as well as `user@host.com`, but not with `<user@host.com` nor `user@host.com>`. Changed in version 3.12: Group id can only contain ASCII digits. In bytes patterns, group name can only contain bytes in the ASCII range (b'x00'-b'x7f').

The special sequences consist of `\` and a character from the list below. If the ordinary character is not an ASCII digit or an ASCII letter, then the resulting RE will match the second character. For example, `\$` matches the character `$`.

\number

Matches the contents of the group of the same number. Groups are numbered starting from 1. For example, `(.+)\1` matches `thethe` or `'5555'`, but not `thethe` (note the space after the group). This special sequence can only be used to match one of the first 99 groups. If the first digit of number is 0, or number is 3 octal digits long, it will not be interpreted as a group match, but as the character with octal value number. Inside the `'[and']` of a character class, all numeric escapes are treated as characters.

\A

Matches only at the start of the string.

\b

Matches the empty string, but only at the beginning or end of a word. A word is defined as a sequence of word characters. Note that formally, `\b` is defined as the boundary between `a\w` and `a\W` character (or vice versa), or between `\w` and the beginning or end of the string. This means that `r'\bat\b'` matches `'at'`, `'at.'`, `'(at)'`, and `'asatay'` but not `'attempt'` or `'atlas'`. The default word characters in Unicode (str) patterns are Unicode alphanumerics and the underscore, but this can be changed by using the `ASCII` flag.

\B

Matches the empty string, but only when it is not at the beginning or end of a word. This means that `r'\at\B'` matches `'athens'`, `'atom'`, `'attorney'`, but not `'at'`, `'at.'`, or `'at!'`. `\B` is the opposite of `\b`, so word characters in Unicode (str) patterns are Unicode alphanumerics or the underscore, although this can be changed by using the `ASCII` flag. Word boundaries are determined by the current locale if the `LOCALE` flag is used. Changed in version 3.14: `\B` now matches empty input string.

\d

For Unicode (str) patterns:

For 8-bit (bytes) patterns:

For Unicode (str) patterns: Matches any Unicode decimal digit (that is, any character in Unicode character category `[Nd]`). This includes `[0-9]`, and also many other digit characters. Matches `[0-9]` if the `ASCII` flag is used. For 8-bit (bytes) patterns: Matches any decimal digit in the ASCII character set; this is equivalent to `[0-9]`.

Matches any Unicode decimal digit (that is, any character in Unicode character category `[Nd]`). This includes `[0-9]`, and also many other digit characters. Matches `[0-9]` if the `ASCII` flag is used.

Matches any decimal digit in the ASCII character set; this is equivalent to `[0-9]`.

\D

Matches any character which is not a decimal digit. This is the opposite of `\d`. Matches `^[^0-9]` if the `ASCII` flag is used.

\s

For Unicode (str) patterns:

For 8-bit (bytes) patterns:

For Unicode (str) patterns: Matches Unicode whitespace characters (as defined by `str.isspace()`). This includes `[\t\n\r\f\v]`, and also many other characters, for example the non-breaking spaces mandated by typography rules in many languages. Matches `[\t\n\r\f\v]` if the `ASCII` flag is used. For 8-bit (bytes) patterns: Matches characters considered whitespace in the ASCII character set; this is equivalent to `[\t\n\r\f\v]`.

Matches Unicode whitespace characters (as defined by `str.isspace()`). This includes `[\t\n\r\f\v]`, and also many other characters, for example the non-breaking spaces mandated by typography rules in many languages. Matches `[\t\n\r\f\v]` if the `ASCII` flag is used.

Matches characters considered whitespace in the ASCII character set; this is equivalent to `[\t\n\r\f\v]`.

\S

Matches any character which is not a whitespace character. This is the opposite of `\s`. Matches `^[^\t\n\r\f\v]` if the `ASCII` flag is used.

\w

For Unicode (str) patterns:

For 8-bit (bytes) patterns:

For Unicode (str) patterns: Matches Unicode word characters; this includes all Unicode alphanumeric characters (as defined by `str.isalnum()`), as well as the underscore (`_`). Matches `[a-zA-Z0-9_]` if the `ASCII` flag is used. For 8-bit (bytes) patterns: Matches characters considered alphanumeric in the ASCII character set; this is equivalent to `[a-zA-Z0-9_]`. If the `LOCALE` flag is used, matches characters considered alphanumeric in the current locale and the underscore.

Matches Unicode word characters; this includes all Unicode alphanumeric characters (as defined by `str.isalnum()`), as well as the underscore (`_`). Matches `[a-zA-Z0-9_]` if the `ASCII` flag is used.

Matches characters considered alphanumeric in the ASCII character set; this is equivalent to `[a-zA-Z0-9_]`. If the `LOCALE` flag is used, matches characters considered alphanumeric in the current locale and the underscore.

\W

Matches any character which is not a word character. This is the opposite of `\w`. By default, matches non-underscore (`_`) characters for which `str.isalnum()` returns `False`. Matches `^[^a-zA-Z0-9_]` if the `ASCII` flag is used. If the `LOCALE` flag is used, matches characters which are neither alphanumeric in the current locale nor the underscore.

\z

\Z

Matches only at the end of the string. Added in version 3.14.

The same as `\z`. For compatibility with old Python versions.

Most of the escape sequences supported by Python string literals are also accepted by the regular expression parser:

`\a` `\b` `\f` `\n`

<code>\N</code>	<code>\r</code>	<code>\t</code>	<code>\u</code>
<code>\U</code>	<code>\v</code>	<code>\x</code>	<code>\\</code>

(Note that `\b` is used to represent word boundaries, and means “backspace” only inside character classes.)

`'u'`, `'U'`, and `'N'` escape sequences are only recognized in Unicode (str) patterns. In bytes patterns they are errors. Unknown escapes of ASCII letters are reserved for future use and treated as errors.

Octal escapes are included in a limited form. If the first digit is a 0, or if there are three octal digits, it is considered an octal escape. Otherwise, it is a group reference. As for string literals, octal escapes are always at most three digits in length.

Changed in version 3.3: The `'u'` and `'U'` escape sequences have been added.

Changed in version 3.6: Unknown escapes consisting of `'\'` and an ASCII letter now are errors.

Changed in version 3.8: The `'N{name}'` escape sequence has been added. As in string literals, it expands to the named Unicode character (e.g. `'N{EMDASH}'`).

Module Contents

The module defines several functions, constants, and an exception. Some of the functions are simplified versions of the full featured methods for compiled regular expressions. Most non-trivial applications always use the compiled form.

Flags

Changed in version 3.6: Flag constants are now instances of `RegexFlag`, which is a subclass of `enum.IntFlag`.

`re.RegexFlag`

An `enum.IntFlag` class containing the regex options listed below. Added in version 3.11: - added to `__all__`

`re.A`

`re.ASCII`

Make `w`, `W`, `b`, `B`, `d`, `D`, `s` and `S` perform ASCII-only matching instead of full Unicode matching. This is only meaningful for Unicode (str) patterns, and is ignored for bytes patterns. Corresponds to the inline flag `(?a)`. Note The `U` flag still exists for backward compatibility, but is redundant in Python 3 since matches are Unicode by default for str patterns, and Unicode matching isn't allowed for bytes patterns. `UNICODE` and the inline flag `(?u)` are similarly redundant.

`re.DEBUG`

Display debug information about compiled expression. No corresponding inline flag.

`re.I`

`re.IGNORECASE`

Perform case-insensitive matching; expressions like `[A-Z]` will also match lowercase letters. Full Unicode matching (such as `ü` matching `ü`) also works unless the `ASCII` flag is used to disable non-ASCII matches. The current locale does not change the effect of this flag unless the `LOCALE` flag is also used. Corresponds to the inline flag `(?i)`. Note that when the Unicode patterns `[a-z]` or `[A-Z]` are used in combination with the `IGNORECASE` flag, they will match the 52 ASCII letters and 4 additional non-ASCII letters: ‘

`re.L`

`re.LOCALE`

Make `w, W, \b, \B` and case-insensitive matching dependent on the current locale. This flag can be used only with bytes patterns. Corresponds to the inline flag `(?L)`. **Warning** This flag is discouraged; consider Unicode matching instead. The locale mechanism is very unreliable as it only handles one “culture” at a time and only works with 8-bit locales. Unicode matching is enabled by default for Unicode (str) patterns and it is able to handle different locales and languages. Changed in version 3.6: `LOCALE` `ca`

re.M

re.MULTILINE

When specified, the pattern character `^` matches at the beginning of the string and at the beginning of each line (immediately following each newline); and the pattern character `$` matches at the end of the string and at the end of each line (immediately preceding each newline). By default, `^` matches only at the beginning of the string, and `$` only at the end of the string and immediately before the newline (if any) at the end of the string. Corresponds to the inline flag `(?m)`.

re.NOFLAG

Indicates no flag being applied, the value is 0. This flag may be used as a default value for a function keyword argument or as a base value that will be conditionally ORed with other flags. Example of use as a default value: `def myfunc(text, flag=re.NOFLAG): return re.match(text, flag)` Added in version 3.11.

re.S

re.DOTALL

Make the `.` special character match any character at all, including a newline; without this flag, `.` will match anything except a newline. Corresponds to the inline flag `(?s)`.

re.U

re.UNICODE

In Python 3, Unicode characters are matched by default for str patterns. This flag is therefore redundant with no effect and is only kept for backward compatibility. See `ASCII` to restrict matching to ASCII characters instead.

re.X

re.VERBOSE

This flag allows you to write regular expressions that look nicer and are more readable by allowing you to visually separate logical sections of the pattern and add comments. Whitespace within the pattern is ignored, except when in a character class, or when preceded by an unescaped backslash, or within tokens like `*?|(?|or|P<...>`. For example, `(?:and*?)` are not allowed. When a line contains a `#` that is not in a character class and is not preceded by an unescaped backslash, all characters from the line

Functions

re.compile(pattern, flags=0)

Compile a regular expression pattern into a regular expression object, which can be used for matching using `match()`, `search()` and other methods, described below. The expression's behaviour can be modified by specifying a flags value. Values can be any of the flags variables, combined using bitwise OR (the `|` operator). The sequence `prog=re.compile(pattern)` `result=prog.match(string)` is equivalent to `result=re.match(pattern, string)` but using `re.compile()` and saving the resulting regular expression object for reuse

re.search(pattern, string, flags=0)

Scan through string looking for the first location where the regular expression pattern produces a match, and return a corresponding `Match`. Return `None` if no position in the string matches the pattern; note that this is different from finding a zero-length match at some point in the string. The expression's behaviour can be

modified by specifying a flags value. Values can be any of the flags variables, combined using bitwise OR (the | operator).

re.match(pattern,string,flags=0)

If zero or more characters at the beginning of string match the regular expression pattern, return a corresponding Match object. Return None if the string does not match the pattern; note that this is different from a zero-length match. Note that even in MULTILINE mode, re.match() will only match at the beginning of the string and not at the beginning of each line. If you want to locate a match anywhere in string, use search() instead (see also search() vs. match()). The expression's behaviour can be modified by specifying a flags value.

re.fullmatch(pattern,string,flags=0)

If the whole string matches the regular expression pattern, return a corresponding Match object. Return None if the string does not match the pattern; note that this is different from a zero-length match. The expression's behaviour can be modified by specifying a flags value. Values can be any of the flags variables, combined using bitwise OR (the | operator). Added in version 3.4.

re.split(pattern,string,maxsplit=0,flags=0)

Split string by the occurrences of pattern. If capturing parentheses are used in pattern, then the text of all groups in the pattern are also returned as part of the resulting list. If maxsplit is nonzero, at most maxsplit splits occur, and the remainder of the string is returned as the final element of the list. >>> re.split(r'\W+', 'Words, words, words.')[0] 'Words', [1] 'words', [2] 'words', [3] '' >>> re.split(r'(\W+)', 'Words, words, words.')[0] 'Words', [1] ',', [2] 'words', [3] ',', [4] 'words', [5] ',', [6] '' >>> re.split(r'\W+', 'Words, w

re.findall(pattern,string,flags=0)

Return all non-overlapping matches of pattern in string, as a list of strings or tuples. The string is scanned left-to-right, and matches are returned in the order found. Empty matches are included in the result. The result depends on the number of capturing groups in the pattern. If there are no groups, return a list of strings matching the whole pattern. If there is exactly one group, return a list of strings matching that group. If multiple groups are present, return a list of tuples of strings

re.finditer(pattern,string,flags=0)

Return an iterator yielding Match objects over all non-overlapping matches for the RE pattern in string. The string is scanned left-to-right, and matches are returned in the order found. Empty matches are included in the result. The expression's behaviour can be modified by specifying a flags value. Values can be any of the flags variables, combined using bitwise OR (the | operator). Changed in version 3.7: Non-empty matches can now start just after a previous empty match.

re.sub(pattern,repl,string,count=0,flags=0)

Return the string obtained by replacing the leftmost non-overlapping occurrences of pattern in string by the replacement repl. If the pattern isn't found, string is returned unchanged. repl can be a string or a function; if it is a string, any backslash escapes in it are processed. That is, \n is converted to a single newline character, \r is converted to a carriage return, and so forth. Unknown escapes of ASCII letters are reserved for future use and treated as errors. Other unknown escapes such as \& are

re.subn(pattern,repl,string,count=0,flags=0)

Perform the same operation as sub(), but return a tuple (new_string, number_of_subs_made). The expression's behaviour can be modified by specifying a flags value. Values can be any of the flags variables, combined using bitwise OR (the | operator).

re.escape(pattern)

Escape special characters in pattern. This is useful if you want to match an arbitrary literal string that may have regular expression metacharacters in it. For example: >>> print(re.escape('https://www.python.org')) https://www\.

```
python\org>>>legal_chars=string.ascii_lowercase+string.digits+"!#$%&'+-.\^_`|~:">>>print('%s'+'%re.escape(l
egal_chars))[abcdefghijklmnopqrstuvwxyz0123456789!\#$%&'*\+|-.\^_`|~:]>>>operators=['+', '-', '*', '/', '**']>>>
print(''.join(map(re.escape,sorted(operators,rev
```

re.purge()

Clear the regular expression cache.

Exceptions

exception`re.PatternError(msg,pattern=None,pos=None)`

msg

pattern

pos

lineno

colno

Exception raised when a string passed to one of the functions here is not a valid regular expression (for example, it might contain unmatched parentheses) or when some other error occurs during compilation or matching. It is never an error if a string contains no match for a pattern. The `PatternError` instance has the following additional attributes: `msg` The unformatted error message. `pattern` The regular expression `pattern`. `pos` The index in `pattern` where compilation failed (may be `None`). `lineno` The line cor

The unformatted error message.

The regular expression pattern.

The index in `pattern` where compilation failed (may be `None`).

The line corresponding to `pos` (may be `None`).

The column corresponding to `pos` (may be `None`).

Regular Expression Objects

class`re.Pattern`

Compiled regular expression object returned by `re.compile()`. Changed in version 3.9: `re.Pattern` supports `[]` to indicate a Unicode (str) or bytes pattern. See `Generic Alias Type`.

Pattern.search(string[,pos[,endpos]])

Scan through `string` looking for the first location where this regular expression produces a match, and return a corresponding `Match`. Return `None` if no position in the string matches the pattern; note that this is different from finding a zero-length match at some point in the string. The optional second parameter `pos` gives an index in the string where the search is to start; it defaults to 0. This is not completely equivalent to slicing the string; the `^` pattern character matches at the real beginning of

Pattern.match(string[,pos[,endpos]])

If zero or more characters at the beginning of `string` match this regular expression, return a corresponding `Match`. Return `None` if the string does not match the pattern; note that this is different from a zero-length match. The optional `pos` and `endpos` parameters have the same meaning as for these `search()` method. `>>> pattern=re.compile("o")>>> pattern.match("dog")` # No match as "o" is not at the start of "dog". `>>> pattern.match("dog",1)` # Match as "o" is the 2nd character of "dog". `<re.Match object; span=(1, 2), match=`

Pattern.fullmatch(string[,pos[,endpos]])

If the whole string matches this regular expression, return a corresponding `Match`. Return `None` if the string does not match the pattern; note that this is different from a zero-length match. The optional `pos` and `endpos` parameters have the same meaning as for the `search()` method. >>> `pattern=re.compile("o[gh]")` >>> `pattern.fullmatch("dog")` # No match as "o" is not at the start of "dog". >>> `pattern.fullmatch("ogre")` # No match as not the full string matches. >>> `pattern.fullmatch("doggie",1,3)` # Matches within given l

Pattern.split(string,maxsplit=0)

Identical to the `split()` function, using the compiled pattern.

Pattern.findall(string[,pos[,endpos]])

Similar to the `findall()` function, using the compiled pattern, but also accepts optional `pos` and `endpos` parameters that limit the search region like `forsearch()`.

Pattern.finditer(string[,pos[,endpos]])

Similar to the `finditer()` function, using the compiled pattern, but also accepts optional `pos` and `endpos` parameters that limit the search region like `forsearch()`.

Pattern.sub(repl,string,count=0)

Identical to the `sub()` function, using the compiled pattern.

Pattern.subn(repl,string,count=0)

Identical to the `subn()` function, using the compiled pattern.

Pattern.flags

The regex matching flags. This is a combination of the flags given to `compile()`, any `(?...)` inline flags in the pattern, and implicit flags such as `UNICODE` if the pattern is a Unicode string.

Pattern.groups

The number of capturing groups in the pattern.

Pattern.groupindex

A dictionary mapping any symbolic group names defined by `(?P<id>)` to group numbers. The dictionary is empty if no symbolic groups were used in the pattern.

Pattern.pattern

The pattern string from which the pattern object was compiled.

Changed in version 3.7: Added support for `copy.copy()` and `copy.deepcopy()`. Compiled regular expression objects are considered atomic.

Match Objects

Match objects always have a boolean value of `True`. Since `match()` and `search()` return `None` when there is no match, you can test whether there was a match with a simple `if` statement:

```
match = re.search(pattern, string)
if match:
    process(match)
```

class re.Match

Match object returned by successful matches and searches. Changed in version 3.9: `re.Match` supports `__to__` to indicate a Unicode (str) or bytes match. See `Generic Alias Type`.

Match.expand(template)

Return the string obtained by doing backslash substitution on the template string `template`, as done by the `sub()` method. Escapes such as `\n` are converted to the appropriate characters, and numeric backreferences (`\1`, `\2`) and named backreferences (`\g<1>`, `\g<name>`) are replaced by the contents of the corresponding group. The backreference `\g<0>` will be replaced by the entire match. Changed in version 3.5: Unmatched groups are replaced with an empty string.

Match.group([group1,...])

Returns one or more subgroups of the match. If there is a single argument, the result is a single string; if there are multiple arguments, the result is a tuple with one item per argument. Without arguments, `group1` defaults to zero (the whole match is returned). If a `groupN` argument is zero, the corresponding return value is the entire matching string; if it is a positive integer, it is the string matching the corresponding parenthesized group. If a group number is negative or larger than the number of groups, the result is an empty string.

Match.__getitem__(g)

This is identical to `m.group(g)`. This allows easier access to an individual group from a match: `>>>m=re.match(r"(w+) (w+)", "Isaac Newton, physicist")>>>m[0]` # The entire match 'Isaac Newton'
`>>>m[1]` # The first parenthesized subgroup. 'Isaac'
`>>>m[2]` # The second parenthesized subgroup. 'Newton'
Named groups are supported as well: `>>>m=re.match(r"(?P<first_name>w+)(?P<last_name>w+)", "Isaac Newton")>>>m['first_name']` 'Isaac'
`>>>m['last_name']` 'Newton'
Added in version 3.6.

Match.groups(default=None)

Return a tuple containing all the subgroups of the match, from 1 up to however many groups are in the pattern. The `default` argument is used for groups that did not participate in the match; it defaults to `None`. For example: `>>>m=re.match(r"(\d+)\.(\d+)", "24.1632")>>>m.groups()` ('24', '1632')
If we make the decimal place and everything after it optional, not all groups might participate in the match. These groups will default to `None` unless the `default` argument is given: `>>>m=re.match(r"(\d+)\.?(d+)?", "24")`

Match.groupdict(default=None)

Return a dictionary containing all the named subgroups of the match, keyed by the subgroup name. The `default` argument is used for groups that did not participate in the match; it defaults to `None`. For example: `>>>m=re.match(r"(?P<first_name>w+) (?P<last_name>w+)", "Malcolm Reynolds")>>>m.groupdict()` {'first_name': 'Malcolm', 'last_name': 'Reynolds'}

Match.start([group])

Match.end([group])

Return the indices of the start and end of the substring matched by `group`; `group` defaults to zero (meaning the whole matched substring). Return -1 if `group` exists but did not contribute to the match. For a match object `m`, and a `group` `g` that did contribute to the match, the substring matched by `group` (equivalent to `m.group(g)`) is `m.string[m.start(g):m.end(g)]`. Note that `m.start(group)` will equal `m.end(group)` if `group` matched a null string. For example, after `m=re.search('b(c?)', 'cba')`, `m.start(0)` is 1, `m.end(0)` is 2, `m.start(1)` is 1, `m.end(1)` is 2.

Match.span([group])

For a match `m`, return the 2-tuple `(m.start(group), m.end(group))`. Note that if `group` did not contribute to the match, this is `(-1, -1)`. `group` defaults to zero, the entire match.

Match.pos

The value of `pos` which was passed to the `search()` or `match()` method of a `regex` object. This is the index into the string at which the RE engine started looking for a match.

Match.endpos

The value of `endpos` which was passed to `search()` or `match()` method of a `regex` object. This is the index into the string beyond which the RE engine will not go.

Match.lastindex

The integer index of the last matched capturing group, or `None` if no group was matched at all. For example, the expressions `(a)b`, `((a)(b))`, and `((ab))` will have `lastindex==1` if applied to the string `'ab'`, while the expression `(a)(b)` will have `lastindex==2`, if applied to the same string.

Match.lastgroup

The name of the last matched capturing group, or `None` if the group didn't have a name, or if no group was matched at all.

Match.re

The regular expression object whose `match()` or `search()` method produced this match instance.

Match.string

The string passed to `match()` or `search()`.

Changed in version 3.7: Added support for `copy.copy()` and `copy.deepcopy()`. Match objects are considered atomic.

Regular Expression Examples

Checking for a Pair

In this example, we'll use the following helper function to display match objects a little more gracefully:

```
def displaymatch(match):
    if match is None:
        return None
    return '<Match: %r, groups=%r>' % (match.group(), match.groups())
```

Suppose you are writing a poker program where a player's hand is represented as a 5-character string with each character representing a card, "a" for ace, "k" for king, "q" for queen, "j" for jack, "t" for 10, and "2" through "9" representing the card with that value.

To see if a given string is a valid hand, one could do the following:

```
>>> valid = re.compile(r"^[a2-9tjqk]{5}$")
>>> displaymatch(valid.match("akt5q")) # Valid.
"<Match: 'akt5q', groups=()>"
>>> displaymatch(valid.match("akt5e")) # Invalid.
>>> displaymatch(valid.match("akt"))   # Invalid.
>>> displaymatch(valid.match("727ak")) # Valid.
"<Match: '727ak', groups=()>"
```

That last hand, "727ak", contained a pair, or two of the same valued cards. To match this with a regular expression, one could use backreferences as such:

```
>>> pair = re.compile(r"^(.*)\1$")
>>> displaymatch(pair.match("717ak")) # Pair of 7s.
"<Match: '717', groups=('7',)>"
>>> displaymatch(pair.match("718ak")) # No pairs.
>>> displaymatch(pair.match("354aa")) # Pair of aces.
"<Match: '354aa', groups=('a',)>"
```

To find out what card the pair consists of, one could use the `group()` method of the match object in the following manner:

```
>>> pair = re.compile(r"^(.*)\1$")
>>> pair.match("717ak").group(1)
```

```
'7'
```

```
# Error because re.match() returns None, which doesn't have a group() method:
```

```
>>> pair.match("718ak").group(1)
```

```
Traceback (most recent call last):
```

```
File "<pyshell#23>", line 1, in <module>
```

```
    re.match(r".*(.)*\1", "718ak").group(1)
```

```
AttributeError: 'NoneType' object has no attribute 'group'
```

```
>>> pair.match("354aa").group(1)
```

```
'a'
```

Simulating scanf()

Python does not currently have an equivalent to `scanf()`. Regular expressions are generally more powerful, though also more verbose, than `scanf()` format strings. The table below offers some more-or-less equivalent mappings between `scanf()` format tokens and regular expressions.

scanf()	Token	Regular Expression
%c	.	%5c
{5}	%d	[-+]? \d+
%e	,	%E
%f	,	%g
[-+]? (\d+(\.\d*)?) \.\d+	([eE][-+]? \d+)?	%i
[-+]? (0[xX][\dA-Fa-f]+ 0[0-7]* \d+)	%o	[-+]? [0-7]+
%s	\S+	%u
\d+	%x	,
%X	[-+]? (0[xX])? [\dA-Fa-f]+	

To extract the filename and numbers from a string like

```
/usr/sbin/sendmail - 0 errors, 4 warnings
```

you would use `scanf()` format like

```
%s - %d errors, %d warnings
```

The equivalent regular expression would be

```
(\S+) - (\d+) errors, (\d+) warnings
```

search() vs. match()

Python offers different primitive operations based on regular expressions:

- `re.match()` checks for a match only at the beginning of the string
- `re.search()` checks for a match anywhere in the string (this is what Perl does by default)
- `re.fullmatch()` checks for entire string to be a match

For example:

```
>>> re.match("c", "abcdef")    # No match
>>> re.search("c", "abcdef")   # Match
<re.Match object; span=(2, 3), match='c'>
>>> re.fullmatch("p.*n", "python") # Match
<re.Match object; span=(0, 6), match='python'>
>>> re.fullmatch("r.*n", "python") # No match
```

Regular expressions beginning with `^` can be used with `search()` to restrict the match at the beginning of the string:

```
>>> re.match("c", "abcdef")    # No match
>>> re.search("^c", "abcdef")  # No match
>>> re.search("^a", "abcdef")  # Match
<re.Match object; span=(0, 1), match='a'>
```

Note however that in `MULTILINE` mode `match()` only matches at the beginning of the string, whereas `search()` with a regular expression beginning with `^` will match at the beginning of each line.

```
>>> re.match("X", "A\nB\nX", re.MULTILINE) # No match
>>> re.search("^X", "A\nB\nX", re.MULTILINE) # Match
<re.Match object; span=(4, 5), match='X'>
```

Making a Phonebook

`split()` splits a string into a list delimited by the passed pattern. The method is invaluable for converting textual data into data structures that can be easily read and modified by Python as demonstrated in the following example that creates a phonebook.

First, here is the input. Normally it may come from a file, here we are using triple-quoted string syntax

```
>>> text = """Ross McFluff: 834.345.1254 155 Elm Street
...
... Ronald Heathmore: 892.345.3428 436 Finley Avenue
... Frank Burger: 925.541.7625 662 South Dogwood Way
...
...
... Heather Albrecht: 548.326.4584 919 Park Place"""
```

The entries are separated by one or more newlines. Now we convert the string into a list with each nonempty line having its own entry:

```
>>> entries = re.split("\n+", text)
>>> entries
['Ross McFluff: 834.345.1254 155 Elm Street',
 'Ronald Heathmore: 892.345.3428 436 Finley Avenue',
 'Frank Burger: 925.541.7625 662 South Dogwood Way',
 'Heather Albrecht: 548.326.4584 919 Park Place']
```

Finally, split each entry into a list with first name, last name, telephone number, and address. We use `themaxsplit` parameter of `split()` because the address has spaces, our splitting pattern, in it:

```
>>> [re.split("?: ", entry, maxsplit=3) for entry in entries]
[['Ross', 'McFluff', '834.345.1254', '155 Elm Street'],
 ['Ronald', 'Heathmore', '892.345.3428', '436 Finley Avenue'],
 ['Frank', 'Burger', '925.541.7625', '662 South Dogwood Way'],
 ['Heather', 'Albrecht', '548.326.4584', '919 Park Place']]
```

The `?:` pattern matches the colon after the last name, so that it does not occur in the result list. With `amaxsplit=4`, we could separate the house number from the street name:

```
>>> [re.split("?: ", entry, maxsplit=4) for entry in entries]
[['Ross', 'McFluff', '834.345.1254', '155', 'Elm Street'],
 ['Ronald', 'Heathmore', '892.345.3428', '436', 'Finley Avenue'],
 ['Frank', 'Burger', '925.541.7625', '662', 'South Dogwood Way'],
 ['Heather', 'Albrecht', '548.326.4584', '919', 'Park Place']]
```

Text Munging

`sub()` replaces every occurrence of a pattern with a string or the result of a function. This example demonstrates using `sub()` with a function to “munge” text, or randomize the order of all the characters in each word of a sentence except for the first and last characters:

```
>>> def repl(m):
...     inner_word = list(m.group(2))
...     random.shuffle(inner_word)
...     return m.group(1) + "".join(inner_word) + m.group(3)
...
>>> text = "Professor Abdolmalek, please report your absences promptly."
>>> re.sub(r"(\w)(\w+)(\w)", repl, text)
'Poefsrosr Aealmlobdk, pslaee reorpt your abnseces plmrptoy.'
>>> re.sub(r"(\w)(\w+)(\w)", repl, text)
'Pofsroser Aodlambelk, plasee reorpt yuor asnebces potlmrpy.'
```

Finding all Adverbs

`findall()` matches all occurrences of a pattern, not just the first one as `search()` does. For example, if a writer wanted to find all of the adverbs in some text, they might use `findall()` in the following manner:

```
>>> text = "He was carefully disguised but captured quickly by police."
>>> re.findall(r"\w+ly\b", text)
['carefully', 'quickly']
```

Finding all Adverbs and their Positions

If one wants more information about all matches of a pattern than the matched text, `finditer()` is useful as it provides `Match` objects instead of strings. Continuing with the previous example, if a writer wanted to find all of the adverbs and their positions in some text, they would use `finditer()` in the following manner:

```
>>> text = "He was carefully disguised but captured quickly by police."
>>> for m in re.finditer(r"\w+ly\b", text):
...     print('%02d-%02d: %s' % (m.start(), m.end(), m.group(0)))
07-16: carefully
40-47: quickly
```

Raw String Notation

Raw string notation (`r"text"`) keeps regular expressions sane. Without it, every backslash (`\`) in a regular expression would have to be prefixed with another one to escape it. For example, the two following lines of code are functionally identical:

```
>>> re.match(r"\W(.)\1\W", " ff ")
<re.Match object; span=(0, 4), match=' ff '>
>>> re.match("\\W(.)\\1\\W", " ff ")
<re.Match object; span=(0, 4), match=' ff '>
```

When one wants to match a literal backslash, it must be escaped in the regular expression. With raw string notation, this means `r"\\"`. Without raw string notation, one must use `\\\"`, making the following lines of code functionally identical:

```
>>> re.match(r"\\", r"\\")
<re.Match object; span=(0, 1), match='\\'>
>>> re.match("\\\\", r"\\")
<re.Match object; span=(0, 1), match='\\'>
```

Writing a Tokenizer

A tokenizer or scanner analyzes a string to categorize groups of characters. This is a useful first step in writing a compiler or interpreter.

The text categories are specified with regular expressions. The technique is to combine those into a single master regular expression and to loop over successive matches:

```
from typing import NamedTuple
import re

class Token(NamedTuple):
    type: str
    value: str
    line: int
    column: int

def tokenize(code):
    keywords = {'IF', 'THEN', 'ENDIF', 'FOR', 'NEXT', 'GOSUB', 'RETURN'}
    token_specification = [
        ('NUMBER', r'\d+(\.\d*)?'), # Integer or decimal number
        ('ASSIGN', r':='),           # Assignment operator
        ('END', r';'),               # Statement terminator
        ('ID', r'[A-Za-z]+'),        # Identifiers
        ('OP', r'[+ \- */]'),        # Arithmetic operators
        ('NEWLINE', r'\n'),          # Line endings
        ('SKIP', r'[ \t]+'),          # Skip over spaces and tabs
        ('MISMATCH', r'.'),          # Any other character
```

```

]
tok_regex = '|'.join('(P<%s>%s)' % pair for pair in token_specification)
line_num = 1
line_start = 0
for mo in re.finditer(tok_regex, code):
    kind = mo.lastgroup
    value = mo.group()
    column = mo.start() - line_start
    if kind == 'NUMBER':
        value = float(value) if '.' in value else int(value)
    elif kind == 'ID' and value in keywords:
        kind = value
    elif kind == 'NEWLINE':
        line_start = mo.end()
        line_num += 1
        continue
    elif kind == 'SKIP':
        continue
    elif kind == 'MISMATCH':
        raise RuntimeError(f'{value!r} unexpected on line {line_num}')
    yield Token(kind, value, line_num, column)

statements = '''
    IF quantity THEN
        total := total + price * quantity;
        tax := price * 0.05;
    ENDIF;
'''

for token in tokenize(statements):
    print(token)

```

The tokenizer produces the following output:

```

Token(type='IF', value='IF', line=2, column=4)
Token(type='ID', value='quantity', line=2, column=7)
Token(type='THEN', value='THEN', line=2, column=16)
Token(type='ID', value='total', line=3, column=8)
Token(type='ASSIGN', value=':=', line=3, column=14)
Token(type='ID', value='total', line=3, column=17)
Token(type='OP', value='+', line=3, column=23)
Token(type='ID', value='price', line=3, column=25)
Token(type='OP', value='*', line=3, column=31)
Token(type='ID', value='quantity', line=3, column=33)
Token(type='END', value=';', line=3, column=41)
Token(type='ID', value='tax', line=4, column=8)
Token(type='ASSIGN', value=':=', line=4, column=12)
Token(type='ID', value='price', line=4, column=15)
Token(type='OP', value='*', line=4, column=21)
Token(type='NUMBER', value=0.05, line=4, column=23)
Token(type='END', value=';', line=4, column=27)
Token(type='ENDIF', value='ENDIF', line=5, column=4)
Token(type='END', value=';', line=5, column=9)

```

Friedl, Jeffrey. Mastering Regular Expressions. 3rd ed., O'Reilly Media, 2009. The third edition of the book no longer covers Python at all, but the first edition covered writing good regular expression patterns in great detail.

difflib— Helpers for computing deltas

Source code: `Lib/difflib.py`

This module provides classes and functions for comparing sequences. It can be used for example, for comparing files, and can produce information about file differences in various formats, including HTML and context and unified diffs. For comparing directories and files, see also, `thefilecmp` module.

class difflib.SequenceMatcher

This is a flexible class for comparing pairs of sequences of any type, so long as the sequence elements are hashable. The basic algorithm predates, and is a little fancier than, an algorithm published in the late 1980's by Ratcliff and Obershelp under the hyperbolic name "gestalt pattern matching." The idea is to find the longest contiguous matching subsequence that contains no "junk" elements; these "junk" elements are ones that are uninteresting in some sense, such as blank lines or whitespace

class difflib.Differ

This is a class for comparing sequences of lines of text, and producing human-readable differences or deltas. Differ uses SequenceMatcher both to compare sequences of lines, and to compare sequences of characters within similar (near-matching) lines. Each line of a Differ delta begins with a two-letter code: Code Meaning - 'line unique to sequence 1' + 'line unique to sequence 2' "line common to both sequences" ? 'line not present in either input sequence' Lines beginning with '?' attempt to guide the eye to

class difflib.HtmlDiff

__init__ (tabsize=8, wrapcolumn=None, linejunk=None, charjunk=IS_CHARACTER_JUNK)

make_file (fromlines, tolines, fromdesc="", todesc="", context=False, numlines=5, *, charset='utf-8')

make_table (fromlines, tolines, fromdesc="", todesc="", context=False, numlines=5)

This class can be used to create an HTML table (or a complete HTML file containing the table) showing a side by side, line by line comparison of text with inter-line and intra-line change highlights. The table can be generated in either full or contextual difference mode. The constructor for this class

is: **__init__** (tabsize=8, wrapcolumn=None, linejunk=None, charjunk=IS_CHARACTER_JUNK) Initializes instance of HtmlDiff. tabsize is an optional keyword argument to specify tab stop spacing and defaults to 8. w

Initializes instance of HtmlDiff. tabsize is an optional keyword argument to specify tab stop spacing and defaults to 8. wrapcolumn is an optional keyword to specify column number where lines are broken and wrapped, defaults to None where lines are not wrapped. linejunk and charjunk are optional keyword arguments passed into **diff()** (used by HtmlDiff to generate the side by side HTML differences). See **diff()** documentation for argument default values and descriptions.

Compares fromlines and tolines (lists of strings) and returns a string which is a complete HTML file containing a table showing line by line differences with inter-line and intra-line changes highlighted. fromdesc and todesc are optional keyword arguments to specify from/to file column header strings (both default to an empty string). context and numlines are both optional keyword arguments. Set context to True when contextual differences are to be shown, else the default is False to show the full files. numlines d

Compares fromlines and tolines (lists of strings) and returns a string which is a complete HTML table showing line by line differences with inter-line and intra-line changes highlighted. The arguments for this method are the same as those for the **make_file()** method.

difflib.context_diff (a, b, fromfile="", tofile="", fromfiledate="", tofiledate="", n=3, lineterm='\n')

Compare a and b (lists of strings); return a delta (a generator generating the delta lines) in context diff format. Context diffs are a compact way of showing just the lines that have changed plus a few lines of context.

The changes are shown in a before/after style. The number of context lines is set by `by` which defaults to three. By default, the diff control lines (those with `***` or `---`) are created with a trailing newline. This is helpful so that inputs created from `io.IOBase.readlines()` result in diffs that

`difflib.get_close_matches(word, possibilities, n=3, cutoff=0.6)`

Return a list of the best “good enough” matches. `word` is a sequence for which close matches are desired (typically a string), and `possibilities` is a list of sequences against which to match `word` (typically a list of strings). Optional argument `n` (default 3) is the maximum number of close matches to return; `n` must be greater than 0. Optional argument `cutoff` (default 0.6) is a float in the range [0, 1]. Possibilities that don’t score at least that similar to `word` are ignored. The best (no more than `n`) matches among the

`difflib.ndiff(a, b, linejunk=None, charjunk=IS_CHARACTER_JUNK)`

Compare `a` and `b` (lists of strings); return a Differ-style delta (a generator generating the delta lines). Optional keyword parameters `linejunk` and `charjunk` are filtering functions (or `None`): `linejunk`: A function that accepts a single string argument, and returns true if the string is junk, or false if not. The default is `None`. There is also a module-level function `IS_LINE_JUNK()`, which filters out lines without visible characters, except for at most one hash character (`#`) – however the underlying `SequenceMatch`

`difflib.restore(sequence, which)`

Return one of the two sequences that generated a delta. Given a sequence produced by `Differ.compare()` or `ndiff()`, extract lines originating from file 1 or 2 (parameter `which`), stripping off line prefixes. Example:

```
>>> diff = ndiff('one\ntwo\nthree\n'.splitlines(keepends=True), ... 'ore\ntree\nemu\n'.splitlines(keepends=True))
>>> diff = list(diff) # materialize the generated delta into a list
>>> print("".join	restore(diff, 1), end="")
onetwothree
>>> print("".join	restore(diff, 2), end="")
oretreeemu
```

`difflib.unified_diff(a, b, fromfile="", tofile="", fromfiledate="", tofiledate="", n=3, lineterm='\n')`

Compare `a` and `b` (lists of strings); return a delta (a generator generating the delta lines) in unified diff format. Unified diffs are a compact way of showing just the lines that have changed plus a few lines of context. The changes are shown in an inline style (instead of separate before/after blocks). The number of context lines is set by `by` which defaults to three. By default, the diff control lines (those with `---`, `+++`, or `@@`) are created with a trailing newline. This is helpful so that inputs created

`difflib.diff_bytes(dfunc, a, b, fromfile=b'', tofile=b'', fromfiledate=b'', tofiledate=b'', n=3, lineterm=b'\n')`

Compare `a` and `b` (lists of bytes objects) using `dfunc`; yield a sequence of delta lines (also bytes) in the format returned by `dfunc`. `dfunc` must be a callable, typically either `unified_diff()` or `context_diff()`. Allows you to compare data with unknown or inconsistent encoding. All inputs except `n` must be bytes objects, not str. Works by losslessly converting all inputs (except `n`) to str, and calling `dfunc(a, b, fromfile, tofile, fromfiledate, tofiledate, n, lineterm)`. The output of `dfunc` is then converted back to bytes, so

`difflib.IS_LINE_JUNK(line)`

Return True for ignorable lines. The line `line` is ignorable if `line` is blank or contains a single `#`, otherwise it is not ignorable. Used as a default for parameter `linejunk` in `ndiff()` in older versions.

`difflib.IS_CHARACTER_JUNK(ch)`

Return True for ignorable characters. The character `ch` is ignorable if `ch` is a space or tab, otherwise it is not ignorable. Used as a default for parameter `charjunk` in `ndiff()`.

See also

Pattern Matching: The Gestalt Approach

Discussion of a similar algorithm by John W. Ratcliff and D. E. Metzener. This was published in Dr. Dobbs's Journal in July, 1988.

SequenceMatcher objects

The `SequenceMatcher` class has this constructor:

```
class difflib.SequenceMatcher(isjunk=None,a="",b="",autojunk=True)
```

```
set_seqs(a,b)
```

```
set_seq1(a)
```

```
set_seq2(b)
```

```
find_longest_match(alo=0,ahi=None,blo=0,bhi=None)
```

```
get_matching_blocks()
```

```
get_opcodes()
```

```
get_grouped_opcodes(n=3)
```

```
ratio()
```

```
quick_ratio()
```

```
real_quick_ratio()
```

Optional argument `isjunk` must be `None` (the default) or a one-argument function that takes a sequence element and returns true if and only if the element is "junk" and should be ignored. Passing `None` for `isjunk` is equivalent to passing `lambda x: False`; in other words, no elements are ignored. For example, pass `lambda x: x in "\t"` if you're comparing lines as sequences of characters, and don't want to synch up on blanks or hard tabs. The optional arguments `a` and `b` are sequences to be compared; both default to empty strings.

Set the two sequences to be compared.

Set the first sequence to be compared. The second sequence to be compared is not changed.

Set the second sequence to be compared. The first sequence to be compared is not changed.

Find longest matching block in `a[alo:ahi]` and `b[blo:bhi]`. If `isjunk` was omitted or `None`, `find_longest_match()` returns `(i,j,k)` such that `a[i:i+k]` is equal to `b[j:j+k]`, where `alo <= i < i+k <= ahi` and `blo <= j < j+k <= bhi`. For all `(i',j',k')` meeting those conditions, the additional conditions `k >= k'`, `i <= i'`, and if `i==i'`, `j <= j'` are also met. In other words, of all maximal matching blocks, return one that starts earliest in `a`, and of all those maximal matching blocks that start earliest in `a`, return the one that starts earliest in `b`. >>>

Return list of triples describing non-overlapping matching subsequences. Each triple is of the form `(i,j,n)`, and means that `a[i:i+n]==b[j:j+n]`. The triples are monotonically increasing in `i` and `j`. The last triple is a dummy, and has the value `(len(a),len(b),0)`. It is the only triple with `n==0`. If `(i,j,n)` and `(i',j',n')` are adjacent triples in the list, and the second is not the last triple in the list, then `i+n < i'` or `j+n < j'`; in other words, adjacent triples always describe non-adjacent equal blocks. >>> `s=Seq`

Return list of 5-tuples describing how to turn `a` into `b`. Each tuple is of the form `(tag,i1,i2,j1,j2)`. The first tuple has `i1==j1==0`, and remaining tuples have `i1` equal to `i2` from the preceding tuple, and, likewise, `j1` equal to the previous `j2`. The `tag` values are strings, with these meanings: `ValueMeaning` 'replace' `a[i1:i2]` should be replaced by `b[j1:j2]`. 'delete' `a[i1:i2]` should be deleted. Note that `j1==j2` in this case. 'insert' `b[j1:j2]` should be inserted at `a[i1:i1]`. Note that `i1==i2` in this case. 'equal' `a[i1:i2]==b[j1:j2]`

Return a generator of groups with up to `lines` of context. Starting with the groups returned by `get_opcodes()`, this method splits out smaller change clusters and eliminates intervening ranges which have no changes. The groups are returned in the same format as `get_opcodes()`.

Return a measure of the sequences' similarity as a float in the range `[0, 1]`. Where `T` is the total number of elements in both sequences, and `M` is the number of matches, this is $2.0 * M / T$. Note that this is `1.0` if the sequences are identical, and `0.0` if they have nothing in common. This is expensive to compute if `get_matching_blocks()` or `get_opcodes()` hasn't already been called, in which case you may want to try `quick_ratio()` or `real_quick_ratio()` first to get an upper bound. Note Caution: The result of `ratio()`

Return an upper bound on `ratio()` relatively quickly.

Return an upper bound on `ratio()` very quickly.

The three methods that return the ratio of matching to total characters can give different results due to differing levels of approximation, although `quick_ratio()` and `real_quick_ratio()` are always at least as large as `ratio()`:

```
>>> s = SequenceMatcher(None, "abcd", "bcde")
>>> s.ratio()
0.75
>>> s.quick_ratio()
0.75
>>> s.real_quick_ratio()
1.0
```

SequenceMatcher examples

This example compares two strings, considering blanks to be "junk":

```
>>> s = SequenceMatcher(lambda x: x == " ",
...                       "private Thread currentThread;",
...                       "private volatile Thread currentThread;")
```

`ratio()` returns a float in `[0, 1]`, measuring the similarity of the sequences. As a rule of thumb, a `ratio()` value over `0.6` means the sequences are close matches:

```
>>> print(round(s.ratio(), 3))
0.866
```

If you're only interested in where the sequences match, `get_matching_blocks()` is handy:

```
>>> for block in s.get_matching_blocks():
...     print("a[%d] and b[%d] match for %d elements" % block)
a[0] and b[0] match for 8 elements
a[8] and b[17] match for 21 elements
a[29] and b[38] match for 0 elements
```

Note that the last tuple returned by `get_matching_blocks()` is always a dummy, `(len(a), len(b), 0)`, and this is the only case in which the last tuple element (number of elements matched) is `0`.

If you want to know how to change the first sequence into the second, use `get_opcodes()`:

```
>>> for opcode in s.get_opcodes():
...     print("%6s a[%d:%d] b[%d:%d]" % opcode)
equal a[0:8] b[0:8]
insert a[8:8] b[8:17]
equal a[8:29] b[17:38]
```

See also

- The `get_close_matches()` function in this module which shows how simple code building on `SequenceMatcher` can be used to do useful work.

- Simple version control recipe for a small application built with `SequenceMatcher`.

Differ objects

Note that `Differ`-generated deltas make no claim to be minimal diffs. To the contrary, minimal diffs are often counter-intuitive, because they synch up anywhere possible, sometimes accidental matches 100 pages apart. Restricting synch points to contiguous matches preserves some notion of locality, at the occasional cost of producing a longer diff.

The `Differ` class has this constructor:

```
class difflib.Differ(linejunk=None, charjunk=None)
```

```
compare(a,b)
```

Optional keyword parameters `linejunk` and `charjunk` are for filter functions (or `None`): `linejunk`: A function that accepts a single string argument, and returns true if the string is junk. The default is `None`, meaning that no line is considered junk. `charjunk`: A function that accepts a single character argument (a string of length 1), and returns true if the character is junk. The default is `None`, meaning that no character is considered junk. These junk-filtering functions speed up matching to find differences.

Compare two sequences of lines, and generate the delta (a sequence of lines). Each sequence must contain individual single-line strings ending with newlines. Such sequences can be obtained from the `readlines()` method of file-like objects. The delta generated also consists of newline-terminated strings, ready to be printed as-is via the `writelines()` method of a file-like object.

Differ example

This example compares two texts. First we set up the texts, sequences of individual single-line strings ending with newlines (such sequences can also be obtained from the `readlines()` method of file-like objects):

```
>>> text1 = ''' 1. Beautiful is better than ugly.
... 2. Explicit is better than implicit.
... 3. Simple is better than complex.
... 4. Complex is better than complicated.
... '''.splitlines(keepends=True)
>>> len(text1)
4
>>> text1[0][-1]
'\n'
>>> text2 = ''' 1. Beautiful is better than ugly.
... 3. Simple is better than complex.
... 4. Complicated is better than complex.
... 5. Flat is better than nested.
... '''.splitlines(keepends=True)
```

Next we instantiate a `Differ` object:

```
>>> d = Differ()
```

Note that when instantiating a `Differ` object we may pass functions to filter out line and character “junk.” See the `Differ()` constructor for details.

Finally, we compare the two:

```
>>> result = list(d.compare(text1, text2))
```

`result` is a list of strings, so let's pretty-print it:

```
>>> from pprint import pprint
```

```
>>> pprint(result)
[' 1. Beautiful is better than ugly.\n',
'- 2. Explicit is better than implicit.\n',
'- 3. Simple is better than complex.\n',
'+ 3.   Simple is better than complex.\n',
'?   ++\n',
'- 4. Complex is better than complicated.\n',
'?      ^              ---- ^\n',
'+ 4. Complicated is better than complex.\n',
'?      ++++ ^              ^\n',
'+ 5. Flat is better than nested.\n']
```

As a single multi-line string it looks like this:

```
>>> import sys
>>> sys.stdout.writelines(result)
 1. Beautiful is better than ugly.
- 2. Explicit is better than implicit.
- 3. Simple is better than complex.
+ 3.   Simple is better than complex.
?   ++
- 4. Complex is better than complicated.
?      ^              ---- ^
+ 4. Complicated is better than complex.
?      ++++ ^              ^
+ 5. Flat is better than nested.
```

A command-line interface to difflib

This example shows how to use difflib to create adiff-like utility.

```
""" Command-line interface to difflib.py providing diffs in four formats:
```

```
* ndiff:    lists every line and highlights interline changes.
* context:  highlights clusters of changes in a before/after format.
* unified:  highlights clusters of changes in an inline format.
* html:     generates side by side comparison with change highlights.
```

```
"""
```

```
import sys, os, difflib, argparse
import datetime as dt
```

```
def file_mtime(path):
    t = dt.datetime.fromtimestamp(os.stat(path).st_mtime,
                                dt.timezone.utc)
    return t.astimezone().isoformat()
```

```
def main():
```

```
    parser = argparse.ArgumentParser()
    parser.add_argument('-c', action='store_true', default=False,
                        help='Produce a context format diff (default)')
    parser.add_argument('-u', action='store_true', default=False,
                        help='Produce a unified format diff')
    parser.add_argument('-m', action='store_true', default=False,
                        help='Produce HTML side by side diff '
                             '(can use -c and -l in conjunction)')
    parser.add_argument('-n', action='store_true', default=False,
                        help='Produce a ndiff format diff')
    parser.add_argument('-l', '--lines', type=int, default=3,
                        help='Set number of context lines (default 3)')
    parser.add_argument('fromfile')
    parser.add_argument('tofile')
```

```

options = parser.parse_args()

n = options.lines
fromfile = options.fromfile
tofile = options.tofile

fromdate = file_mtime(fromfile)
todate = file_mtime(tofile)
with open(fromfile) as ff:
    fromlines = ff.readlines()
with open(tofile) as tf:
    tolines = tf.readlines()

if options.u:
    diff = difflib.unified_diff(fromlines, tolines, fromfile, tofile, fromdate, todate, n=n)
elif options.n:
    diff = difflib.ndiff(fromlines, tolines)
elif options.m:
    diff = difflib.HtmlDiff().make_file(fromlines, tolines, fromfile, tofile, context=options.c, numlines=n)
else:
    diff = difflib.context_diff(fromlines, tolines, fromfile, tofile, fromdate, todate, n=n)

sys.stdout.writelines(diff)

if __name__ == '__main__':
    main()

```

ndiff example

This example shows how to use `difflib.ndiff()`.

```

"""ndiff [-q] file1 file2
    or
ndiff (-r1 | -r2) < ndiff_output > file1_or_file2

```

Print a human-friendly file difference report to stdout. Both inter- and intra-line differences are noted. In the second form, recreate file1 (-r1) or file2 (-r2) on stdout, from an ndiff report on stdin.

In the first form, if -q ("quiet") is not specified, the first two lines of output are

```

-: file1
+: file2

```

Each remaining line begins with a two-letter code:

```

"- "    line unique to file1
"+ "    line unique to file2
"  "    line common to both files
"? "    line not present in either input file

```

Lines beginning with "? " attempt to guide the eye to intraline differences, and were not present in either input file. These lines can be confusing if the source files contain tab characters.

The first file can be recovered by retaining only lines that begin with " " or "- ", and deleting those 2-character prefixes; use ndiff with -r1.

The second file can be recovered similarly, but by retaining only " " and "+ " lines; use ndiff with -r2; or, on Unix, the second file can be recovered by piping the output through

```
sed -n '/^[+ ] /s/^..//p'
"""
```

```
__version__ = 1, 7, 0
```

```
import difflib, sys
```

```
def fail(msg):
    out = sys.stderr.write
    out(msg + "\n\n")
    out(__doc__)
    return 0
```

```
# open a file & return the file object; gripe and return 0 if it
# couldn't be opened
```

```
def fopen(fname):
    try:
        return open(fname)
    except IOError as detail:
        return fail("couldn't open " + fname + ": " + str(detail))
```

```
# open two files & spray the diff to stdout; return false iff a problem
```

```
def fcompare(f1name, f2name):
    f1 = fopen(f1name)
    f2 = fopen(f2name)
    if not f1 or not f2:
        return 0

    a = f1.readlines(); f1.close()
    b = f2.readlines(); f2.close()
    for line in difflib.ndiff(a, b):
        print(line, end=' ')
```

```
    return 1
```

```
# crack args (sys.argv[1:] is normal) & compare;
# return false iff a problem
```

```
def main(args):
    import getopt
    try:
        opts, args = getopt.getopt(args, "qr:")
    except getopt.error as detail:
        return fail(str(detail))
    noisy = 1
    qseen = rseen = 0
    for opt, val in opts:
        if opt == "-q":
```

textwrap— Text wrapping and filling

Source code: `Lib/textwrap.py`

The `textwrap` module provides some convenience functions, as well as `TextWrapper`, the class that does all the work. If you're just wrapping or filling one or two text strings, the convenience functions should be good enough; otherwise, you should use an instance of `TextWrapper` for efficiency.

`textwrap.wrap(text,width=70,* ,initial_indent="",subsequent_indent="",expand_tabs=True,replace_whitespace=True,fix_sentence_endings=False,break_long_words=True,drop_whitespace=True,break_on_hyphens=True,tabsize=8,max_lines=None,placeholder='[...]')`

Wraps the single paragraph `text` (a string) so every line is at most `width` characters long. Returns a list of output lines, without final newlines. Optional keyword arguments correspond to the instance attributes of `TextWrapper`, documented below. See the `TextWrapper.wrap()` method for additional details on how `wrap()` behaves.

`textwrap.fill(text,width=70,* ,initial_indent="",subsequent_indent="",expand_tabs=True,replace_whitespace=True,fix_sentence_endings=False,break_long_words=True,drop_whitespace=True,break_on_hyphens=True,tabsize=8,max_lines=None,placeholder='[...]')`

Wraps the single paragraph `text`, and returns a single string containing the wrapped paragraph. `fill()` is shorthand for `"\n".join(wrap(text,...))`. In particular, `fill()` accepts exactly the same keyword arguments as `wrap()`.

`textwrap.shorten(text,width,* ,fix_sentence_endings=False,break_long_words=True,break_on_hyphens=True,placeholder='[...]')`

Collapse and truncate the given `text` to fit in the given `width`. First the whitespace in `text` is collapsed (all whitespace is replaced by single spaces). If the result fits in the `width`, it is returned. Otherwise, enough words are dropped from the end so that the remaining words plus the `placeholder` fit within `width`:
>>> `textwrap.shorten("Hello world!",width=12)` 'Hello world!'
>>> `textwrap.shorten("Hello world!",width=11)` 'Hello [...]'
>>> `textwrap.shorten("Hello world",width=10,placeholder="...")` 'Hello... 'Opti

`textwrap.dedent(text)`

Remove any common leading whitespace from every line in `text`. This can be used to make triple-quoted strings line up with the left edge of the display, while still presenting them in the source code in indented form. Note that tabs and spaces are both treated as whitespace, but they are not equal: the lines `"hello"` and `"\thello"` are considered to have no common leading whitespace. Lines containing only whitespace are ignored in the input and normalized to a single newline character in the output. For example:

`textwrap.indent(text,prefix,predicate=None)`

Add `prefix` to the beginning of selected lines in `text`. Lines are separated by calling `text.splitlines(True)`. By default, `prefix` is added to all lines that do not consist solely of whitespace (including any line endings). For example:
>>> `s='hello\n\n\nworld'`
>>> `indent(s,' ')` ' hello\n\n\n world'
The optional `predicate` argument can be used to control which lines are indented. For example, it is easy to add `prefix` to even empty and whitespace-only lines:
>>> `print(indent(s,' ',lambda line: True))`
+ hello+++ worldAd

`wrap()`, `fill()` and `shorten()` work by creating a `TextWrapper` instance and calling a single method on it. That instance is not reused, so for applications that process many text strings using `wrap()` and/or `fill()`, it may be more efficient to create your own `TextWrapper` object.

Text is preferably wrapped on whitespaces and right after the hyphens in hyphenated words; only then will long words be broken if necessary, unless `TextWrapper.break_long_words` is set to `false`.

`class textwrap.TextWrapper(**kwargs)`

width

expand_tabs

tabsize

replace_whitespace

drop_whitespace

initial_indent

subsequent_indent

fix_sentence_endings

break_long_words

break_on_hyphens

max_lines

placeholder

wrap(text)

fill(text)

The `TextWrapper` constructor accepts a number of optional keyword arguments. Each keyword argument corresponds to an instance attribute, so for example `wrapper=TextWrapper(initial_indent="* ")` is the same as `wrapper=TextWrapper(); wrapper.initial_indent="* "`. You can reuse the same `TextWrapper` object many times, and you can change any of its options through direct assignment to instance attributes between uses. The `TextWrapper` instance attributes (and keyword arguments to the constructor) are as follows:

width (default: 70) The maximum length of wrapped lines. As long as there are no individual words in the input text longer than `width`, `TextWrapper` guarantees that no output line will be longer than `width` characters.

expand_tabs (default: True) If true, then all tab characters in `text` will be expanded to spaces using the `expandtabs()` method of `text`.

tabsize (default: 8) If `expand_tabs` is true, then all tab characters in `text` will be expanded to zero or more spaces, depending on the current column and the given tab size. Added in version 3.3.

replace_whitespace (default: True) If true, after tab expansion but before wrapping, the `wrap()` method will replace each whitespace character with a single space. The whitespace characters replaced are as follows: tab, newline, vertical tab, form feed, and carriage return (`'\t\n\v\f\r'`). Note if `expand_tabs` is false and `replace_whitespace` is true, each tab character will be replaced by a single space, which is not the same as tab expansion. Note if `replace_whitespace` is false, newlines may appear in the middle of a line and cause

drop_whitespace (default: True) If true, whitespace at the beginning and ending of every line (after wrapping but before indenting) is dropped. Whitespace at the beginning of the paragraph, however, is not dropped if non-whitespace follows it. If whitespace being dropped takes up an entire line, the whole line is dropped.

initial_indent (default: "") String that will be prepended to the first line of wrapped output. Counts towards the length of the first line. The empty string is not indented.

subsequent_indent (default: "") String that will be prepended to all lines of wrapped output except the first. Counts towards the length of each line except the first.

fix_sentence_endings (default: False) If true, `TextWrapper` attempts to detect sentence endings and ensure that sentences are always separated by exactly two spaces. This is generally desired for text in a monospaced font. However, the sentence detection algorithm is imperfect: it assumes that a sentence ending consists of a lowercase letter followed by one of `','`, `','`, or `'?'`, possibly followed by one of `'''` or `"""`, followed by a space. One problem with this

algorithm is that it is unable to detect the difference between “

(default:True) If true, then words longer thanwidthwill be broken in order to ensure that no lines are longer thanwidth. If it is false, long words will not be broken, and some lines may be longer thanwidth. (Long words will be put on a line by themselves, in order to minimize the amount by whichwidththis exceeded.)

(default:True) If true, wrapping will occur preferably on whitespaces and right after hyphens in compound words, as it is customary in English. If false, only whitespaces will be considered as potentially good places for line breaks, but you need to setbreak_long_wordsto false if you want truly insecable words. Default behaviour in previous versions was to always allow breaking hyphenated words.

(default:None) If notNone, then the output will contain at mostmax_lineslines, withplaceholderappearing at the end of the output.Added in version 3.4.

(default:'[...]') String that will appear at the end of the output text if it has been truncated.Added in version 3.4.

Wraps the single paragraph intext(a string) so every line is at mostwidthcharacters long. All wrapping options are taken from instance attributes of theTextWrapperinstance. Returns a list of output lines, without final newlines. If the wrapped output has no content, the returned list is empty.

Wraps the single paragraph intext, and returns a single string containing the wrapped paragraph.

unicodedata— Unicode Database

This module provides access to the Unicode Character Database (UCD) which defines character properties for all Unicode characters. The data contained in this database is compiled from the UCD version 16.0.0.

The module uses the same names and symbols as defined by Unicode Standard Annex #44, “Unicode Character Database”. It defines the following functions:

See also

The Unicode HOWTO for more information about Unicode and how to use this module.

unicodedata.lookup(name)

Look up character by name. If a character with the given name is found, return the corresponding character. If not found, `KeyError` is raised. For example: `>>>unicodedata.lookup('LEFT CURLY BRACKET')'` `{` The characters returned by this function are the same as those produced by `\N` escape sequence in string literals. For example: `>>>unicodedata.lookup('MIDDLE DOT')=='\N{MIDDLE DOT}'` `True` Changed in version 3.3: Support for name aliases [1] and named sequences [2] has been added.

unicodedata.name(chr,default=None,/)

Returns the name assigned to the character `chr` as a string. If no name is defined, `default` is returned, or, if not given, `ValueError` is raised. For example: `>>>unicodedata.name('½')` `VULGAR FRACTION ONE HALF'` `>>>unicodedata.name('\uFFFF','fallback')` `'fallback'`

unicodedata.decimal(chr,default=None,/)

Returns the decimal value assigned to the character `chr` as integer. If no such value is defined, `default` is returned, or, if not given, `ValueError` is raised. For example: `>>>unicodedata.decimal('\N{ARABIC-INDIC DIGIT NINE}')` `9` `>>>unicodedata.decimal('\N{SUPERSCRIPT NINE}')` `-1`

unicodedata.digit(chr,default=None,/)

Returns the digit value assigned to the character `chr` as integer. If no such value is defined, `default` is returned, or, if not given, `ValueError` is raised: `>>>unicodedata.digit('\N{SUPERSCRIPT NINE}')` `9`

unicodedata.numeric(chr,default=None,/)

Returns the numeric value assigned to the character `chr` as float. If no such value is defined, `default` is returned, or, if not given, `ValueError` is raised: `>>>unicodedata.numeric('½')` `0.5`

unicodedata.category(chr)

Returns the general category assigned to the character `chr` as string. General category names consist of two letters. See the General Category Values section of the Unicode Character Database documentation for a list of category codes. For example: `>>>unicodedata.category('A')` `# 'L'etter, 'u'ppercase'Lu'`

unicodedata.bidirectional(chr)

Returns the bidirectional class assigned to the character `chr` as string. If no such value is defined, an empty string is returned. See the Bidirectional Class Values section of the Unicode Character Database documentation for a list of bidirectional codes. For example: `>>>unicodedata.bidirectional('\N{ARABIC-INDIC DIGIT SEVEN}')` `# 'A'rabic, 'N'umber'AN'`

unicodedata.combining(chr)

Returns the canonical combining class assigned to the character `chr` as integer. Returns `0` if no combining class is defined. See the Canonical Combining Class Values section of the Unicode Character Database for more information.

unicodedata.east_asian_width(chr)

Returns the east asian width assigned to the character `chr` as string. For a list of widths and or more information, see the [Unicode Standard Annex #11](#).

unicodedata.mirrored(chr)

Returns the mirrored property assigned to the character `chr` as integer. Returns 1 if the character has been identified as a “mirrored” character in bidirectional text, 0 otherwise. For example: `>>>unicodedata.mirrored('>')1`

unicodedata.decomposition(chr)

Returns the character decomposition mapping assigned to the character `chr` as string. An empty string is returned in case no such mapping is defined. For example: `>>>unicodedata.decomposition('Ã')'0041 0303'`

unicodedata.normalize(form,unistr)

Return the normal form `form` for the Unicode string `unistr`. Valid values for `form` are 'NFC', 'NFKC', 'NFD', and 'NFKD'. The Unicode standard defines various normalization forms of a Unicode string, based on the definition of canonical equivalence and compatibility equivalence. In Unicode, several characters can be expressed in various way. For example, the character U+00C7 (LATIN CAPITAL LETTER C WITH CEDILLA) can also be expressed as the sequence U+0043 (LATIN CAPITAL LETTER C) U+0327 (COMBINING CEDIL

unicodedata.is_normalized(form,unistr)

Return whether the Unicode string `unistr` is in the normal form `form`. Valid values for `form` are 'NFC', 'NFKC', 'NFD', and 'NFKD'. Added in version 3.8.

In addition, the module exposes the following constant:

unicodedata.unidata_version

The version of the Unicode database used in this module.

unicodedata.ucd_3_2_0

This is an object that has the same methods as the entire module, but uses the Unicode database version 3.2 instead, for applications that require this specific version of the Unicode database (such as IDNA).

Footnotes

<https://www.unicode.org/Public/16.0.0/ucd/NameAliases.txt>

<https://www.unicode.org/Public/16.0.0/ucd/NamedSequences.txt>

stringprep— Internet String Preparation

Source code: `Lib/stringprep.py`

When identifying things (such as host names) in the internet, it is often necessary to compare such identifications for “equality”. Exactly how this comparison is executed may depend on the application domain, e.g. whether it should be case-insensitive or not. It may be also necessary to restrict the possible identifications, to allow only identifications consisting of “printable” characters.

RFC 3454 defines a procedure for “preparing” Unicode strings in internet protocols. Before passing strings onto the wire, they are processed with the preparation procedure, after which they have a certain normalized form. The RFC defines a set of tables, which can be combined into profiles. Each profile must define which tables it uses, and what other optional parts of the stringprep procedure are part of the profile. One example of a stringprep profile is `isnameprep`, which is used for internationalized domain names.

The module `stringprep` only exposes the tables from RFC 3454. As these tables would be very large to represent as dictionaries or lists, the module uses the Unicode character database internally. The module source code itself was generated using the `mkstringprep.py` utility.

As a result, these tables are exposed as functions, not as data structures. There are two kinds of tables in the RFC: sets and mappings. For a set, `stringprep` provides the “characteristic function”, i.e. a function that returns `True` if the parameter is part of the set. For mappings, it provides the mapping function: given the key, it returns the associated value. Below is a list of all functions available in the module.

`stringprep.in_table_a1(code)`

Determine whether `code` is in table A.1 (Unassigned code points in Unicode 3.2).

`stringprep.in_table_b1(code)`

Determine whether `code` is in table B.1 (Commonly mapped to nothing).

`stringprep.map_table_b2(code)`

Return the mapped value for `code` according to table B.2 (Mapping for case-folding used with NFKC).

`stringprep.map_table_b3(code)`

Return the mapped value for `code` according to table B.3 (Mapping for case-folding used with no normalization).

`stringprep.in_table_c11(code)`

Determine whether `code` is in table C.1.1 (ASCII space characters).

`stringprep.in_table_c12(code)`

Determine whether `code` is in table C.1.2 (Non-ASCII space characters).

`stringprep.in_table_c11_c12(code)`

Determine whether `code` is in table C.1 (Space characters, union of C.1.1 and C.1.2).

`stringprep.in_table_c21(code)`

Determine whether `code` is in table C.2.1 (ASCII control characters).

`stringprep.in_table_c22(code)`

Determine whether `code` is in table C.2.2 (Non-ASCII control characters).

`stringprep.in_table_c21_c22(code)`

Determine whether `code` is in table C.2 (Control characters, union of C.2.1 and C.2.2).

stringprep.in_table_c3(code)

Determine whethercodeis in tableC.3 (Private use).

stringprep.in_table_c4(code)

Determine whethercodeis in tableC.4 (Non-character code points).

stringprep.in_table_c5(code)

Determine whethercodeis in tableC.5 (Surrogate codes).

stringprep.in_table_c6(code)

Determine whethercodeis in tableC.6 (Inappropriate for plain text).

stringprep.in_table_c7(code)

Determine whethercodeis in tableC.7 (Inappropriate for canonical representation).

stringprep.in_table_c8(code)

Determine whethercodeis in tableC.8 (Change display properties or are deprecated).

stringprep.in_table_c9(code)

Determine whethercodeis in tableC.9 (Tagging characters).

stringprep.in_table_d1(code)

Determine whethercodeis in tableD.1 (Characters with bidirectional property “R” or “AL”).

stringprep.in_table_d2(code)

Determine whethercodeis in tableD.2 (Characters with bidirectional property “L”).

readline— GNU readline interface

The `readline` module defines a number of functions to facilitate completion and reading/writing of history files from the Python interpreter. This module can be used directly, or via the `rlcompleter` module, which supports completion of Python identifiers at the interactive prompt. Settings made using this module affect the behaviour of both the interpreter's interactive prompt and the prompts offered by the built-in `input()` function.

Readline keybindings may be configured via an initialization file, typically `.inputrc` in your home directory. See `Readline Init File` in the GNU Readline manual for information about the format and allowable constructs of that file, and the capabilities of the Readline library in general.

Availability: not Android, not iOS, not WASI.

This module is not supported on mobile platforms or WebAssembly platforms.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see `Requirements for optional modules`.

Availability: Unix.

■ **Nota:** Note The underlying Readline library API may be implemented by the `editline` (`libedit`) library instead of GNU `readline`. On macOS the `readline` module detects which library is being used at run time. The configuration file for `editline` is different from that of GNU `readline`. If you programmatically load configuration strings you can use `backend` to determine which library is being used. If you use `editline/libedit` emulation on macOS, the initialization file located in your home directory is named `.editrc`. For example, the following content in `~/.editrc` will turn ON vi keybindings and TAB completion: `python:`

`readline.backend`

The name of the underlying Readline library being used, either `"readline"` or `"editline"`. Added in version 3.13.

Init file

The following functions relate to the init file and user configuration:

`readline.parse_and_bind(string)`

Execute the init line provided in the `string` argument. This calls `rl_parse_and_bind()` in the underlying library.

`readline.read_init_file(filename)`

Execute a readline initialization file. The default filename is the last filename used. This calls `rl_read_init_file()` in the underlying library. It raises an `auditing` event `open` with the file name if given, and `"<readline_init_file>"` otherwise, regardless of which file the library resolves. Changed in version 3.14: The `auditing` event was added.

Line buffer

The following functions operate on the line buffer:

`readline.get_line_buffer()`

Return the current contents of the line buffer (`rl_line_buffer` in the underlying library).

`readline.insert_text(string)`

Insert text into the line buffer at the cursor position. This calls `rl_insert_text()` in the underlying library, but ignores the return value.

readline.redisplay()

Change what's displayed on the screen to reflect the current contents of the line buffer. This calls `rl_redisplay()` in the underlying library.

History file

The following functions operate on a history file:

readline.read_history_file([filename])

Load a readline history file, and append it to the history list. The default filename is `~/.history`. This calls `read_history_file()` in the underlying library and raises an `auditopen` event with the file name if given and `~/.history` otherwise. Changed in version 3.14: The auditing event was added.

readline.write_history_file([filename])

Save the history list to a readline history file, overwriting any existing file. The default filename is `~/.history`. This calls `write_history_file()` in the underlying library and raises an `auditopen` event with the file name if given and `~/.history` otherwise. Changed in version 3.14: The auditing event was added.

readline.append_history_file(nelements[,filename])

Append the last `nelements` items of history to a file. The default filename is `~/.history`. The file must already exist. This calls `append_history_file()` in the underlying library. This function only exists if Python was compiled for a version of the library that supports it. It raises an `auditopen` event with the file name if given and `~/.history` otherwise. Added in version 3.5. Changed in version 3.14: The auditing event was added.

readline.get_history_length()

readline.set_history_length(length)

Set or return the desired number of lines to save in the history file. The `write_history_file()` function uses this value to truncate the history file, by calling `history_truncate_file()` in the underlying library. Negative values imply unlimited history file size.

History list

The following functions operate on a global history list:

readline.clear_history()

Clear the current history. This calls `clear_history()` in the underlying library. The Python function only exists if Python was compiled for a version of the library that supports it.

readline.get_current_history_length()

Return the number of items currently in the history. (This is different from `get_history_length()`, which returns the maximum number of lines that will be written to a history file.)

readline.get_history_item(index)

Return the current contents of history item at `index`. The item index is one-based. This calls `history_get()` in the underlying library.

readline.remove_history_item(pos)

Remove history item specified by its position from the history. The position is zero-based. This calls `remove_history()` in the underlying library.

`readline.replace_history_item(pos,line)`

Replace history item specified by its position with `line`. The position is zero-based. This calls `replace_history_entry()` in the underlying library.

`readline.add_history(line)`

Append `line` to the history buffer, as if it was the last line typed. This calls `sadd_history()` in the underlying library.

`readline.set_auto_history(enabled)`

Enable or disable automatic calls to `add_history()` when reading input via `readline`. The `enabled` argument should be a Boolean value that when true, enables auto history, and that when false, disables auto history. Added in version 3.6. CPython implementation detail: Auto history is enabled by default, and changes to this do not persist across multiple sessions.

Startup hooks

`readline.set_startup_hook([function])`

Set or remove the function invoked by `th_rl_startup_hook` callback of the underlying library. If `function` is specified, it will be used as the new hook function; if omitted or `None`, any function already installed is removed. The hook is called with no arguments just before `readline` prints the first prompt.

`readline.set_pre_input_hook([function])`

Set or remove the function invoked by `th_rl_pre_input_hook` callback of the underlying library. If `function` is specified, it will be used as the new hook function; if omitted or `None`, any function already installed is removed. The hook is called with no arguments after the first prompt has been printed and just before `readline` starts reading input characters. This function only exists if Python was compiled for a version of the library that supports it.

Completion

The following functions relate to implementing a custom word completion function. This is typically operated by the Tab key, and can suggest and automatically complete a word being typed. By default, `Readline` is set up to be used by `rlcompleter` to complete Python identifiers for the interactive interpreter. If the `readline` module is to be used with a custom completer, a different set of word delimiters should be set.

`readline.set_completer([function])`

Set or remove the completer function. If `function` is specified, it will be used as the new completer function; if omitted or `None`, any completer function already installed is removed. The completer function is called as `function(text,state)`, for `state` in `0,1,2, ...`, until it returns a non-string value. It should return the next possible completion starting with `text`. The installed completer function is invoked by the `entry_func` callback passed to `rl_completion_matches()` in the underlying library. The `text` is

`readline.get_completer()`

Get the completer function, or `None` if no completer function has been set.

`readline.get_completion_type()`

Get the type of completion being attempted. This returns the `th_rl_completion_type` variable in the underlying library as an integer.

readline.get_begidx()

readline.get_endidx()

Get the beginning or ending index of the completion scope. These indexes are the `start` and `end` arguments passed to the `rl_attempted_completion_function` callback of the underlying library. The values may be different in the same input editing scenario based on the underlying C readline implementation. Ex: `libedit` is known to behave differently than `libreadline`.

readline.set_completer_delims(string)

readline.get_completer_delims()

Set or get the word delimiters for completion. These determine the start of the word to be considered for completion (the completion scope). These functions access the `rl_completer_word_break_characters` variable in the underlying library.

readline.set_completion_display_matches_hook([function])

Set or remove the completion display function. If `function` is specified, it will be used as the new completion display function; if omitted or `None`, any completion display function already installed is removed. This sets or clears the `rl_completion_display_matches_hook` callback in the underlying library. The completion display function is called as `function(substitution, [matches], longest_match_length)` once each time matches need to be displayed.

Example

The following example demonstrates how to use the `readline` module's history reading and writing functions to automatically load and save a history file named `python_history` from the user's home directory. The code below would normally be executed automatically during interactive sessions from the user's `PYTHONSTARTUP` file.

```
import atexit
import os
import readline

histfile = os.path.join(os.path.expanduser("~"), ".python_history")
try:
    readline.read_history_file(histfile)
    # default history len is -1 (infinite), which may grow unruly
    readline.set_history_length(1000)
except FileNotFoundError:
    pass

atexit.register(readline.write_history_file, histfile)
```

This code is actually automatically run when Python is run in interactive mode (see [Readline configuration](#)).

The following example achieves the same goal but supports concurrent interactive sessions, by only appending the new history.

```
import atexit
import os
import readline

histfile = os.path.join(os.path.expanduser("~"), ".python_history")

try:
    readline.read_history_file(histfile)
    h_len = readline.get_current_history_length()
except FileNotFoundError:
    open(histfile, 'wb').close()
    h_len = 0
```

```
def save(prev_h_len, histfile):
    new_h_len = readline.get_current_history_length()
    readline.set_history_length(1000)
    readline.append_history_file(new_h_len - prev_h_len, histfile)
atexit.register(save, h_len, histfile)
```

The following example extends the `code.InteractiveConsole` class to support history save/restore.

```
import atexit
import code
import os
import readline

class HistoryConsole(code.InteractiveConsole):
    def __init__(self, locals=None, filename="<console>",
                 histfile=os.path.expanduser("~/console-history")):
        code.InteractiveConsole.__init__(self, locals, filename)
        self.init_history(histfile)

    def init_history(self, histfile):
        readline.parse_and_bind("tab: complete")
        if hasattr(readline, "read_history_file"):
            try:
                readline.read_history_file(histfile)
            except FileNotFoundError:
                pass
            atexit.register(self.save_history, histfile)

    def save_history(self, histfile):
        readline.set_history_length(1000)
        readline.write_history_file(histfile)
```

■ **Nota:** NoteThe newREPLintroduced in version 3.13 doesn't support readline. However, readline can still be used by setting the `PYTHON_BASIC_REPL` environment variable.

rlcompleter— Completion function for GNU readline

Source code: `Lib/rlcompleter.py`

The `rlcompleter` module defines a completion function suitable to be passed to `set_completer()` in the `readline` module.

When this module is imported on a Unix platform with the `readline` module available, an instance of the `Completer` class is automatically created and its `complete()` method is set as the readline completer. The method provides completion of valid Python identifiers and keywords.

Example:

```
>>> import rlcompleter
>>> import readline
>>> readline.parse_and_bind("tab: complete")
>>> readline. <TAB PRESSED>
readline.__doc__      readline.get_line_buffer( readline.read_init_file(
readline.__file__     readline.insert_text(      readline.set_completer(
readline.__name__     readline.parse_and_bind(
>>> readline.
```

The `rlcompleter` module is designed for use with Python's interactive mode. Unless Python is run with the `-S` option, the module is automatically imported and configured (see `Readline configuration`).

On platforms without `readline`, the `Completer` class defined by this module can still be used for custom purposes.

class `rlcompleter.Completer`

`complete(text, state)`

Completer objects have the following method: `complete(text, state)` Return the next possible completion for `text`. When called by the `readline` module, this method is called successively with `state==0,1,2,...` until the method returns `None`. If called for `text` that doesn't include a period character ('.'), it will complete from names currently defined in `__main__`, builtins and keywords (as defined by the `keyword` module). If called for a dotted name, it will try to evaluate anything without obvious side-effects (function

Return the next possible completion for `text`. When called by the `readline` module, this method is called successively with `state==0,1,2,...` until the method returns `None`. If called for `text` that doesn't include a period character ('.'), it will complete from names currently defined in `__main__`, builtins and keywords (as defined by the `keyword` module). If called for a dotted name, it will try to evaluate anything without obvious side-effects (functions will not be evaluated, but it can generate calls to `__getattr__`

Binary Data Services

The modules described in this chapter provide some basic services operations for manipulation of binary data. Other operations on binary data, specifically in relation to file formats and network protocols, are described in the relevant sections.

Some libraries described under [Text Processing Services](#) also work with either ASCII-compatible binary formats (for example, [re](#)) or all binary data (for example, [difflib](#)).

In addition, see the documentation for Python's built-in binary data types in [Binary Sequence Types](#) — [bytes](#), [bytearray](#), [memoryview](#).

- [struct](#)— Interpret bytes as packed binary data [Functions and Exceptions](#) [Format Strings](#) [Byte Order, Size, and Alignment](#) [Format Characters](#) [Examples](#) [Applications](#) [Native Formats](#) [Standard Formats](#) [Classes](#)
- [codecs](#)— Codec registry and base classes [Codec Base Classes](#) [Error Handlers](#) [Stateless Encoding and Decoding](#) [Incremental Encoding and Decoding](#) [Incremental Encoder Objects](#) [Incremental Decoder Objects](#) [Stream Encoding and Decoding](#) [StreamWriter Objects](#) [StreamReader Objects](#) [StreamReaderWriter Objects](#) [StreamRecoder Objects](#) [Encodings and Unicode](#) [Standard Encodings](#) [Python Specific Encodings](#) [Text Encodings](#) [Binary Transforms](#) [Sta](#)

struct— Interpret bytes as packed binary data

Source code: `Lib/struct.py`

This module converts between Python values and C structs represented as Python `bytes` objects. Compact format strings describe the intended conversions to/from Python values. The module's functions and objects can be used for two largely distinct applications, data exchange with external sources (files or network connections), or data transfer between the Python application and the C layer.

■ **Nota:** Note When no prefix character is given, native mode is the default. It packs or unpacks data based on the platform and compiler on which the Python interpreter was built. The result of packing a given C struct includes pad bytes which maintain proper alignment for the C types involved; similarly, alignment is taken into account when unpacking. In contrast, when communicating data between external sources, the programmer is responsible for defining byte ordering and padding between elements. See [Byte Order, Size, and Alignment](#) for details.

Several `struct` functions (and methods of `Struct`) take a `buffer` argument. This refers to objects that implement the `Buffer` Protocol and provide either a readable or read-writable buffer. The most common types used for that purpose are `bytes` and `bytearray`, but many other types that can be viewed as an array of bytes implement the buffer protocol, so that they can be read/filled without additional copying from a `bytes` object.

Functions and Exceptions

The module defines the following exception and functions:

exception `struct.error`

Exception raised on various occasions; argument is a string describing what is wrong.

struct.pack(format, v1, v2, ...)

Return a `bytes` object containing the values `v1, v2, ...` packed according to the format string `format`. The arguments must match the values required by the format exactly.

struct.pack_into(format, buffer, offset, v1, v2, ...)

Pack the values `v1, v2, ...` according to the format string `format` and write the packed bytes into the writable buffer `buffer` starting at position `offset`. Note that `offset` is a required argument.

struct.unpack(format, buffer)

Unpack from the buffer `buffer` (presumably packed by `pack(format, ...)`) according to the format string `format`. The result is a tuple even if it contains exactly one item. The buffer's size in bytes must match the size required by the format, as reflected by `calcsizesize()`.

struct.unpack_from(format, /, buffer, offset=0)

Unpack from `buffer` starting at position `offset`, according to the format string `format`. The result is a tuple even if it contains exactly one item. The buffer's size in bytes, starting at position `offset`, must be at least the size required by the format, as reflected by `calcsizesize()`.

struct.iter_unpack(format, buffer)

Iteratively unpack from the buffer `buffer` according to the format string `format`. This function returns an iterator which will read equally sized chunks from the buffer until all its contents have been consumed. The buffer's size in bytes must be a multiple of the size required by the format, as reflected by `calcsizesize()`. Each iteration yields a tuple as specified by the format string. Added in version 3.4.

struct.calcsizesize(format)

Return the size of the struct (and hence of the bytes object produced by `byunpack(format,...)`) corresponding to the format string `format`.

Format Strings

Format strings describe the data layout when packing and unpacking data. They are built up from format characters, which specify the type of data being packed/unpacked. In addition, special characters control the byte order, size and alignment. Each format string consists of an optional prefix character which describes the overall properties of the data and one or more format characters which describe the actual data values and padding.

Byte Order, Size, and Alignment

By default, C types are represented in the machine's native format and byte order, and properly aligned by skipping pad bytes if necessary (according to the rules used by the C compiler). This behavior is chosen so that the bytes of a packed struct correspond exactly to the memory layout of the corresponding C struct. Whether to use native byte ordering and padding or standard formats depends on the application.

Alternatively, the first character of the format string can be used to indicate the byte order, size and alignment of the packed data, according to the following table:

Character	Byte order	Size	Alignment	@	native	native	native	=	native	standard
none	<	little-endian	standard	none	>	big-endian	standard	none	!	network (= big-endian)
										standard
										none

If the first character is not one of these, '@' is assumed.

■ Nota: NoteThe number 1023 (0x3ff in hexadecimal) has the following byte representations: 03ff in big-endian (>)ff03 in little-endian (<)Python example:>>>import struct>>>struct.pack('>h',1023)b'\x03\xff'>>>struct.pack('<h',1023)b'\xff\x03'

Native byte order is big-endian or little-endian, depending on the host system. For example, Intel x86, AMD64 (x86-64), and Apple M1 are little-endian; IBM z and many legacy architectures are big-endian.

Use `sys.byteorder` to check the endianness of your system.

Native size and alignment are determined using the C compiler's `sizeof expression`. This is always combined with native byte order.

Standard size depends only on the format character; see the table in the `Format Characters` section.

Note the difference between '@' and '=': both use native byte order, but the size and alignment of the latter is standardized.

The form '!' represents the network byte order which is always big-endian as defined in `IETF RFC 1700`.

There is no way to indicate non-native byte order (force byte-swapping); use the appropriate choice of '<' or '>'.

Notes:

- Padding is only automatically added between successive structure members. No padding is added at the beginning or the end of the encoded struct.
- No padding is added when using non-native size and alignment, e.g. with '<', '>', '=', and '!'.
- To align the end of a structure to the alignment requirement of a particular type, end the format with the code for that type with a repeat count of zero. See `Examples`.

Format Characters

Format characters have the following meaning; the conversion between C and Python values should be obvious given their types. The 'Standard size' column refers to the size of the packed value in bytes when using

standard size; that is, when the format string starts with one of '<','>','!' or '='. When using native size, the size of the packed value is platform-dependent.

Format	C Type	Python type	Standard size	Notes	x	pad byte	no value	(7)	c	char	
	bytes of length 1	1	b	signed	char	integer	1	(1), (2)	B	unsigned	char
	integer	1	(2)	?	_Bool	bool	1	(1)	h	short	integer
	short	integer	2	(2)	H	unsigned	short	integer	2	(2)	I
	i	int	integer	4	(2)	L	unsigned	long	integer	4	(2)
	l	long	integer	4	(2)	Q	unsigned	long	long	integer	8
	Q	unsigned	long	long	integer	8	(2)	n	ssize_t	integer	(3)
	N	size_t	integer	(3)	e	_Float16	float	2	(4), (6)	f	float
	d	double	float	8	(4)	F	float	complex	complex	8	(10)
	complex	16	(10)	s	char						

Changed in version 3.3: Added support for the 'n' and 'N' formats.

Changed in version 3.6: Added support for the 'e' format.

Changed in version 3.14: Added support for the 'F' and 'D' formats.

See also

The `array` and `ctypes` modules, as well as third-party modules like `numpy`, use similar – but slightly different – type codes.

Notes:

- The '?' conversion code corresponds to the `_Bool` type defined by C standards since C99. In standard mode, it is represented by one byte.
- When attempting to pack a non-integer using any of the integer conversion codes, if the non-integer has a `__index__()` method then that method is called to convert the argument to an integer before packing. Changed in version 3.2: Added use of the `__index__()` method for non-integers.
- The 'n' and 'N' conversion codes are only available for the native size (selected as the default or with the '@' byte order character). For the standard size, you can use whichever of the other integer formats fits your application.
- For the 'f', 'd' and 'e' conversion codes, the packed representation uses the IEEE 754 binary32, binary64 or binary16 format (for 'f', 'd' or 'e' respectively), regardless of the floating-point format used by the platform.
- The 'P' format character is only available for the native byte ordering (selected as the default or with the '@' byte order character). The byte order character '=' chooses to use little- or big-endian ordering based on the host system. The `struct` module does not interpret this as native ordering, so the 'P' format is not available.
- The IEEE 754 binary16 “half precision” type was introduced in the 2008 revision of the IEEE 754 standard. It has a sign bit, a 5-bit exponent and 11-bit precision (with 10 bits explicitly stored), and can represent numbers between approximately 6.1×10^{-5} and 6.5×10^4 at full precision. This type is not widely supported by C compilers: it's available as `_Float16` type, if the compiler supports the Annex H of
- When packing, 'x' inserts one NUL byte.
- The 'p' format character encodes a “Pascal string”, meaning a short variable-length string stored in a fixed number of bytes, given by the count. The first byte stored is the length of the string, or 255, whichever is smaller. The bytes of the string follow. If the string passed in `topack()` is too long (longer than the count minus 1), only the leading `count-1` bytes of the string are stored. If the st
- For the 's' format character, the count is interpreted as the length of the bytes, not a repeat count like for the other format characters; for example, '10s' means a single 10-byte string mapping to or from a single Python byte string, while '10c' means 10 separate one byte character elements (e.g., 'cccccccccc') mapping to or from ten different Python byte objects. (See Examples for a concrete demonstration)
- For the 'F' and 'D' format characters, the packed representation uses the IEEE 754 binary32 and binary64 format for components of the complex number, regardless of the floating-point format used by the platform. Note that

complex types (F and D) are available unconditionally, despite complex types being an optional feature in C. As specified in the C11 standard, each complex type is represented by a two

A format character may be preceded by an integral repeat count. For example, the format string '4h' means exactly the same as 'hhhh'.

Whitespace characters between formats are ignored; a count and its format must not contain whitespace though.

When packing a value using one of the integer formats ('b', 'B', 'h', 'H', 'i', 'I', 'l', 'L', 'q', 'Q'), if it is outside the valid range for that format then `struct.error` is raised.

Changed in version 3.1: Previously, some of the integer formats wrapped out-of-range values and raised `DeprecationWarning` instead of `struct.error`.

For the '?' format character, the return value is either `True` or `False`. When packing, the truth value of the argument object is used. Either 0 or 1 in the native or standard bool representation will be packed, and any non-zero value will be `True` when unpacking.

Examples

■ Nota: Note Native byte order examples (designated by the '@' format prefix or lack of any prefix character) may not match what the reader's machine produces as that depends on the platform and compiler.

Pack and unpack integers of three different sizes, using big endian ordering:

```
>>> from struct import *
>>> pack(">bhl", 1, 2, 3)
b'\x01\x00\x02\x00\x00\x00\x03'
>>> unpack('>bhl', b'\x01\x00\x02\x00\x00\x00\x03')
(1, 2, 3)
>>> calcsize('>bhl')
7
```

Attempt to pack an integer which is too large for the defined field:

```
>>> pack(">h", 99999)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
struct.error: 'h' format requires -32768 <= number <= 32767
```

Demonstrate the difference between 's' and 'c' format characters:

```
>>> pack("@ccc", b'1', b'2', b'3')
b'123'
>>> pack("@3s", b'123')
b'123'
```

Unpacked fields can be named by assigning them to variables or by wrapping the result in a named tuple:

```
>>> record = b'raymond \x32\x12\x08\x01\x08'
>>> name, serialnum, school, gradelevel = unpack('<10sHHb', record)

>>> from collections import namedtuple
>>> Student = namedtuple('Student', 'name serialnum school gradelevel')
>>> Student._make(unpack('<10sHHb', record))
Student(name=b'raymond ', serialnum=4658, school=264, gradelevel=8)
```

The ordering of format characters may have an impact on size in native mode since padding is implicit. In standard mode, the user is responsible for inserting any desired padding. Note in the first pack call below that three NUL bytes were added after the packed '#' to align the following integer on a four-byte boundary. In this example, the output was produced on a little endian machine:

```
>>> pack('@ci', b'##', 0x12131415)
b'##\x00\x00\x00\x15\x14\x13\x12'
```



```
>>> pack('@ic', 0x12131415, b'#')
b'\x15\x14\x13\x12#'
>>> calcsize('@ci')
8
>>> calcsize('@ic')
5
```

The following format `'llh0l'` results in two pad bytes being added at the end, assuming the platform's longs are aligned on 4-byte boundaries:

```
>>> pack('@llh0l', 1, 2, 3)
b'\x00\x00\x00\x01\x00\x00\x00\x02\x00\x03\x00\x00'
```

See also

Modulearray

Modulejson

Modulepickle

Packed binary storage of homogeneous data.

JSON encoder and decoder.

Python object serialization.

Applications

Two main applications for the `struct` module exist, data interchange between Python and C code within an application or another application compiled using the same compiler (native formats), and data interchange between applications using agreed upon data layout (standard formats). Generally speaking, the format strings constructed for these two domains are distinct.

Native Formats

When constructing format strings which mimic native layouts, the compiler and machine architecture determine byte ordering and padding. In such cases, the `@` format character should be used to specify native byte ordering and data sizes. Internal pad bytes are normally inserted automatically. It is possible that a zero-repeat format code will be needed at the end of a format string to round up to the correct byte boundary for proper alignment of consecutive chunks of data.

Consider these two simple examples (on a 64-bit, little-endian machine):

```
>>> calcsize('@lhl')
24
>>> calcsize('@llh')
18
```

Data is not padded to an 8-byte boundary at the end of the second format string without the use of extra padding. A zero-repeat format code solves that problem:

```
>>> calcsize('@llh0l')
24
```

The `'x'` format code can be used to specify the repeat, but for native formats it is better to use a zero-repeat format like `'0l'`.

By default, native byte ordering and alignment is used, but it is better to be explicit and use the `'@'` prefix character.

Standard Formats

When exchanging data beyond your process such as networking or storage, be precise. Specify the exact byte order, size, and alignment. Do not assume they match the native order of a particular machine. For example, network byte order is big-endian, while many popular CPUs are little-endian. By defining this explicitly, the user need not care about the specifics of the platform their code is running on. The first character should typically be < or > (or !). Padding is the responsibility of the programmer. The zero-repeat format character won't work. Instead, the user must explicitly add 'x' pad bytes where needed. Revisiting the examples from the previous section, we have:

```
>>> calcsize('<qh6xq')
24
>>> pack('<qh6xq', 1, 2, 3) == pack('@lhl', 1, 2, 3)
True
>>> calcsize('@llh')
18
>>> pack('@llh', 1, 2, 3) == pack('<qqh', 1, 2, 3)
True
>>> calcsize('<qqh6x')
24
>>> calcsize('@llh0l')
24
>>> pack('@llh0l', 1, 2, 3) == pack('<qqh6x', 1, 2, 3)
True
```

The above results (executed on a 64-bit machine) aren't guaranteed to match when executed on different machines. For example, the examples below were executed on a 32-bit machine:

```
>>> calcsize('<qqh6x')
24
>>> calcsize('@llh0l')
12
>>> pack('@llh0l', 1, 2, 3) == pack('<qqh6x', 1, 2, 3)
False
```

Classes

The `struct` module also defines the following type:

`struct.Struct(format)`

`pack(v1,v2,...)`

`pack_into(buffer,offset,v1,v2,...)`

`unpack(buffer)`

`unpack_from(buffer,offset=0)`

`iter_unpack(buffer)`

`format`

`size`

Return a new `Struct` object which writes and reads binary data according to the format string `format`. Creating a `Struct` object once and calling its methods is more efficient than calling module-level functions with the same format since the format string is only compiled once. Note the compiled versions of the most recent format strings passed to the module-level functions are cached, so programs that use only a few format strings needn't worry about reusing a single `Struct` instance. Compiled `Struct` obj

Identical to the `pack()` function, using the compiled format. (`len(result)` will equal `size`.)

Identical to the `pack_into()` function, using the compiled format.

Identical to the `unpack()` function, using the compiled format. The buffer's size in bytes must equal `size`.

Identical to the `unpack_from()` function, using the compiled format. The buffer's size in bytes, starting at `position` `offset`, must be at least `size`.

Identical to the `iter_unpack()` function, using the compiled format. The buffer's size in bytes must be a multiple of `size`. Added in version 3.4.

The format string used to construct this Struct object. Changed in version 3.7: The format string type is `nowstr` instead of `bytes`.

The calculated size of the struct (and hence of the bytes object produced by the `pack()` method) corresponding to `format`.

codecs— Codec registry and base classes

Source code: `Lib/codecs.py`

This module defines base classes for standard Python codecs (encoders and decoders) and provides access to the internal Python codec registry, which manages the codec and error handling lookup process. Most standard codecs are text encodings, which encode text to bytes (and decode bytes to text), but there are also codecs provided that encode text to text, and bytes to bytes. Custom codecs may encode and decode between arbitrary types, but some module features are restricted to be used specifically with text encodings or with codecs that encode to bytes.

The module defines the following functions for encoding and decoding with any codec:

`codecs.encode(obj, encoding='utf-8', errors='strict')`

Encodes `obj` using the codec registered for `encoding`. `Errors` may be given to set the desired error handling scheme. The default error handler is 'strict' meaning that encoding errors raise `ValueError` (or a more codec specific subclass, such as `UnicodeEncodeError`). Refer to `Codec Base Classes` for more information on codec error handling.

`codecs.decode(obj, encoding='utf-8', errors='strict')`

Decodes `obj` using the codec registered for `encoding`. `Errors` may be given to set the desired error handling scheme. The default error handler is 'strict' meaning that decoding errors raise `ValueError` (or a more codec specific subclass, such as `UnicodeDecodeError`). Refer to `Codec Base Classes` for more information on codec error handling.

`codecs.charmap_build(string)`

Return a mapping suitable for encoding with a custom single-byte encoding. Given a `string` of up to 256 characters representing a decoding table, returns either a compact internal mapping object `EncodingMap` or a dictionary mapping character ordinals to byte values. Raises a `TypeError` on invalid input.

The full details for each codec can also be looked up directly:

`codecs.lookup(encoding, /)`

Looks up the codec info in the Python codec registry and returns a `CodecInfo` object as defined below. Encodings are first looked up in the registry's cache. If not found, the list of registered search functions is scanned. If no `CodecInfo` object is found, a `LookupError` is raised. Otherwise, the `CodecInfo` object is stored in the cache and returned to the caller.

`class codecs.CodecInfo(encode, decode, streamreader=None, streamwriter=None, incrementalencoder=None, incrementaldecoder=None, name=None)`

`name`

`encode`

`decode`

`incrementalencoder`

`incrementaldecoder`

`streamwriter`

`streamreader`

Codec details when looking up the codec registry. The constructor arguments are stored in attributes of the same name: `name` The name of the encoding. `encode` `decode` The stateless encoding and decoding functions.

These must be functions or methods which have the same interface as the `encode()` and `decode()` methods of Codec instances (see `Codec Interface`). The functions or methods are expected to work in a stateless mode. `incrementalencoder` `incrementaldecoder` `Incremental encoder` and `decoder` classes or factory functions

The name of the encoding.

The stateless encoding and decoding functions. These must be functions or methods which have the same interface as the `encode()` and `decode()` methods of Codec instances (see `Codec Interface`). The functions or methods are expected to work in a stateless mode.

Incremental encoder and decoder classes or factory functions. These have to provide the interface defined by the base classes `IncrementalEncoder` and `IncrementalDecoder`, respectively. Incremental codecs can maintain state.

Stream writer and reader classes or factory functions. These have to provide the interface defined by the base classes `StreamWriter` and `StreamReader`, respectively. Stream codecs can maintain state.

To simplify access to the various codec components, the module provides these additional functions which use `lookup()` for the codec lookup:

`codecs.getencoder(encoding)`

Look up the codec for the given encoding and return its encoder function. Raises a `LookupError` in case the encoding cannot be found.

`codecs.getdecoder(encoding)`

Look up the codec for the given encoding and return its decoder function. Raises a `LookupError` in case the encoding cannot be found.

`codecs.getincrementalencoder(encoding)`

Look up the codec for the given encoding and return its incremental encoder class or factory function. Raises a `LookupError` in case the encoding cannot be found or the codec doesn't support an incremental encoder.

`codecs.getincrementaldecoder(encoding)`

Look up the codec for the given encoding and return its incremental decoder class or factory function. Raises a `LookupError` in case the encoding cannot be found or the codec doesn't support an incremental decoder.

`codecs.getreader(encoding)`

Look up the codec for the given encoding and return its `StreamReader` class or factory function. Raises a `LookupError` in case the encoding cannot be found.

`codecs.getwriter(encoding)`

Look up the codec for the given encoding and return its `StreamWriter` class or factory function. Raises a `LookupError` in case the encoding cannot be found.

Custom codecs are made available by registering a suitable codec search function:

`codecs.register(search_function, /)`

Register a codec search function. Search functions are expected to take one argument, being the encoding name in all lower case letters with hyphens and spaces converted to underscores, and return a `CodecInfo` object. In case a search function cannot find a given encoding, it should return `None`. Changed in version 3.9: Hyphens and spaces are converted to underscore.

`codecs.unregister(search_function, /)`

Unregister a codec search function and clear the registry's cache. If the search function is not registered, do nothing. Added in version 3.10.

While the builtin `open()` and the associated `io` module are the recommended approach for working with encoded text files, this module provides additional utility functions and classes that allow the use of a wider range of codecs when working with binary files:

`codecs.open(filename, mode='r', encoding=None, errors='strict', buffering=-1)`

Open an encoded file using the given `mode` and return an instance of `StreamReaderWriter`, providing transparent encoding/decoding. The default file mode is `'r'`, meaning to open the file in read mode. Note if `encoding` is not `None`, then the underlying encoded files are always opened in binary mode. No automatic conversion of `'\n'` is done on reading and writing. The `mode` argument may be any binary mode acceptable to the built-in `open()` function; the `'b'` is automatically added. `encoding` specifies the encoding which is t

`codecs.EncodedFile(file, data_encoding, file_encoding=None, errors='strict')`

Return a `StreamRecoder` instance, a wrapped version of `file` which provides transparent transcoding. The original file is closed when the wrapped version is closed. Data written to the wrapped file is decoded according to the given `data_encoding` and then written to the original file as bytes using `file_encoding`. Bytes read from the original file are decoded according to `file_encoding`, and the result is encoded using `data_encoding`. If `file_encoding` is not given, it defaults to `data_encoding`. `errors` may be given to

`codecs.iterencode(iterator, encoding, errors='strict', **kwargs)`

Uses an incremental encoder to iteratively encode the input provided by `iterator`. `iterator` must yield `str` objects. This function is a generator. The `errors` argument (as well as any other keyword argument) is passed through to the incremental encoder. This function requires that the codec accept `text` `str` objects to encode. Therefore it does not support bytes-to-bytes encoders such as `base64_codec`.

`codecs.iterdecode(iterator, encoding, errors='strict', **kwargs)`

Uses an incremental decoder to iteratively decode the input provided by `iterator`. `iterator` must yield `bytes` objects. This function is a generator. The `errors` argument (as well as any other keyword argument) is passed through to the incremental decoder. This function requires that the codec accept `bytes` objects to decode. Therefore it does not support text-to-text encoders such as `rot_13`, although `rot_13` may be used equivalently with `iterencode()`.

`codecs.readbuffer_encode(buffer, errors=None, /)`

Return a `tuple` containing the raw bytes of `buffer`, a `buffer-compatible` object or `str` (encoded to UTF-8 before processing), and their length in bytes. The `errors` argument is ignored. >>> `codecs.readbuffer_encode(b"Zito")(b'Zito', 4)`

The module also provides the following constants which are useful for reading and writing to platform dependent files:

`codecs.BOM`

`codecs.BOM_BE`

`codecs.BOM_LE`

`codecs.BOM_UTF8`

`codecs.BOM_UTF16`

`codecs.BOM_UTF16_BE`

`codecs.BOM_UTF16_LE`

`codecs.BOM_UTF32`

`codecs.BOM_UTF32_BE`

`codecs.BOM_UTF32_LE`

These constants define various byte sequences, being Unicode byte order marks (BOMs) for several encodings. They are used in UTF-16 and UTF-32 data streams to indicate the byte order used, and in UTF-8 as a Unicode signature. `BOM_UTF16` is either `BOM_UTF16_BE` or `BOM_UTF16_LE` depending on the platform's native byte order, `BOM` is an alias for `BOM_UTF16`, `BOM_LE` for `BOM_UTF16_LE` and `BOM_BE` for `BOM_UTF16_BE`. The others represent the BOM in UTF-8 and UTF-32 encodings.

Codec Base Classes

The `codecs` module defines a set of base classes which define the interfaces for working with codec objects, and can also be used as the basis for custom codec implementations.

Each codec has to define four interfaces to make it usable as codec in Python: stateless encoder, stateless decoder, stream reader and stream writer. The stream reader and writers typically reuse the stateless encoder/decoder to implement the file protocols. Codec authors also need to define how the codec will handle encoding and decoding errors.

Error Handlers

To simplify and standardize error handling, codecs may implement different error handling schemes by accepting the `errors` string argument:

```
>>> 'German ß, ■'.encode(encoding='ascii', errors='backslashreplace')
b'German \\xdf, \\u266c'
>>> 'German ß, ■'.encode(encoding='ascii', errors='xmlcharrefreplace')
b'German &#223;, &#9836;'
```

The following error handlers can be used with all Python Standard Encodings codecs:

Value	Meaning
'strict'	Raise <code>UnicodeError</code> (or a subclass), this is the default. Implemented in <code>strict_errors()</code> .
'ignore'	Ignore the malformed data and continue without further notice. Implemented in <code>ignore_errors()</code> .
'replace'	Replace with a replacement marker. On encoding, use ? (ASCII character). On decoding, use ■ (<code>U+FFFD</code> , the official REPLACEMENT CHARACTER). Implemented in <code>replace_errors()</code> .
'backslashreplace'	Replace with backslashed escape sequences. On encoding, use hexadecimal form of Unicode code point with format \x hh \u xxxx \U xxxxxxxx . On decoding, use hexadecimal form of byte value with format \x hh . Implemented in <code>backslashreplace_errors()</code> .
'surrogateescape'	On decoding, replace byte with individual

The following error handlers are only applicable to encoding (with text encodings):

Value	Meaning
'xmlcharrefreplace'	Replace with XML/HTML numeric character reference, which is a decimal form of Unicode code point with format &# num ; . Implemented in <code>xmlcharrefreplace_errors()</code> .
'namereplace'	Replace with \N{...} escape sequences, what appears in the braces is the Name property from Unicode Character Database. Implemented in <code>namereplace_errors()</code> .

In addition, the following error handler is specific to the given codecs:

Value	Codecs	Meaning
'surrogatepass'	utf-8, utf-16, utf-32, utf-16-be, utf-16-le, utf-32-be, utf-32-le	Allow encoding and decoding surrogate code point (<code>U+D800</code> - <code>U+DFFF</code>) as normal code point. Otherwise these codecs treat the presence of surrogate code point in str as an error.

Added in version 3.1: The 'surrogateescape' and 'surrogatepass' error handlers.

Changed in version 3.4: The 'surrogatepass' error handler now works with utf-16* and utf-32* codecs.

Added in version 3.5: The 'namereplace' error handler.

Changed in version 3.5: The 'backslashreplace' error handler now works with decoding and translating.

The set of allowed values can be extended by registering a new named error handler:

codecs.register_error(name,error_handler,)

Register the error handling function `error_handler` under the name `name`. The `error_handler` argument will be called during encoding and decoding in case of an error, when `name` is specified as the `errors` parameter. For encoding, `error_handler` will be called with a `UnicodeEncodeError` instance, which contains information about the location of the error. The error handler must either raise this or a different exception, or return a tuple with a replacement for the unencodable part of the input and a position where

Previously registered error handlers (including the standard error handlers) can be looked up by name:

codecs.lookup_error(name,)

Return the error handler previously registered under the name `name`. Raises a `LookupError` in case the handler cannot be found.

The following standard error handlers are also made available as module level functions:

codecs.strict_errors(exception)

Implements the 'strict' error handling. Each encoding or decoding error raises a `UnicodeError`.

codecs.ignore_errors(exception)

Implements the 'ignore' error handling. Malformed data is ignored; encoding or decoding is continued without further notice.

codecs.replace_errors(exception)

Implements the 'replace' error handling. Substitutes `'?'` (ASCII character) for encoding errors or `■` (U+FFFD, the official REPLACEMENT CHARACTER) for decoding errors.

codecs.backslashreplace_errors(exception)

Implements the 'backslashreplace' error handling. Malformed data is replaced by a backslashed escape sequence. On encoding, use the hexadecimal form of Unicode code point with format `\xhh\uxxxx\Uxxxxxxxx`. On decoding, use the hexadecimal form of byte value with format `\xhh`. Changed in version 3.5: Works with decoding and translating.

codecs.xmlcharrefreplace_errors(exception)

Implements the 'xmlcharrefreplace' error handling (for encoding with `text encoding` only). The unencodable character is replaced by an appropriate XML/HTML numeric character reference, which is a decimal form of Unicode code point with format `&#num;`.

codecs.namereplace_errors(exception)

Implements the 'namereplace' error handling (for encoding with `text encoding` only). The unencodable character is replaced by a `\N{...}` escape sequence. The set of characters that appear in the braces is the Name property from Unicode Character Database. For example, the German lowercase letter 'ß' will be converted to byte sequence `\N{LATINSMALLLETTERSHARPS}`. Added in version 3.5.

Stateless Encoding and Decoding

The base `Codec` class defines these methods which also define the function interfaces of the stateless encoder and decoder:

class codecs.Codec

encode(input,errors='strict')

decode(input,errors='strict')

`encode(input,errors='strict')` Encodes the object `input` and returns a tuple (output object, length consumed). For instance, `text encoding` converts a string object to a bytes object using a particular character set encoding

(e.g.,cp1252oriso-8859-1).Theerrorsargument defines the error handling to apply. It defaults to'strict'handling.The method may not store state in theCodecinstance. UseStreamWriterfor codecs which have to keep state in order to make encoding efficient.The encoder must be able to hand

Encodes the objectinputand returns a tuple (output object, length consumed). For instance,text encodingconverts a string object to a bytes object using a particular character set encoding (e.g.,cp1252oriso-8859-1).Theerrorsargument defines the error handling to apply. It defaults to'strict'handling.The method may not store state in theCodecinstance. UseStreamWriterfor codecs which have to keep state in order to make encoding efficient.The encoder must be able to handle zero length input and retu

Decodes the objectinputand returns a tuple (output object, length consumed). For instance, for atext encoding, decoding converts a bytes object encoded using a particular character set encoding to a string object.For text encodings and bytes-to-bytes codecs,inputmust be a bytes object or one which provides the read-only buffer interface – for example, buffer objects and memory mapped files.Theerrorsargument defines the error handling to apply. It defaults to'strict'handling.The method may not st

Incremental Encoding and Decoding

TheIncrementalEncoderandIncrementalDecoderclasses provide the basic interface for incremental encoding and decoding. Encoding/decoding the input isn't done with one call to the stateless encoder/decoder function, but with multiple calls to theencode()/decode()method of the incremental encoder/decoder. The incremental encoder/decoder keeps track of the encoding/decoding process during method calls.

The joined output of calls to theencode()/decode()method is the same as if all the single inputs were joined into one, and this input was encoded/decoded with the stateless encoder/decoder.

IncrementalEncoder Objects

TheIncrementalEncoderclass is used for encoding an input in multiple steps. It defines the following methods which every incremental encoder must define in order to be compatible with the Python codec registry.

classcodecs.IncrementalEncoder(errors='strict')

encode(object,final=False)

reset()

getstate()

setstate(state)

Constructor for anIncrementalEncoderinstance.All incremental encoders must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry.TheIncrementalEncodermay implement different error handling schemes by providing theerrorskeyword argument. SeeError Handlersfor possible values.Theerrorsargument will be assigned to an attribute of the same name. Assigning to this attribute makes it possible to switch

Encodesobject(taking the current state of the encoder into account) and returns the resulting encoded object. If this is the last call toencode()finalmust be true (the default is false).

Reset the encoder to the initial state. The output is discarded: call.encode(object,final=True), passing an empty byte or text string if necessary, to reset the encoder and to get the output.

Return the current state of the encoder which must be an integer. The implementation should make sure that0is the most common state. (States that are more complicated than integers can be converted into an integer by marshaling/pickling the state and encoding the bytes of the resulting string into an integer.)

Set the state of the encoder tostate.statemust be an encoder state returned bygetstate().

IncrementalDecoder Objects

The `IncrementalDecoder` class is used for decoding an input in multiple steps. It defines the following methods which every incremental decoder must define in order to be compatible with the Python codec registry.

`class codecs.IncrementalDecoder(errors='strict')`

`decode(object, final=False)`

`reset()`

`getstate()`

`setstate(state)`

Constructor for an `IncrementalDecoder` instance. All incremental decoders must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry. The `IncrementalDecoder` may implement different error handling schemes by providing the `errors` keyword argument. See [Error Handlers](#) for possible values. The `errors` argument will be assigned to an attribute of the same name. Assigning to this attribute makes it possible to switch

`decode(object)` (taking the current state of the decoder into account) and returns the resulting decoded object. If this is the last call to `decode()` `final` must be true (the default is false). If `final` is true the decoder must decode the input completely and must flush all buffers. If this isn't possible (e.g. because of incomplete byte sequences at the end of the input) it must initiate error handling just like in the stateless case (which might raise an exception).

Reset the decoder to the initial state.

Return the current state of the decoder. This must be a tuple with two items, the first must be the buffer containing the still undecoded input. The second must be an integer and can be additional state info. (The implementation should make sure that 0 is the most common additional state info.) If this additional state info is 0 it must be possible to set the decoder to the state which has no input buffered and 0 as the additional state info, so that feeding the previously buffered input to the `decode`

Set the state of the decoder to `state`. `state` must be a decoder state returned by `getstate()`.

Stream Encoding and Decoding

The `StreamWriter` and `StreamReader` classes provide generic working interfaces which can be used to implement new encoding submodules very easily. See [encodings.utf_8](#) for an example of how this is done.

StreamWriter Objects

The `StreamWriter` class is a subclass of `Codec` and defines the following methods which every stream writer must define in order to be compatible with the Python codec registry.

`class codecs.StreamWriter(stream, errors='strict')`

`write(object)`

`writelines(list)`

`reset()`

Constructor for a `StreamWriter` instance. All stream writers must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry. The `stream` argument must be a file-like object open for writing text or binary data, as appropriate for the specific codec. The `StreamWriter` may implement different error handling schemes by providing the `errors` keyword argument. See [Error Handlers](#) for the standard error handlers the underl

Writes the object's contents encoded to the stream.

Writes the concatenated iterable of strings to the stream (possibly by reusing the `write()` method). Infinite or very large iterables are not supported. The standard bytes-to-bytes codecs do not support this method.

Resets the codec buffers used for keeping internal state. Calling this method should ensure that the data on the output is put into a clean state that allows appending of new fresh data without having to rescan the whole stream to recover state.

In addition to the above methods, the `StreamWriter` must also inherit all other methods and attributes from the underlying stream.

StreamReader Objects

The `StreamReader` class is a subclass of `Codec` and defines the following methods which every stream reader must define in order to be compatible with the Python codec registry.

`class codecs.StreamReader(stream, errors='strict')`

`read(size=-1, chars=-1, firstline=False)`

`readline(size=None, keepends=True)`

`readlines(sizehint=None, keepends=True)`

`reset()`

Constructor for a `StreamReader` instance. All stream readers must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry. The `stream` argument must be a file-like object open for reading text or binary data, as appropriate for the specific codec. The `StreamReader` may implement different error handling schemes by providing the `errors` keyword argument. See [Error Handlers](#) for the standard error handlers the underl

Decodes data from the stream and returns the resulting object. The `chars` argument indicates the number of decoded code points or bytes to return. The `read()` method will never return more data than requested, but it might return less, if there is not enough available. The `size` argument indicates the approximate maximum number of encoded bytes or code points to read for decoding. The decoder can modify this setting as appropriate. The default value `-1` indicates to read and decode as much as possible. This

Read one line from the input stream and return the decoded data. `size`, if given, is passed as `size` argument to the stream's `read()` method. If `keepends` is `false` line-endings will be stripped from the lines returned.

Read all lines available on the input stream and return them as a list of lines. Line-endings are implemented using the codec's `decode()` method and are included in the list entries if `keepends` is `true`. `sizehint`, if given, is passed as the `size` argument to the stream's `read()` method.

Resets the codec buffers used for keeping internal state. Note that no stream repositioning should take place. This method is primarily intended to be able to recover from decoding errors.

In addition to the above methods, the `StreamReader` must also inherit all other methods and attributes from the underlying stream.

StreamReaderWriter Objects

The `StreamReaderWriter` is a convenience class that allows wrapping streams which work in both read and write modes.

The design is such that one can use the factory functions returned by the `lookup()` function to construct the instance.

`class codecs.StreamReaderWriter(stream, Reader, Writer, errors='strict')`

Creates a `StreamReaderWriter` instance. `stream` must be a file-like object. `Reader` and `Writer` must be factory functions or classes providing the `StreamReader` and `StreamWriter` interface resp. Error handling is done in the same way as defined for the stream readers and writers.

`StreamReaderWriter` instances define the combined interfaces of `StreamReader` and `StreamWriter` classes. They inherit all other methods and attributes from the underlying stream.

StreamRecoder Objects

The `StreamRecoder` translates data from one encoding to another, which is sometimes useful when dealing with different encoding environments.

The design is such that one can use the factory functions returned by the `lookup()` function to construct the instance.

`class codecs.StreamRecoder(stream, encode, decode, Reader, Writer, errors='strict')`

Creates a `StreamRecoder` instance which implements a two-way conversion: `encode` and `decode` work on the frontend — the data visible to code calling `read()` and `write()`, while `Reader` and `Writer` work on the backend — the data in stream. You can use these objects to do transparent transcodings, e.g., from Latin-1 to UTF-8 and back. The `stream` argument must be a file-like object. The `encode` and `decode` arguments must adhere to the `Codec` interface. `Reader` and `Writer` must be factory functions or classes providing objects of the `Stream`

`StreamRecoder` instances define the combined interfaces of `StreamReader` and `StreamWriter` classes. They inherit all other methods and attributes from the underlying stream.

Encodings and Unicode

Strings are stored internally as sequences of code points in range `U+0000–U+10FFFF`. (See PEP 393 for more details about the implementation.) Once a string object is used outside of CPU and memory, endianness and how these arrays are stored as bytes become an issue. As with other codecs, serialising a string into a sequence of bytes is known as `encoding`, and recreating the string from the sequence of bytes is known as `decoding`. There are a variety of different text serialisation codecs, which are collectively referred to as `text encodings`.

The simplest text encoding (called 'latin-1' or 'iso-8859-1') maps the code points 0–255 to the bytes `0x0–0xff`, which means that a string object that contains code points above `U+00FF` can't be encoded with this codec. Doing so will raise a `UnicodeEncodeError` that looks like the following (although the details of the error message may differ): `UnicodeEncodeError: 'latin-1' codec can't encode character '\u0123' in position 3: ordinal not in range(256)`.

There's another group of encodings (the so called `charmap` encodings) that choose a different subset of all Unicode code points and how these code points are mapped to the bytes `0x0–0xff`. To see how this is done simply open e.g. `encodings/cp1252.py` (which is an encoding that is used primarily on Windows). There's a string constant with 256 characters that shows you which character is mapped to which byte value.

All of these encodings can only encode 256 of the 1114112 code points defined in Unicode. A simple and straightforward way that can store each Unicode code point, is to store each code point as four consecutive bytes. There are two possibilities: store the bytes in big endian or in little endian order. These two encodings are called `UTF-32-BE` and `UTF-32-LE` respectively. Their disadvantage is that if, for example, you use `UTF-32-BE` on a little endian machine you will always have to swap bytes on encoding and decoding.

Python's `UTF-16` and `UTF-32` codecs avoid this problem by using the platform's native byte order when no BOM is present. Python follows prevailing platform practice, so native-endian data round-trips without redundant byte swapping, even though the Unicode Standard defaults to big-endian when the byte order is unspecified. When these bytes are read by a CPU with a different endianness, the bytes have to be swapped. To be able to detect the endianness of a `UTF-16` or `UTF-32` byte sequence, a BOM ("Byte Order Mark") is used. This is the Unicode character `U+FEFF`. This character can be prepended to every `UTF-16` or `UTF-32` byte sequence. The byte swapped version of this character (`0xFFFE`) is an illegal character that may not appear in a Unicode text. When the first character of a `UTF-16` or `UTF-32` byte sequence is `U+FFFE`, the bytes have to be swapped on decoding.

Unfortunately the character `U+FEFF` had a second purpose as `aZEROWIDTHNO-BREAKSPACE`: a character that has no width and doesn't allow a word to be split. It can e.g. be used to give hints to a ligature algorithm. With Unicode 4.0 using `U+FEFF` as `aZEROWIDTHNO-BREAKSPACE` has been deprecated (with `U+2060` (`WORDJOINER`) assuming this role). Nevertheless Unicode software still must be able to handle `U+FEFF` in both roles: as a BOM it's a device to determine the storage layout of the encoded bytes, and vanishes once the byte sequence has been decoded into a string; as `aZEROWIDTHNO-BREAKSPACE` it's a normal character that will be decoded like any other.

There's another encoding that is able to encode the full range of Unicode characters: UTF-8. UTF-8 is an 8-bit encoding, which means there are no issues with byte order in UTF-8. Each byte in a UTF-8 byte sequence consists of two parts: marker bits (the most significant bits) and payload bits. The marker bits are a sequence of zero to four 1 bits followed by a 0 bit. Unicode characters are encoded like this (with x being payload bits, which when concatenated give the Unicode character):

Range	Encoding	U-00000000	...	U-0000007F	0xxxxxxx	U-00000080	...	U-000007FF	110xxxxx 10xxxxxx
U-00000800	...	U-0000FFFF	1110xxxx 10xxxxxx 10xxxxxx	U-00010000	...	U-0010FFFF	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx		

The least significant bit of the Unicode character is the rightmost x bit.

As UTF-8 is an 8-bit encoding no BOM is required and any `U+FEFF` character in the decoded string (even if it's the first character) is treated as `aZEROWIDTHNO-BREAKSPACE`.

Without external information it's impossible to reliably determine which encoding was used for encoding a string. Each charmap encoding can decode any random byte sequence. However that's not possible with UTF-8, as UTF-8 byte sequences have a structure that doesn't allow arbitrary byte sequences. To increase the reliability with which a UTF-8 encoding can be detected, Microsoft invented a variant of UTF-8 (that Python calls "utf-8-sig") for its Notepad program: Before any of the Unicode characters is written to the file, a UTF-8 encoded BOM (which looks like this as a byte sequence: `0xef,0xbb,0xbf`) is written. As it's rather improbable that any charmap encoded file starts with these byte values (which would e.g. map to

in iso-8859-1), this increases the probability that utf-8-sig encoding can be correctly guessed from the byte sequence. So here the BOM is not used to be able to determine the byte order used for generating the byte sequence, but as a signature that helps in guessing the encoding. On encoding the utf-8-sig codec will write `0xef,0xbb,0xbf` as the first three bytes to the file. On decoding utf-8-sig will skip those three bytes if they appear as the first three bytes in the file. In UTF-8, the use of the BOM is discouraged and should generally be avoided.

Standard Encodings

Python comes with a number of codecs built-in, either implemented as C functions or with dictionaries as mapping tables. The following table lists the codecs by name, together with a few common aliases, and the languages for which the encoding is likely used. Neither the list of aliases nor the list of languages is meant to be exhaustive. Notice that spelling alternatives that only differ in case or use a hyphen instead of an underscore are also valid aliases because they are equivalent when normalized by `normalize_encoding()`. For example, 'utf-8' is a valid alias for the 'utf_8' codec.

■ Nota: Note The below table lists the most common aliases, for a complete list refer to the source `aliases.pyfile`.

On Windows, cpXXX codecs are available for all code pages. But only codecs listed in the following table are guaranteed to exist on other platforms.

CPython implementation detail: Some common encodings can bypass the codecs lookup machinery to improve performance. These optimization opportunities are only recognized by CPython for a limited set of (case insensitive) aliases: `utf-8`, `utf8`, `latin-1`, `latin1`, `iso-8859-1`, `iso8859-1`, `mbcs` (Windows only), `ascii`, `us-ascii`, `utf-16`, `utf16`, `utf-32`, `utf32`, and the same using underscores instead of dashes. Using alternative aliases for these

encodings may result in slower execution.

Changed in version 3.6: Optimization opportunity recognized for us-ascii.

Many of the character sets support the same languages. They vary in individual characters (e.g. whether the EURO SIGN is supported or not), and in the assignment of characters to code positions. For the European languages in particular, the following variants typically exist:

- an ISO 8859 codeset
- a Microsoft Windows code page, which is typically derived from an 8859 codeset, but replaces control characters with additional graphic characters
- an IBM EBCDIC code page
- an IBM PC code page, which is ASCII compatible

```
Codec | Aliases | Languages | ascii | 646, us-ascii | English | big5 | big5-tw, csbig5 |  
Traditional Chinese | big5hkscs | big5-hkscs, hkscs | Traditional Chinese | cp037 | IBM037,  
IBM039 | English | cp273 | 273, IBM273, csIBM273 | German | Added in version 3.4. | cp424 |  
EBCDIC-CP-HE, IBM424 | Hebrew | cp437 | 437, IBM437 | English | cp500 | EBCDIC-CP-BE,  
EBCDIC-CP-CH, IBM500 | Western Europe | cp720 | Arabic | cp737 | Greek | cp775 | IBM775 | Baltic  
languages | cp850 | 850, IBM850 | Western Europe | cp852 | 852, IBM852 | Central and Eastern  
Europe | cp855 | 855, IBM855 | Belarusian, Bulgarian, Macedonian, Russian, Serbian | cp856 |  
Hebrew | cp857 | 857, IBM857 | Turkish | cp858 | 858, IBM858 | Western Europe | cp860 | 860,  
IBM860 | Portuguese | cp861 | 861, CP-IS, IBM861 | Icelandic | cp
```

Changed in version 3.4: The utf-16* and utf-32* encoders no longer allow surrogate code points (U+D800–U+DFFF) to be encoded. The utf-32* decoders no longer decode byte sequences that correspond to surrogate code points.

Changed in version 3.8: cp65001 is now an alias for utf_8.

Changed in version 3.14: On Windows, cpXXX codecs are now available for all code pages.

Python Specific Encodings

A number of predefined codecs are specific to Python, so their codec names have no meaning outside Python. These are listed in the tables below based on the expected input and output types (note that while text encodings are the most common use case for codecs, the underlying codec infrastructure supports arbitrary data transforms rather than just text encodings). For asymmetric codecs, the stated meaning describes the encoding direction.

Text Encodings

The following codecs provide str to bytes encoding and bytes-like object to str decoding, similar to the Unicode text encodings.

```
Codec | Aliases | Meaning | idna | Implement | RFC 3490 | , see also | encodings.idna | . Only |  
errors='strict' | is supported. | mbc | ansi, dbcs | Windows only: Encode the operand according  
to the ANSI codepage (CP_ACP). | oem | Windows only: Encode the operand according to the OEM  
codepage (CP_OEMCP). | Added in version 3.6. | palm | Encoding of PalmOS 3.5. | punycode |  
Implement | RFC 3492 | . Stateful codecs are not supported. | raw_unicode_escape | Latin-1  
encoding with | \u | XXXX | and | \U | XXXXXXXX | for other code points. Existing backslashes  
are not escaped in any way. It is used in the Python pickle protocol. | undefined | This Codec  
should only be used for testing purposes. | Raise an exception for all conversions, even empty  
strings. The error handler is ignored. | unic
```

Changed in version 3.8: “unicode_internal” codec is removed.

Binary Transforms

The following codecs provide binary transforms: bytes-like object to bytes mappings. They are not supported by `bytes.decode()` (which only produces `str` output).

```
Codec | Aliases | Meaning | Encoder / decoder | base64_codec | [ | 1 | ] | base64, base_64 |
Convert the operand to multiline MIME base64 (the result always includes a trailing | '\n' | ).
| Changed in version 3.4: | accepts any | bytes-like object | as input for encoding and decoding
| base64.encodebytes() | / | base64.decodebytes() | bz2_codec | bz2 | Compress the operand using
bz2. | bz2.compress() | / | bz2.decompress() | hex_codec | hex | Convert the operand to
hexadecimal representation, with two digits per byte. | binascii.b2a_hex() | / |
binascii.a2b_hex() | quopri_codec | quopri, quotedprintable, quoted_printable | Convert the
operand to MIME quoted printable. | quopri.encode() | with | quotetabs=True | / |
quopri.decode() | uu_codec | uu | Convert the operand using uuencode. | z1
```

In addition to bytes-like objects, `'base64_codec'` also accepts ASCII-only instances of `str` for decoding.

Added in version 3.2: Restoration of the binary transforms.

Changed in version 3.4: Restoration of the aliases for the binary transforms.

Standalone Codec Functions

The following functions provide encoding and decoding functionality similar to codecs, but are not available as named codecs through `codecs.encode()` or `codecs.decode()`. They are used internally (for example, by `pickle`) and behave similarly to the `string_escape` codec that was removed in Python 3.

`codecs.escape_encode(input, errors=None)`

Encode `input` using escape sequences. Similar to `howrepr()` on bytes produces escaped byte values. `input` must be a bytes object. Returns a tuple (`output`, `length`) where `output` is a bytes object and `length` is the number of bytes consumed.

`codecs.escape_decode(input, errors=None)`

Decode `input` from escape sequences back to the original bytes. `input` must be a bytes-like object. Returns a tuple (`output`, `length`) where `output` is a bytes object and `length` is the number of bytes consumed.

Text Transforms

The following codec provides a text transform: `astr to str` mapping. It is not supported by `str.encode()` (which only produces bytes output).

```
Codec | Aliases | Meaning | rot_13 | rot13 | Return the Caesar-cypher encryption of the operand.
```

Added in version 3.2: Restoration of the `rot_13` text transform.

Changed in version 3.4: Restoration of the `rot13` alias.

encodings— Encodings package

This module implements the following functions:

`encodings.normalize_encoding(encoding)`

Normalize encoding name `encoding`. Normalization works as follows: all non-alphanumeric characters except the dot used for Python package names are collapsed and replaced with a single underscore, leading and trailing underscores are removed. For example, `'-;#'` becomes `'_'`. Note that `encodings` should be ASCII only.

■ **Nota:** Note The following functions should not be used directly, except for testing purposes; `codecs.lookup()` should be used instead.

`encodings.search_function(encoding)`

Search for the codec module corresponding to the given encoding name `encoding`. This function first normalizes the encoding using `normalize_encoding()`, then looks for a corresponding alias. It attempts to import a codec module from the `encodings` package using either the alias or the normalized name. If the module is found and defines a valid `getregentry()` function that returns a `codecs.CodecInfo` object, the codec is cached and returned. If the codec module defines a `getaliases()` function any returned aliases

`encodings.win32_code_page_search_function(encoding)`

Search for a Windows code page encoding of the form `mcpXXXX`. If the code page is valid and supported, return a `codecs.CodecInfo` object for it. Availability: Windows. Added in version 3.14.

This module implements the following exception:

`exception encodings.CodecRegistryError`

Raised when a codec is invalid or incompatible.

`encodings.idna`— Internationalized Domain Names in Applications

This module implements RFC 3490 (Internationalized Domain Names in Applications) and RFC 3492 (Nameprep: A Stringprep Profile for Internationalized Domain Names (IDN)). It builds upon the `punycode` encoding and `stringprep`.

If you need the IDNA 2008 standard from RFC 5891 and RFC 5895, use the third-party `idna` module.

These RFCs together define a protocol to support non-ASCII characters in domain names. A domain name containing non-ASCII characters (such as `www.Alliancefrançaise.nu`) is converted into an ASCII-compatible encoding (ACE, such as `www.xn--alliancefranaise-npb.nu`). The ACE form of the domain name is then used in all places where arbitrary characters are not allowed by the protocol, such as DNS queries, HTTP host fields, and so on. This conversion is carried out in the application; if possible invisible to the user: The application should transparently convert Unicode domain labels to IDNA on the wire, and convert back ACE labels to Unicode before presenting them to the user.

Python supports this conversion in several ways: the `idna` codec performs conversion between Unicode and ACE, separating an input string into labels based on the separator characters defined in section 3.1 of RFC 3490 and converting each label to ACE as required, and conversely separating an input byte string into labels based on the separator and converting any ACE labels found into Unicode. Furthermore, the `socket` module transparently converts Unicode host names to ACE, so that applications need not be concerned about converting host names themselves when they pass them to the socket module. On top of that, modules that have host names as function parameters, such as `http.client` and `ftplib`, accept Unicode host names (`http.client` then also transparently sends an IDNA hostname in the `Host` field if it sends that field at all).

When receiving host names from the wire (such as in reverse name lookup), no automatic conversion to Unicode is performed: applications wishing to present such host names to the user should decode them to Unicode.

The module `encodings.idna` also implements the nameprep procedure, which performs certain normalizations on host names, to achieve case-insensitivity of international domain names, and to unify similar characters. The nameprep functions can be used directly if desired.

`encodings.idna.nameprep(label)`

Return the nameprepped version of `label`. The implementation currently assumes query strings, so `allow_unassigned` is `true`.

`encodings.idna.ToASCII(label)`

Convert a label to ASCII, as specified in RFC 3490. Use `STD3ASCIIRules` is assumed to be `false`.

encodings.idna.ToUnicode(label)

Convert a label to Unicode, as specified in RFC 3490.

encodings.mbcs— Windows ANSI codepage

This module implements the ANSI codepage (CP_ACP).

Availability: Windows.

Changed in version 3.2: Before 3.2, the `errors` argument was ignored; 'replace' was always used to encode, and 'ignore' to decode.

Changed in version 3.3: Support any error handler.

encodings.utf_8_sig— UTF-8 codec with BOM signature

This module implements a variant of the UTF-8 codec. On encoding, a UTF-8 encoded BOM will be prepended to the UTF-8 encoded bytes. For the stateful encoder this is only done once (on the first write to the byte stream). On decoding, an optional UTF-8 encoded BOM at the start of the data will be skipped.

Data Types

The modules described in this chapter provide a variety of specialized data types such as dates and times, fixed-type arrays, heap queues, double-ended queues, and enumerations.

Python also provides some built-in data types, in particular, `dict`, `list`, `set` and `frozenset`, and `tuple`. The `str` class is used to hold Unicode strings, and the `bytes` and `bytearray` classes are used to hold binary data.

The following modules are documented in this chapter:

- [datetime](#)— Basic date and time typesAware and naive objectsConstantsAvailable typesCommon propertiesDetermining if an object is aware or naive`timedelta`objectsExamples of usage:`timedelta`dateobjectsExamples of usage:`date``datetime`objectsExamples of usage:`datetime``time`objectsExamples of usage:`time``tzinfo`objects`time`zoneobjects`strptime()`and`strftime()`behavior`strptime()`and`strftime()`format codesTechnical detail
- [zoneinfo](#)— IANA time zone supportUsingZoneInfoData sourcesConfiguring the data sourcesCompile-time configurationEnvironment configurationRuntime configurationTheZoneInfoclassString representationsPickle serializationFunctionsGlobalsExceptions and warnings
- [calendar](#)— General calendar-related functionsCommand-line usage
- [collections](#)— Container datatypesChainMapobjectsChainMapExamples and RecipesCounterobjectsdequeobjectsdequeRecipesdefaultdictobjectsdefaultdictExamplesnamedtuple()Factory Function for Tuples with Named FieldsOrderedDictobjectsOrderedDictExamples and RecipesUserDictobjectsUserListobjectsUserStringobjects
- [collections.abc](#)— Abstract Base Classes for ContainersCollections Abstract Base ClassesCollections Abstract Base Classes – Detailed DescriptionsExamples and Recipes
- [heapq](#)— Heap queue algorithmBasic ExamplesOther ApplicationsPriority Queue Implementation NotesTheory
- [bisect](#)— Array bisection algorithmPerformance NotesSearching Sorted ListsExamples
- [array](#)— Efficient arrays of numeric values
- [weakref](#)— Weak referencesWeak Reference ObjectsExampleFinalizer ObjectsComparing finalizers with`__del__()`methods
- [types](#)— Dynamic type creation and names for built-in typesDynamic Type CreationStandard Interpreter TypesAdditional Utility Classes and FunctionsCoroutine Utility Functions
- [copy](#)— Shallow and deep copy operations
- [pprint](#)— Data pretty printerFunctionsPrettyPrinter ObjectsExample
- [reprlib](#)— Alternaterepr()implementationRepr ObjectsSubclassing Repr Objects
- [enum](#)— Support for enumerationsModule contentsData typesSupported`__dunder__`namesSupported`__sunder__`namesUtilities and decoratorsNotes
- [graphlib](#)— Functionality to operate with graph-like structuresExceptions

datetime— Basic date and time types

Source code: `Lib/datetime.py`

The `datetime` module supplies classes for manipulating dates and times.

While date and time arithmetic is supported, the focus of the implementation is on efficient attribute extraction for output formatting and manipulation.

Tip

Skip to the format codes.

See also

Module `calendar`

Module `time`

Module `zoneinfo`

Package `dateutil`

Package `DateType`

General calendar related functions.

Time access and conversions.

Concrete time zones representing the IANA time zone database.

Third-party library with expanded time zone and parsing support.

Third-party library that introduces distinct static types to for example, allow static type checkers to differentiate between naive and aware datetimes.

Aware and naive objects

Date and time objects may be categorized as “aware” or “naive” depending on whether or not they include time zone information.

With sufficient knowledge of applicable algorithmic and political time adjustments, such as time zone and daylight saving time information, an aware object can locate itself relative to other aware objects. An aware object represents a specific moment in time that is not open to interpretation.^[1]

A naive object does not contain enough information to unambiguously locate itself relative to other date/time objects. Whether a naive object represents Coordinated Universal Time (UTC), local time, or time in some other time zone is purely up to the program, just like it is up to the program whether a particular number represents metres, miles, or mass. Naive objects are easy to understand and to work with, at the cost of ignoring some aspects of reality.

For applications requiring aware objects, `datetime` and `time` objects have an optional time zone information attribute, `tzinfo`, that can be set to an instance of a subclass of the abstract `tzinfo` class. These `tzinfo` objects capture information about the offset from UTC time, the time zone name, and whether daylight saving time is in effect.

Only one concrete `tzinfo` class, the `timezone` class, is supplied by the `datetime` module. The `timezone` class can represent simple time zones with fixed offsets from UTC, such as UTC itself or North American EST and EDT time zones. Supporting time zones at deeper levels of detail is up to the application. The rules for time adjustment across the world are more political than rational, change frequently, and there is no standard

suitable for every application aside from UTC.

Constants

The `datetime` module exports the following constants:

`datetime.MINYEAR`

The smallest year number allowed in a `date` or `datetime` object. `MINYEAR` is 1.

`datetime.MAXYEAR`

The largest year number allowed in a `date` or `datetime` object. `MAXYEAR` is 9999.

`datetime.UTC`

Alias for the UTC time zone singleton `datetime.timezone.utc`. Added in version 3.11.

Available types

`class datetime.date`

An idealized naive date, assuming the current Gregorian calendar always was, and always will be, in effect. Attributes: `year`, `month`, and `day`.

`class datetime.time`

An idealized time, independent of any particular day, assuming that every day has exactly 24*60*60 seconds. (There is no notion of “leap seconds” here.) Attributes: `hour`, `minute`, `second`, `microsecond`, and `tzinfo`.

`class datetime.datetime`

A combination of a date and a time. Attributes: `year`, `month`, `day`, `hour`, `minute`, `second`, `microsecond`, and `tzinfo`.

`class datetime.timedelta`

A duration expressing the difference between two `datetime` or `date` instances to microsecond resolution.

`class datetime.tzinfo`

An abstract base class for time zone information objects. These are used by the `datetime` and `time` classes to provide a customizable notion of time adjustment (for example, to account for time zone and/or daylight saving time).

`class datetime.timezone`

A class that implements the `tzinfo` abstract base class as a fixed offset from the UTC. Added in version 3.2.

Objects of these types are immutable.

Subclass relationships:

Common properties

The `date`, `datetime`, `time`, and `timezone` types share these common features:

- Objects of these types are immutable.
- Objects of these types are hashable, meaning that they can be used as dictionary keys.
- Objects of these types support efficient pickling via the `pickle` module.

Determining if an object is aware or naive

Objects of the `datetime` type are always naive.

An object of type `datetime.datetime` may be aware or naive.

A `datetime.datetime` object is aware if both of the following hold:

- `d.tzinfo` is not `None`
- `d.tzinfo.utcoffset(d)` does not return `None`

Otherwise, `d` is naive.

A `datetime.time` object is aware if both of the following hold:

- `t.tzinfo` is not `None`
- `t.tzinfo.utcoffset(None)` does not return `None`.

Otherwise, `t` is naive.

The distinction between aware and naive doesn't apply to `timedelta` objects.

timedelta objects

A `timedelta` object represents a duration, the difference between two `datetime` or `date` instances.

`datetime.datetime.timedelta(days=0,seconds=0,microseconds=0,milliseconds=0,minutes=0,hours=0,weeks=0)`

All arguments are optional and default to 0. Arguments may be integers or floats, and may be positive or negative. Only `days`, `seconds` and `microseconds` are stored internally. Arguments are converted to those units: A millisecond is converted to 1000 microseconds. A minute is converted to 60 seconds. An hour is converted to 3600 seconds. A week is converted to 7 days. `days`, `seconds` and `microseconds` are then normalized so that the representation is unique, with $0 \leq \text{microseconds} < 1000000$ and $0 \leq \text{seconds} < 3600 \times 24$ (the

Class attributes:

`timedelta.min`

The most negative `timedelta` object, `timedelta(-999999999)`.

`timedelta.max`

The most positive `timedelta` object, `timedelta(days=999999999,hours=23,minutes=59,seconds=59,microseconds=999999)`.

`timedelta.resolution`

The smallest possible difference between non-equal `timedelta` objects, `timedelta(microseconds=1)`.

Note that, because of normalization, `timedelta.max` is greater than `-timedelta.min`. `timedelta.max` is not representable as a `timedelta` object.

Instance attributes (read-only):

`timedelta.days`

Between -999,999,999 and 999,999,999 inclusive.

`timedelta.seconds`

Between 0 and 86,399 inclusive. Caution! It is a somewhat common bug for code to unintentionally use this attribute when it is actually intended to get `total_seconds()` value instead:

```
>>> import datetime as dt
>>> duration = dt.timedelta(seconds=11235813)
>>> duration.days, duration.seconds(130,
```

```
3813)>>>duration.total_seconds()11235813.0
```

timedelta.microseconds

Between 0 and 999,999 inclusive.

Supported operations:

```
Operation | Result | t1 | = | t2 | + | t3 | Sum of | t2 | and | t3 | . Afterwards | t1 | - | t2 |
| == | t3 | and | t1 | - | t3 | == | t2 | are true. (1) | t1 | = | t2 | - | t3 | Difference of |
t2 | and | t3 | . Afterwards | t1 | == | t2 | - | t3 | and | t2 | == | t1 | + | t3 | are true.
(1)(6) | t1 | = | t2 | * | i | or | t1 | = | i | * | t2 | Delta multiplied by an integer.
Afterwards | t1 | // | i | == | t2 | is true, provided | i | != | 0 | . | In general, | t1 | * |
i | == | t1 | * | (i-1) | + | t1 | is true. (1) | t1 | = | t2 | * | f | or | t1 | = | f | * | t2 |
Delta multiplied by a float. The result is rounded to the nearest multiple of
timedelta.resolution using round-half-to-even. | f | = | t2 | / | t3 | Division (3) of overall
duration | t2 | by interval unit | t3 | . Returns a |
```

Notes:

- This is exact but may overflow.
- This is exact and cannot overflow.
- Division by zero raises `ZeroDivisionError`.
- `-timedelta.max` is not representable as a `timedelta` object.
- String representations of `timedelta` objects are normalized similarly to their internal representation. This leads to somewhat unusual results for negative `timedeltas`. For example:

```
>>>timedelta(hours=-5)+datetime.timedelta(days=-1, seconds=68400)>>>print(_)-1 day, 19:00:00
```
- The expression `t2-t3` will always be equal to the expression `t2+(-t3)` except when `t3` is equal to `timedelta.max`; in that case the former will produce a result while the latter will overflow.

In addition to the operations listed above, `timedelta` objects support certain additions and subtractions with `date` and `datetime` objects (see below).

Changed in version 3.2: Floor division and true division of a `timedelta` object by another `timedelta` object are now supported, as are remainder operations and the `divmod()` function. True division and multiplication of a `timedelta` object by a `float` object are now supported.

`timedelta` objects support equality and order comparisons.

In Boolean contexts, a `timedelta` object is considered to be true if and only if it isn't equal to `timedelta(0)`.

Instance methods:

timedelta.total_seconds()

Return the total number of seconds contained in the duration. Equivalent to `td.total_seconds()`. For interval units other than seconds, use the division form directly (for example, `td/timedelta(seconds=1)`). Note that for very large time intervals (greater than 270 years on most platforms) this method will lose microsecond accuracy. Added in version 3.2.

Examples of usage:timedelta

An additional example of normalization:

```
>>> # Components of another_year add up to exactly 365 days
>>> import datetime as dt
>>> year = dt.timedelta(days=365)
>>> another_year = dt.timedelta(weeks=40, days=84, hours=23,
...                             minutes=50, seconds=600)
>>> year == another_year
True
```

```
>>> year.total_seconds()
31536000.0
```

Examples of timedelta arithmetic:

```
>>> import datetime as dt
>>> year = dt.timedelta(days=365)
>>> ten_years = 10 * year
>>> ten_years
datetime.timedelta(days=3650)
>>> ten_years.days // 365
10
>>> nine_years = ten_years - year
>>> nine_years
datetime.timedelta(days=3285)
>>> three_years = nine_years // 3
>>> three_years, three_years.days // 365
(datetime.timedelta(days=1095), 3)
```

dateobjects

A date object represents a date (year, month and day) in an idealized calendar, the current Gregorian calendar indefinitely extended in both directions.

January 1 of year 1 is called day number 1, January 2 of year 1 is called day number 2, and so on.[2]

class datetime.date(year, month, day)

All arguments are required. Arguments must be integers, in the following ranges: MINYEAR ≤ year ≤ MAXYEAR, 1 ≤ month ≤ 12, 1 ≤ day ≤ number of days in the given month and year. If an argument outside those ranges is given, ValueError is raised.

Other constructors, all class methods:

classmethod date.today()

Return the current local date. This is equivalent to date.fromtimestamp(time.time()).

classmethod date.fromtimestamp(timestamp)

Return the local date corresponding to the POSIX timestamp, such as is returned by time.time(). This may raise OverflowError, if the timestamp is out of the range of values supported by the platform C localtime() function, and OSError on localtime() failure. It's common for this to be restricted to years from 1970 through 2038. Note that on non-POSIX systems that include leap seconds in their notion of a timestamp, leap seconds are ignored by fromtimestamp(). Changed in version 3.3: Raise OverflowError instead

classmethod date.fromordinal(ordinal)

Return the date corresponding to the proleptic Gregorian ordinal, where January 1 of year 1 has ordinal 1. ValueError is raised unless 1 ≤ ordinal ≤ date.max.toordinal(). For any dated, date.fromordinal(d.toordinal()) == d.

classmethod date.fromisoformat(date_string)

Return a date corresponding to date_string given in any valid ISO 8601 format, with the following exceptions: Reduced precision dates are not currently supported (YYYY-MM, YYYY). Extended date representations are not currently supported (±YYYYYY-MM-DD). Ordinal dates are not currently supported (YYYY-OOO). Examples: >>> import datetime as dt >>> dt.date.fromisoformat('2019-12-04') datetime.date(2019, 12, 4) >>> dt.date.fromisoformat('20191204') datetime.date(2019, 12, 4) >>> dt.date.fromisoformat('2021-W01-1') datet

classmethod date.fromisocalendar(year, week, day)

Return a date corresponding to the ISO calendar date specified by year, week and day. This is the inverse of the function `date.isocalendar()`. Added in version 3.8.

class method `date.strptime(date_string, format)`

Return a date corresponding to `date_string`, parsed according to `format`. This is equivalent to: `date(*(time.strptime(date_string, format)[0:3]))`. `ValueError` is raised if the `date_string` and `format` can't be parsed by `time.strptime()` or if it returns a value which isn't a time tuple. See also `strptime()` and `strptime()` behavior and `date.fromisoformat()`. Note if `format` specifies a day of month without a year a `DeprecationWarning` is emitted. This is to avoid a quadrennial leap year bug in code seeking to parse only a m

Class attributes:

`date.min`

The earliest representable date, `date(MINYEAR, 1, 1)`.

`date.max`

The latest representable date, `date(MAXYEAR, 12, 31)`.

`date.resolution`

The smallest possible difference between non-equal date objects, `timedelta(days=1)`.

Instance attributes (read-only):

`date.year`

Between `MINYEAR` and `MAXYEAR` inclusive.

`date.month`

Between 1 and 12 inclusive.

`date.day`

Between 1 and the number of days in the given month of the given year.

Supported operations:

```
Operation | Result | date2 | = | date1 | + | timedelta | date2 | will be | timedelta.days | days
after | date1 | . (1) | date2 | = | date1 | - | timedelta | Computes | date2 | such that | date2
| + | timedelta | == | date1 | . (2) | timedelta | = | date1 | - | date2 | (3) | date1 | == |
date2 | date1 | != | date2 | Equality comparison. (4) | date1 | < | date2 | date1 | > | date2 |
date1 | <= | date2 | date1 | >= | date2 | Order comparison. (5)
```

Notes:

- `date2` is moved forward in time if `timedelta.days > 0`, or backward if `timedelta.days < 0`. Afterward `date2 - date1 == timedelta.days * timedelta.seconds + timedelta.microseconds` are ignored. `OverflowError` is raised if `date2.year` would be smaller than `MINYEAR` or larger than `MAXYEAR`.
- `timedelta.seconds` and `timedelta.microseconds` are ignored.
- This is exact, and cannot overflow. `timedelta.seconds` and `timedelta.microseconds` are 0, and `date2 + timedelta == date1` after.
- `date` objects are equal if they represent the same date. `date` objects that are not also `datetime` instances are never equal to `datetime` objects, even if they represent the same date.
- `date1` is considered less than `date2` when `date1` precedes `date2` in time. In other words, `date1 < date2` if and only if `date1.toordinal() < date2.toordinal()`. Order comparison between a `date` object that is not also a `datetime` instance and a `datetime` object raises `TypeError`.

Changed in version 3.13: Comparison between `datetimeobject` and an instance of the `date` subclass that is not a `datetime` subclass no longer converts the latter to `date`, ignoring the time part and the time zone. The default behavior can be changed by overriding the special comparison methods in subclasses.

In Boolean contexts, all `date` objects are considered to be true.

Instance methods:

`date.replace(year=self.year, month=self.month, day=self.day)`

Return a new `date` object with the same values, but with specified parameters updated. Example: `>>>import datetime as dt>>>d=dt.date(2002,12,31)>>>d.replace(day=26)datetime.date(2002, 12, 26)` The generic function `copy.replace()` also supports `date` objects.

`date.timetuple()`

Return a `time.struct_time` such as returned by `time.localtime()`. The hours, minutes and seconds are 0, and the DST flag is -1. `d.timetuple()` is equivalent to `time.struct_time((d.year,d.month,d.day,0,0,0,d.weekday(),yday,-1))` where `yday=d.toordinal()-date(d.year,1,1).toordinal()+1` is the day number within the current year starting with 1 for January 1st.

`date.toordinal()`

Return the proleptic Gregorian ordinal of the date, where January 1 of year 1 has ordinal 1. For any `date` object `d`, `date.fromordinal(d.toordinal())==d`.

`date.weekday()`

Return the day of the week as an integer, where Monday is 0 and Sunday is 6. For example, `date(2002,12,4).weekday()==2`, a Wednesday. See also `isoweekday()`.

`date.isoweekday()`

Return the day of the week as an integer, where Monday is 1 and Sunday is 7. For example, `date(2002,12,4).isoweekday()==3`, a Wednesday. See also `weekday()`, `isocalendar()`.

`date.isocalendar()`

Return a named tuple object with three components: year, week and weekday. The ISO calendar is a widely used variant of the Gregorian calendar.[3] The ISO year consists of 52 or 53 full weeks, and where a week starts on a Monday and ends on a Sunday. The first week of an ISO year is the first (Gregorian) calendar week of a year containing a Thursday. This is called week number 1, and the ISO year of that Thursday is the same as its Gregorian year. For example, 2004 begins on a Thursday, so the first week

`date.isoformat()`

Return a string representing the date in ISO 8601 format, YYYY-MM-DD: `>>>import datetime as dt>>>dt.date(2002,12,4).isoformat()'2002-12-04'`

`date.__str__()`

For a `date` `d`, `str(d)` is equivalent to `d.isoformat()`.

`date.ctime()`

Return a string representing the date: `>>>import datetime as dt>>>dt.date(2002,12,4).ctime()'Wed Dec 4 00:00:00 2002'` `d.ctime()` is equivalent to `time.ctime(time.mktime(d.timetuple()))` on platforms where the native `Ctime()` function (which `time.ctime()` invokes, but which `date.ctime()` does not invoke) conforms to the C standard.

`date.strftime(format)`

Return a string representing the date, controlled by an explicit format string. Format codes referring to hours, minutes or seconds will see 0 values. See also `strftime()` and `strptime()` behavior and `date.isoformat()`.

date.__format__(format)

Same as `date.strftime()`. This makes it possible to specify a format string for a date object in a formatted string literal and when using `str.format()`. See also `strftime()` and `strptime()` behavior and `date.isoformat()`.

Examples of usage: date

Example of counting days to an event:

```
>>> import time
>>> import datetime as dt
>>> today = dt.date.today()
>>> today
datetime.date(2007, 12, 5)
>>> today == dt.date.fromtimestamp(time.time())
True
>>> my_birthday = dt.date(today.year, 6, 24)
>>> if my_birthday < today:
...     my_birthday = my_birthday.replace(year=today.year + 1)
...
>>> my_birthday
datetime.date(2008, 6, 24)
>>> time_to_birthday = abs(my_birthday - today)
>>> time_to_birthday.days
202
```

More examples of working with `date`:

```
>>> import datetime as dt
>>> d = dt.date.fromordinal(730920) # 730920th day after 1. 1. 0001
>>> d
datetime.date(2002, 3, 11)

>>> # Methods related to formatting string output
>>> d.isoformat()
'2002-03-11'
>>> d.strftime("%d/%m/%y")
'11/03/02'
>>> d.strftime("%A %d. %B %Y")
'Monday 11. March 2002'
>>> d.ctime()
'Mon Mar 11 00:00:00 2002'
>>> 'The {1} is {0:%d}, the {2} is {0:%B}.'.format(d, "day", "month")
'The day is 11, the month is March.'

>>> # Methods for extracting 'components' under different calendars
>>> t = d.timetuple()
>>> for i in t:
...     print(i)
2002          # year
3             # month
11            # day
0
0
0
0             # weekday (0 = Monday)
70            # 70th day in the year
-1

>>> ic = d.isocalendar()
>>> for i in ic:
...     print(i)
2002          # ISO year
11            # ISO week number
1             # ISO day number ( 1 = Monday )
```

```
>>> # A date object is immutable; all operations produce a new object
>>> d.replace(year=2005)
datetime.date(2005, 3, 11)
```

datetimeobjects

A `datetimeobject` is a single object containing all the information from `adateobject` and `atimeobject`.

Like `adateobject`, `datetime` assumes the current Gregorian calendar extended in both directions; like `atimeobject`, `datetime` assumes there are exactly 3600×24 seconds in every day.

Constructor:

`class datetime.datetime(year, month, day, hour=0, minute=0, second=0, microsecond=0, tzinfo=None, *, fold=0)`

The `year`, `month` and `day` arguments are required. `tzinfo` may be `None`, or an instance of `tzinfo` subclass. The remaining arguments must be integers in the following ranges: $\text{MINYEAR} \leq \text{year} \leq \text{MAXYEAR}$, $1 \leq \text{month} \leq 12$, $1 \leq \text{day} \leq \text{number of days in the given month and year}$, $0 \leq \text{hour} < 24$, $0 \leq \text{minute} < 60$, $0 \leq \text{second} < 60$, $0 \leq \text{microsecond} < 1000000$, `fold` in `[0, 1]`. If an argument outside those ranges is given, `ValueError` is raised. Changed in version 3.6: Added the `fold` parameter.

Other constructors, all class methods:

`classmethod datetime.today()`

Return the current local date and time, with `tzinfo=None`. Equivalent to `datetime.fromtimestamp(time.time())`. See also `now()`, `fromtimestamp()`. This method is functionally equivalent to `now()`, but without `tz` parameter.

`classmethod datetime.now(tz=None)`

Return the current local date and time. If optional argument `tz` is `None` or not specified, this is like `today()`, but, if possible, supplies more precision than can be gotten from going through `time.time()` (for example, this may be possible on platforms supplying the `Cgettimeofday()` function). If `tz` is not `None`, it must be an instance of `tzinfo` subclass, and the current date and time are converted to `tz`'s time zone. This function is preferred over `today()` and `utcnow()`. Note: Subsequent calls to `datetime.now`

`classmethod datetime.utcnow()`

Return the current UTC date and time, with `tzinfo=None`. This is like `now()`, but returns the current UTC date and time, as a naive `datetimeobject`. An aware current UTC `datetime` can be obtained by calling `datetime.now(timezone.utc)`. See also `now()`. Warning: Because naive `datetimeobjects` are treated by many `datetime` methods as local times, it is preferred to use aware `datetimes` to represent times in UTC. As such, the recommended way to create an object representing the current time in UTC is by calling `datetime`.

`classmethod datetime.fromtimestamp(timestamp, tz=None)`

Return the local date and time corresponding to the POSIX timestamp, such as is returned by `time.time()`. If optional argument `tz` is `None` or not specified, the timestamp is converted to the platform's local date and time, and the returned `datetimeobject` is naive. If `tz` is not `None`, it must be an instance of `tzinfo` subclass, and the timestamp is converted to `tz`'s time zone. `fromtimestamp()` may raise `OverflowError`, if the timestamp is out of the range of values supported by the platform `Clocaltime()` or `gmtime()` function.

`classmethod datetime.utcfromtimestamp(timestamp)`

Return the UTC `datetime` corresponding to the POSIX timestamp, with `tzinfo=None`. (The resulting object is naive.) This may raise `OverflowError`, if the timestamp is out of the range of values supported by the platform `Cgmtime()` function, and `OS error` on `gmtime()` failure. It's common for this to be restricted to years in 1970 through 2038. To get an aware `datetimeobject`, call `fromtimestamp(): datetime.fromtimestamp(timestamp, timezone.utc)`. On

the POSIX compliant platforms, it is equivalent to the following expressi

classmethoddatetime.fromordinal(ordinal)

Return the `datetime` corresponding to the proleptic Gregorian ordinal, where January 1 of year 1 has ordinal 1. `ValueError` is raised unless $1 \leq \text{ordinal} \leq \text{datetime.max.toordinal}()$. The hour, minute, second and microsecond of the result are all 0, and `tzinfo` is `None`.

classmethoddatetime.combine(date,time,tzinfo=time.tzinfo)

Return a new `datetime` object whose date components are equal to the given `date` object's, and whose time components are equal to the given `time` object's. If the `tzinfo` argument is provided, its value is used to set the `tzinfo` attribute of the result, otherwise the `tzinfo` attribute of the `time` argument is used. If the `date` argument is a `datetime` object, its time components and `tzinfo` attributes are ignored. For any `datetime` object `d`, `d == datetime.combine(d.date(), d.time(), d.tzinfo)`. Changed in version 3.6: Added the `tzinfo` argument.

classmethoddatetime.fromisoformat(date_string)

Return a `datetime` corresponding to a `date_string` in any valid ISO 8601 format, with the following exceptions: Time zone offsets may have fractional seconds. The `T` separator may be replaced by any single unicode character. Fractional hours and minutes are not supported. Reduced precision dates are not currently supported (YYYY-MM, YYYY). Extended date representations are not currently supported ($\pm$ YYYYYY-MM-DD). Ordinal dates are not currently supported (YYYY-OOO). Examples: `>>> import datetime as dt`
`>>> dt.datetime.fromisoformat('2012-12-25T15:00')`

classmethoddatetime.fromisocalendar(year,week,day)

Return a `datetime` corresponding to the ISO calendar date specified by `year`, `week` and `day`. The non-date components of the `datetime` are populated with their normal default values. This is the inverse of the function `datetime.isocalendar()`. Added in version 3.8.

classmethoddatetime.strptime(date_string,format)

Return a `datetime` corresponding to a `date_string`, parsed according to `format`. If `format` does not contain microseconds or time zone information, this is equivalent to: `datetime(*(time.strptime(date_string,format)[0:6]))`. `ValueError` is raised if the `date_string` and `format` can't be parsed by `time.strptime()` or if it returns a value which isn't a time tuple. See also `strptime()` and `strptime()` behavior and `datetime.fromisoformat()`. Changed in version 3.13: If `format` specifies a day of month without a year, a `DeprecationWarning` is raised.

Class attributes:

datetime.min

The earliest representable `datetime`, `datetime(MINYEAR,1,1,tzinfo=None)`.

datetime.max

The latest representable `datetime`, `datetime(MAXYEAR,12,31,23,59,59,999999,tzinfo=None)`.

datetime.resolution

The smallest possible difference between non-equal `datetime` objects, `timedelta(microseconds=1)`.

Instance attributes (read-only):

datetime.year

Between `MINYEAR` and `MAXYEAR` inclusive.

datetime.month

Between 1 and 12 inclusive.

datetime.day

Between 1 and the number of days in the given month of the given year.

datetime.hour

Inrange(24).

datetime.minute

Inrange(60).

datetime.second

Inrange(60).

datetime.microsecond

Inrange(1000000).

datetime.tzinfo

The object passed as the `tzinfo` argument to the `datetime` constructor, or `None` if none was passed.

datetime.fold

In[0,1]. Used to disambiguate wall times during a repeated interval. (A repeated interval occurs when clocks are rolled back at the end of daylight saving time or when the UTC offset for the current zone is decreased for political reasons.) The values 0 and 1 represent, respectively, the earlier and later of the two moments with the same wall time representation. Added in version 3.6.

Supported operations:

```
Operation | Result | datetime2 | = | datetime1 | + | timedelta | (1) | datetime2 | = | datetime1 | - | timedelta | (2) | timedelta | = | datetime1 | - | datetime2 | (3) | datetime1 | == | datetime2 | datetime1 | != | datetime2 | Equality comparison. (4) | datetime1 | < | datetime2 | datetime1 | > | datetime2 | datetime1 | <= | datetime2 | datetime1 | >= | datetime2 | Order comparison. (5)
```

- `datetime2` is a duration of `timedelta` removed from `datetime1`, moving forward in time if `timedelta.days > 0`, or backward if `timedelta.days < 0`. The result has the same `tzinfo` attribute as the input `datetime`, and `datetime2 - datetime1 == timedelta` after. `OverflowError` is raised if `datetime2.year` would be smaller than `MINYEAR` or larger than `MAXYEAR`. Note that no time zone adjustments are done even if the input is an aware obj
- Computes the `datetime2` such that `datetime2 + timedelta == datetime1`. As for addition, the result has the same `tzinfo` attribute as the input `datetime`, and no time zone adjustments are done even if the input is aware.
- Subtraction of a `datetime` from a `datetime` is defined only if both operands are naive, or if both are aware. If one is aware and the other is naive, `TypeError` is raised. If both are naive, or both are aware and have the same `tzinfo` attribute, the `tzinfo` attributes are ignored, and the result is a `timedelta` object such that `datetime2 + t == datetime1`. No time zone adjustments are done in this case. If both are aware and have different `tzinfo` attributes, the comparison acts as if the comparands were first converted to a common time zone.
- `datetime` objects are equal if they represent the same date and time, taking into account the time zone. Naive and aware `datetime` objects are never equal. If both comparands are aware, and have the same `tzinfo` attribute, the `tzinfo` and `fold` attributes are ignored and the base datetimes are compared. If both comparands are aware and have different `tzinfo` attributes, the comparison acts as if the comparands were first converted to a common time zone.
- `datetime1` is considered less than `datetime2` when `datetime1` precedes `datetime2` in time, taking into account the time zone. Order comparison between naive and aware `datetime` objects raises `TypeError`. If both comparands are aware, and have the same `tzinfo` attribute, the `tzinfo` and `fold` attributes are ignored and the base datetimes are compared. If both comparands are aware and have different `tzinfo` attributes, the comparison acts as if the comparands were first converted to a common time zone.

Changed in version 3.3: Equality comparisons between aware and naive `datetime` instances don't raise `TypeError`.

Changed in version 3.13: Comparison between `datetime.datetime` object and an instance of the `datetime.date` subclass that is not a `datetime.datetime` subclass no longer converts the latter to a `date`, ignoring the time part and the time zone. The default behavior can be changed by overriding the special comparison methods in subclasses.

Instance methods:

`datetime.date()`

Return `datetime.datetime` object with same year, month and day.

`datetime.time()`

Return `datetime.time` object with same hour, minute, second, microsecond and fold. `tzinfo` is `None`. See also `datetime.datetime.timetz()`. Changed in version 3.6: The fold value is copied to the returned `time` object.

`datetime.timetz()`

Return `datetime.time` object with same hour, minute, second, microsecond, fold, and `tzinfo` attributes. See also `datetime.datetime.time()`. Changed in version 3.6: The fold value is copied to the returned `time` object.

`datetime.replace(year=self.year, month=self.month, day=self.day, hour=self.hour, minute=self.minute, second=self.second, microsecond=self.microsecond, tzinfo=self.tzinfo, *, fold=0)`

Return a new `datetime.datetime` object with the same attributes, but with specified parameters updated. Note that `tzinfo=None` can be specified to create a naive `datetime` from an aware `datetime` with no conversion of date and time data. `datetime` objects are also supported by generic `functioncopy.replace()`. Changed in version 3.6: Added the `fold` parameter.

`datetime.astimezone(tz=None)`

Return a `datetime.datetime` object with new `tzinfo` attribute `tz`, adjusting the date and time data so the result is the same UTC time as `self`, but in `tz`'s local time. If provided, `tz` must be an instance of `datetime.tzinfo` subclass, and its `utcoffset()` and `dst()` methods must not return `None`. If `self` is naive, it is presumed to represent time in the system time zone. If called without arguments (or with `tz=None`) the system local time zone is assumed for the target time zone. The `tzinfo` attribute of the converted `datetime` instance will be

`datetime.utcoffset()`

If `tzinfo` is `None`, returns `None`, else returns `self.tzinfo.utcoffset(self)`, and raises an exception if the latter doesn't return `None` or a `datetime.timedelta` object with magnitude less than one day. Changed in version 3.7: The UTC offset is not restricted to a whole number of minutes.

`datetime.dst()`

If `tzinfo` is `None`, returns `None`, else returns `self.tzinfo.dst(self)`, and raises an exception if the latter doesn't return `None` or a `datetime.timedelta` object with magnitude less than one day. Changed in version 3.7: The DST offset is not restricted to a whole number of minutes.

`datetime.tzname()`

If `tzinfo` is `None`, returns `None`, else returns `self.tzinfo.tzname(self)`, raises an exception if the latter doesn't return `None` or a string object,

`datetime.timetuple()`

Return a `time.struct_time` such as returned by `time.localtime()`. `datetime.timetuple()` is equivalent to `time.struct_time((d.year, d.month, d.day, d.hour, d.minute, d.second, d.weekday(), yday, dst))` where `yday=d.toordinal()-date(d.year,1,1).toordinal()+1` is the day number within the current year starting with 1 for January 1st. The `tm_isdst` flag of the result is set according to the `dst()` method: `tzinfo` is `None` or `dst()` returns `None`, `tm_isdst` is set to -1; else if `dst()` returns a non-zero value, `tm_isdst` is set to 1; else `tm_isdst` is set to

`datetime.utctimetuple()`

If `datetime.datetime.isnaive()` is naive, this is the same as `datetime.datetime.fromtimestamp(t, tz=None)` except that `tm_isdst` is forced to 0 regardless of what `d.dst()` returns. DST is never in effect for a UTC time. If `d` is aware, `d` is normalized to UTC time, by subtracting `d.utcoffset()`, and `datetime.datetime.fromtimestamp(t, tz=None)` for the normalized time is returned. `tm_isdst` is forced to 0. Note that an `OverflowError` may be raised if `d.year` was `MINYEAR` or `MAXYEAR` and UTC adjustment spills over a year boundary. Warning: Because naive `datetime` objects are treated by many `datetime` methods

datetime.toordinal()

Return the proleptic Gregorian ordinal of the date. The same as `self.date().toordinal()`.

datetime.timestamp()

Return POSIX timestamp corresponding to the `datetime` instance. The return value is a float similar to that returned by `time.time()`. Naive `datetime` instances are assumed to represent local time and this method relies on the platform `C` `mktime()` function to perform the conversion. Since `datetime` supports wider range of values than `mktime()` on many platforms, this method may raise `OverflowError` or `OSError` for times far in the past or far in the future. For aware `datetime` instances, the return value is computed as: `(dt-da`

datetime.weekday()

Return the day of the week as an integer, where Monday is 0 and Sunday is 6. The same as `self.date().weekday()`. See also `isoweekday()`.

datetime.isoweekday()

Return the day of the week as an integer, where Monday is 1 and Sunday is 7. The same as `self.date().isoweekday()`. See also `weekday()`, `isocalendar()`.

datetime.isocalendar()

Return a named tuple with three components: year, week and weekday. The same as `self.date().isocalendar()`.

datetime.isoformat(sep='T', timespec='auto')

Return a string representing the date and time in ISO 8601 format: `YYYY-MM-DDTHH:MM:SS.ffffff`, if `microsecond` is not 0; `YYYY-MM-DDTHH:MM:SS`, if `microsecond` is 0; if `utcoffset()` does not return `None`, a string is appended, giving the UTC offset: `YYYY-MM-DDTHH:MM:SS.ffffff+HH:MM[:SS[.ffffff]]`, if `microsecond` is not 0; `YYYY-MM-DDTHH:MM:SS+HH:MM[:SS[.ffffff]]`, if `microsecond` is 0. Examples: `>>>import datetime as dt>>>dt.datetime(2019,5,18,15,17,8,132263).isoformat()` `'2019-05-18T15:17:08.132263'` `>>>dt.datetime(2019,5,18,15,17,`

datetime.__str__()

For a `datetime` instance `d`, `str(d)` is equivalent to `d.isoformat()`.

datetime.ctime()

Return a string representing the date and time: `>>>import datetime as dt>>>dt.datetime(2002,12,4,20,30,40).ctime()` `'Wed Dec 4 20:30:40 2002'` The output string will not include time zone information, regardless of whether the input is aware or naive. `d.ctime()` is equivalent to `time.ctime(time.mktime(d.timetuple()))` on platforms where the native `C` `ctime()` function (which `time.ctime()` invokes, but which `datetime.ctime()` does not invoke) conforms to the C standard.

datetime.strftime(format)

Return a string representing the date and time, controlled by an explicit format string. See also `strftime()` and `strptime()` behavior and `datetime.isoformat()`.

datetime.__format__(format)

Same as `datetime.strftime()`. This makes it possible to specify a format string for a `datetime` object in a formatted string literal and when using `str.format()`. See also `strftime()` and `strptime()` behavior and `datetime.isoformat()`.

Examples of usage:datetime

Examples of working withdatetimeobjects:

```
>>> import datetime as dt

>>> # Using datetime.combine()
>>> d = dt.date(2005, 7, 14)
>>> t = dt.time(12, 30)
>>> dt.datetime.combine(d, t)
datetime.datetime(2005, 7, 14, 12, 30)

>>> # Using datetime.now()
>>> dt.datetime.now()
datetime.datetime(2007, 12, 6, 16, 29, 43, 79043) # GMT +1
>>> dt.datetime.now(dt.timezone.utc)
datetime.datetime(2007, 12, 6, 15, 29, 43, 79060, tzinfo=datetime.timezone.utc)

>>> # Using datetime.strptime()
>>> my_datetime = dt.datetime.strptime("21/11/06 16:30", "%d/%m/%y %H:%M")
>>> my_datetime
datetime.datetime(2006, 11, 21, 16, 30)

>>> # Using datetime.timetuple() to get tuple of all attributes
>>> tt = my_datetime.timetuple()
>>> for it in tt:
...     print(it)
...
2006    # year
11      # month
21      # day
16      # hour
30      # minute
0       # second
1       # weekday (0 = Monday)
325     # number of days since 1st January
-1      # dst - method tzinfo.dst() returned None

>>> # Date in ISO format
>>> ic = my_datetime.isocalendar()
>>> for it in ic:
...     print(it)
...
2006    # ISO year
47      # ISO week
2       # ISO weekday

>>> # Formatting a datetime
>>> my_datetime.strftime("%A, %d. %B %Y %I:%M%p")
'Tuesday, 21. November 2006 04:30PM'
>>> 'The {1} is {0:%d}, the {2} is {0:%B}, the {3} is {0:%I:%M%p}.'.format(my_datetime, "day", "month", "time")
'The day is 21, the month is November, the time is 04:30PM.'
```

The example below defines atzinfosubclass capturing time zone information for Kabul, Afghanistan, which used +4 UTC until 1945 and then +4:30 UTC thereafter:

```
import datetime as dt

class KabulTz(dt.tzinfo):
    # Kabul used +4 until 1945, when they moved to +4:30
    UTC_MOVE_DATE = dt.datetime(1944, 12, 31, 20, tzinfo=dt.timezone.utc)

    def utcoffset(self, when):
        if when.year < 1945:
            return dt.timedelta(hours=4)
```



```

elif (1945, 1, 1, 0, 0) <= when.timetuple()[5] < (1945, 1, 1, 0, 30):
    # An ambiguous ("imaginary") half-hour range representing
    # a 'fold' in time due to the shift from +4 to +4:30.
    # If when falls in the imaginary range, use fold to decide how
    # to resolve. See PEP 495.
    return dt.timedelta(hours=4, minutes=(30 if when.fold else 0))
else:
    return dt.timedelta(hours=4, minutes=30)

def fromutc(self, when):
    # Follow same validations as in datetime.tzinfo
    if not isinstance(when, dt.datetime):
        raise TypeError("fromutc() requires a datetime argument")
    if when.tzinfo is not self:
        raise ValueError("when.tzinfo is not self")

    # A custom implementation is required for fromutc as
    # the input to this function is a datetime with utc values
    # but with a tzinfo set to self.
    # See datetime.astimezone or fromtimestamp.
    if when.replace(tzinfo=dt.timezone.utc) >= self.UTC_MOVE_DATE:
        return when + dt.timedelta(hours=4, minutes=30)
    else:
        return when + dt.timedelta(hours=4)

def dst(self, when):
    # Kabul does not observe daylight saving time.
    return dt.timedelta(0)

def tzname(self, when):
    if when >= self.UTC_MOVE_DATE:
        return "+04:30"
    return "+04"

```

Usage ofKabulTzfrom above:

```

>>> tz1 = KabulTz()

>>> # Datetime before the change
>>> dt1 = dt.datetime(1900, 11, 21, 16, 30, tzinfo=tz1)
>>> print(dt1.utcoffset())
4:00:00

>>> # Datetime after the change
>>> dt2 = dt.datetime(2006, 6, 14, 13, 0, tzinfo=tz1)
>>> print(dt2.utcoffset())
4:30:00

>>> # Convert datetime to another time zone
>>> dt3 = dt2.astimezone(dt.timezone.utc)
>>> dt3
datetime.datetime(2006, 6, 14, 8, 30, tzinfo=datetime.timezone.utc)
>>> dt2
datetime.datetime(2006, 6, 14, 13, 0, tzinfo=KabulTz())
>>> dt2 == dt3
True

```

timeobjects

Atimeobject represents a (local) time of day, independent of any particular day, and subject to adjustment via atzinfoobject.

classdatetime.time(hour=0,minute=0,second=0,microsecond=0,tzinfo=None,*,fold=0)

All arguments are optional. `tzinfo` may be `None`, or an instance of `tzinfo` subclass. The remaining arguments must be integers in the following ranges: $0 \leq \text{hour} < 24$, $0 \leq \text{minute} < 60$, $0 \leq \text{second} < 60$, $0 \leq \text{microsecond} < 1000000$, `fold` in `[0,1]`. If an argument outside those ranges is given, `ValueError` is raised. All default to 0 except `tzinfo`, which defaults to `None`.

Class attributes:

time.min

The earliest representable time, `time(0,0,0,0)`.

time.max

The latest representable time, `time(23,59,59,999999)`.

time.resolution

The smallest possible difference between non-equal time objects, `timedelta(microseconds=1)`, although note that arithmetic on time objects is not supported.

Instance attributes (read-only):

time.hour

In range(24).

time.minute

In range(60).

time.second

In range(60).

time.microsecond

In range(1000000).

time.tzinfo

The object passed as the `tzinfo` argument to the time constructor, or `None` if none was passed.

time.fold

In `[0,1]`. Used to disambiguate wall times during a repeated interval. (A repeated interval occurs when clocks are rolled back at the end of daylight saving time or when the UTC offset for the current zone is decreased for political reasons.) The values 0 and 1 represent, respectively, the earlier and later of the two moments with the same wall time representation. Added in version 3.6.

Time objects support equality and order comparisons, where `a` is considered less than `b` when `a` precedes `b` in time.

Naive and aware time objects are never equal. Order comparison between naive and aware time objects raises `TypeError`.

If both comparands are aware, and have the same `tzinfo` attribute, the `tzinfo` and `fold` attributes are ignored and the base times are compared. If both comparands are aware and have different `tzinfo` attributes, the comparands are first adjusted by subtracting their UTC offsets (obtained from `self.utcoffset()`).

Changed in version 3.3: Equality comparisons between aware and naive time instances don't raise `TypeError`.

In Boolean contexts, a time object is always considered to be true.

Changed in version 3.5: Before Python 3.5, a time object was considered to be false if it represented midnight in UTC. This behavior was considered obscure and error-prone and has been removed in Python 3.5.

See [bpo-13936](#) for more information.

Other constructors:

classmethod`time.fromisoformat(time_string)`

Return a `time` corresponding to `time_string` in any valid ISO 8601 format, with the following exceptions: Time zone offsets may have fractional seconds. The leading `T`, normally required in cases where there may be ambiguity between a date and a time, is not required. Fractional seconds may have any number of digits (anything beyond 6 will be truncated). Fractional hours and minutes are not supported. Examples:

```
>>>import datetime as dt>>>dt.time.fromisoformat('04:23:01')datetime.time(4, 23, 1)>>>dt.time.fromis
```

classmethod`time.strptime(date_string,format)`

Return a `time` corresponding to `date_string`, parsed according to `format`. If `format` does not contain microseconds or timezone information, this is equivalent to `time(*(time.strptime(date_string,format)[3:6]))`. `ValueError` is raised if the `date_string` and `format` cannot be parsed by `time.strptime()` or if it returns a value which is not a time tuple. See also `strptime()` and `strptime()` behavior and `time.fromisoformat()`. Added in version 3.14.

Instance methods:

time.replace(hour=self.hour,minute=self.minute,second=self.second,microsecond=self.microsecond,tzinfo=self.tzinfo,*,fold=0)

Return a new `time` with the same values, but with specified parameters updated. Note that `tzinfo=None` can be specified to create a naive `time` from an aware `time`, without conversion of the time data. `time` objects are also supported by generic function `copy.replace()`. Changed in version 3.6: Added the `fold` parameter.

time.isoformat(timespec='auto')

Return a string representing the time in ISO 8601 format, one of: `HH:MM:SS.ffffff`, if `microsecond` is not `0`; `HH:MM:SS`, if `microsecond` is `0`; `HH:MM:SS.ffffff+HH:MM[:SS[.ffffff]]`, if `utcoffset()` does not return `None`; `HH:MM:SS+HH:MM[:SS[.ffffff]]`, if `microsecond` is `0` and `utcoffset()` does not return `None`. The optional argument `timespec` specifies the number of additional components of the time to include (the default is `'auto'`). It can be one of the following: `'auto'`: Same as `'seconds'` if `microsecond` is `0`, same as `'microseconds'` otherwise.

time.__str__()

For a `time`, `str(t)` is equivalent to `t.isoformat()`.

time.strftime(format)

Return a string representing the time, controlled by an explicit format string. See also `strptime()` and `strptime()` behavior and `time.isoformat()`.

time.__format__(format)

Same as `time.strftime()`. This makes it possible to specify a format string for a `time` object in formatted string literals and when using `str.format()`. See also `strptime()` and `strptime()` behavior and `time.isoformat()`.

time.utcoffset()

If `tzinfo` is `None`, returns `None`, else returns `self.tzinfo.utcoffset(None)`, and raises an exception if the latter doesn't return `None` or a `timedelta` object with magnitude less than one day. Changed in version 3.7: The UTC offset is not restricted to a whole number of minutes.

time.dst()

If `tzinfo` is `None`, returns `None`, else returns `self.tzinfo.dst(None)`, and raises an exception if the latter doesn't return `None`, or a `timedelta` object with magnitude less than one day. Changed in version 3.7: The DST offset is not restricted to a whole number of minutes.

time.tzname()

if tzinfo is None, returns None, else returns self.tzinfo.tzname(None), or raises an exception if the latter doesn't return None or a string object.

Examples of usage:time

Examples of working with a time object:

```
>>> import datetime as dt
>>> class TZ1(dt.tzinfo):
...     def utcoffset(self, when):
...         return dt.timedelta(hours=1)
...     def dst(self, when):
...         return dt.timedelta(0)
...     def tzname(self, when):
...         return "+01:00"
...     def __repr__(self):
...         return f"{self.__class__.__name__}()"
...
>>> t = dt.time(12, 10, 30, tzinfo=TZ1())
>>> t
datetime.time(12, 10, 30, tzinfo=TZ1())
>>> t.isoformat()
'12:10:30+01:00'
>>> t.dst()
datetime.timedelta(0)
>>> t.tzname()
'+01:00'
>>> t.strftime("%H:%M:%S %Z")
'12:10:30 +01:00'
>>> 'The {} is {:%H:%M}'.format("time", t)
'The time is 12:10.'
```

tzinfoobjects

class datetime.tzinfo

This is an abstract base class, meaning that this class should not be instantiated directly. Define a subclass of `tzinfo` to capture information about a particular time zone. An instance of (a concrete subclass of) `tzinfo` can be passed to the constructors for `datetime` and `time` objects. The latter objects view their attributes as being in local time, and the `tzinfo` object supports methods revealing offset of local time from UTC, the name of the time zone, and DST offset, all relative to a date or time object.

tzinfo.utcoffset(dt)

Return offset of local time from UTC, as a `timedelta` object that is positive east of UTC. If local time is west of UTC, this should be negative. This represents the total offset from UTC; for example, if a `tzinfo` object represents both time zone and DST adjustments, `utcoffset()` should return their sum. If the UTC offset isn't known, return `None`. Else the value returned must be a `timedelta` object strictly between `-timedelta(hours=24)` and `timedelta(hours=24)` (the magnitude of the offset must be less than one day).

tzinfo.dst(dt)

Return the daylight saving time (DST) adjustment, as a `timedelta` object or `None` if DST information isn't known. Return `timedelta(0)` if DST is not in effect. If DST is in effect, return the offset as a `timedelta` object (see `utcoffset()` for details). Note that DST offset, if applicable, has already been added to the UTC offset returned by `utcoffset()`, so there's no need to consult `dst()` unless you're interested in obtaining DST info separately. For example, `datetime.time tuple()` calls its `tzinfo` attribute's `dst()` method.

tzinfo.tzname(dt)

Return the time zone name corresponding to the `datetimeobject` `dt`, as a string. Nothing about string names is defined by the `datetime` module, and there's no requirement that it mean anything in particular. For example, "GMT", "UTC", "-500", "-5:00", "EDT", "US/Eastern", "America/NewYork" are all valid replies. Return `None` if a string name isn't known. Note that this is a method rather than a fixed string primarily because some `tzinfo` subclasses will wish to return different names depending on the specific value.

These methods are called by `datetime` or `time` object, in response to their methods of the same names. `datetime` object passes itself as the argument, and `time` object passes `None` as the argument. `tzinfo` subclass's methods should therefore be prepared to accept an argument of `None`, or of class `datetime`.

When `None` is passed, it's up to the class designer to decide the best response. For example, returning `None` is appropriate if the class wishes to say that time objects don't participate in the `tzinfo` protocols. It may be more useful for `utcoffset(None)` to return the standard UTC offset, as there is no other convention for discovering the standard offset.

When `datetime` object is passed in response to `datetime` method, `dt.tzinfo` is the same object as `self.tzinfo`. Methods can rely on this, unless user code calls `tzinfo` methods directly. The intent is that the `tzinfo` methods interpret `dt` as being in local time, and not need worry about objects in other time zones.

There is one more `tzinfo` method that a subclass may wish to override:

`tzinfo.fromutc(dt)`

This is called from the default `datetime.astimezone()` implementation. When called from that, `dt.tzinfo` is `self`, and `dt`'s date and time data are to be viewed as expressing a UTC time. The purpose of `fromutc()` is to adjust the date and time data, returning an equivalent `datetime` in `self`'s local time. Most `tzinfo` subclasses should be able to inherit the default `fromutc()` implementation without problems. It's strong enough to handle fixed-offset time zones, and time zones accounting for both standard and daylight

In the following `tzinfo_examples.py` file there are some examples of `tzinfo` classes:

```
import datetime as dt

# A class capturing the platform's idea of local time.
# (May result in wrong values on historical times in
# timezones where UTC offset and/or the DST rules had
# changed in the past.)
import time

ZERO = dt.timedelta(0)
HOUR = dt.timedelta(hours=1)
SECOND = dt.timedelta(seconds=1)

STDOFFSET = dt.timedelta(seconds=-time.timezone)
if time.daylight:
    DSTOFFSET = dt.timedelta(seconds=-time.altzone)
else:
    DSTOFFSET = STDOFFSET

DSTDIFF = DSTOFFSET - STDOFFSET

class LocalTimezone(dt.tzinfo):

    def fromutc(self, when):
        assert when.tzinfo is self
        stamp = (when - dt.datetime(1970, 1, 1, tzinfo=self)) // SECOND
        args = time.localtime(stamp)[:6]
        dst_diff = DSTDIFF // SECOND
        # Detect fold
        fold = (args == time.localtime(stamp - dst_diff))
        return dt.datetime(*args, microsecond=when.microsecond,
```

```

        tzinfo=self, fold=fold)

def utcoffset(self, when):
    if self._isdst(when):
        return DSTOFFSET
    else:
        return STDOFFSET

def dst(self, when):
    if self._isdst(when):
        return DSTDIFF
    else:
        return ZERO

def tzname(self, when):
    return time.tzname[self._isdst(when)]

def _isdst(self, when):
    tt = (when.year, when.month, when.day,
          when.hour, when.minute, when.second,
          when.weekday(), 0, 0)
    stamp = time.mktime(tt)
    tt = time.localtime(stamp)
    return tt.tm_isdst > 0

Local = LocalTimezone()

# A complete implementation of current DST rules for major US time zones.

def first_sunday_on_or_after(when):
    days_to_go = 6 - when.weekday()
    if days_to_go:
        when += dt.timedelta(days_to_go)
    return when

# US DST Rules
#
# This is a simplified (i.e., wrong for a few cases) set of rules for US
# DST start and end times. For a complete and up-to-date set of DST rules
# and timezone definitions, visit the Olson Database (or try pytz):
# http://www.twinsun.com/tz/tz-link.htm
# https://sourceforge.net/projects/pytz/ (might not be up-to-date)
#
# In the US, since 2007, DST starts at 2am (standard time) on the second
# Sunday in March, which is the first Sunday on or after Mar 8.
DSTSTART_2007 = dt.datetime(1, 3, 8, 2)

```

Note that there are unavoidable subtleties twice per year in `atziposubclass` accounting for both standard and daylight time, at the DST transition points. For concreteness, consider US Eastern (UTC -0500), where EDT begins the minute after 1:59 (EST) on the second Sunday in March, and ends the minute after 1:59 (EDT) on the first Sunday in November:

UTC	3:MM	4:MM	5:MM	6:MM	7:MM	8:MM
EST	22:MM	23:MM	0:MM	1:MM	2:MM	3:MM
EDT	23:MM	0:MM	1:MM	2:MM	3:MM	4:MM
start	22:MM	23:MM	0:MM	1:MM	3:MM	4:MM
end	23:MM	0:MM	1:MM	1:MM	2:MM	3:MM

When DST starts (the “start” line), the local wall clock leaps from 1:59 to 3:00. A wall time of the form 2:MM doesn’t really make sense on that day, so `astimezone(Eastern)` won’t deliver a result with `hour==2` on the day DST begins. For example, at the Spring forward transition of 2016, we get:

```
>>> import datetime as dt
>>> from tzinfo_examples import HOUR, Eastern
>>> u0 = dt.datetime(2016, 3, 13, 5, tzinfo=dt.timezone.utc)
>>> for i in range(4):
...     u = u0 + i*HOUR
...     t = u.astimezone(Eastern)
...     print(u.time(), 'UTC =', t.time(), t.tzname())
...
05:00:00 UTC = 00:00:00 EST
06:00:00 UTC = 01:00:00 EST
07:00:00 UTC = 03:00:00 EDT
08:00:00 UTC = 04:00:00 EDT
```

When DST ends (the “end” line), there’s a potentially worse problem: there’s an hour that can’t be spelled unambiguously in local wall time: the last hour of daylight time. In Eastern, that’s times of the form 5:MM UTC on the day daylight time ends. The local wall clock leaps from 1:59 (daylight time) back to 1:00 (standard time) again. Local times of the form 1:MM are ambiguous. `astimezone()` mimics the local clock’s behavior by mapping two adjacent UTC hours into the same local hour then. In the Eastern example, UTC times of the form 5:MM and 6:MM both map to 1:MM when converted to Eastern, but earlier times have the `fold` attribute set to 0 and the later times have it set to 1. For example, at the Fall back transition of 2016, we get:

```
>>> import datetime as dt
>>> from tzinfo_examples import HOUR, Eastern
>>> u0 = dt.datetime(2016, 11, 6, 4, tzinfo=dt.timezone.utc)
>>> for i in range(4):
...     u = u0 + i*HOUR
...     t = u.astimezone(Eastern)
...     print(u.time(), 'UTC =', t.time(), t.tzname(), t.fold)
...
04:00:00 UTC = 00:00:00 EDT 0
05:00:00 UTC = 01:00:00 EDT 0
06:00:00 UTC = 01:00:00 EST 1
07:00:00 UTC = 02:00:00 EST 0
```

Note that the `datetime` instances that differ only by the value of the `fold` attribute are considered equal in comparisons.

Applications that can’t bear wall-time ambiguities should explicitly check the value of the `fold` attribute or avoid using hybrid `tzinfo` subclasses; there are no ambiguities when using `timezone`, or any other fixed-offset `tzinfo` subclass (such as a class representing only EST (fixed offset -5 hours), or only EDT (fixed offset -4 hours)).

See also

zoneinfo

The `datetime` module has a `basictimezone` class (for handling arbitrary fixed offsets from UTC) and its `timezone.utc` attribute (a `UTC` `timezone` instance). `zoneinfo` brings the IANA time zone database (also known as the Olson database) to Python, and its usage is recommended.

IANA time zone database

The Time Zone Database (often called `tz`, `tzdata` or `zoneinfo`) contains code and data that represent the history of local time for many representative locations around the globe. It is updated periodically to reflect changes made by political bodies to time zone boundaries, UTC offsets, and daylight-saving rules.

timezoneobjects

The `timezone` class is a subclass of `tzinfo`, each instance of which represents a time zone defined by a fixed offset from UTC.

Objects of this class cannot be used to represent time zone information in the locations where different offsets are used in different days of the year or where historical changes have been made to civil time.

`class datetime.timezone(offset, name=None)`

The `offset` argument must be specified as a `timedelta` object representing the difference between the local time and UTC. It must be strictly between `-timedelta(hours=24)` and `timedelta(hours=24)`, otherwise `ValueError` is raised. The `name` argument is optional. If specified it must be a string that will be used as the value returned by the `datetime.timezone.tzname()` method. Added in version 3.2. Changed in version 3.7: The UTC offset is not restricted to a whole number of minutes.

`timezone.utcoffset(dt)`

Return the fixed value specified when the `timezone` instance is constructed. The `dt` argument is ignored. The return value is a `timedelta` instance equal to the difference between the local time and UTC. Changed in version 3.7: The UTC offset is not restricted to a whole number of minutes.

`timezone.tzname(dt)`

Return the fixed value specified when the `timezone` instance is constructed. If `name` is not provided in the constructor, the name returned by `tzname(dt)` is generated from the value of the `offset` as follows. If `offset` is `timedelta(0)`, the name is "UTC", otherwise it is a string in the format `UTC±HH:MM`, where $\pm$ is the sign of `offset`, `HH` and `MM` are two digits of `offset.hours` and `offset.minutes` respectively. Changed in version 3.6: Name generated from `offset=timedelta(0)` is now plain 'UTC', not 'UTC+00:00'.

`timezone.dst(dt)`

Always returns `None`.

`timezone.fromutc(dt)`

Return `dt+offset`. The `dt` argument must be an aware `datetime` instance, with `tzinfo` set to self.

Class attributes:

`timezone.utc`

The UTC time zone, `timezone(timedelta(0))`.

strptime() and strftime() behavior

`date`, `datetime`, and `time` objects all support a `strftime(format)` method, to create a string representing the time under the control of an explicit format string.

Conversely, the `date.strptime()`, `datetime.strptime()` and `time.strptime()` class methods create an object from a string representing the time and a corresponding format string.

The table below provides a high-level comparison of `strftime()` versus `strptime()`:

strftime	strptime	Usage	Convert object to a string according to a given format	Parse a string into an object given a corresponding format	Type of method	Instance method	Class method
method	Signature	strftime(format)	strptime(date_string, format)				

strftime() and strptime() format codes

These methods accept format codes that can be used to parse and format dates:


```
>>> import datetime as dt
>>> dt.datetime.strptime('31/01/22 23:59:59.999999',
...                       '%d/%m/%y %H:%M:%S.%f')
datetime.datetime(2022, 1, 31, 23, 59, 59, 999999)
>>> _.strftime('%a %d %b %Y, %I:%M%p')
'Mon 31 Jan 2022, 11:59PM'
```

The following is a list of all the format codes that the 1989 C standard requires, and these work on all platforms with a standard C implementation.

Directive	Meaning	Example	Notes
%a	Weekday as locale's abbreviated name.	Sun, Mon, ..., Sat (en_US); So, Mo, ..., Sa (de_DE)	(1)
%A	Weekday as locale's full name.	Sunday, Monday, ..., Saturday (en_US); Sonntag, Montag, ..., Samstag (de_DE)	(1)
%w	Weekday as a decimal number, where 0 is Sunday and 6 is Saturday.	0, 1, ..., 6	(9)
%d	Day of the month as a zero-padded decimal number.	01, 02, ..., 31	(9)
%b	Month as locale's abbreviated name.	Jan, Feb, ..., Dec (en_US); Jan, Feb, ..., Dez (de_DE)	(1)
%B	Month as locale's full name.	January, February, ..., December (en_US); Januar, Februar, ..., Dezember (de_DE)	(1)
%m	Month as a zero-padded decimal number.	01, 02, ..., 12	(9)
%y	Year without century as a zero-padded decimal number.	00, 01, ..., 99	(6)

Several additional directives not required by the C89 standard are included for convenience. These parameters all correspond to ISO 8601 date values.

Directive	Meaning	Example	Notes
%G	ISO 8601 year with century representing the year that contains the greater part of the ISO week (%V).	0001, 0002, ..., 2013, 2014, ..., 9998, 9999	(8)
%u	ISO 8601 weekday as a decimal number where 1 is Monday.	1, 2, ..., 7	(7)
%V	ISO 8601 week as a decimal number with Monday as the first day of the week. Week 01 is the week containing Jan 4.	01, 02, ..., 53	(8), (9)
%:z	UTC offset in the form ±HH:MM[:SS[.ffffff]] (empty string if the object is naive).	(empty), +00:00, -04:00, +10:30, +06:34:15, -03:07:12.345216	(6)

These may not be available on all platforms when used with the `strftime()` method. The ISO 8601 year and ISO 8601 week directives are not interchangeable with the year and week number directives above.

Calling `strptime()` with incomplete or ambiguous ISO 8601 directives will raise a `ValueError`.

The full set of format codes supported varies across platforms, because Python calls the platform C library's `strftime()` function, and platform variations are common. To see the full set of format codes supported on your platform, consult the `strftime(3)` documentation. There are also differences between platforms in handling of unsupported format specifiers.

Added in version 3.6: `%G, %u` and `%V` were added.

Added in version 3.12: `%:z` was added.

Technical detail

Broadly speaking, `d.strftime(fmt)` acts like `time.strptime(fmt, d.timetuple())` although not all objects support a `timetuple()` method.

For the `datetime.strptime()` and `date.strptime()` class methods, the default value is `1900-01-01T00:00:00.000`: any components not specified in the format string will be pulled from the default value.

■ **Note:** Format strings without separators can be ambiguous for parsing. For example, with `%Y%m%d`, the string `2026111` may be parsed either as `2026-11-01` or as `2026-01-11`. Use separators to ensure the input is parsed as intended.

■ **Note:** When used to parse partial dates lacking a year, `datetime.strptime()` and `date.strptime()` will raise an error when encountering February 29 because the default year of 1900 is not a leap year. Always add a default leap year to partial date strings before parsing.

```
>>> import datetime as dt
>>> value = "2/29"
>>> dt.datetime.strptime(value, "%m/%d")
Traceback (most recent call last):
...
ValueError: time data '2/29' does not match format '%m/%d' (ValueError)
```

```

ValueError: day 29 must be in range 1..28 for month 2 in year 1900
>>> dt.datetime.strptime(f"1904 {value}", "%Y %m/%d")
datetime.datetime(1904, 2, 29, 0, 0)

```

Using `datetime.strptime(date_string, format)` is equivalent to:

```
datetime(*(time.strptime(date_string, format)[0:6]))
```

except when the format includes sub-second components or time zone offset information, which are supported in `datetime.strptime` but are discarded by `time.strptime`.

For `time` objects, the format codes for year, month, and day should not be used, as `time` objects have no such values. If they're used anyway, 1900 is substituted for the year, and 1 for the month and day.

For `date` objects, the format codes for hours, minutes, seconds, and microseconds should not be used, as `date` objects have no such values. If they're used anyway, 0 is substituted for them.

For the same reason, handling of format strings containing Unicode code points that can't be represented in the charset of the current locale is also platform-dependent. On some platforms such code points are preserved intact in the output, while on others `strptime` may raise `UnicodeError` or return an empty string instead.

Notes:

- Because the format depends on the current locale, care should be taken when making assumptions about the output value. Field orderings will vary (for example, "month/day/year" versus "day/month/year"), and the output may contain non-ASCII characters.
- The `strptime()` method can parse years in the full [1, 9999] range, but years < 1000 must be zero-filled to 4-digit width. Changed in version 3.2: In previous versions, `strptime()` method was restricted to years >= 1900. Changed in version 3.3: In version 3.2, `strptime()` method was restricted to years >= 1000.
- When used with the `strptime()` method, the `%p` directive only affects the output hour field if the `%I` directive is used to parse the hour.
- Unlike the `time` module, the `datetime` module does not support leap seconds.
- When used with the `strptime()` method, the `%f` directive accepts from one to six digits and zero pads on the right. `%f` is an extension to the set of format characters in the C standard (but implemented separately in `datetime` objects, and therefore always available).
- For a naive object, the `%z`, `%:z` and `%Z` format codes are replaced by empty strings. For an aware object: `%z` to `utcoffset()` is transformed into a string of the form `±HHMM[SS[.ffffff]]`, where `HH` is a 2-digit string giving the number of UTC offset hours, `MM` is a 2-digit string giving the number of UTC offset minutes, `SS` is a 2-digit string giving the number of UTC offset seconds and `ffffff` is a 6-digit string giving the nanoseconds.
- When used with the `strptime()` method, `%U` and `%W` are only used in calculations when the day of the week and the calendar year (`%Y`) are specified.
- Similar to `%U` and `%W`, `%V` is only used in calculations when the day of the week and the ISO year (`%G`) are specified in `strptime()` format string. Also note that `%G` and `%Y` are not interchangeable.
- When used with the `strptime()` method, the leading zero is optional for formats `%d`, `%m`, `%H`, `%I`, `%M`, `%S`, `%j`, `%U`, `%W`, and `%V`. Format `%y` does require a leading zero.
- When parsing a month and day using `strptime()`, always include a year in the format. If the value you need to parse lacks a year, append an explicit dummy leap year. Otherwise your code will raise an exception when it encounters leap day because the default year used by the parser (1900) is not a leap year. Users run into that bug every leap year. `>>> month_day="02/29">>> dt.datetime.strptime(f"{mon`

Footnotes

If, that is, we ignore the effects of relativity.

This matches the definition of the “proleptic Gregorian” calendar in Dershowitz and Reingold’s book *Calendrical Calculations*, where it’s the base calendar for all computations. See the book for algorithms for converting between proleptic Gregorian ordinals and many other calendar systems.

See R. H. van Gent’s *guide to the mathematics of the ISO 8601 calendar* for a good explanation.

zoneinfo— IANA time zone support

Added in version 3.9.

Source code: [Lib/zoneinfo](#)

The `zoneinfo` module provides a concrete time zone implementation to support the IANA time zone database as originally specified in PEP 615. By default, `zoneinfo` uses the system's time zone data if available; if no system time zone data is available, the library will fall back to using the first-party `tzdata` package available on PyPI.

See also

Module: `datetime`

Package: `tzdata`

Provides the `time` and `datetime` types with which the `ZoneInfo` class is designed to be used.

First-party package maintained by the CPython core developers to supply time zone data via PyPI.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

Using `ZoneInfo`

`ZoneInfo` is a concrete implementation of the `datetime.tzinfo` abstract base class, and is intended to be attached to `tzinfo`, either via the constructor, the `datetime.replace` method or `datetime.astimezone`:

```
>>> from zoneinfo import ZoneInfo
>>> import datetime as dt

>>> when = dt.datetime(2020, 10, 31, 12, tzinfo=ZoneInfo("America/Los_Angeles"))
>>> print(when)
2020-10-31 12:00:00-07:00

>>> when.tzname()
'PDT'
```

Datetimes constructed in this way are compatible with `datetime` arithmetic and handle daylight saving time transitions with no further intervention:

```
>>> when_add = when + dt.timedelta(days=1)

>>> print(when_add)
2020-11-01 12:00:00-08:00

>>> when_add.tzname()
'PST'
```

These time zones also support the `fold` attribute introduced in PEP 495. During offset transitions which induce ambiguous times (such as a daylight saving time to standard time transition), the offset from before the transition is used when `fold=0`, and the offset after the transition is used when `fold=1`, for example:

```
>>> when = dt.datetime(2020, 11, 1, 1, tzinfo=ZoneInfo("America/Los_Angeles"))
>>> print(when)
2020-11-01 01:00:00-07:00

>>> print(when.replace(fold=1))
2020-11-01 01:00:00-08:00
```

When converting from another time zone, the fold will be set to the correct value:

```
>>> LOS_ANGELES = ZoneInfo("America/Los_Angeles")
>>> when_utc = dt.datetime(2020, 11, 1, 8, tzinfo=dt.timezone.utc)

>>> # Before the PDT -> PST transition
>>> print(when_utc.astimezone(LOS_ANGELES))
2020-11-01 01:00:00-07:00

>>> # After the PDT -> PST transition
>>> print((when_utc + dt.timedelta(hours=1)).astimezone(LOS_ANGELES))
2020-11-01 01:00:00-08:00
```

Data sources

The `zoneinfo` module does not directly provide time zone data, and instead pulls time zone information from the system time zone database or the first-party PyPI package `tzdata`, if available. Some systems, including notably Windows systems, do not have an IANA database available, and so for projects targeting cross-platform compatibility that require time zone data, it is recommended to declare a dependency on `tzdata`. If neither system data nor `tzdata` are available, all calls to `ZoneInfo` will raise `ZoneInfoNotFoundError`.

Configuring the data sources

When `ZoneInfo(key)` is called, the constructor first searches the directories specified in `TZPATH` for a file matching `key`, and on failure looks for a match in the `tzdata` package. This behavior can be configured in three ways:

- The default `TZPATH` when not otherwise specified can be configured at compile time.
- `TZPATH` can be configured using an environment variable.
- At runtime, the search path can be manipulated using the `reset_tzpath()` function.

Compile-time configuration

The default `TZPATH` includes several common deployment locations for the time zone database (except on Windows, where there are no “well-known” locations for time zone data). On POSIX systems, downstream distributors and those building Python from source who know where their system time zone data is deployed may change the default time zone path by specifying the compile-time option `TZPATH` (or, more likely, the configure flag `--with-tzpath`), which should be a string delimited by `os.pathsep`.

On all platforms, the configured value is available as the `TZPATH` key in `sysconfig.get_config_var()`.

Environment configuration

When initializing `TZPATH` (either at import time or whenever `reset_tzpath()` is called with no arguments), the `zoneinfo` module will use the environment variable `PYTHONTZPATH`, if it exists, to set the search path.

PYTHONTZPATH

This is an `os.pathsep`-separated string containing the time zone search path to use. It must consist of only absolute rather than relative paths. Relative components specified in `PYTHONTZPATH` will not be used, but otherwise the behavior when a relative path is specified is implementation-defined; CPython will raise `InvalidTZPathWarning`, but other implementations are free to silently ignore the erroneous component or raise an exception.

To set the system to ignore the system data and use the `tzdata` package instead, set `PYTHONTZPATH=""`.

Runtime configuration

The TZ search path can also be configured at runtime using `reset_tzpath()` function. This is generally not an advisable operation, though it is reasonable to use it in test functions that require the use of a specific time zone path (or require disabling access to the system time zones).

The `ZoneInfo` class

`class zoneinfo.ZoneInfo(key)`

A concrete `datetime.tzinfo` subclass that represents an IANA time zone specified by the string `key`. Calls to the primary constructor will always return objects that compare identically; put another way, barring cache invalidation via `ZoneInfo.clear_cache()`, for all values of `key`, the following assertion will always be true: `a=ZoneInfo(key)b=ZoneInfo(key)assert a is b` `key` must be in the form of a relative, normalized POSIX path, with no up-level references. The constructor will raise `ValueError` if a non-conformant

The `ZoneInfo` class has two alternate constructors:

`classmethod ZoneInfo.from_file(file_obj, key=None)`

Constructs a `ZoneInfo` object from a file-like object returning bytes (e.g. a file opened in binary mode or `io.BytesIO` object). Unlike the primary constructor, this always constructs a new object. The `key` parameter sets the name of the zone for the purposes of `__str__()` and `__repr__()`. Objects created via this constructor cannot be pickled (see pickling). `ValueError` is raised if the data read from `file_obj` is not a valid TZif file.

`classmethod ZoneInfo.no_cache(key)`

An alternate constructor that bypasses the constructor's cache. It is identical to the primary constructor, but returns a new object on each call. This is most likely to be useful for testing or demonstration purposes, but it can also be used to create a system with a different cache invalidation strategy. Objects created via this constructor will also bypass the cache of a deserializing process when unpickled. Caution Using this constructor may change the semantics of your datetimes in surprising

The following class methods are also available:

`classmethod ZoneInfo.clear_cache(*, only_keys=None)`

A method for invalidating the cache on the `ZoneInfo` class. If no arguments are passed, all caches are invalidated and the next call to the primary constructor for each key will return a new instance. If an iterable of key names is passed to the `only_keys` parameter, only the specified keys will be removed from the cache. Keys passed to `only_keys` but not found in the cache are ignored. Warning Invoking this function may change the semantics of datetimes using `ZoneInfo` in surprising ways; this modifies module

The class has one attribute:

`ZoneInfo.key`

This is a read-only attribute that returns the value of `key` passed to the constructor, which should be a lookup key in the IANA time zone database (e.g. `America/New_York`, `Europe/Paris` or `Asia/Tokyo`). For zones constructed from file without specifying a `key` parameter, this will be set to `None`. Note Although it is a somewhat common practice to expose these to end users, these values are designed to be primary keys for representing the relevant zones and not necessarily user-facing elements. Projects like CLDR

String representations

The string representation returned when calling `str()` on a `ZoneInfo` object defaults to using the `ZoneInfo.key` attribute (see the note on usage in the attribute documentation):

```
>>> zone = ZoneInfo("Pacific/Kwajalein")
>>> str(zone)
'Pacific/Kwajalein'
```

```
>>> when = dt.datetime(2020, 4, 1, 3, 15, tzinfo=zone)
>>> f"{when.isoformat()} [{when.tzinfo}]"
'2020-04-01T03:15:00+12:00 [Pacific/Kwajalein]'
```

For objects constructed from a file without specifying a key parameter, `str` falls back to calling `repr()`. `ZoneInfo`'s `repr` implementation is defined and not necessarily stable between versions, but it is guaranteed not to be a valid `ZoneInfo` key.

Pickle serialization

Rather than serializing all transition data, `ZoneInfo` objects are serialized by key, and `ZoneInfo` objects constructed from files (even those with a value for `keys` specified) cannot be pickled.

The behavior of a `ZoneInfo` file depends on how it was constructed:

- `ZoneInfo(key)`: When constructed with the primary constructor, a `ZoneInfo` object is serialized by key, and when deserialized, the deserializing process uses the primary and thus it is expected that these are the same object as other references to the same time zone. For example, if `europa_berlin_pkl` is a string containing a pickle constructed from `ZoneInfo("Europe/Berlin")`, one would expect the following behavior:


```
>>> a=ZoneInfo("Europe/Berlin")
>>> b=pickle.loads(pkl)
>>> a is b
True
```
- `ZoneInfo.no_cache(key)`: When constructed from the cache-bypassing constructor, the `ZoneInfo` object is also serialized by key, but when deserialized, the deserializing process uses the cache bypassing constructor. If `europa_berlin_pkl_nc` is a string containing a pickle constructed from `ZoneInfo.no_cache("Europe/Berlin")`, one would expect the following behavior:


```
>>> a=ZoneInfo.no_cache("Europe/Berlin")
>>> b=pickle.loads(pkl)
>>> a is b
False
```
- `ZoneInfo.from_file(file_obj, key=None)`: When constructed from a file, the `ZoneInfo` object raises an exception on pickling. If an end user wants to pickle a `ZoneInfo` constructed from a file, it is recommended that they use a wrapper type or a custom serialization function: either serializing by key or storing the contents of the file object and serializing that.

This method of serialization requires that the time zone data for the required key be available on both the serializing and deserializing side, similar to the way that references to classes and functions are expected to exist in both the serializing and deserializing environments. It also means that no guarantees are made about the consistency of results when unpickling a `ZoneInfo` pickled in an environment with a different version of the time zone data.

Functions

`zoneinfo.available_timezones()`

Get a set containing all the valid keys for IANA time zones available anywhere on the time zone path. This is recalculated on every call to the function. This function only includes canonical zone names and does not include “special” zones such as those under `posix` and `right` directories, or the `posixrules` zone. **Caution** This function may open a large number of files, as the best way to determine if a file on the time zone path is a valid time zone is to read the “magic string” at the beginning. **Note** T

`zoneinfo.reset_tzpath(to=None)`

Sets or resets the time zone search path (TZPATH) for the module. When called with no arguments, TZPATH is set to the default value. Calling `reset_tzpath` will not invalidate the `ZoneInfo` cache, and so calls to the primary `ZoneInfo` constructor will only use the new TZPATH in the case of a cache miss. The `to` parameter must be a sequence of strings or `os.PathLike` and not a string, all of which must be absolute paths. `ValueError` will be raised if something other than an absolute path is passed.

Globals

zoneinfo.TZPATH

A read-only sequence representing the time zone search path – when constructing a `ZoneInfo` from a key, the key is joined to each entry in the `TZPATH`, and the first file found is used. `TZPATH` may contain only absolute paths, never relative paths, regardless of how it is configured. The object that `zoneinfo.TZPATH` points to may change in response to a call to `reset_tzpath()`, so it is recommended to use `zoneinfo.TZPATH` rather than importing `TZPATH` from `zoneinfo` or assigning a long-lived variable to `zoneinfo.TZPATH`.

Exceptions and warnings

exception zoneinfo.ZoneInfoNotFoundError

Raised when construction of a `ZoneInfo` object fails because the specified key could not be found on the system. This is a subclass of `KeyError`.

exception zoneinfo.InvalidTZPathWarning

Raised when `PYTHON_TZPATH` contains an invalid component that will be filtered out, such as a relative path.

calendar— General calendar-related functions

Source code: `Lib/calendar.py`

This module allows you to output calendars like the Unix `cal` program, and provides additional useful functions related to the calendar. By default, these calendars have Monday as the first day of the week, and Sunday as the last (the European convention). Use `setfirstweekday()` to set the first day of the week to Sunday (6) or to any other weekday. Parameters that specify dates are given as integers. For related functionality, see also `datetime` and `time` modules.

The functions and classes defined in this module use an idealized calendar, the current Gregorian calendar extended indefinitely in both directions. This matches the definition of the “proleptic Gregorian” calendar in Dershowitz and Reingold’s book “Calendrical Calculations”, where it’s the base calendar for all computations. Zero and negative years are interpreted as prescribed by the ISO 8601 standard. Year 0 is 1 BC, year -1 is 2 BC, and so on.

`class calendar.Calendar(firstweekday=0)`

`firstweekday`

`getfirstweekday()`

`setfirstweekday(firstweekday)`

`iterweekdays()`

`itermonthdates(year, month)`

`itermonthdays(year, month)`

`itermonthdays2(year, month)`

`itermonthdays3(year, month)`

`itermonthdays4(year, month)`

`monthdatescalendar(year, month)`

`monthdays2calendar(year, month)`

`monthdayscalendar(year, month)`

`yeardatescalendar(year, width=3)`

`yeardays2calendar(year, width=3)`

`yeardayscalendar(year, width=3)`

Creates a `Calendar` object. `firstweekday` is an integer specifying the first day of the week. `MONDAY` is 0 (the default), `SUNDAY` is 6. A `Calendar` object provides several methods that can be used for preparing the calendar data for formatting. This class doesn’t do any formatting itself. This is the job of subclasses. `Calendar` instances have the following methods and attributes: `firstweekday` The first weekday as an integer (0–6). This property can also be set and read using `setfirstweekday()` and `getfirstweekday()` respectively.

The first weekday as an integer (0–6). This property can also be set and read using `setfirstweekday()` and `getfirstweekday()` respectively.

Return an `int` for the current first weekday (0–6). Identical to reading the `firstweekday` property.

Set the first weekday to `firstweekday`, passed as an `int` (0–6). Identical to setting the `firstweekday` property.

Return an iterator for the week day numbers that will be used for one week. The first value from the iterator will be the same as the value of the `firstweekday` property.

Return an iterator for the monthmonth(1–12) in the yearyear. This iterator will return all days (asdatetime.dateobjects) for the month and all days before the start of the month or after the end of the month that are required to get a complete week.

Return an iterator for the monthmonthin the yearyearsimilar toitermonthdates(), but not restricted by thedatetime.daterange. Days returned will simply be day of the month numbers. For the days outside of the specified month, the day number is0.

Return an iterator for the monthmonthin the yearyearsimilar toitermonthdates(), but not restricted by thedatetime.daterange. Days returned will be tuples consisting of a day of the month number and a week day number.

Return an iterator for the monthmonthin the yearyearsimilar toitermonthdates(), but not restricted by thedatetime.daterange. Days returned will be tuples consisting of a year, a month and a day of the month numbers.Added in version 3.7.

Return an iterator for the monthmonthin the yearyearsimilar toitermonthdates(), but not restricted by thedatetime.daterange. Days returned will be tuples consisting of a year, a month, a day of the month, and a day of the week numbers.Added in version 3.7.

Return a list of the weeks in the monthmonthof theyearas full weeks. Weeks are lists of sevendatetime.dateobjects.

Return a list of the weeks in the monthmonthof theyearas full weeks. Weeks are lists of seven tuples of day numbers and weekday numbers.

Return a list of the weeks in the monthmonthof theyearas full weeks. Weeks are lists of seven day numbers.

Return the data for the specified year ready for formatting. The return value is a list of month rows. Each month row contains up towidthmonths (defaulting to 3). Each month contains between 4 and 6 weeks and each week contains 1–7 days. Days aredatetime.dateobjects.

Return the data for the specified year ready for formatting (similar toyeardatescalendar()). Entries in the week lists are tuples of day numbers and weekday numbers. Day numbers outside this month are zero.

Return the data for the specified year ready for formatting (similar toyeardatescalendar()). Entries in the week lists are day numbers. Day numbers outside this month are zero.

classcalendar.TextCalendar(firstweekday=0)

formatday(theday,weekday,width)

formatweek(theweek,w=0)

formatweekday(weekday,width)

formatweekheader(width)

formatmonth(theyear,themonth,w=0,l=0)

formatmonthname(theyear,themonth,width=0,withyear=True)

prmonth(theyear,themonth,w=0,l=0)

formatyear(theyear,w=2,l=1,c=6,m=3)

pryear(theyear,w=2,l=1,c=6,m=3)

This class can be used to generate plain text calendars.TextCalendarinstances have the following methods:formatday(theday,weekday,width)Return a string representing a single day formatted with the givenwidth. Ifthedayis0, return a string of spaces of the specified width, representing an empty day. Theweekdayparameter is unused.formatweek(theweek,w=0)Return a single week in a string with no newline. Ifwis provided, it specifies the width of the date columns, which are centered. Depends on the fir

Return a string representing a single day formatted with the givenwidth. Ifthedayis0, return a string of spaces of the specified width, representing an empty day. Theweekdayparameter is unused.

Return a single week in a string with no newline. Ifwis provided, it specifies the width of the date columns, which are centered. Depends on the first weekday as specified in the constructor or set by thesetfirstweekday()method.

Return a string representing the name of a single weekday formatted to the specifiedwidth. Theweekdayparameter is an integer representing the day of the week, where0is Monday and6is Sunday.

Return a string containing the header row of weekday names, formatted with the givenwidthfor each column. The names depend on the locale settings and are padded to the specified width.

Return a month's calendar in a multi-line string. Ifwis provided, it specifies the width of the date columns, which are centered. Iflis given, it specifies the number of lines that each week will use. Depends on the first weekday as specified in the constructor or set by thesetfirstweekday()method.

Return a string representing the month's name centered within the specifiedwidth. IfwithyearisTrue, include the year in the output. Thetheyearandthemothparameters specify the year and month for the name to be formatted respectively.

Print a month's calendar as returned byformatmonth().

Return am-column calendar for an entire year as a multi-line string. Optional parametersw,l, andcare for date column width, lines per week, and number of spaces between month columns, respectively. Depends on the first weekday as specified in the constructor or set by thesetfirstweekday()method. The earliest year for which a calendar can be generated is platform-dependent.

Print the calendar for an entire year as returned byformatyear().

classcalendar.HTMLCalendar(firstweekday=0)

formatmonth(theyear,themoth,withyear=True)

formatyear(theyear,width=3)

formatyearpage(theyear,width=3,css='calendar.css',encoding=None)

formatmonthname(theyear,themoth,withyear=True)

cssclasses

cssclass_noday

cssclasses_weekday_head

cssclass_month_head

cssclass_month

cssclass_year

cssclass_year_head

This class can be used to generate HTML calendars.HTMLCalendarinstances have the following methods:formatmonth(theyear,themoth,withyear=True)Return a month's calendar as an HTML table. Ifwithyearis true the year will be included in the header, otherwise just the month name will be used.formatyear(theyear,width=3)Return a year's calendar as an HTML table.width(defaulting to 3) specifies the number of months per row.formatyearpage(theyear,width=3,css='calendar.css',encoding=None)Return a year's c

Return a month's calendar as an HTML table. Ifwithyearis true the year will be included in the header, otherwise just the month name will be used.

Return a year's calendar as an HTML table. `width` (defaulting to 3) specifies the number of months per row.

Return a year's calendar as a complete HTML page. `width` (defaulting to 3) specifies the number of months per row. `css` is the name for the cascading style sheet to be used. `None` can be passed if no style sheet should be used. `encoding` specifies the encoding to be used for the output (defaulting to the system default encoding).

Return a month name as an HTML table row. If `withyear` is `true` the year will be included in the row, otherwise just the month name will be used.

A list of CSS classes used for each weekday. The default class list is: `cssclasses=["mon","tue","wed","thu","fri","sat","sun"]` more styles can be added for each day: `cssclasses=["mon text-bold","tue","wed","thu","fri","sat","sun red"]` Note that the length of this list must be seven items.

The CSS class for a weekday occurring in the previous or coming month. Added in version 3.7.

A list of CSS classes used for weekday names in the header row. The default is the same as `cssclasses`. Added in version 3.7.

The month's head CSS class (used by `formatmonthname()`). The default value is `"month"`. Added in version 3.7.

The CSS class for the whole month's table (used by `formatmonth()`). The default value is `"month"`. Added in version 3.7.

The CSS class for the whole year's table of tables (used by `formatyear()`). The default value is `"year"`. Added in version 3.7.

The CSS class for the table head for the whole year (used by `formatyear()`). The default value is `"year"`. Added in version 3.7.

`calendar.LocaleTextCalendar(firstweekday=0,locale=None)`

This subclass of `TextCalendar` can be passed a locale name in the constructor and will return month and weekday names in the specified locale.

`calendar.LocaleHTMLCalendar(firstweekday=0,locale=None)`

This subclass of `HTMLCalendar` can be passed a locale name in the constructor and will return month and weekday names in the specified locale.

■ Nota: Note The constructor, `formatweekday()` and `formatmonthname()` methods of these two classes temporarily change the `LC_TIME` locale to the given locale. Because the current locale is a process-wide setting, they are not thread-safe.

For simple text calendars this module provides the following functions.

`calendar.setfirstweekday(weekday)`

Sets the weekday (0 is Monday, 6 is Sunday) to start each week. The values `MONDAY`, `TUESDAY`, `WEDNESDAY`, `THURSDAY`, `FRIDAY`, `SATURDAY`, and `SUNDAY` are provided for convenience. For example, to set the first weekday to Sunday: `import calendar; calendar.setfirstweekday(calendar.SUNDAY)`

`calendar.firstweekday()`

Returns the current setting for the weekday to start each week.

`calendar.isleap(year)`

Returns `True` if `year` is a leap year, otherwise `False`.

`calendar.leapdays(y1,y2)`

Returns the number of leap years in the range from `y1` to `y2` (exclusive), where `y1` and `y2` are years. This function works for ranges spanning a century change.

calendar.weekday(year,month,day)

Returns the day of the week (0 is Monday) for year(1970–...), month(1–12), day(1–31).

calendar.weekheader(n)

Return a header containing abbreviated weekday names. n specifies the width in characters for one weekday.

calendar.monthrange(year,month)

Returns weekday of first day of the month and number of days in month, for the specified year and month.

calendar.monthcalendar(year,month)

Returns a matrix representing a month's calendar. Each row represents a week; days outside of the month are represented by zeros. Each week begins with Monday unless set by `setfirstweekday()`.

calendar.prmonth(theyear,themoth,w=0,l=0)

Prints a month's calendar as returned by `month()`.

calendar.month(theyear,themoth,w=0,l=0)

Returns a month's calendar in a multi-line string using the `formatmonth()` of the `TextCalendar` class.

calendar.prcal(year,w=0,l=0,c=6,m=3)

Prints the calendar for an entire year as returned by `calendar()`.

calendar.calendar(year,w=2,l=1,c=6,m=3)

Returns a 3-column calendar for an entire year as a multi-line string using the `formatyear()` of the `TextCalendar` class.

calendar.timegm(tuple)

An unrelated but handy function that takes a time tuple such as returned by the `gmtime()` function in the `time` module, and returns the corresponding Unix timestamp value, assuming an epoch of 1970, and the POSIX encoding. In fact, `time.gmtime()` and `timegm()` are each others' inverse.

The `calendar` module exports the following data attributes:

calendar.day_name

A sequence that represents the days of the week in the current locale, where Monday is day number 0. >>> `import calendar` >>> `list(calendar.day_name)` ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']

calendar.day_abbr

A sequence that represents the abbreviated days of the week in the current locale, where Mon is day number 0. >>> `import calendar` >>> `list(calendar.day_abbr)` ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

calendar.MONDAY**calendar.TUESDAY****calendar.WEDNESDAY****calendar.THURSDAY****calendar.FRIDAY****calendar.SATURDAY****calendar.SUNDAY**

Aliases for the days of the week, where `MONDAY` is 0 and `SUNDAY` is 6. Added in version 3.12.

classcalendar.Day

Enumeration defining days of the week as integer constants. The members of this enumeration are exported to the module scope asMONDAYthroughSUNDAY.Added in version 3.12.

calendar.month_name

A sequence that represents the months of the year in the current locale. This follows normal convention of January being month number 1, so it has a length of 13 andmonth_name[0]is the empty string.>>>importcalendar>>>list(calendar.month_name)[", 'January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']

calendar.month_abbr

A sequence that represents the abbreviated months of the year in the current locale. This follows normal convention of January being month number 1, so it has a length of 13 andmonth_abbr[0]is the empty string.>>>importcalendar>>>list(calendar.month_abbr)[", 'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

calendar.JANUARY

calendar.FEBRUARY

calendar.MARCH

calendar.APRIL

calendar.MAY

calendar.JUNE

calendar.JULY

calendar.AUGUST

calendar.SEPTEMBER

calendar.OCTOBER

calendar.NOVEMBER

calendar.DECEMBER

Aliases for the months of the year, whereJANUARYis1andDECEMBERis12.Added in version 3.12.

classcalendar.Month

Enumeration defining months of the year as integer constants. The members of this enumeration are exported to the module scope asJANUARYthroughDECEMBER.Added in version 3.12.

Thecalendarmodule defines the following exceptions:

exceptioncalendar.IllegalMonthError(month)

month

A subclass ofValueError, raised when the given month number is outside of the range 1-12 (inclusive).monthThe invalid month number.

The invalid month number.

exceptioncalendar.IllegalWeekdayError(weekday)

weekday

A subclass ofValueError, raised when the given weekday number is outside of the range 0-6 (inclusive).weekdayThe invalid weekday number.

The invalid weekday number.

See also

Moduledatetime

Moduletime

Object-oriented interface to dates and times with similar functionality to thetimemodule.

Low-level time related functions.

Command-line usage

Added in version 2.5.

Thecalendarmodule can be executed as a script from the command line to interactively print a calendar.

```
python -m calendar [-h] [-L LOCALE] [-e ENCODING] [-t {text,html}]
                  [-w WIDTH] [-l LINES] [-s SPACING] [-m MONTHS] [-c CSS]
                  [-f FIRST_WEEKDAY] [year] [month]
```

For example, to print a calendar for the year 2000:

```
$ python -m calendar 2000
```

2000

January							February							March						
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su
					1	2		1	2	3	4	5	6			1	2	3	4	5
3	4	5	6	7	8	9	7	8	9	10	11	12	13	6	7	8	9	10	11	12
10	11	12	13	14	15	16	14	15	16	17	18	19	20	13	14	15	16	17	18	19
17	18	19	20	21	22	23	21	22	23	24	25	26	27	20	21	22	23	24	25	26
24	25	26	27	28	29	30	28	29						27	28	29	30	31		
31																				

April							May							June						
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su
					1	2		1	2	3	4	5	6				1	2	3	4
3	4	5	6	7	8	9	8	9	10	11	12	13	14	5	6	7	8	9	10	11
10	11	12	13	14	15	16	15	16	17	18	19	20	21	12	13	14	15	16	17	18
17	18	19	20	21	22	23	22	23	24	25	26	27	28	19	20	21	22	23	24	25
24	25	26	27	28	29	30	29	30	31					26	27	28	29	30		

July							August							September						
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su
					1	2		1	2	3	4	5	6					1	2	3
3	4	5	6	7	8	9	7	8	9	10	11	12	13	4	5	6	7	8	9	10
10	11	12	13	14	15	16	14	15	16	17	18	19	20	11	12	13	14	15	16	17
17	18	19	20	21	22	23	21	22	23	24	25	26	27	18	19	20	21	22	23	24
24	25	26	27	28	29	30	28	29	30	31				25	26	27	28	29	30	
31																				

October							November							December						
Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa	Su
					1			1	2	3	4	5						1	2	3
2	3	4	5	6	7	8	6	7	8	9	10	11	12	4	5	6	7	8	9	10
9	10	11	12	13	14	15	13	14	15	16	17	18	19	11	12	13	14	15	16	17
16	17	18	19	20	21	22	20	21	22	23	24	25	26	18	19	20	21	22	23	24
23	24	25	26	27	28	29	27	28	29	30				25	26	27	28	29	30	31
30	31																			

The following options are accepted:

--help,-h

Show the help message and exit.

--locale**LOCALE**,**-l****LOCALE**

The locale to use for month and weekday names. Defaults to English.

--encoding**ENCODING**,**-e****ENCODING**

The encoding to use for output. `--encoding` is required if `--locale` is set.

--type**{text,html}**,**-t****{text,html}**

Print the calendar to the terminal as text, or as an HTML document.

--first-weekday**FIRST_WEEKDAY**,**-f****FIRST_WEEKDAY**

The weekday to start each week. Must be a number between 0 (Monday) and 6 (Sunday). Defaults to 0. Added in version 3.13.

year

The year to print the calendar for. Defaults to the current year.

month

The month of the specified `year` to print the calendar for. Must be a number between 1 and 12, and may only be used in text mode. Defaults to printing a calendar for the full year.

Text-mode options:

--width**WIDTH**,**-w****WIDTH**

The width of the date column in terminal columns. The date is printed centred in the column. Any value lower than 2 is ignored. Defaults to 2.

--lines**LINES**,**-l****LINES**

The number of lines for each week in terminal rows. The date is printed top-aligned. Any value lower than 1 is ignored. Defaults to 1.

--spacing**SPACING**,**-s****SPACING**

The space between months in columns. Any value lower than 2 is ignored. Defaults to 6.

--months**MONTHS**,**-m****MONTHS**

The number of months printed per row. Defaults to 3.

Changed in version 3.14: By default, today's date is highlighted in color and can be controlled using environment variables.

HTML-mode options:

--css**CSS**,**-c****CSS**

The path of a CSS stylesheet to use for the calendar. This must either be relative to the generated HTML, or an absolute HTTP or file:///URL.

collections— Container datatypes

Source code: `Lib/collections/__init__.py`

This module implements specialized container datatypes providing alternatives to Python's general purpose built-in containers, `dict`, `list`, `set`, and `tuple`.

```
namedtuple() | factory function for creating tuple subclasses with named fields | deque |
list-like container with fast appends and pops on either end | ChainMap | dict-like class for
creating a single view of multiple mappings | Counter | dict subclass for counting | hashable |
objects | OrderedDict | dict subclass that remembers the order entries were added | defaultdict
| dict subclass that calls a factory function to supply missing values | UserDict | wrapper
around dictionary objects for easier dict subclassing | UserList | wrapper around list objects
for easier list subclassing | UserString | wrapper around string objects for easier string
subclassing
```

ChainMapobjects

Added in version 3.3.

A `ChainMap` class is provided for quickly linking a number of mappings so they can be treated as a single unit. It is often much faster than creating a new dictionary and running multiple `update()` calls.

The class can be used to simulate nested scopes and is useful in templating.

`collections.ChainMap(*maps)`

maps

`new_child(m=None, **kwargs)`

parents

A `ChainMap` groups multiple dicts or other mappings together to create a single, updateable view. If no maps are specified, a single empty dictionary is provided so that a new chain always has at least one mapping. The underlying mappings are stored in a list. That list is public and can be accessed or updated using the `maps` attribute. There is no other state. Lookups search the underlying mappings successively until a key is found. In contrast, writes, updates, and deletions only operate on the first

A user updateable list of mappings. The list is ordered from first-searched to last-searched. It is the only stored state and can be modified to change which mappings are searched. The list should always contain at least one mapping.

Returns a new `ChainMap` containing a new map followed by all of the maps in the current instance. If `m` is specified, it becomes the new map at the front of the list of mappings; if not specified, an empty dict is used, so that a call to `new_child()` is equivalent to `ChainMap({}, *d.maps)`. If any keyword arguments are specified, they update passed map or new empty dict. This method is used for creating subcontexts that can be updated without altering values in any of the parent mappings. Changed in version 3.9

Property returning a new `ChainMap` containing all of the maps in the current instance except the first one. This is useful for skipping the first map in the search. Use cases are similar to those for the `nonlocal` keyword used in nested scopes. The use cases also parallel those for the built-in `super()` function. A reference to `parents` is equivalent to `ChainMap(*d.maps[1:])`.

See also

- The `MultiContext` class in the `EnthoughtCodeTools` package has options to support writing to any mapping in the chain.

- Django's Context class for templating is a read-only chain of mappings. It also features pushing and popping of contexts similar to the new_child() method and the parents property.
- The Nested Contexts recipe has options to control whether writes and other mutations apply only to the first mapping or to any mapping in the chain.
- A greatly simplified read-only version of Chainmap.

ChainMap Examples and Recipes

This section shows various approaches to working with chained maps.

Example of simulating Python's internal lookup chain:

```
import builtins
pylookup = ChainMap(locals(), globals(), vars(builtins))
```

Example of letting user specified command-line arguments take precedence over environment variables which in turn take precedence over default values:

```
import os, argparse

defaults = {'color': 'red', 'user': 'guest'}

parser = argparse.ArgumentParser()
parser.add_argument('-u', '--user')
parser.add_argument('-c', '--color')
namespace = parser.parse_args()
command_line_args = {k: v for k, v in vars(namespace).items() if v is not None}

combined = ChainMap(command_line_args, os.environ, defaults)
print(combined['color'])
print(combined['user'])
```

Example patterns for using the ChainMap class to simulate nested contexts:

```
c = ChainMap()           # Create root context
d = c.new_child()        # Create nested child context
e = c.new_child()        # Child of c, independent from d
e.maps[0]                # Current context dictionary -- like Python's locals()
e.maps[-1]               # Root context -- like Python's globals()
e.parents                # Enclosing context chain -- like Python's nonlocals

d['x'] = 1               # Set value in current context
d['x']                   # Get first key in the chain of contexts
del d['x']                # Delete from current context
list(d)                  # All nested values
k in d                   # Check all nested values
len(d)                   # Number of nested values
d.items()                # All nested items
dict(d)                  # Flatten into a regular dictionary
```

The ChainMap class only makes updates (writes and deletions) to the first mapping in the chain while lookups will search the full chain. However, if deep writes and deletions are desired, it is easy to make a subclass that updates keys found deeper in the chain:

```
class DeepChainMap(ChainMap):
    'Variant of ChainMap that allows direct updates to inner scopes'

    def __setitem__(self, key, value):
        for mapping in self.maps:
            if key in mapping:
                mapping[key] = value
                return
        self.maps[0][key] = value
```

```

def __delitem__(self, key):
    for mapping in self.maps:
        if key in mapping:
            del mapping[key]
    return
    raise KeyError(key)

>>> d = DeepChainMap({'zebra': 'black'}, {'elephant': 'blue'}, {'lion': 'yellow'})
>>> d['lion'] = 'orange'          # update an existing key two levels down
>>> d['snake'] = 'red'           # new keys get added to the topmost dict
>>> del d['elephant']            # remove an existing key one level down
>>> d                            # display result
DeepChainMap({'zebra': 'black', 'snake': 'red'}, {}, {'lion': 'orange'})

```

Counterobjects

A counter tool is provided to support convenient and rapid tallies. For example:

```

>>> # Tally occurrences of words in a list
>>> cnt = Counter()
>>> for word in ['red', 'blue', 'red', 'green', 'blue', 'blue']:
...     cnt[word] += 1
...
>>> cnt
Counter({'blue': 3, 'red': 2, 'green': 1})

>>> # Find the ten most common words in Hamlet
>>> import re
>>> words = re.findall(r'\w+', open('hamlet.txt').read().lower())
>>> Counter(words).most_common(10)
[('the', 1143), ('and', 966), ('to', 762), ('of', 669), ('i', 631),
 ('you', 554), ('a', 546), ('my', 514), ('hamlet', 471), ('in', 451)]

```

classcollections.Counter(kwargs)**

classcollections.Counter(iterable,kwargs)**

classcollections.Counter(mapping,kwargs)**

elements()

most_common(n=None)

subtract(kwargs)**

subtract(iterable,kwargs)**

subtract(mapping,kwargs)**

total()

fromkeys(iterable)

update(kwargs)**

update(iterable,kwargs)**

update(mapping,kwargs)**

A Counter is a dict subclass for counting hashable objects. It is a collection where elements are stored as dictionary keys and their counts are stored as dictionary values. Counts are allowed to be any integer value including zero or negative counts. The Counter class is similar to bags or multisets in other languages. Elements are counted from an iterable or initialized from another mapping (or counter):

```
>>> c = Counter() # a new, empty counter
>>> c = Counter('gallahad') # a new counter from an iterable
>>> c = Count
```

Return an iterator over elements repeating each as many times as its count. Elements are returned in the order first encountered. If an element's count is less than one, `elements()` will ignore it. `>>>c=Counter(a=4,b=2,c=0,d=-2)>>>sorted(c.elements())['a', 'a', 'a', 'a', 'b', 'b']`

Return a list of the most common elements and their counts from the most common to the least. If `n` is omitted or `None`, `most_common()` returns all elements in the counter. Elements with equal counts are ordered in the order first encountered: `>>>Counter('abracadabra').most_common(3)[('a', 5), ('b', 2), ('r', 2)]`

Elements are subtracted from an iterable or from another mapping (or counter). Like `dict.update()` but subtracts counts instead of replacing them. Both inputs and outputs may be zero or negative. `>>>c=Counter(a=4,b=2,c=0,d=-2)>>>d=Counter(a=1,b=2,c=3,d=4)>>>c.subtract(d)>>>c` Counter({'a': 3, 'b': 0, 'c': -3, 'd': -6}) Added in version 3.2.

Compute the sum of the counts. `>>>c=Counter(a=10,b=5,c=0)>>>c.total()` 15 Added in version 3.10.

This class method is not implemented for Counter objects.

Elements are counted from an iterable or added-in from another mapping (or counter). Like `dict.update()` but adds counts instead of replacing them. Also, the iterable is expected to be a sequence of elements, not a sequence of (key,value) pairs.

Counters support rich comparison operators for equality, subset, and superset relationships: `==`, `!=`, `<`, `<=`, `>`, `>=`. All of those tests treat missing elements as having zero counts so that `Counter(a=1)==Counter(a=1,b=0)` returns `true`.

Changed in version 3.10: Rich comparison operations were added.

Changed in version 3.10: In equality tests, missing elements are treated as having zero counts.

Formerly, `Counter(a=3)` and `Counter(a=3,b=0)` were considered distinct.

Common patterns for working with Counter objects:

```
c.total()           # total of all counts
c.clear()           # reset all counts
list(c)             # list unique elements
set(c)              # convert to a set
dict(c)             # convert to a regular dictionary
c.items()           # access the (elem, cnt) pairs
Counter(dict(list_of_pairs)) # convert from a list of (elem, cnt) pairs
c.most_common()[:n] # n least common elements
+c                 # remove zero and negative counts
```

Several mathematical operations are provided for combining Counter objects to produce multisets (counters that have counts greater than zero). Addition and subtraction combine counters by adding or subtracting the counts of corresponding elements. Intersection and union return the minimum and maximum of corresponding counts. Equality and inclusion compare corresponding counts. Each operation can accept inputs with signed counts, but the output will exclude results with counts of zero or less.

```
>>> c = Counter(a=3, b=1)
>>> d = Counter(a=1, b=2)
>>> c + d           # add two counters together:  c[x] + d[x]
Counter({'a': 4, 'b': 3})
>>> c - d           # subtract (keeping only positive counts)
Counter({'a': 2})
>>> c & d           # intersection:  min(c[x], d[x])
Counter({'a': 1, 'b': 1})
>>> c | d           # union:  max(c[x], d[x])
Counter({'a': 3, 'b': 2})
>>> c == d          # equality:  c[x] == d[x]
False
>>> c <= d          # inclusion:  c[x] <= d[x]
False
```

Unary addition and subtraction are shortcuts for adding an empty counter or subtracting from an empty counter.

```
>>> c = Counter(a=2, b=-4)
>>> +c
Counter({'a': 2})
>>> -c
Counter({'b': 4})
```

Added in version 3.3: Added support for unary plus, unary minus, and in-place multiset operations.

■ Nota: NoteCounters were primarily designed to work with positive integers to represent running counts; however, care was taken to not unnecessarily preclude use cases needing other types or negative values. To help with those use cases, this section documents the minimum range and type restrictions. The Counter class itself is a dictionary subclass with no restrictions on its keys and values. The values are intended to be numbers representing counts, but you could store anything in the value field. The most_common() method requires only that the values be orderable. For in-place operations such as asc[key]+=1

See also

- Bag class in Smalltalk.
- Wikipedia entry for Multisets.
- C++ multiset tutorial with examples.
- For mathematical operations on multisets and their use cases, see Knuth, Donald. The Art of Computer Programming Volume II, Section 4.6.3, Exercise 19.
- To enumerate all distinct multisets of a given size over a given set of elements, see `itertools.combinations_with_replacement():map(Counter, combinations_with_replacement('ABC', 2))` # --> AA AB AC BB BC CC

deque objects

`collections.deque([iterable[, maxlen]])`

`append(item, /)`

`appendleft(item, /)`

`clear()`

`copy()`

`count(value, /)`

`extend(iterable, /)`

`extendleft(iterable, /)`

`index(value[, start[, stop]])`

`insert(index, value, /)`

`pop()`

`popleft()`

`remove(value, /)`

`reverse()`

`rotate(n=1, /)`

`maxlen`

```
>>> from collections import deque
>>> d = deque('ghi') # make a new deque with three items
>>> for elem in d: # iterate over the deque's elements
...     print(elem.upper())
G
H
I

>>> d.append('j') # add a new entry to the right side
>>> d.appendleft('f') # add a new entry to the left side
>>> d # show the representation of the deque
deque(['f', 'g', 'h', 'i', 'j'])

>>> d.pop() # return and remove the rightmost item
```

```

'j'
>>> d.popleft()           # return and remove the leftmost item
'f'
>>> list(d)               # list the contents of the deque
['g', 'h', 'i']
>>> d[0]                  # peek at leftmost item
'g'
>>> d[-1]                 # peek at rightmost item
'i'

>>> list(reversed(d))     # list the contents of a deque in reverse
['i', 'h', 'g']
>>> 'h' in d              # search the deque
True
>>> d.extend('jkl')       # add multiple elements at once
>>> d
deque(['g', 'h', 'i', 'j', 'k', 'l'])
>>> d.rotate(1)           # right rotation
>>> d
deque(['l', 'g', 'h', 'i', 'j', 'k'])
>>> d.rotate(-1)          # left rotation
>>> d
deque(['g', 'h', 'i', 'j', 'k', 'l'])

>>> deque(reversed(d))    # make a new deque in reverse order
deque(['l', 'k', 'j', 'i', 'h', 'g'])
>>> d.clear()             # empty the deque
>>> d.pop()               # cannot pop from an empty deque
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    d.pop()
IndexError: pop from an empty deque

>>> d.extendleft('abc')   # extendleft() reverses the input order
>>> d
deque(['c', 'b', 'a'])

```

dequeRecipes

This section shows various approaches to working with deques.

Bounded length deques provide functionality similar to `thetailfilter` in Unix:

```

def tail(filename, n=10):
    'Return the last n lines of a file'
    with open(filename) as f:
        return deque(f, n)

```

Another approach to using deques is to maintain a sequence of recently added elements by appending to the right and popping to the left:

```

def moving_average(iterable, n=3):
    # moving_average([40, 30, 50, 46, 39, 44]) --> 40.0 42.0 45.0 43.0
    # https://en.wikipedia.org/wiki/Moving_average
    it = iter(iterable)
    d = deque(itertools.islice(it, n-1))
    d.appendleft(0)
    s = sum(d)
    for elem in it:
        s += elem - d.popleft()
        d.append(elem)
        yield s / n

```

Around-robin scheduler can be implemented with input iterators stored in a deque. Values are yielded from the active iterator in position zero. If that iterator is exhausted, it can be removed with `popleft()`; otherwise, it can be

cycled back to the end with the `rotate()` method:

```
def roundrobin(*iterables):
    "roundrobin('ABC', 'D', 'EF') --> A D E B F C"
    iterators = deque(map(iter, iterables))
    while iterators:
        try:
            while True:
                yield next(iterators[0])
                iterators.rotate(-1)
            except StopIteration:
                # Remove an exhausted iterator.
                iterators.popleft()
```

The `rotate()` method provides a way to implement deque slicing and deletion. For example, a pure Python implementation of `del d[n]` relies on the `rotate()` method to position elements to be popped:

```
def delete_nth(d, n):
    d.rotate(-n)
    d.popleft()
    d.rotate(n)
```

To implement deque slicing, use a similar approach applying `rotate()` to bring a target element to the left side of the deque. Remove old entries with `popleft()`, add new entries with `extend()`, and then reverse the rotation. With minor variations on that approach, it is easy to implement Forth style stack manipulations such as `dup`, `drop`, `swap`, `over`, `pick`, `rot`, and `roll`.

defaultdict objects

`collections.defaultdict(default_factory=..., **kwargs)`

`collections.defaultdict(default_factory, mapping, **kwargs)`

`collections.defaultdict(default_factory, iterable, **kwargs)`

`__missing__(key,)`

`default_factory`

Return a new dictionary-like object. `defaultdict` is a subclass of the built-in `dict` class. It overrides one method and adds one writable instance variable. The remaining functionality is the same as for the `dict` class and is not documented here. The first argument provides the initial value for the `default_factory` attribute; it defaults to `None`. All remaining arguments are treated the same as if they were passed to the `dict` constructor, including keyword arguments. `defaultdict` objects support the following

If the `default_factory` attribute is `None`, this raises a `KeyError` exception with the `key` as argument. If `default_factory` is not `None`, it is called without arguments to provide a default value for the given `key`, this value is inserted in the dictionary for the `key`, and returned. If calling `default_factory` raises an exception this exception is propagated unchanged. This method is called by the `__getitem__()` method of the `dict` class when the requested key is not found; whatever it returns or raises is then returned or r

This attribute is used by the `__missing__()` method; it is initialized from the first argument to the constructor, if present, or to `None`, if absent.

defaultdict Examples

Using lists as the `default_factory`, it is easy to group a sequence of key-value pairs into a dictionary of lists:

```
>>> s = [('yellow', 1), ('blue', 2), ('yellow', 3), ('blue', 4), ('red', 1)]
>>> d = defaultdict(list)
>>> for k, v in s:
...     d[k].append(v)
```



```
...
>>> sorted(d.items())
[('blue', [2, 4]), ('red', [1]), ('yellow', [1, 3])]
```

When each key is encountered for the first time, it is not already in the mapping; so an entry is automatically created using `thedefault_factoryfunction` which returns an empty list. The `thelist.append()` operation then attaches the value to the new list. When keys are encountered again, the look-up proceeds normally (returning the list for that key) and the `thelist.append()` operation adds another value to the list. This technique is simpler and faster than an equivalent technique using `dict.setdefault()`:

```
>>> d = {}
>>> for k, v in s:
...     d.setdefault(k, []).append(v)
...
>>> sorted(d.items())
[('blue', [2, 4]), ('red', [1]), ('yellow', [1, 3])]
```

Setting `thedefault_factory` to `int` makes `thedefaultdict` useful for counting (like a bag or multiset in other languages):

```
>>> s = 'mississippi'
>>> d = defaultdict(int)
>>> for k in s:
...     d[k] += 1
...
>>> sorted(d.items())
[('i', 4), ('m', 1), ('p', 2), ('s', 4)]
```

When a letter is first encountered, it is missing from the mapping, so `thedefault_factory` function calls `int()` to supply a default count of zero. The increment operation then builds up the count for each letter.

The function `int()` which always returns zero is just a special case of constant functions. A faster and more flexible way to create constant functions is to use a lambda function which can supply any constant value (not just zero):

```
>>> def constant_factory(value):
...     return lambda: value
...
>>> d = defaultdict(constant_factory('<missing>'))
>>> d.update(name='John', action='ran')
>>> '%(name)s %(action)s to %(object)s' % d
'John ran to <missing>'
```

Setting `thedefault_factory` to `set` makes `thedefaultdict` useful for building a dictionary of sets:

```
>>> s = [('red', 1), ('blue', 2), ('red', 3), ('blue', 4), ('red', 1), ('blue', 4)]
>>> d = defaultdict(set)
>>> for k, v in s:
...     d[k].add(v)
...
>>> sorted(d.items())
[('blue', {2, 4}), ('red', {1, 3})]
```

namedtuple() Factory Function for Tuples with Named Fields

Named tuples assign meaning to each position in a tuple and allow for more readable, self-documenting code. They can be used wherever regular tuples are used, and they add the ability to access fields by name instead of position index.

`collections.namedtuple(typename, field_names, *, rename=False, defaults=None, module=None)`

Returns a new tuple subclass named `typename`. The new subclass is used to create tuple-like objects that have fields accessible by attribute lookup as well as being indexable and iterable. Instances of the subclass also have a helpful docstring (with `typename` and `field_names`) and a helpful `__repr__()` method which lists the tuple

contents in `aname=valueformat`. The `field_names` are a sequence of strings such as `['x','y']`. Alternatively, `field_names` can be a single string with each field name separated by white

```
>>> # Basic example
>>> Point = namedtuple('Point', ['x', 'y'])
>>> p = Point(11, y=22)      # instantiate with positional or keyword arguments
>>> p[0] + p[1]              # indexable like the plain tuple (11, 22)
33
>>> x, y = p                 # unpack like a regular tuple
>>> x, y
(11, 22)
>>> p.x + p.y                # fields also accessible by name
33
>>> p                        # readable __repr__ with a name=value style
Point(x=11, y=22)
```

Named tuples are especially useful for assigning field names to result tuples returned by the `csv` and `sqlite3` modules:

```
EmployeeRecord = namedtuple('EmployeeRecord', 'name, age, title, department, paygrade')
```

```
import csv
for emp in map(EmployeeRecord._make, csv.reader(open("employees.csv", "rb"))):
    print(emp.name, emp.title)

import sqlite3
conn = sqlite3.connect('/companydata')
cursor = conn.cursor()
cursor.execute('SELECT name, age, title, department, paygrade FROM employees')
for emp in map(EmployeeRecord._make, cursor.fetchall()):
    print(emp.name, emp.title)
```

In addition to the methods inherited from tuples, named tuples support three additional methods and two attributes. To prevent conflicts with field names, the method and attribute names start with an underscore.

`somenamedtuple._make(iterable, /)`

Class method that makes a new instance from an existing sequence or iterable. `>>>t=[11,22]>>>Point._make(t)Point(x=11, y=22)`

`somenamedtuple._asdict()`

Return a new dict which maps field names to their corresponding values: `>>>p=Point(x=11,y=22)>>>p._asdict(){'x': 11, 'y': 22}` Changed in version 3.1: Returns an `OrderedDict` instead of a regular dict. Changed in version 3.8: Returns a regular dict instead of an `OrderedDict`. As of Python 3.7, regular dicts are guaranteed to be ordered. If the extra features of `OrderedDict` are required, the suggested remediation is to cast the result to the desired type: `OrderedDict(nt._asdict())`.

`somenamedtuple._replace(**kwargs)`

Return a new instance of the named tuple replacing specified fields with new values: `>>>p=Point(x=11,y=22)>>>p._replace(x=33)Point(x=33, y=22)>>>for partnum, record in inventory.items():
..inventory[partnum]=record._replace(price=newprices[partnum], timestamp=time.now())` Named tuples are also supported by generic function `copy.replace()`. Changed in version 3.13: Raise `TypeError` instead of `ValueError` for invalid keyword arguments.

`somenamedtuple._fields`

Tuple of strings listing the field names. Useful for introspection and for creating new named tuple types from existing named tuples. `>>>p._fields# view the field names('x', 'y')>>>Color=namedtuple('Color','red green blue')>>>Pixel=namedtuple('Pixel',Point._fields+Color._fields)>>>Pixel(11,22,128,255,0)Pixel(x=11, y=22, red=128, green=255, blue=0)`

somenamedtuple._field_defaults

Dictionary mapping field names to default

```
values.>>>Account=namedtuple('Account',['type','balance'],defaults=[0])>>>Account._field_defaults{'balance':0}>>>Account('premium')Account(type='premium', balance=0)
```

To retrieve a field whose name is stored in a string, use the `getattr()` function:

```
>>> getattr(p, 'x')
11
```

To convert a dictionary to a named tuple, use the double-star-operator (as described in [Unpacking Argument Lists](#)):

```
>>> d = {'x': 11, 'y': 22}
>>> Point(**d)
Point(x=11, y=22)
```

Since a named tuple is a regular Python class, it is easy to add or change functionality with a subclass. Here is how to add a calculated field and a fixed-width print format:

```
>>> class Point(namedtuple('Point', ['x', 'y'])):
...     __slots__ = ()
...     @property
...     def hypot(self):
...         return (self.x ** 2 + self.y ** 2) ** 0.5
...     def __str__(self):
...         return 'Point: x=%6.3f y=%6.3f hypot=%6.3f' % (self.x, self.y, self.hypot)

>>> for p in Point(3, 4), Point(14, 5/7):
...     print(p)
Point: x= 3.000 y= 4.000 hypot= 5.000
Point: x=14.000 y= 0.714 hypot=14.018
```

The subclass shown above sets `__slots__` to an empty tuple. This helps keep memory requirements low by preventing the creation of instance dictionaries.

Subclassing is not useful for adding new, stored fields. Instead, simply create a new named tuple type from the `_fields` attribute:

```
>>> Point3D = namedtuple('Point3D', Point._fields + ('z',))
```

Docstrings can be customized by making direct assignments to the `__doc__` fields:

```
>>> Book = namedtuple('Book', ['id', 'title', 'authors'])
>>> Book.__doc__ += ': Hardcover book in active collection'
>>> Book.id.__doc__ = '13-digit ISBN'
>>> Book.title.__doc__ = 'Title of first printing'
>>> Book.authors.__doc__ = 'List of authors sorted by last name'
```

Changed in version 3.5: Property docstrings became writeable.

See also

- `Seetyping.NamedTuple` for a way to add type hints for named tuples. It also provides an elegant notation using the `classkeyword`: `class Component(NamedTuple): part_number: int weight: float description: Optional[str] = None`
- `Seetypes.SimpleNamespace()` for a mutable namespace based on an underlying dictionary instead of a tuple.
- The `dataclasses` module provides a decorator and functions for automatically adding generated special methods to user-defined classes.

OrderedDict objects

Ordered dictionaries are just like regular dictionaries but have some extra capabilities relating to ordering operations. They have become less important now that the built-in `dict` class gained the ability to remember insertion order (this new behavior became guaranteed in Python 3.7).

Some differences from `dict` still remain:

- The regular `dict` was designed to be very good at mapping operations. Tracking insertion order was secondary.
- The `OrderedDict` was designed to be good at reordering operations. Space efficiency, iteration speed, and the performance of update operations were secondary.
- The `OrderedDict` algorithm can handle frequent reordering operations better than `dict`. As shown in the recipes below, this makes it suitable for implementing various kinds of LRU caches.
- The equality operation for `OrderedDict` checks for matching order. A regular `dict` can emulate the order sensitive equality test with `p==q and all(k1==k2 for k1,k2 in zip(p,q))`.
- The `popitem()` method of `OrderedDict` has a different signature. It accepts an optional argument to specify which item is popped. A regular `dict` can emulate `OrderedDict`'s `popitem(last=True)` with `d.popitem()` which is guaranteed to pop the rightmost (last) item. A regular `dict` can emulate `OrderedDict`'s `popitem(last=False)` with `(k:=next(iter(d)),d.pop(k))` which will return and remove the leftmost (first) item if
- `OrderedDict` has a `move_to_end()` method to efficiently reposition an element to an endpoint. A regular `dict` can emulate `OrderedDict`'s `move_to_end(k,last=True)` with `d[k]=d.pop(k)` which will move the key and its associated value to the rightmost (last) position. A regular `dict` does not have an efficient equivalent for `OrderedDict`'s `move_to_end(k,last=False)` which moves the key and its associated value to the
- Until Python 3.8, `dict` lacked a `__reversed__()` method.

`collections.OrderedDict(**kwargs)`

`collections.OrderedDict(mapping,/,**kwargs)`

`collections.OrderedDict(iterable,/,**kwargs)`

`popitem(last=True)`

`move_to_end(key,last=True)`

Return an instance of `dict` subclass that has methods specialized for rearranging dictionary order. Added in version 3.1. `popitem(last=True)` The `popitem()` method for ordered dictionaries returns and removes a (key, value) pair. The pairs are returned in LIFO order if `last` is true or FIFO order if false. `move_to_end(key,last=True)` Move an existing key to either end of an ordered dictionary. The item is moved to the right end if `last` is true (the default) or to the beginning if `last` is false. Raises `KeyError` if the key

The `popitem()` method for ordered dictionaries returns and removes a (key, value) pair. The pairs are returned in LIFO order if `last` is true or FIFO order if false.

Move an existing key to either end of an ordered dictionary. The item is moved to the right end if `last` is true (the default) or to the beginning if `last` is false. Raises `KeyError` if the key does not exist: `>>>d=OrderedDict.fromkeys('abcde')>>>d.move_to_end('b')>>>".join(d)'acdeb'>>>d.move_to_end('b',last=False)>>>".join(d)'bacde'` Added in version 3.2.

In addition to the usual mapping methods, ordered dictionaries also support reverse iteration using `reversed()`.

Equality tests between `OrderedDict` objects are order-sensitive and are roughly equivalent to `list(od1.items())==list(od2.items())`.

Equality tests between `OrderedDict` objects and other `Mapping` objects are order-insensitive like regular dictionaries. This allows `OrderedDict` objects to be substituted anywhere a regular dictionary is used.

Changed in version 3.5: The items, keys, and values views of `OrderedDict` now support reverse iteration using `reversed()`.

Changed in version 3.6: With the acceptance of PEP 468, order is retained for keyword arguments passed to the `OrderedDict` constructor and its `update()` method.

Changed in version 3.9: Added merge (`|`) and update (`|=`) operators, specified in PEP 584.

OrderedDict Examples and Recipes

It is straightforward to create an ordered dictionary variant that remembers the order the keys were last inserted. If a new entry overwrites an existing entry, the original insertion position is changed and moved to the end:

```
class LastUpdatedOrderedDict(OrderedDict):
    'Store items in the order the keys were last added'

    def __setitem__(self, key, value):
        super().__setitem__(key, value)
        self.move_to_end(key)
```

An `OrderedDict` would also be useful for implementing variants of `functools.lru_cache()`:

```
from collections import OrderedDict
from time import time

class TimeBoundedLRU:
    "LRU Cache that invalidates and refreshes old entries."

    def __init__(self, func, maxsize=128, maxage=30):
        self.cache = OrderedDict() # { args : (timestamp, result) }
        self.func = func
        self.maxsize = maxsize
        self.maxage = maxage

    def __call__(self, *args):
        if args in self.cache:
            self.cache.move_to_end(args)
            timestamp, result = self.cache[args]
            if time() - timestamp <= self.maxage:
                return result
        result = self.func(*args)
        self.cache[args] = time(), result
        if len(self.cache) > self.maxsize:
            self.cache.popitem(last=False)
        return result

class MultiHitLRUCache:
    """ LRU cache that defers caching a result until
        it has been requested multiple times.

        To avoid flushing the LRU cache with one-time requests,
        we don't cache until a request has been made more than once.

    """

    def __init__(self, func, maxsize=128, maxrequests=4096, cache_after=1):
        self.requests = OrderedDict() # { uncached_key : request_count }
        self.cache = OrderedDict() # { cached_key : function_result }
        self.func = func
        self.maxrequests = maxrequests # max number of uncached requests
        self.maxsize = maxsize # max number of stored return values
        self.cache_after = cache_after

    def __call__(self, *args):
        if args in self.cache:
```

```

        self.cache.move_to_end(args)
        return self.cache[args]
    result = self.func(*args)
    self.requests[args] = self.requests.get(args, 0) + 1
    if self.requests[args] <= self.cache_after:
        self.requests.move_to_end(args)
        if len(self.requests) > self.maxrequests:
            self.requests.popitem(last=False)
    else:
        self.requests.pop(args, None)
        self.cache[args] = result
        if len(self.cache) > self.maxsize:
            self.cache.popitem(last=False)
    return result

```

UserDictobjects

The class,UserDictacts as a wrapper around dictionary objects. The need for this class has been partially supplanted by the ability to subclass directly fromdict; however, this class can be easier to work with because the underlying dictionary is accessible as an attribute.

classcollections.UserDict(kwargs)**

classcollections.UserDict(mapping,/,kwargs)**

classcollections.UserDict(iterable,/,kwargs)**

data

Class that simulates a dictionary. The instance's contents are kept in a regular dictionary, which is accessible via thedataattribute ofUserDictinstances. If arguments are provided, they are used to initialize data, like a regular dictionary. In addition to supporting the methods and operations of mappings,UserDictinstances provide the following attribute: dataA real dictionary used to store the contents of theUserDictclass.

A real dictionary used to store the contents of theUserDictclass.

UserListobjects

This class acts as a wrapper around list objects. It is a useful base class for your own list-like classes which can inherit from them and override existing methods or add new ones. In this way, one can add new behaviors to lists.

The need for this class has been partially supplanted by the ability to subclass directly fromlist; however, this class can be easier to work with because the underlying list is accessible as an attribute.

classcollections.UserList([list])

data

Class that simulates a list. The instance's contents are kept in a regular list, which is accessible via thedataattribute ofUserListinstances. The instance's contents are initially set to a copy oflist, defaulting to the empty list[].listcan be any iterable, for example a real Python list or aUserListobject. In addition to supporting the methods and operations of mutable sequences,UserListinstances provide the following attribute: dataA real listobject used to store the contents of theUserListclass.

A real listobject used to store the contents of theUserListclass.

Subclassing requirements: Subclasses ofUserListare expected to offer a constructor which can be called with either no arguments or one argument. List operations which return a new sequence attempt to create an

instance of the actual implementation class. To do so, it assumes that the constructor can be called with a single parameter, which is a sequence object used as a data source.

If a derived class does not wish to comply with this requirement, all of the special methods supported by this class will need to be overridden; please consult the sources for information about the methods which need to be provided in that case.

UserStringobjects

The class, `UserString` acts as a wrapper around string objects. The need for this class has been partially supplanted by the ability to subclass directly from `str`; however, this class can be easier to work with because the underlying string is accessible as an attribute.

`collections.UserString(seq)`

data

Class that simulates a string object. The instance's content is kept in a regular string object, which is accessible via the `data` attribute of `UserString` instances. The instance's contents are initially set to a copy of `seq`. The `seq` argument can be any object which can be converted into a string using the built-in `str()` function. In addition to supporting the methods and operations of strings, `UserString` instances provide the following attribute: `data` A `str` object used to store the contents of the `UserString` class.

A `str` object used to store the contents of the `UserString` class.

collections.abc— Abstract Base Classes for Containers

Added in version 3.3: Formerly, this module was part of the `collections` module.

Source code: `Lib/_collections_abc.py`

This module provides abstract base classes that can be used to test whether a class provides a particular interface; for example, whether it is hashable or whether it is a mapping.

An `issubclass()` or `isinstance()` test for an interface works in one of three ways.

- A newly written class can inherit directly from one of the abstract base classes. The class must supply the required abstract methods. The remaining mixin methods come from inheritance and can be overridden if desired. Other methods may be added as needed:

```
class C(Sequence): # Direct inheritance
    def __init__(self): ... # Extra method not required by the ABC
    def __getitem__(self, index): ... # Required abstract method
```
- Existing classes and built-in classes can be registered as “virtual subclasses” of the ABCs. Those classes should define the full API including all of the abstract methods and all of the mixin methods. This lets users rely on `issubclass()` or `isinstance()` tests to determine whether the full interface is supported. The exception to this rule is for methods that are automatically inferred from the rest
- Some simple interfaces are directly recognizable by the presence of the required methods (unless those methods have been set to `None`):

```
class E:
    def __iter__(self): ...
    def __next__(self): ...
>>> issubclass(E, Iterable)
True
>>> isinstance(E(), Iterable)
True
```

Complex interfaces do not support this last technique because an interface is more than just the presence of method names. Interfaces specify semantics and r

Added in version 3.9: These abstract classes now support `[]`. See [Generic Alias Type](#) and [PEP 585](#).

Collections Abstract Base Classes

The `collections` module offers the following ABCs:

```
ABC | Inherits from | Abstract Methods | Mixin Methods | Container | [ | 1 | ] | __contains__ |
Hashable | [ | 1 | ] | __hash__ | Iterable | [ | 1 | ] | [ | 2 | ] | __iter__ | Iterator | [ | 1 | ] |
Iterable | __next__ | __iter__ | Reversible | [ | 1 | ] | Iterable | __reversed__ |
Generator | [ | 1 | ] | Iterator | send | , | throw | close | , | __iter__ | , | __next__ |
Sized | [ | 1 | ] | __len__ | Callable | [ | 1 | ] | __call__ | Collection | [ | 1 | ] | Sized | ,
| Iterable | , | Container | __contains__ | , | __iter__ | , | __len__ | Sequence | Reversible
| , | Collection | __getitem__ | , | __len__ | __contains__ | , | __iter__ | , | __reversed__ | ,
| index | , and | count | MutableSequence | Sequence | __getitem__ | , | __setitem__ | , |
__delitem__ | , | __len__ | , | insert | I
```

Footnotes

These ABCs override `__subclasshook__()` to support testing an interface by verifying the required methods are present and have not been set to `None`. This only works for simple interfaces. More complex interfaces require registration or direct subclassing.

Checking `isinstance(obj, Iterable)` detects classes that are registered as `Iterable` or that have an `__iter__()` method, but it does not detect classes that iterate with the `__getitem__()` method. The only reliable way to determine whether an object is iterable is to call `iter(obj)`.

Collections Abstract Base Classes – Detailed Descriptions

`class collections.abc.Container`

ABC for classes that provide the `__contains__()` method.

collections.abc.Hashable

ABC for classes that provide the `__hash__()` method.

collections.abc.Sized

ABC for classes that provide the `__len__()` method.

collections.abc.Callable

ABC for classes that provide the `__call__()` method. See [Annotating callable objects](#) for details on how to use `Callable` in type annotations.

collections.abc.Iterable

ABC for classes that provide the `__iter__()` method. `Checkingisinstance(obj, Iterable)` detects classes that are registered as iterable or that have an `__iter__()` method, but it does not detect classes that iterate with the `__getitem__()` method. The only reliable way to determine whether an object is iterable is to call `iter(obj)`.

collections.abc.Collection

ABC for sized iterable container classes. Added in version 3.6.

collections.abc.Iterator

ABC for classes that provide the `__iter__()` and `__next__()` methods. See also the definition of `iterator`.

collections.abc.Reversible

ABC for iterable classes that also provide the `__reversed__()` method. Added in version 3.6.

collections.abc.Generator

ABC for generator classes that implement the protocol defined in PEP 342 that extends iterators with the `send()`, `throw()` and `close()` methods. See [Annotating generators and coroutines](#) for details on using `Generator` in type annotations. Added in version 3.5.

collections.abc.Sequence

collections.abc.MutableSequence

collections.abc.ByteString

index(value, start=0, stop=None)

ABCs for read-only and mutable sequences. Implementation note: Some of the mixin methods, such as `__iter__()`, `__reversed__()`, and `index()` make repeated calls to the underlying `__getitem__()` method. Consequently, if `__getitem__()` is implemented with constant access speed, the mixin methods will have linear performance; however, if the underlying method is linear (as it would be with a linked list), the mixins will have quadratic performance and will likely need to be overridden. `index(value, start=0, stop=None)`

Return first index of `value`. Raises `ValueError` if the value is not present. Supporting the `start` and `stop` arguments is optional, but recommended. Changed in version 3.5: The `index()` method gained support for the `stop` and `start` arguments.

collections.abc.Set

collections.abc.MutableSet

ABCs for read-only and mutable sets.

collections.abc.Mapping

collections.abc.MutableMapping

ABCs for read-only and mutable mappings.

collections.abc.MappingView

collections.abc.ItemsView

collections.abc.KeysView

collections.abc.ValuesView

ABCs for mapping, items, keys, and values views.

collections.abc.Awaitable

ABC for awaitable objects, which can be used in await expressions. Custom implementations must provide the `__await__()` method. Coroutine objects and instances of the `CoroutineABC` are all instances of this ABC. Note in CPython, generator-based coroutines (generators decorated with `@types.coroutine`) are awaitables, even though they do not have an `__await__()` method. Using `isinstance(gencoro, Awaitable)` for them will return `False`. Use `inspect.isawaitable()` to detect them. Added in version 3.5.

collections.abc.Coroutine

ABC for coroutine-compatible classes. These implement the following methods, defined in `Coroutine Objects: send(), throw(), and close()`. Custom implementations must also implement `__await__()`. All `Coroutine` instances are also instances of `Awaitable`. Note in CPython, generator-based coroutines (generators decorated with `@types.coroutine`) are awaitables, even though they do not have an `__await__()` method. Using `isinstance(gencoro, Coroutine)` for them will return `False`. Use `inspect.isawaitable()` to detect them. See [Anno](#)

collections.abc.AsyncIterable

ABC for classes that provide an `__aiter__` method. See also the definition of asynchronous iterable. Added in version 3.5.

collections.abc.AsyncIterator

ABC for classes that provide `__aiter__` and `__anext__` methods. See also the definition of asynchronous iterator. Added in version 3.5.

collections.abc.AsyncGenerator

ABC for asynchronous generator classes that implement the protocol defined in [PEP 525](#) and [PEP 492](#). See [Annotating generators and coroutines](#) for details on using `AsyncGenerator` in type annotations. Added in version 3.6.

collections.abc.Buffer

ABC for classes that provide the `__buffer__()` method, implementing the `buffer` protocol. See [PEP 688](#). Added in version 3.12.

Examples and Recipes

ABCs allow us to ask classes or instances if they provide particular functionality, for example:

```
size = None
if isinstance(myvar, collections.abc.Sized):
    size = len(myvar)
```

Several of the ABCs are also useful as mixins that make it easier to develop classes supporting container APIs. For example, to write a class supporting the `fullSet` API, it is only necessary to supply the three underlying abstract methods: `__contains__()`, `__iter__()`, and `__len__()`. The ABC supplies the remaining methods such

as `__and__()` and `disjoint()`:

```
class ListBasedSet(collections.abc.Set):
    ''' Alternate set implementation favoring space over speed
        and not requiring the set elements to be hashable. '''
    def __init__(self, iterable):
        self.elements = lst = []
        for value in iterable:
            if value not in lst:
                lst.append(value)

    def __iter__(self):
        return iter(self.elements)

    def __contains__(self, value):
        return value in self.elements

    def __len__(self):
        return len(self.elements)

s1 = ListBasedSet('abcdef')
s2 = ListBasedSet('defghi')
overlap = s1 & s2          # The __and__() method is supported automatically
```

Notes on using `Set` and `MutableSet` as a mixin:

- Since some set operations create new sets, the default mixin methods need a way to create new instances from an iterable. The class constructor is assumed to have a signature in the form `ClassName(iterable)`. That assumption is factored-out to an internal class method called `_from_iterable()` which calls `cls(iterable)` to produce a new set. If the `Set` mixin is being used in a class with a different constructor
- To override the comparisons (presumably for speed, as the semantics are fixed), redefine `__le__()` and `__ge__()`, then the other operations will automatically follow suit.
- The `Set` mixin provides a `_hash()` method to compute a hash value for the set; however, `__hash__()` is not defined because not all sets are hashable or immutable. To add set hashability using mixins, inherit from both `Set()` and `Hashable()`, then define `__hash__=Set._hash`.

See also

- `OrderedSet` recipe for an example built on `MutableSet`.
- For more about ABCs, see the `abc` module and PEP 3119.

heapq—Heap queue algorithm

Source code: `Lib/heapq.py`

This module provides an implementation of the heap queue algorithm, also known as the priority queue algorithm.

Min-heaps are binary trees for which every parent node has a value less than or equal to any of its children. We refer to this condition as the heap invariant.

For min-heaps, this implementation uses lists for which $\text{heap}[k] \leq \text{heap}[2*k+1]$ and $\text{heap}[k] \leq \text{heap}[2*k+2]$ for all k for which the compared elements exist. Elements are counted from zero. The interesting property of a min-heap is that its smallest element is always the root, `heap[0]`.

Max-heaps satisfy the reverse invariant: every parent node has a value greater than any of its children. These are implemented as lists for which $\text{maxheap}[2*k+1] \leq \text{maxheap}[k]$ and $\text{maxheap}[2*k+2] \leq \text{maxheap}[k]$ for all k for which the compared elements exist. The root, `maxheap[0]`, contains the largest element; `heap.sort(reverse=True)` maintains the max-heap invariant.

The `heapq` API differs from textbook heap algorithms in two aspects: (a) We use zero-based indexing. This makes the relationship between the index for a node and the indexes for its children slightly less obvious, but is more suitable since Python uses zero-based indexing. (b) Textbooks often focus on max-heaps, due to their suitability for in-place sorting. Our implementation favors min-heaps as they better correspond to Python lists.

These two aspects make it possible to view the heap as a regular Python list without surprises: `heap[0]` is the smallest item, and `heap.sort()` maintains the heap invariant!

Like `list.sort()`, this implementation uses only the `<` operator for comparisons, for both min-heaps and max-heaps.

In the API below, and in this documentation, the unqualified term `heap` generally refers to a min-heap. The API for max-heaps is named using a `_max` suffix.

To create a heap, use a list initialized as `[]`, or transform an existing list into a min-heap or max-heap using the `heapify()` or `heapify_max()` functions, respectively.

The following functions are provided for min-heaps:

`heapq.heapify(x)`

Transform `list` into a min-heap, in-place, in linear time.

`heapq.heappush(heap, item)`

Push the value `item` onto the heap, maintaining the min-heap invariant.

`heapq.heappop(heap)`

Pop and return the smallest item from the heap, maintaining the min-heap invariant. If the heap is empty, `IndexError` is raised. To access the smallest item without popping it, use `heap[0]`.

`heapq.heappushpop(heap, item)`

Push `item` on the heap, then pop and return the smallest item from the heap. The combined action runs more efficiently than `heappush()` followed by a separate call to `heappop()`.

`heapq.heapreplace(heap, item)`

Pop and return the smallest item from the heap, and also push the new `item`. The heap size doesn't change. If the heap is empty, `IndexError` is raised. This one step operation is more efficient than `heappop()` followed by `heappush()` and can be more appropriate when using a fixed-size heap. The pop/push combination always returns an element from the heap and replaces it with `item`. The value returned may be larger than the item added.

If that isn't desired, consider using `heapq.heappushpop()` instead. Its push/pop com

For max-heaps, the following functions are provided:

`heapq.heapify_max(x)`

Transform list `x` into a max-heap, in-place, in linear time. Added in version 3.14.

`heapq.heappush_max(heap, item)`

Push the value `item` onto the max-heap `heap`, maintaining the max-heap invariant. Added in version 3.14.

`heapq.heappop_max(heap)`

Pop and return the largest item from the max-heap `heap`, maintaining the max-heap invariant. If the max-heap is empty, `IndexError` is raised. To access the largest item without popping it, use `maxheap[0]`. Added in version 3.14.

`heapq.heappushpop_max(heap, item)`

Push `item` on the max-heap `heap`, then pop and return the largest item from `heap`. The combined action runs more efficiently than `heappush_max()` followed by a separate call to `heappop_max()`. Added in version 3.14.

`heapq.heapreplace_max(heap, item)`

Pop and return the largest item from the max-heap `heap` and also push the new `item`. The max-heap size doesn't change. If the max-heap is empty, `IndexError` is raised. The value returned may be smaller than the `item` added. Refer to the analogous function `heapreplace()` for detailed usage notes. Added in version 3.14.

The module also offers three general purpose functions based on heaps.

`heapq.merge(*iterables, key=None, reverse=False)`

Merge multiple sorted inputs into a single sorted output (for example, merge timestamped entries from multiple log files). Returns an iterator over the sorted values. Similar to `sorted(itertools.chain(*iterables))` but returns an iterable, does not pull the data into memory all at once, and assumes that each of the input streams is already sorted (smallest to largest). Has two optional arguments which must be specified as keyword arguments. `key` specifies a key function of one argument that is used to extract

`heapq.nlargest(n, iterable, key=None)`

Return a list with the `n` largest elements from the dataset defined by `iterable`. `key`, if provided, specifies a function of one argument that is used to extract a comparison key from each element in `iterable` (for example, `key=str.lower`). Equivalent to: `sorted(iterable, key=key, reverse=True)[:n]`.

`heapq.nsmallest(n, iterable, key=None)`

Return a list with the `n` smallest elements from the dataset defined by `iterable`. `key`, if provided, specifies a function of one argument that is used to extract a comparison key from each element in `iterable` (for example, `key=str.lower`). Equivalent to: `sorted(iterable, key=key)[:n]`.

The latter two functions perform best for smaller values of `n`. For larger values, it is more efficient to use the `sorted()` function. Also, when `n==1`, it is more efficient to use the built-in `min()` and `max()` functions. If repeated usage of these functions is required, consider turning the iterable into an actual heap.

Basic Examples

A `heapsort` can be implemented by pushing all values onto a heap and then popping off the smallest values one at a time:

```
>>> def heapsort(iterable):  
...     h = []
```

```

...     for value in iterable:
...         heappush(h, value)
...     return [heappop(h) for i in range(len(h))]
...
>>> heapsort([1, 3, 5, 7, 9, 2, 4, 6, 8, 0])
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

```

This is similar to `sorted(iterable)`, but unlike `sorted()`, this implementation is not stable.

Heap elements can be tuples. This is useful for assigning comparison values (such as task priorities) alongside the main record being tracked:

```

>>> h = []
>>> heappush(h, (5, 'write code'))
>>> heappush(h, (7, 'release product'))
>>> heappush(h, (1, 'write spec'))
>>> heappush(h, (3, 'create tests'))
>>> heappop(h)
(1, 'write spec')

```

Other Applications

Medians are a measure of central tendency for a set of numbers. In distributions skewed by outliers, the median provides a more stable estimate than an average (arithmetic mean). A running median is an online algorithm that updates continuously as new data arrives.

A running median can be efficiently implemented by balancing two heaps, a max-heap for values at or below the midpoint and a min-heap for values above the midpoint. When the two heaps have the same size, the new median is the average of the tops of the two heaps; otherwise, the median is at the top of the larger heap:

```

def running_median(iterable):
    "Yields the cumulative median of values seen so far."

    lo = [] # max-heap
    hi = [] # min-heap (same size as or one smaller than lo)

    for x in iterable:
        if len(lo) == len(hi):
            heappush_max(lo, heappushpop(hi, x))
            yield lo[0]
        else:
            heappush(hi, heappushpop_max(lo, x))
            yield (lo[0] + hi[0]) / 2

```

For example:

```

>>> list(running_median([5.0, 9.0, 4.0, 12.0, 8.0, 9.0]))
[5.0, 7.0, 5.0, 7.0, 8.0, 8.5]

```

Priority Queue Implementation Notes

A priority queue is common use for a heap, and it presents several implementation challenges:

- Sort stability: how do you get two tasks with equal priorities to be returned in the order they were originally added?
- Tuple comparison breaks for (priority, task) pairs if the priorities are equal and the tasks do not have a default comparison order.
- If the priority of a task changes, how do you move it to a new position in the heap?

- Or if a pending task needs to be deleted, how do you find it and remove it from the queue?

A solution to the first two challenges is to store entries as 3-element list including the priority, an entry count, and the task. The entry count serves as a tie-breaker so that two tasks with the same priority are returned in the order they were added. And since no two entry counts are the same, the tuple comparison will never attempt to directly compare two tasks.

Another solution to the problem of non-comparable tasks is to create a wrapper class that ignores the task item and only compares the priority field:

```
from dataclasses import dataclass, field
from typing import Any
```

```
@dataclass(order=True)
class PrioritizedItem:
    priority: int
    item: Any=field(compare=False)
```

The remaining challenges revolve around finding a pending task and making changes to its priority or removing it entirely. Finding a task can be done with a dictionary pointing to an entry in the queue.

Removing the entry or changing its priority is more difficult because it would break the heap structure invariants. So, a possible solution is to mark the entry as removed and add a new entry with the revised priority:

```
pq = [] # list of entries arranged in a heap
entry_finder = {} # mapping of tasks to entries
REMOVED = '<removed-task>' # placeholder for a removed task
counter = itertools.count() # unique sequence count
```

```
def add_task(task, priority=0):
    'Add a new task or update the priority of an existing task'
    if task in entry_finder:
        remove_task(task)
    count = next(counter)
    entry = [priority, count, task]
    entry_finder[task] = entry
    heappush(pq, entry)
```

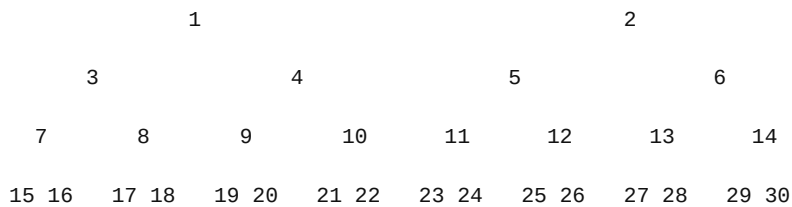
```
def remove_task(task):
    'Mark an existing task as REMOVED. Raise KeyError if not found.'
    entry = entry_finder.pop(task)
    entry[-1] = REMOVED
```

```
def pop_task():
    'Remove and return the lowest priority task. Raise KeyError if empty.'
    while pq:
        priority, count, task = heappop(pq)
        if task is not REMOVED:
            del entry_finder[task]
            return task
    raise KeyError('pop from an empty priority queue')
```

Theory

Heaps are arrays for which $a[k] \leq a[2k+1]$ and $a[k] \leq a[2k+2]$ for all k , counting elements from 0. For the sake of comparison, non-existing elements are considered to be infinite. The interesting property of a heap is that $a[0]$ is always its smallest element.

The strange invariant above is meant to be an efficient memory representation for a tournament. The numbers below are k , not $a[k]$:



In the tree above, each cell is topped 2^k+1 and 2^k+2 . In a usual binary tournament we see in sports, each cell is the winner over the two cells it tops, and we can trace the winner down the tree to see all opponents s/he had. However, in many computer applications of such tournaments, we do not need to trace the history of a winner. To be more memory efficient, when a winner is promoted, we try to replace it by something else at a lower level, and the rule becomes that a cell and the two cells it tops contain three different items, but the top cell “wins” over the two topped cells.

If this heap invariant is protected at all time, index 0 is clearly the overall winner. The simplest algorithmic way to remove it and find the “next” winner is to move some loser (let’s say cell 30 in the diagram above) into the 0 position, and then percolate this new 0 down the tree, exchanging values, until the invariant is re-established. This is clearly logarithmic on the total number of items in the tree. By iterating over all items, you get an $O(n \log n)$ sort.

A nice feature of this sort is that you can efficiently insert new items while the sort is going on, provided that the inserted items are not “better” than the last 0th element you extracted. This is especially useful in simulation contexts, where the tree holds all incoming events, and the “win” condition means the smallest scheduled time. When an event schedules other events for execution, they are scheduled into the future, so they can easily go into the heap. So, a heap is a good structure for implementing schedulers (this is what I used for my MIDI sequencer :-).

Various structures for implementing schedulers have been extensively studied, and heaps are good for this, as they are reasonably speedy, the speed is almost constant, and the worst case is not much different than the average case. However, there are other representations which are more efficient overall, yet the worst cases might be terrible.

Heaps are also very useful in big disk sorts. You most probably all know that a big sort implies producing “runs” (which are pre-sorted sequences, whose size is usually related to the amount of CPU memory), followed by a merging passes for these runs, which merging is often very cleverly organised[1]. It is very important that the initial sort produces the longest runs possible. Tournaments are a good way to achieve that. If, using all the memory available to hold a tournament, you replace and percolate items that happen to fit the current run, you’ll produce runs which are twice the size of the memory for random input, and much better for input fuzzily ordered.

Moreover, if you output the 0th item on disk and get an input which may not fit in the current tournament (because the value “wins” over the last output value), it cannot fit in the heap, so the size of the heap decreases. The freed memory could be cleverly reused immediately for progressively building a second heap, which grows at exactly the same rate the first heap is melting. When the first heap completely vanishes, you switch heaps and start a new run. Clever and quite effective!

In a word, heaps are useful memory structures to know. I use them in a few applications, and I think it is good to keep a ‘heap’ module around. :-)

Footnotes

The disk balancing algorithms which are current, nowadays, are more annoying than clever, and this is a consequence of the seeking capabilities of the disks. On devices which cannot seek, like big tape drives, the story was quite different, and one had to be very clever to ensure (far in advance) that each tape movement will be the most effective possible (that is, will best participate at “progressing” the merge). Some tapes were even able to read backwards, and this was also used to avoid the rewinding time. Believe me, real good tape sorts were quite spectacular to watch! From all times, sorting has always been a Great Art! :-)

bisect— Array bisection algorithm

Source code: `Lib/bisect.py`

This module provides support for maintaining a list in sorted order without having to sort the list after each insertion. For long lists of items with expensive comparison operations, this can be an improvement over linear searches or frequent resorting.

The module is called `bisect` because it uses a basic bisection algorithm to do its work. Unlike other bisection tools that search for a specific value, the functions in this module are designed to locate an insertion point.

Accordingly, the functions never call an `__eq__()` method to determine whether a value has been found. Instead, the functions only call the `__lt__()` method and will return an insertion point between values in an array.

■ **Nota:** Note The functions in this module are not thread-safe. If multiple threads concurrently use `bisect` functions on the same sequence, this may result in undefined behaviour. Likewise, if the provided sequence is mutated by a different thread while a `bisect` function is operating on it, the result is undefined. For example, using `insert_left()` on the same list from multiple threads may result in the list becoming unsorted.

The following functions are provided:

`bisect.bisect_left(a,x,lo=0,hi=len(a),*,key=None)`

Locate the insertion point for `x` into `a` to maintain sorted order. The parameters `lo` and `hi` may be used to specify a subset of the list which should be considered; by default the entire list is used. If `x` is already present in `a`, the insertion point will be before (to the left of) any existing entries. The return value is suitable for use as the first parameter to `list.insert()` assuming that `a` is already sorted. The returned insertion point `i` partitions the array `a` into two slices such that `all(elem < x for elem in a[lo:i])`

`bisect.bisect_right(a,x,lo=0,hi=len(a),*,key=None)`

`bisect.bisect(a,x,lo=0,hi=len(a),*,key=None)`

Similar to `bisect_left()`, but returns an insertion point which comes after (to the right of) any existing entries of `x` in `a`. The returned insertion point `i` partitions the array `a` into two slices such that `all(elem <= x for elem in a[lo:i])` is true for the left slice and `all(elem > x for elem in a[i:hi])` is true for the right slice. Changed in version 3.10: Added the `key` parameter.

`bisect.insort_left(a,x,lo=0,hi=len(a),*,key=None)`

Insert `x` into `a` in sorted order. This function first runs `bisect_left()` to locate an insertion point. Next, it runs the `insert()` method on `a` to insert `x` at the appropriate position to maintain sort order. To support inserting records in a table, the `key` function (if any) is applied to `x` for the search step but not for the insertion step. Keep in mind that the $O(\log n)$ search is dominated by the slow $O(n)$ insertion step. Changed in version 3.10: Added the `key` parameter.

`bisect.insort_right(a,x,lo=0,hi=len(a),*,key=None)`

`bisect.insort(a,x,lo=0,hi=len(a),*,key=None)`

Similar to `insort_left()`, but inserting `x` into `a` after any existing entries of `x`. This function first runs `bisect_right()` to locate an insertion point. Next, it runs the `insert()` method on `a` to insert `x` at the appropriate position to maintain sort order. To support inserting records in a table, the `key` function (if any) is applied to `x` for the search step but not for the insertion step. Keep in mind that the $O(\log n)$ search is dominated by the slow $O(n)$ insertion step. Changed in version 3.10: Added the `key` parameter.

Performance Notes

When writing time sensitive code using `bisect()` and `insort()`, keep these thoughts in mind:

- Bisection is effective for searching ranges of values. For locating specific values, dictionaries are more performant.
- The `insort()` functions are $O(n)$ because the logarithmic search step is dominated by the linear time insertion step.
- The search functions are stateless and discard key function results after they are used. Consequently, if the search functions are used in a loop, the key function may be called again and again on the same array elements. If the key function isn't fast, consider wrapping it with `functools.cache()` to avoid duplicate computations. Alternatively, consider searching an array of precomputed keys to locate

See also

- `Sorted Collections` is a high performance module that uses `bisect` to manage sorted collections of data.
- The `SortedCollection` recipe uses `bisect` to build a full-featured collection class with straight-forward search methods and support for a key-function. The keys are precomputed to save unnecessary calls to the key function during searches.

Searching Sorted Lists

The above `bisect` functions are useful for finding insertion points but can be tricky or awkward to use for common searching tasks. The following five functions show how to transform them into the standard lookups for sorted lists:

```
def index(a, x):
    'Locate the leftmost value exactly equal to x'
    i = bisect_left(a, x)
    if i != len(a) and a[i] == x:
        return i
    raise ValueError

def find_lt(a, x):
    'Find rightmost value less than x'
    i = bisect_left(a, x)
    if i:
        return a[i-1]
    raise ValueError

def find_le(a, x):
    'Find rightmost value less than or equal to x'
    i = bisect_right(a, x)
    if i:
        return a[i-1]
    raise ValueError

def find_gt(a, x):
    'Find leftmost value greater than x'
    i = bisect_right(a, x)
    if i != len(a):
        return a[i]
    raise ValueError

def find_ge(a, x):
    'Find leftmost item greater than or equal to x'
    i = bisect_left(a, x)
    if i != len(a):
        return a[i]
    raise ValueError
```

Examples

The `bisect()` function can be useful for numeric table lookups. This example uses `bisect()` to look up a letter grade for an exam score (say) based on a set of ordered numeric breakpoints: 90 and up is an 'A', 80 to 89 is a 'B', and so on:

```
>>> def grade(score)
...     i = bisect([60, 70, 80, 90], score)
...     return "FDCBA"[i]
...
>>> [grade(score) for score in [33, 99, 77, 70, 89, 90, 100]]
['F', 'A', 'C', 'C', 'B', 'A', 'A']
```

The `bisect()` and `insort()` functions also work with lists of tuples. The `key` argument can serve to extract the field used for ordering records in a table:

```
>>> from collections import namedtuple
>>> from operator import attrgetter
>>> from bisect import bisect, insort
>>> from pprint import pprint

>>> Movie = namedtuple('Movie', ('name', 'released', 'director'))

>>> movies = [
...     Movie('Jaws', 1975, 'Spielberg'),
...     Movie('Titanic', 1997, 'Cameron'),
...     Movie('The Birds', 1963, 'Hitchcock'),
...     Movie('Aliens', 1986, 'Cameron')
... ]

>>> # Find the first movie released after 1960
>>> by_year = attrgetter('released')
>>> movies.sort(key=by_year)
>>> movies[bisect(movies, 1960, key=by_year)]
Movie(name='The Birds', released=1963, director='Hitchcock')

>>> # Insert a movie while maintaining sort order
>>> romance = Movie('Love Story', 1970, 'Hiller')
>>> insort(movies, romance, key=by_year)
>>> pprint(movies)
[Movie(name='The Birds', released=1963, director='Hitchcock'),
 Movie(name='Love Story', released=1970, director='Hiller'),
 Movie(name='Jaws', released=1975, director='Spielberg'),
 Movie(name='Aliens', released=1986, director='Cameron'),
 Movie(name='Titanic', released=1997, director='Cameron')]
```

If the key function is expensive, it is possible to avoid repeated function calls by searching a list of precomputed keys to find the index of a record:

```
>>> data = [('red', 5), ('blue', 1), ('yellow', 8), ('black', 0)]
>>> data.sort(key=lambda r: r[1])          # Or use operator.itemgetter(1).
>>> keys = [r[1] for r in data]           # Precompute a list of keys.
>>> data[bisect_left(keys, 0)]
('black', 0)
>>> data[bisect_left(keys, 1)]
('blue', 1)
>>> data[bisect_left(keys, 5)]
('red', 5)
>>> data[bisect_left(keys, 8)]
('yellow', 8)
```

array— Efficient arrays of numeric values

This module defines an object type which can compactly represent an array of basic values: characters, integers, floating-point numbers. Arrays are mutable sequence types and behave very much like lists, except that the type of objects stored in them is constrained. The type is specified at object creation time by using a type code, which is a single character. The following type codes are defined:

Type code	C Type	Python Type	Minimum size in bytes	Notes
'b'	signed char	int	1	'B'
'B'	unsigned char	int	1	'u'
'u'	wchar_t	Unicode character	2	(1)
'w'	Py_UCS4	Unicode character	4	(2)
'h'	signed short	int	2	'H'
'H'	unsigned short	int	2	'i'
'i'	signed int	int	2	'I'
'I'	unsigned int	int	2	'l'
'l'	signed long	int	4	'L'
'L'	unsigned long	int	4	'q'
'q'	signed long long	int	8	'Q'
'Q'	unsigned long long	int	8	'f'
'f'	float	float	4	'd'
'd'	double	float	8	

Notes:

- It can be 16 bits or 32 bits depending on the platform. Changed in version 3.9: `array('u')` now uses `wchar_t` as C type instead of deprecated `Py_UNICODE`. This change doesn't affect its behavior because `Py_UNICODE` is alias of `wchar_t` since Python 3.3. Deprecated since version 3.3, will be removed in version 3.16: Please migrate to `'w'` typecode.
- Added in version 3.13.

See also

The `types` and `struct` modules, as well as third-party modules like `numpy`, use similar – but slightly different – type codes.

The actual representation of values is determined by the machine architecture (strictly speaking, by the C implementation). The actual size can be accessed through the `array.itemsize` attribute.

The module defines the following item:

array.typecodes

A string with all available type codes.

The module defines the following type:

class `array.array(typecode[,initializer])`

typecode

itemsize

append(value, /)

buffer_info()

byteswap()

count(value, /)

extend(iterable, /)

frombytes(buffer, /)

fromfile(f, n, /)

fromlist(list, /)

fromunicode(ustr, /)

index(value[,start[,stop]])

insert(index,value,/)

pop(index=-1,/)

remove(value,/)

clear()

reverse()

tobytes()

tofile(f,/)

tolist()

tounicode()

A new array whose items are restricted by `typecode`, and initialized from the optional `initializervalue`, which must be a `bytes` or `bytearray` object, a Unicode string, or iterable over elements of the appropriate type. If given a `bytes` or `bytearray` object, the initializer is passed to the new array's `frombytes()` method; if given a Unicode string, the initializer is passed to the `fromunicode()` method; otherwise, the initializer's iterator is passed to the `extend()` method to add initial items to the array. Array object

The `typecode` character used to create the array.

The length in bytes of one array item in the internal representation.

Append a new item with the specified value to the end of the array.

Return a tuple `(address,length)` giving the current memory address and the length in elements of the buffer used to hold array's contents. The size of the memory buffer in bytes can be computed as `array.buffer_info()[1]*array.itemsize`. This is occasionally useful when working with low-level (and inherently unsafe) I/O interfaces that require memory addresses, such as certain `ioctl()` operations. The returned numbers are valid as long as the array exists and no length-changing operations are applied

"Byteswap" all items of the array. This is only supported for values which are 1, 2, 4, or 8 bytes in size; for other types of values, `RuntimeError` is raised. It is useful when reading data from a file written on a machine with a different byte order.

Return the number of occurrences of `value` in the array.

Append items from `iterable` to the end of the array. If `iterable` is another array, it must have exactly the same type code; if not, `TypeError` will be raised. If `iterable` is not an array, it must be iterable and its elements must be the right type to be appended to the array.

Appends items from the `bytes`-like object, interpreting its content as an array of machine values (as if it had been read from a file using the `fromfile()` method). Added in version 3.2: `fromstring()` is renamed to `frombytes()` for clarity.

Read `n` items (as machine values) from the `file` object `f` and append them to the end of the array. If less than `n` items are available, `EOFError` is raised, but the items that were available are still inserted into the array.

Append items from the list. This is equivalent to `for x in list: a.append(x)` except that if there is a type error, the array is unchanged.

Extends this array with data from the given Unicode string. The array must have type code `'u'` or `'w'`; otherwise a `ValueError` is raised. Use `array.frombytes(unicodestring.encode(enc))` to append Unicode data to an array of some other type.

Return the smallest `i` such that `i` is the index of the first occurrence of `value` in the array. The optional arguments `start` and `stop` can be specified to search for `value` within a subsection of the array. Raise `ValueError` if `value` is not found. Changed in version 3.10: Added optional `start` and `stop` parameters.

Insert a new item `value` in the array before position `index`. Negative values are treated as being relative to the end of the array.

Removes the item with the index `i` from the array and returns it. The optional argument defaults to -1, so that by default the last item is removed and returned.

Remove the first occurrence of `value` from the array.

Remove all elements from the array. Added in version 3.13.

Reverse the order of the items in the array.

Convert the array to an array of machine values and return the bytes representation (the same sequence of bytes that would be written to a file by the `tofile()` method.) Added in version 3.2: `tostring()` is renamed to `tobytes()` for clarity.

Write all items (as machine values) to the file object `f`.

Convert the array to an ordinary list with the same items.

Convert the array to a Unicode string. The array must have a type 'u' or 'w'; otherwise a `ValueError` is raised. Use `array.tobytes().decode(enc)` to obtain a Unicode string from an array of some other type.

The string representation of array objects has the form `array(typecode, initializer)`. The `initializer` is omitted if the array is empty, otherwise it is a Unicode string if the `typecode` is 'u' or 'w', otherwise it is a list of numbers. The string representation is guaranteed to be able to be converted back to an array with the same type and value using `eval()`, so long as the `array` class has been imported using `from array import array`. Variables `inf` and `nan` must also be defined if it contains corresponding floating-point values. Examples:

```
array('l')
array('w', 'hello \u2641')
array('l', [1, 2, 3, 4, 5])
array('d', [1.0, 2.0, 3.14, -inf, nan])
```

See also

Module struct

NumPy

Packing and unpacking of heterogeneous binary data.

The NumPy package defines another array type.

weakref— Weak references

Source code: `Lib/weakref.py`

The `weakref` module allows the Python programmer to create weak references to objects.

In the following, the term `referent` means the object which is referred to by a weak reference.

A weak reference to an object is not enough to keep the object alive: when the only remaining references to a referent are weak references, garbage collection is free to destroy the referent and reuse its memory for something else. However, until the object is actually destroyed the weak reference may return the object even if there are no strong references to it.

A primary use for weak references is to implement caches or mappings holding large objects, where it's desired that a large object not be kept alive solely because it appears in a cache or mapping.

For example, if you have a number of large binary image objects, you may wish to associate a name with each. If you used a Python dictionary to map names to images, or images to names, the image objects would remain alive just because they appeared as values or keys in the dictionaries.

The `WeakKeyDictionary` and `WeakValueDictionary` classes supplied by the `weakref` module are an alternative, using weak references to construct mappings that don't keep objects alive solely because they appear in the mapping objects. If, for example, an image object is a value in a `WeakValueDictionary`, then when the last remaining references to that image object are the weak references held by weak mappings, garbage collection can reclaim the object, and its corresponding entries in weak mappings are simply deleted.

`WeakKeyDictionary` and `WeakValueDictionary` use weak references in their implementation, setting up callback functions on the weak references that notify the weak dictionaries when a key or value has been reclaimed by garbage collection. `WeakSet` implements the `set` interface, but keeps weak references to its elements, just like a `WeakKeyDictionary` does.

`finalize` provides a straight forward way to register a cleanup function to be called when an object is garbage collected. This is simpler to use than setting up a callback function on a raw weak reference, since the module automatically ensures that the finalizer remains alive until the object is collected.

Most programs should find that using one of these weak container types or `finalize` is all they need – it's not usually necessary to create your own weak references directly. The low-level machinery is exposed by the `weakref` module for the benefit of advanced uses.

Not all objects can be weakly referenced. Objects which support weak references include class instances, functions written in Python (but not in C), instance methods, sets, frozensets, some file objects, generators, type objects, sockets, arrays, dequeues, regular expression pattern objects, and code objects.

Changed in version 3.2: Added support for `thread.lock`, `threading.Lock`, and code objects.

Several built-in types such as `list` and `dict` do not directly support weak references but can add support through subclassing:

```
class Dict(dict):
    pass
```

```
obj = Dict(red=1, green=2, blue=3)  # this object is weak referenceable
```

CPython implementation detail: Other built-in types such as `tuple` and `int` do not support weak references even when subclassed.

Extension types can easily be made to support weak references; see [Weak Reference Support](#).

When `__slots__` are defined for a given type, weak reference support is disabled unless a `'__weakref__'` string is also present in the sequence of strings in the `__slots__` declaration. See `__slots__` documentation for details.

class weakref.ref(object[, callback])

__callback__

Return a weak reference to object. The original object can be retrieved by calling the reference object if the referent is still alive; if the referent is no longer alive, calling the reference object will cause None to be returned. If callback is provided and not None, and the returned weakref object is still alive, the callback will be called when the object is about to be finalized; the weak reference object will be passed as the only parameter to the callback; the referent will no longer be available.

This read-only attribute returns the callback currently associated to the weakref. If there is no callback or if the referent of the weakref is no longer alive then this attribute will have value None.

weakref.proxy(object[, callback])

Return a proxy to object which uses a weak reference. This supports use of the proxy in most contexts instead of requiring the explicit dereferencing used with weak reference objects. The returned object will have a type of either ProxyType or CallableProxyType, depending on whether object is callable. Proxy objects are not hashable regardless of the referent; this avoids a number of problems related to their fundamentally mutable nature, and prevents their use as dictionary keys. callback is the same as for weakref.ref.

weakref.getweakrefcount(object)

Return the number of weak references and proxies which refer to object.

weakref.getweakrefs(object)

Return a list of all weak reference and proxy objects which refer to object.

class weakref.WeakKeyDictionary([dict])

Mapping class that references keys weakly. Entries in the dictionary will be discarded when there is no longer a strong reference to the key. This can be used to associate additional data with an object owned by other parts of an application without adding attributes to those objects. This can be especially useful with objects that override attribute accesses. Note that when a key with equal value to an existing key (but not equal identity) is inserted into the dictionary, it replaces the value.

WeakKeyDictionary objects have an additional method that exposes the internal references directly. The references are not guaranteed to be “live” at the time they are used, so the result of calling the references needs to be checked before being used. This can be used to avoid creating references that will cause the garbage collector to keep the keys around longer than needed.

WeakKeyDictionary.keyrefs()

Return an iterable of the weak references to the keys.

class weakref.WeakValueDictionary([dict])

Mapping class that references values weakly. Entries in the dictionary will be discarded when no strong reference to the value exists any more. Changed in version 3.9: Added support for |and|= operators, as specified in PEP 584.

WeakValueDictionary objects have an additional method that has the same issues as the WeakKeyDictionary.keyrefs() method.

WeakValueDictionary.valuerefs()

Return an iterable of the weak references to the values.

class weakref.WeakSet([elements])

Set class that keeps weak references to its elements. An element will be discarded when no strong reference to it exists any more.

classweakref.WeakMethod(method[,callback])

A customrefsubclass which simulates a weak reference to a bound method (i.e., a method defined on a class and looked up on an instance). Since a bound method is ephemeral, a standard weak reference cannot keep hold of it. WeakMethod has special code to recreate the bound method until either the object or the original function dies:>>>class C: ... def method(self): ... print("method called!") ...>>>c=C()>>>r=weakref.ref(c.method)>>>r()>>>r=weakref.WeakMethod(c.method)>>>r()<bound method C.method of <__mai

classweakref.finalize(obj,func,/,*args,kwargs)**

__call__()

detach()

peek()

alive

atexit

Return a callable finalizer object which will be called when obj is garbage collected. Unlike an ordinary weak reference, a finalizer will always survive until the reference object is collected, greatly simplifying lifecycle management. A finalizer is considered alive until it is called (either explicitly or at garbage collection), and after that it is dead. Calling a live finalizer returns the result of evaluating func(*arg,**kwargs), whereas calling a dead finalizer returns None. Exceptions raised by

If self is alive then mark it as dead and return the result of calling func(*args,**kwargs). If self is dead then return None.

If self is alive then mark it as dead and return the tuple(obj,func,args,kwags). If self is dead then return None.

If self is alive then return the tuple(obj,func,args,kwags). If self is dead then return None.

Property which is true if the finalizer is alive, false otherwise.

A writable boolean property which by default is true. When the program exits, it calls all remaining live finalizers for which atexit is true. They are called in reverse order of creation.

weakref.ReferenceType

The type object for weak references objects.

weakref.ProxyType

The type object for proxies of objects which are not callable.

weakref.CallableProxyType

The type object for proxies of callable objects.

weakref.ProxyTypes

Sequence containing all the type objects for proxies. This can make it simpler to test if an object is a proxy without being dependent on naming both proxy types.

See also

PEP 205- Weak References

The proposal and rationale for this feature, including links to earlier implementations and information about similar features in other languages.

Weak Reference Objects

Weak reference objects have no methods and no attributes besides `ref.__callback__`. A weak reference object allows the referent to be obtained, if it still exists, by calling it:

```
>>> import weakref
>>> class Object:
...     pass
...
>>> o = Object()
>>> r = weakref.ref(o)
>>> o2 = r()
>>> o is o2
True
```

If the referent no longer exists, calling the reference object returns `None`:

```
>>> del o, o2
>>> print(r())
None
```

Testing that a weak reference object is still live should be done using the expression `ref() is not None`. Normally, application code that needs to use a reference object should follow this pattern:

```
# r is a weak reference object
o = r()
if o is None:
    # referent has been garbage collected
    print("Object has been deallocated; can't frobnicate.")
else:
    print("Object is still live!")
    o.do_something_useful()
```

Using a separate test for “liveness” creates race conditions in threaded applications; another thread can cause a weak reference to become invalidated before the weak reference is called; the idiom shown above is safe in threaded applications as well as single-threaded applications.

Specialized versions of `ref` objects can be created through subclassing. This is used in the implementation of `WeakValueDictionary` to reduce the memory overhead for each entry in the mapping. This may be most useful to associate additional information with a reference, but could also be used to insert additional processing on calls to retrieve the referent.

This example shows how a subclass of `ref` can be used to store additional information about an object and affect the value that's returned when the referent is accessed:

```
import weakref

class ExtendedRef(weakref.ref):
    def __init__(self, ob, callback=None, /, **annotations):
        super().__init__(ob, callback)
        self.__counter = 0
        for k, v in annotations.items():
            setattr(self, k, v)

    def __call__(self):
        """Return a pair containing the referent and the number of
        times the reference has been called.
        """
        ob = super().__call__()
        if ob is not None:
            self.__counter += 1
            ob = (ob, self.__counter)
        return ob
```

Example

This simple example shows how an application can use object IDs to retrieve objects that it has seen before. The IDs of the objects can then be used in other data structures without forcing the objects to remain alive, but the objects can still be retrieved by ID if they do.

```
import weakref

_id2obj_dict = weakref.WeakValueDictionary()

def remember(obj):
    oid = id(obj)
    _id2obj_dict[oid] = obj
    return oid

def id2obj(oid):
    return _id2obj_dict[oid]
```

Finalizer Objects

The main benefit of using `finalize` is that it makes it simple to register a callback without needing to preserve the returned finalizer object. For instance

```
>>> import weakref
>>> class Object:
...     pass
...
>>> kenny = Object()
>>> weakref.finalize(kenny, print, "You killed Kenny!")
<finalize object at ...; for 'Object' at ...>
>>> del kenny
You killed Kenny!
```

The finalizer can be called directly as well. However the finalizer will invoke the callback at most once.

```
>>> def callback(x, y, z):
...     print("CALLBACK")
...     return x + y + z
...
>>> obj = Object()
>>> f = weakref.finalize(obj, callback, 1, 2, z=3)
>>> assert f.alive
>>> assert f() == 6
CALLBACK
>>> assert not f.alive
>>> f()                                # callback not called because finalizer dead
>>> del obj                             # callback not called because finalizer dead
```

You can unregister a finalizer using its `detach()` method. This kills the finalizer and returns the arguments passed to the constructor when it was created.

```
>>> obj = Object()
>>> f = weakref.finalize(obj, callback, 1, 2, z=3)
>>> f.detach()
(<...Object object ...>, <function callback ...>, (1, 2), {'z': 3})
>>> newobj, func, args, kwargs = _
>>> assert not f.alive
>>> assert newobj is obj
>>> assert func(*args, **kwargs) == 6
CALLBACK
```

Unless you set the `exit` attribute to `False`, a finalizer will be called when the program exits if it is still alive. For instance

```
>>> obj = Object()
>>> weakref.finalize(obj, print, "obj dead or exiting")
<finalize object at ...; for 'Object' at ...>
>>> exit()
obj dead or exiting
```

Comparing finalizers with `__del__()` methods

Suppose we want to create a class whose instances represent temporary directories. The directories should be deleted with their contents when the first of the following events occurs:

- the object is garbage collected,
- the object's `remove()` method is called, or
- the program exits.

We might try to implement the class using a `__del__()` method as follows:

```
class TempDir:
    def __init__(self):
        self.name = tempfile.mkdtemp()

    def remove(self):
        if self.name is not None:
            shutil.rmtree(self.name)
            self.name = None

    @property
    def removed(self):
        return self.name is None

    def __del__(self):
        self.remove()
```

Starting with Python 3.4, `__del__()` methods no longer prevent reference cycles from being garbage collected, and module globals are no longer forced to `None` during interpreter shutdown. So this code should work without any issues on CPython.

However, handling of `__del__()` methods is notoriously implementation specific, since it depends on internal details of the interpreter's garbage collector implementation.

A more robust alternative can be to define a finalizer which only references the specific functions and objects that it needs, rather than having access to the full state of the object:

```
class TempDir:
    def __init__(self):
        self.name = tempfile.mkdtemp()
        self._finalizer = weakref.finalize(self, shutil.rmtree, self.name)

    def remove(self):
        self._finalizer()

    @property
    def removed(self):
        return not self._finalizer.alive
```

Defined like this, our finalizer only receives a reference to the details it needs to clean up the directory appropriately. If the object never gets garbage collected the finalizer will still be called at exit.

The other advantage of weakref based finalizers is that they can be used to register finalizers for classes where the definition is controlled by a third party, such as running code when a module is unloaded:

```
import weakref, sys
def unloading_module():
    # implicit reference to the module globals from the function body
weakref.finalize(sys.modules[__name__], unloading_module)
```

■ Nota: NoteIf you create a finalizer object in a daemon thread just as the program exits then there is the possibility that the finalizer does not get called at exit. However, in a daemon thread `atexit.register()`, `try:...finally:...andwith:...do not guarantee that cleanup occurs either.`

types— Dynamic type creation and names for built-in types

Source code: `Lib/types.py`

This module defines utility functions to assist in dynamic creation of new types.

It also defines names for some object types that are used by the standard Python interpreter, but not exposed as builtins like `int` or `str`.

Finally, it provides some additional type-related utility classes and functions that are not fundamental enough to be builtins.

Dynamic Type Creation

`types.new_class(name, bases=(), kwds=None, exec_body=None)`

Creates a class object dynamically using the appropriate metaclass. The first three arguments are the components that make up a class definition header: the class name, the base classes (in order), the keyword arguments (such as `metaclass`). The `exec_body` argument is a callback that is used to populate the freshly created class namespace. It should accept the class namespace as its sole argument and update the namespace directly with the class contents. If no callback is provided, it has the same effect as `exec`.

`types.prepare_class(name, bases=(), kwds=None)`

Calculates the appropriate metaclass and creates the class namespace. The arguments are the components that make up a class definition header: the class name, the base classes (in order) and the keyword arguments (such as `metaclass`). The return value is a 3-tuple: `metaclass, namespace, kwds`. `metaclass` is the appropriate metaclass, `namespace` is the prepared class namespace and `kwds` is an updated copy of the passed `kwds` argument with any `'metaclass'` entry removed. If no `kwds` argument is passed in, this will be an empty dictionary.

See also

Metaclasses

PEP 3115- Metaclasses in Python 3000

Full details of the class creation process supported by these functions

Introduced the `__prepare__` namespace hook

`types.resolve_bases(bases)`

Resolve MRO entries dynamically as specified by PEP 560. This function looks for items in `bases` that are not instances of `type`, and returns a tuple where each such object that has an `__mro_entries__()` method is replaced with an unpacked result of calling this method. If `bases` item is an instance of `type`, or it doesn't have an `__mro_entries__()` method, then it is included in the return tuple unchanged. Added in version 3.7.

`types.get_original_bases(cls, /)`

Return the tuple of objects originally given as the bases of `cls` before the `__mro_entries__()` method has been called on any bases (following the mechanisms laid out in PEP 560). This is useful for introspecting Generics. For classes that have an `__orig_bases__` attribute, this function returns the value of `cls.__orig_bases__`. For classes without the `__orig_bases__` attribute, `cls.__bases__` is returned. Examples: `from typing import TypeVar, Generic, NamedTuple, TypedDict`
`T = TypeVar("T")`
`class Foo(Generic[T]):...`
`class Bar(Foo):...`

See also

Standard Interpreter Types

This module provides names for many of the types that are required to implement a Python interpreter. It deliberately avoids including some of the types that arise only incidentally during processing such as `thelistiterator` type.

Typical use of these names is for `isinstance()` or `issubclass()` checks.

If you instantiate any of these types, note that signatures may vary between Python versions.

Standard names are defined for the following types:

types.NoneType

The type of `None`. Added in version 3.10.

types.FunctionType

types.LambdaType

The type of user-defined functions and functions created by `lambda` expressions. Raises an auditing event `function.__new__` with argument `code`. The audit event only occurs for direct instantiation of function objects, and is not raised for normal compilation.

types.GeneratorType

The type of generator-iterator objects, created by generator functions.

types.CoroutineType

The type of `coroutine` objects, created by `async def` functions. Added in version 3.5.

types.AsyncGeneratorType

The type of asynchronous generator-iterator objects, created by asynchronous generator functions. Added in version 3.6.

types.CodeType(kwargs)**

The type of `code` objects such as returned by `compile()`. Raises an auditing event `code.__new__` with arguments `code`, `filename`, `name`, `argcount`, `posonlyargcount`, `kwonlyargcount`, `nlocals`, `stacksize`, `flags`. Note that the audited arguments may not match the names or positions required by the initializer. The audit event only occurs for direct instantiation of code objects, and is not raised for normal compilation.

types.CellType

The type for cell objects: such objects are used as containers for a function's closure variables. Added in version 3.8.

types.MethodType

The type of methods of user-defined class instances.

types.BuiltinFunctionType

types.BuiltinMethodType

The type of built-in functions like `len()` or `sys.exit()`, and methods of built-in classes. (Here, the term “built-in” means “written in C”.)

types.WrapperDescriptorType

The type of methods of some built-in data types and base classes such as `object.__init__()` or `object.__lt__()`. Added in version 3.7.

types.MethodWrapperType

The type of bound methods of some built-in data types and base classes. For example it is the type of `object().__str__`. Added in version 3.7.

types.NotImplementedType

The type of `NotImplemented`. Added in version 3.10.

types.MethodDescriptorType

The type of methods of some built-in data types such as `str.join()`. Added in version 3.7.

types.ClassMethodDescriptorType

The type of unbound class methods of some built-in data types such as `dict.__dict__['fromkeys']`. Added in version 3.7.

types.ModuleType(name, doc=None)

Documentation on module objects

importlib.util.module_from_spec()

The type of modules. The constructor takes the name of the module to be created and optionally its docstring. See also [Documentation on module objects](#). Provides details on the special attributes that can be found on instances of `ModuleType`. `importlib.util.module_from_spec()` Modules created using the `ModuleType` constructor are created with many of their special attributes unset or set to default values. `module_from_spec()` provides a more robust way of creating `ModuleType` instances which ensures the various attributes are set appropriately.

Provides details on the special attributes that can be found on instances of `ModuleType`.

Modules created using the `ModuleType` constructor are created with many of their special attributes unset or set to default values. `module_from_spec()` provides a more robust way of creating `ModuleType` instances which ensures the various attributes are set appropriately.

types.EllipsisType

The type of `Ellipsis`. Added in version 3.10.

types.GenericAlias(t_origin, t_args)

Generic Alias Types

PEP 585- Type Hinting Generics In Standard Collections

The type of parameterized generics such as `list[int]`. `t_origin` should be a non-parameterized generic class, such as `list`, `tuple` or `dict`. `t_args` should be a tuple (possibly of length 1) of types which parameterize `t_origin`: `>>> from types import GenericAlias >>> list[int] == GenericAlias(list, (int,)) True >>> dict[str, int] == GenericAlias(dict, (str, int)) True` Added in version 3.9. Changed in version 3.9.2: This type can now be subclassed. See also [Generic Alias Types](#). In-depth documentation on instances of `types.GenericAlias` [PEP 585-](#)

In-depth documentation on instances of `types.GenericAlias`

Introducing the `types.GenericAlias` class

types.UnionType

The type of union type expressions. Added in version 3.10. Changed in version 3.14: This is now an alias for `typing.Union`.

classtypes.TracebackType(tb_next,tb_frame,tb_lasti,tb_lineno)

The type of traceback objects such as found in `sys.exception().__traceback__`. See the language reference for details of the available attributes and operations, and guidance on creating tracebacks dynamically.

types.FrameType

The type of frame objects such as found in `tb.tb_frame` if `tb` is a traceback object.

types.GetSetDescriptorType

The type of objects defined in extension modules with `PyGetSetDef`, such as `FrameType.f_locals` or `array.array.typecode`. This type is used as descriptor for object attributes; it has the same purpose as the `property` type, but for classes defined in extension modules.

types.MemberDescriptorType

The type of objects defined in extension modules with `PyMemberDef`, such as `datetime.timedelta.days`. This type is used as descriptor for simple C data members which use standard conversion functions; it has the same purpose as the `property` type, but for classes defined in extension modules. In addition, when a class is defined with a `__slots__` attribute, then for each slot, an instance of `MemberDescriptorType` will be added as an attribute on the class. This allows the slot to appear in the class's `__dict__`.

classtypes.MappingProxyType(mapping)

keyinproxy

proxy[key]

iter(proxy)

len(proxy)

copy()

get(key[,default])

items()

keys()

values()

reversed(proxy)

hash(proxy)

Read-only proxy of a mapping. It provides a dynamic view on the mapping's entries, which means that when the mapping changes, the view reflects these changes. Added in version 3.3. Changed in version 3.9: Updated to support the new union (`|`) operator from PEP 584, which simply delegates to the underlying mapping. `keyinproxy` Return `True` if the underlying mapping has a key `key`, else `False`. `proxy[key]` Return the item of the underlying mapping with key `key`. Raises a `KeyError` if `key` is not in the underlying mapping.

Return `True` if the underlying mapping has a key `key`, else `False`.

Return the item of the underlying mapping with key `key`. Raises a `KeyError` if `key` is not in the underlying mapping.

Return an iterator over the keys of the underlying mapping. This is a shortcut for `iter(proxy.keys())`.

Return the number of items in the underlying mapping.

Return a shallow copy of the underlying mapping.

Return the value for `key` if `key` is in the underlying mapping, else `default`. If `default` is not given, it defaults to `None`, so that this method never raises a `KeyError`.

Return a new view of the underlying mapping's items ((key,value)pairs).

Return a new view of the underlying mapping's keys.

Return a new view of the underlying mapping's values.

Return a reverse iterator over the keys of the underlying mapping.Added in version 3.9.

Return a hash of the underlying mapping.Added in version 3.12.

classtypes.CapsuleType

The type ofcapsule objects.Added in version 3.13.

Additional Utility Classes and Functions

classtypes.SimpleNamespace

A simpleobjectsubclass that provides attribute access to its namespace, as well as a meaningful repr.Unlikeobject, withSimpleNamespaceyou can add and remove attributes.SimpleNamespaceobjects may be initialized in the same way asdict: either with keyword arguments, with a single positional argument, or with both. When initialized with keyword arguments, those are directly added to the underlying namespace. Alternatively, when initialized with a positional argument, the underlying namespace will b

types.DynamicClassAttribute(fget=None,fset=None,fdel=None,doc=None)

Route attribute access on a class to __getattr__.This is a descriptor, used to define attributes that act differently when accessed through an instance and through a class. Instance access remains normal, but access to an attribute through a class will be routed to the class's __getattr__ method; this is done by raising AttributeError.This allows one to have properties active on an instance, and have virtual attributes on the class with the same name (seeenum.Enumfor an example).Added in versio

Coroutine Utility Functions

types.coroutine(gen_func)

This function transforms ageneratorfunction into acoroutine functionwhich returns a generator-based coroutine. The generator-based coroutine is still agenerator iterator, but is also considered to be acoroutineobject and isawaitable. However, it may not necessarily implement the__await__()method.Ifgen_funcis a generator function, it will be modified in-place.Ifgen_funcis not a generator function, it will be wrapped. If it returns an instance ofcollections.abc.Generator, the instance will be wra

copy— Shallow and deep copy operations

Source code: `Lib/copy.py`

Assignment statements in Python do not copy objects, they create bindings between a target and an object. For collections that are mutable or contain mutable items, a copy is sometimes needed so one can change one copy without changing the other. This module provides generic shallow and deep copy operations (explained below).

Interface summary:

copy.copy(obj)

Return a shallow copy of obj.

copy.deepcopy(obj[,memo])

Return a deep copy of obj.

copy.replace(obj, **changes)

Creates a new object of the same type as obj, replacing fields with values from changes. Added in version 3.13.

exceptioncopy.Error

Raised for module specific errors.

The difference between shallow and deep copying is only relevant for compound objects (objects that contain other objects, like lists or class instances):

- A shallow copy constructs a new compound object and then (to the extent possible) inserts references into it to the objects found in the original.
- A deep copy constructs a new compound object and then, recursively, inserts copies into it of the objects found in the original.

Two problems often exist with deep copy operations that don't exist with shallow copy operations:

- Recursive objects (compound objects that, directly or indirectly, contain a reference to themselves) may cause a recursive loop.
- Because deep copy copies everything it may copy too much, such as data which is intended to be shared between copies.

The `deepcopy()` function avoids these problems by:

- keeping a memo dictionary of objects already copied during the current copying pass; and
- letting user-defined classes override the copying operation or the set of components copied.

This module does not copy types like module, method, stack trace, stack frame, file, socket, window, or any similar types. It does “copy” functions and classes (shallow and deeply), by returning the original object unchanged; this is compatible with the way these are treated by the pickle module.

Shallow copies of dictionaries can be made using `dict.copy()`, and of lists by assigning a slice of the entire list, for example, `copied_list=original_list[:]`.

Classes can use the same interfaces to control copying that they use to control pickling. See the description of module pickle for information on these methods. In fact, the copy module uses the registered pickle functions from the copyreg module.

In order for a class to define its own copy implementation, it can define special methods `__copy__()` and `__deepcopy__()`.

object.__copy__(self)

Called to implement the shallow copy operation; no additional arguments are passed.

object.__deepcopy__(self, memo)

Called to implement the deep copy operation; it is passed one argument, the memo dictionary. If the `__deepcopy__` implementation needs to make a deep copy of a component, it should call the `deepcopy()` function with the component as first argument and the memo dictionary as second argument. The memo dictionary should be treated as an opaque object.

Function `copy.replace()` is more limited than `copy()` and `deepcopy()`, and only supports named tuples created by `namedtuple()`, dataclasses, and other classes which define method `__replace__()`.

object.__replace__(self, /, **changes)

This method should create a new object of the same type, replacing fields with values from `changes`. Added in version 3.13.

See also

Module pickle

Discussion of the special methods used to support object state retrieval and restoration.

pprint— Data pretty printer

Source code: `Lib/pprint.py`

The `pprint` module provides a capability to “pretty-print” arbitrary Python data structures in a form which can be used as input to the interpreter. If the formatted structures include objects which are not fundamental Python types, the representation may not be loadable. This may be the case if objects such as files, sockets or classes are included, as well as many other objects which are not representable as Python literals.

The formatted representation keeps objects on a single line if it can, and breaks them onto multiple lines if they don't fit within the allowed width, adjustable by the `width` parameter defaulting to 80 characters.

Changed in version 3.9: Added support for pretty-printing `types.SimpleNamespace`.

Changed in version 3.10: Added support for pretty-printing `dataclasses.dataclass`.

Functions

`pprint.pp(object, stream=None, indent=1, width=80, depth=None, *, compact=False, sort_dicts=False, underscore_numbers=False)`

Parameters:

Prints the formatted representation of `object`, followed by a newline. This function may be used in the interactive interpreter instead of the `print()` function for inspecting values. Tip: you can reassign `print=pprint.pp` for use within a scope. Parameters: `object` – The object to be printed. `stream` (file-like object| None) – A file-like object to which the output will be written by calling its `write()` method. If None (the default), `sys.stdout` is used. `indent` (int) – The amount of indentation added for each nesting

`object` – The object to be printed. `stream` (file-like object| None) – A file-like object to which the output will be written by calling its `write()` method. If None (the default), `sys.stdout` is used. `indent` (int) – The amount of indentation added for each nesting level. `width` (int) – The desired maximum number of characters per line in the output. If a structure cannot be formatted within the width constraint, a best effort will be made. `depth` (int| None) – The number of nesting levels which may be printed. If th

`pprint.pprint(object, stream=None, indent=1, width=80, depth=None, *, compact=False, sort_dicts=True, underscore_numbers=False)`

Alias for `pp()` with `sort_dicts` set to `True` by default, which would automatically sort the dictionaries' keys, you might want to use `pp()` instead where it is `False` by default.

`pprint.pformat(object, indent=1, width=80, depth=None, *, compact=False, sort_dicts=True, underscore_numbers=False)`

Return the formatted representation of `object` as a string. `indent`, `width`, `depth`, `compact`, `sort_dicts` and `underscore_numbers` are passed to the `PrettyPrinter` constructor as formatting parameters and their meanings are as described in the documentation above.

`pprint.isreadable(object)`

Determine if the formatted representation of `object` is “readable”, or can be used to reconstruct the value using `eval()`. This always returns `False` for recursive objects. `>>> pprint.isreadable(stuff) False`

`pprint.isrecursive(object)`

Determine if `object` requires a recursive representation. This function is subject to the same limitations as noted in `safe_repr()` below and may raise an `RecursionError` if it fails to detect a recursive object.

pprint.saferepr(object)

Return a string representation of `object`, protected against recursion in some common data structures, namely instances of `dict`, `list` and `tuple` or subclasses whose `__repr__` has not been overridden. If the representation of `object` exposes a recursive entry, the recursive reference will be represented as `<Recursion on type name with id=number>`. The representation is not otherwise formatted. `>>> pprint.saferepr(stuff)` `[<Recursion on list with id=...>, 'spam', 'eggs', 'lumberjack', 'knights', 'ni']`

PrettyPrinter Objects

`class pprint.PrettyPrinter(indent=1, width=80, depth=None, stream=None, *, compact=False, sort_dicts=True, underscore_numbers=False)`

Construct a `PrettyPrinter` instance. Arguments have the same meaning as for `pp()`. Note that they are in a different order, and that `sort_dicts` defaults to `True`. `>>> import pprint`
`>>> stuff = ['spam', 'eggs', 'lumberjack', 'knights', 'ni']`
`>>> stuff.insert(0, stuff[:])`
`>>> pp = pprint.PrettyPrinter(indent=4)`
`>>> pp.pprint(stuff)` `[ 'spam', 'eggs', 'lumberjack', 'knights', 'ni', 'spam', 'eggs', 'lumberjack', 'knights', 'ni']`
`>>> pp = pprint.PrettyPrinter(width=41, compact=True)`
`>>> pp.pprint(stuff)` `[ 'spam', 'eggs', 'lumberjack', 'knigh`

`PrettyPrinter` instances have the following methods:

PrettyPrinter.pformat(object)

Return the formatted representation of `object`. This takes into account the options passed to the `PrettyPrinter` constructor.

PrettyPrinter.pprint(object)

Print the formatted representation of `object` on the configured stream, followed by a newline.

The following methods provide the implementations for the corresponding functions of the same names. Using these methods on an instance is slightly more efficient since new `PrettyPrinter` objects don't need to be created.

PrettyPrinter.isreadable(object)

Determine if the formatted representation of the object is “readable,” or can be used to reconstruct the value using `eval()`. Note that this returns `False` for recursive objects. If the `depth` parameter of the `PrettyPrinter` is set and the object is deeper than allowed, this returns `False`.

PrettyPrinter.isrecursive(object)

Determine if the object requires a recursive representation.

This method is provided as a hook to allow subclasses to modify the way objects are converted to strings. The default implementation uses the internals of the `saferepr()` implementation.

PrettyPrinter.format(object, context, maxlevels, level)

Returns three values: the formatted version of `object` as a string, a flag indicating whether the result is readable, and a flag indicating whether recursion was detected. The first argument is the object to be presented. The second is a dictionary which contains the `id()` of objects that are part of the current presentation context (direct and indirect containers for `object` that are affecting the presentation) as the keys; if an object needs to be presented which is already represented in context, the

Example

To demonstrate several uses of the `pp()` function and its parameters, let's fetch information about a project from PyPI:

```
>>> import json
>>> import pprint
>>> from urllib.request import urlopen
>>> with urlopen('https://pypi.org/pypi/sampleproject/1.2.0/json') as resp:
...     project_info = json.load(resp)['info']
```

In its basic form,pp()shows the whole object:

```
>>> pprint.pp(project_info)
{'author': 'The Python Packaging Authority',
 'author_email': 'pypa-dev@googlegroups.com',
 'bugtrack_url': None,
 'classifiers': ['Development Status :: 3 - Alpha',
                  'Intended Audience :: Developers',
                  'License :: OSI Approved :: MIT License',
                  'Programming Language :: Python :: 2',
                  'Programming Language :: Python :: 2.6',
                  'Programming Language :: Python :: 2.7',
                  'Programming Language :: Python :: 3',
                  'Programming Language :: Python :: 3.2',
                  'Programming Language :: Python :: 3.3',
                  'Programming Language :: Python :: 3.4',
                  'Topic :: Software Development :: Build Tools'],
 'description': 'A sample Python project\n'
                '=====\n'
                '\n'
                'This is the description file for the project.\n'
                '\n'
                'The file should use UTF-8 encoding and be written using '
                'ReStructured Text. It\n'
                'will be used to generate the project webpage on PyPI, and '
                'should be written for\n'
                'that purpose.\n'
                '\n'
                'Typical contents for this file would include an overview of '
                'the project, basic\n'
                'usage examples, etc. Generally, including the project '
                'changelog in here is not\n'
                'a good idea, although a simple "What\'s New" section for the '
                'most recent version\n'
                'may be appropriate.',
 'description_content_type': None,
 'docs_url': None,
 'download_url': 'UNKNOWN',
 'downloads': {'last_day': -1, 'last_month': -1, 'last_week': -1},
 'home_page': 'https://github.com/pypa/sampleproject',
 'keywords': 'sample setuptools development',
 'license': 'MIT',
 'maintainer': None,
 'maintainer_email': None,
 'name': 'sampleproject',
 'package_url': 'https://pypi.org/project/sampleproject/',
 'platform': 'UNKNOWN',
 'project_url': 'https://pypi.org/project/sampleproject/',
 'project_urls': {'Download': 'UNKNOWN',
                  'Homepage': 'https://github.com/pypa/sampleproject'},
 'release_url': 'https://pypi.org/project/sampleproject/1.2.0/',
 'requires_dist': None,
 'requires_python': None,
 'summary': 'A sample Python project',
 'version': '1.2.0'}
```

The result can be limited to a certain depth(ellipsis is used for deeper contents):

```
>>> pprint.pp(project_info, depth=1)
{'author': 'The Python Packaging Authority',
```

```

'author_email': 'pypa-dev@googlegroups.com',
'bugtrack_url': None,
'classifiers': [...],
'description': 'A sample Python project\n'
               '=====\n'
               '\n'
               'This is the description file for the project.\n'
               '\n'
               'The file should use UTF-8 encoding and be written using '
               'ReStructured Text. It\n'
               'will be used to generate the project webpage on PyPI, and '
               'should be written for\n'
               'that purpose.\n'
               '\n'
               'Typical contents for this file would include an overview of '
               'the project, basic\n'
               'usage examples, etc. Generally, including the project '
               'changelog in here is not\n'
               'a good idea, although a simple "What\'s New" section for the '
               'most recent version\n'
               'may be appropriate.',
'description_content_type': None,
'docs_url': None,
'download_url': 'UNKNOWN',
'downloads': {...},
'home_page': 'https://github.com/pypa/sampleproject',
'keywords': 'sample setuptools development',
'license': 'MIT',
'maintainer': None,
'maintainer_email': None,
'name': 'sampleproject',
'package_url': 'https://pypi.org/project/sampleproject/',
'platform': 'UNKNOWN',
'project_url': 'https://pypi.org/project/sampleproject/',
'project_urls': {...},
'release_url': 'https://pypi.org/project/sampleproject/1.2.0/',
'requires_dist': None,
'requires_python': None,
'summary': 'A sample Python project',
'version': '1.2.0'}

```

Additionally, maximum characterwidth can be suggested. If a long object cannot be split, the specified width will be exceeded:

```

>>> pprint.pp(project_info, depth=1, width=60)
{'author': 'The Python Packaging Authority',
 'author_email': 'pypa-dev@googlegroups.com',
 'bugtrack_url': None,
 'classifiers': [...],
 'description': 'A sample Python project\n'
               '=====\n'
               '\n'
               'This is the description file for the '
               'project.\n'
               '\n'
               'The file should use UTF-8 encoding and be '
               'written using ReStructured Text. It\n'
               'will be used to generate the project '
               'webpage on PyPI, and should be written '
               'for\n'
               'that purpose.\n'
               '\n'
               'Typical contents for this file would '
               'include an overview of the project, '
               'basic\n'

```



```
        'usage examples, etc. Generally, including '
        'the project changelog in here is not\n'
        'a good idea, although a simple "What\'s '
        'New" section for the most recent version\n'
        'may be appropriate.',
'description_content_type': None,
'docs_url': None,
'download_url': 'UNKNOWN',
'downloads': {...},
'home_page': 'https://github.com/pypa/sampleproject',
'keywords': 'sample setuptools development',
'license': 'MIT',
'maintainer': None,
'maintainer_email': None,
'name': 'sampleproject',
'package_url': 'https://pypi.org/project/sampleproject/',
'platform': 'UNKNOWN',
'project_url': 'https://pypi.org/project/sampleproject/',
'project_urls': {...},
'release_url': 'https://pypi.org/project/sampleproject/1.2.0/',
'requires_dist': None,
'requires_python': None,
'summary': 'A sample Python project',
'version': '1.2.0'}
```

reprlib— Alternaterepr()implementation

Source code:Lib/reprlib.py

The reprlib module provides a means for producing object representations with limits on the size of the resulting strings. This is used in the Python debugger and may be useful in other contexts as well.

This module provides a class, an instance, and a function:

```
class reprlib.Repr(*, maxlevel=6, maxtuple=6, maxlist=6, maxarray=5, maxdict=4, maxset=6, maxfrozenset=6, maxdeque=6, maxstring=30, maxlong=40, maxother=30, fillvalue='...', indent=None)
```

Class which provides formatting services useful in implementing functions similar to the built-in repr(); size limits for different object types are added to avoid the generation of representations which are excessively long. The keyword arguments of the constructor can be used as a shortcut to set the attributes of the Repr instance. Which means that the following initialization: aRepr = reprlib.Repr(maxlevel=3) is equivalent to: aRepr = reprlib.Repr(aRepr.maxlevel=3) See section Repr Objects for more info.

reprlib.aRepr

This is an instance of Repr which is used to provide the repr() function described below. Changing the attributes of this object will affect the size limits used by repr() and the Python debugger.

reprlib.repr(obj)

This is the repr() method of a Repr. It returns a string similar to that returned by the built-in function of the same name, but with limits on most sizes.

In addition to size-limiting tools, the module also provides a decorator for detecting recursive calls to __repr__() and substituting a placeholder string instead.

@reprlib.recursive_repr(fillvalue='...')

Decorator for __repr__() methods to detect recursive calls within the same thread. If a recursive call is made, the fillvalue is returned, otherwise, the usual __repr__() call is made. For example: >>> from reprlib import recursive_repr >>> class MyList(list): ... @recursive_repr() ... def __repr__(self): ... return '<' + '|'.join(map(repr, self)) + '>' ... >>> m = MyList('abc') >>> m.append(m) >>> m.append('x') >>> print(m) <'a'|'b'|'c'|...'x'> Added in version 3.2.

Repr Objects

Repr instances provide several attributes which can be used to provide size limits for the representations of different object types, and methods which format specific object types.

Repr.fillvalue

This string is displayed for recursive references. It defaults to '...'. Added in version 3.11.

Repr.maxlevel

Depth limit on the creation of recursive representations. The default is 6.

Repr.maxdict

Repr.maxlist

Repr.maxtuple

Repr.maxset

Repr.maxfrozenset

Repr.maxdeque

Repr.maxarray

Limits on the number of entries represented for the named object type. The default is 4 for maxdict, 5 for maxarray, and 6 for the others.

Repr.maxlong

Maximum number of characters in the representation for an integer. Digits are dropped from the middle. The default is 40.

Repr.maxstring

Limit on the number of characters in the representation of the string. Note that the “normal” representation of the string is used as the character source: if escape sequences are needed in the representation, these may be mangled when the representation is shortened. The default is 30.

Repr.maxother

This limit is used to control the size of object types for which no specific formatting method is available on the `Repr` object. It is applied in a similar manner as `maxstring`. The default is 20.

Repr.indent

If this attribute is set to `None` (the default), the output is formatted with no line breaks or indentation, like the standard `repr()`. For example:

```
>>> example = [...1, 'spam', {'a': 2, 'b': 'spam eggs', 'c': {3: 4.5, 6: []}}, 'ham']
>>> import reprlib
>>> aRepr = reprlib.Repr()
>>> print(aRepr.repr(example))
1, 'spam', {'a': 2, 'b': 'spam eggs', 'c': {3: 4.5, 6: []}}, 'ham'
```

 If `indent` is set to a string, each recursion level is placed on its own line, indented by that string:

```
>>> aRepr.indent = '-->'
>>> print(aRepr.repr(example))
--> 1, 'spam', {'a': 2, 'b': 'spam eggs', 'c': {3: 4.5, 6: []}}, 'ham'
```

Repr.repr(obj)

The equivalent to the built-in `repr()` that uses the formatting imposed by the instance.

Repr.repr1(obj, level)

Recursive implementation used by `repr()`. This uses the type of `obj` to determine which formatting method to call, passing it `obj` and `level`. The type-specific methods should call `repr1()` to perform recursive formatting, with `level-1` for the value of `level` in the recursive call.

Repr.repr_TYPE(obj, level)

Formatting methods for specific types are implemented as methods with a name based on the type name. In the method name, `TYPE` is replaced by `'_'.join(type(obj).__name__.split())`. Dispatch to these methods is handled by `repr1()`. Type-specific methods which need to recursively format a value should call `self.repr1(subobj, level-1)`.

Subclassing Repr Objects

The use of dynamic dispatching by `Repr.repr1()` allows subclasses of `Repr` to add support for additional built-in object types or to modify the handling of types already supported. This example shows how special support for file objects could be added:

```
import reprlib
import sys

class MyRepr(reprlib.Repr):

    def repr_TextIOWrapper(self, obj, level):
        if obj.name in {'<stdin>', '<stdout>', '<stderr>'}:
            return obj.name
        return repr(obj)
```

```
aRepr = MyRepr()  
print(aRepr.repr(sys.stdin))      # prints '<stdin>'  
  
<stdin>
```

enum— Support for enumerations

Added in version 3.4.

Source code: [Lib/enum.py](#)

Important

This page contains the API reference information. For tutorial information and discussion of more advanced topics, see

- [Basic Tutorial](#)
- [Advanced Tutorial](#)
- [Enum Cookbook](#)

An enumeration:

- is a set of symbolic names (members) bound to unique values
- can be iterated over to return its canonical (i.e. non-alias) members in definition order
- uses `callsyntax` to return members by value
- uses `indexsyntax` to return members by name

Enumerations are created either by using `classsyntax`, or by using function-call syntax:

```
>>> from enum import Enum

>>> # class syntax
>>> class Color(Enum):
...     RED = 1
...     GREEN = 2
...     BLUE = 3

>>> # functional syntax
>>> Color = Enum('Color', [('RED', 1), ('GREEN', 2), ('BLUE', 3)])
```

Even though we can use `classsyntax` to create Enums, Enums are not normal Python classes. See [How are Enums different?](#) for more details.

■ **Note:** *Nomenclature* The class `Color` is an enumeration (or enum). The attributes `Color.RED`, `Color.GREEN`, etc., are enumeration members (or members) and are functionally constants. The enum members have names and values (the name of `Color.RED` is `RED`, the value of `Color.BLUE` is 3, etc.)

Module contents

`EnumType`

The type for Enum and its subclasses.

`Enum`

Base class for creating enumerated constants.

`IntEnum`

Base class for creating enumerated constants that are also subclasses of `int`. (Notes)

`StrEnum`

Base class for creating enumerated constants that are also subclasses of `str`. (Notes)

Flag

Base class for creating enumerated constants that can be combined using the bitwise operations without losing their flag membership.

IntFlag

Base class for creating enumerated constants that can be combined using the bitwise operators without losing their int flag membership. IntFlag members are also subclasses of int. (Notes)

ReprEnum

Used by IntEnum, StrEnum, and IntFlag to keep the str() of the mixed-in type.

EnumCheck

An enumeration with the values CONTINUOUS, NAMED_FLAGS, and UNIQUE, for use with verify() to ensure various constraints are met by a given enumeration.

FlagBoundary

An enumeration with the values STRICT, CONFORM, EJECT, and KEEP which allows for more fine-grained control over how invalid values are dealt with in an enumeration.

EnumDict

A subclass of dict for use when subclassing EnumType.

auto

Instances are replaced with an appropriate value for Enum members. StrEnum defaults to the lower-cased version of the member name, while other Enums default to 1 and increase from there.

property()

Allows Enum members to have attributes without conflicting with member names. The value and name attributes are implemented this way.

unique()

Enum class decorator that ensures only one name is bound to any one value.

verify()

Enum class decorator that checks user-selectable constraints on an enumeration.

member()

Make obj a member. Can be used as a decorator.

nonmember()

Do not make obj a member. Can be used as a decorator.

global_enum()

Modify the str() and repr() of an enum to show its members as belonging to the module instead of its class, and export the enum members to the global namespace.

show_flag_values()

Return a list of all power-of-two integers contained in a flag.

enum.bin()

Like built-in bin(), except negative values are represented in two's complement, and the leading bit always indicates sign (0 implies positive, 1 implies negative).

Added in version 3.6:Flag,IntFlag,auto

Added in version 3.11:StrEnum,EnumCheck,ReprEnum,FlagBoundary,property,member,nonmember,global_enum,show_flag_values

Added in version 3.13:EnumDict

Data types

classenum.EnumType

`__call__(cls,value, names=None, *, module=None, qualname=None, type=None, start=1, boundary=None)`

cls:

value:

cls:

value:

names:

module:

qualname:

type:

start:

boundary:

`__contains__(cls,member)`

`__dir__(cls)`

`__getitem__(cls,name)`

`__iter__(cls)`

`__len__(cls)`

`__members__`

`__reversed__(cls)`

EnumType is the metaclass for enum enumerations. It is possible to subclass EnumType— see Subclassing EnumType for details. EnumType is responsible for setting the correct `__repr__()`, `__str__()`, `__format__()`, and `__reduce__()` methods on the final enum, as well as creating the enum members, properly handling duplicates, providing iteration over the enum class, etc. Added in version 3.11: Before 3.11 EnumType was called EnumMeta, which is still available as an alias. `__call__(cls,value, names=None, *, module=None, qualnam`

This method is called in two different ways: to look up an existing member: `cls`: The enum class being called. `value`: The value to lookup. to use the `cls` enum to create a new enum (only if the existing enum does not have any members): `cls`: The enum class being called. `value`: The name of the new Enum to create. `names`: The names/values of the members for the new Enum. `module`: The name of the module the new Enum is created in. `qualname`: The actual location in the module where this Enum can be found. `type`: A mix-in type

The enum class being called.

The value to lookup.

The enum class being called.

The name of the new Enum to create.

The names/values of the members for the new Enum.

The name of the module the new Enum is created in.

The actual location in the module where this Enum can be found.

A mix-in type for the new Enum.

The first integer value for the Enum (used by `auto`).

How to handle out-of-range values from bit operations (`Flag` only).

Returns `True` if member belongs to

the cls: `>>> some_var = Color.RED >>> some_var in Color` `True >>> Color.RED.value in Color` `True` Changed in version 3.12: Before Python 3.12, a `TypeError` is raised if a non-Enum-member is used in a containment check.

Returns `['__class__', '__doc__', '__members__', '__module__']` and the names of the members
incls: `>>> dir(Color)` `['BLUE', 'GREEN', 'RED', '__class__', '__contains__', '__doc__', '__getitem__', '__init_subclass__', '__iter__', '__len__', '__members__', '__module__', '__name__', '__qualname__']`

Returns the Enum member incl matching name, or raises a `KeyError`: `>>> Color['BLUE']` `<Color.BLUE: 3>`

Returns each member incl in definition order: `>>> list(Color)` `<Color.RED: 1>, <Color.GREEN: 2>, <Color.BLUE: 3>]`

Returns the number of member incls: `>>> len(Color)` `3`

Returns a mapping of every enum name to its member, including aliases

Returns each member incl in reverse definition order: `>>> list(reversed(Color))` `<Color.BLUE: 3>, <Color.GREEN: 2>, <Color.RED: 1>]`

`class enum.Enum`

name

value

`__name__`

`__value__`

`__order__`

`__ignore__`

`__dir__(self)`

`__generate_next_value__(name, start, count, last_values)`

name:

start:

count:

last_values:

`__init__(self, *args, **kwargs)`

`__init_subclass__(cls, **kwargs)`

`__missing__(cls, value)`

`__new__(cls, *args, **kwargs)`

`__repr__(self)`

`__str__(self)`

`__format__(self)`

`_add_alias_()`

`_add_value_alias_()`

Enum is the base class for all enumerations. The name used to define the Enum member: `>>> Color.BLUE.name` 'BLUE' value The value given to the Enum member: `>>> Color.RED.value` 1 Value of the member, can be set in `__new__()`. Note Enum member values can be anything: int, str, etc. If the exact value is unimportant you may use `autoinstances` and an appropriate value will be chosen for you. See `auto` for the details. While mutable/unhashable values, such as `dict`, `list` or a mutable `dataclass`, can be used, the

The name used to define the Enum member: `>>> Color.BLUE.name` 'BLUE'

The value given to the Enum member: `>>> Color.RED.value` 1 Value of the member, can be set in `__new__()`. Note Enum member values can be anything: int, str, etc. If the exact value is unimportant you may use `autoinstances` and an appropriate value will be chosen for you. See `auto` for the details. While mutable/unhashable values, such as `dict`, `list` or a mutable `dataclass`, can be used, they will have a quadratic performance impact during creation relative to the total number of mutable/unhashable values

Name of the member.

Value of the member, can be set in `__new__()`.

No longer used, kept for backward compatibility. (class attribute, removed during class creation). The `_order_` attribute can be provided to help keep Python 2 / Python 3 code in sync. It will be checked against the actual order of the enumeration and raise an error if the two do not match: `>>> class Color(Enum): ... _order_ = 'RED GREEN BLUE' ... RED = 1 ... BLUE = 3 ... GREEN = 2 ...` Traceback (most recent call last): ... `TypeError: member order does not match _order_: ['RED', 'BLUE', 'GREEN'] ['RED', 'GREEN', 'BLUE']` Note!

`_ignore_` is only used during creation and is removed from the enumeration once creation is complete. `_ignore_` is a list of names that will not become members, and whose names will also be removed from the completed enumeration. See `TimePeriod` for an example. Added in version 3.7.

Returns `['__class__', '__doc__', '__module__', 'name', 'value']` and any public methods defined on self. `__class__`: `>>> from enum import Enum` `>>> import datetime as dt` `>>> class Weekday(Enum): ... MONDAY = 1 ... TUESDAY = 2 ... WEDNESDAY = 3 ... THURSDAY = 4 ... FRIDAY = 5 ... SATURDAY = 6 ... SUNDAY = 7 ...` @classmethod ... `def today(cls): ... print(f'today is {cls(dt.date.today().isoweekday()).name}') ...` `>>> dir(Weekday.SATURDAY)` `['__class__', '__doc__', '__eq__', '__hash__', '__module__', 'name', 'today', 'value']`

name: The name of the member being defined (e.g. 'RED'). start: The start value for the Enum; the default is 1. count: The number of members currently defined, not including this one. last_values: A list of the previous values. `astaticmethod` that is used to determine the next value returned by `auto`. Note For standard Enum classes the next value chosen is the highest value seen incremented by one. For Flag classes the next value chosen will be the next highest power-of-two. This method may be overridden, e.g.: `>>> f`

The name of the member being defined (e.g. 'RED').

The start value for the Enum; the default is 1.

The number of members currently defined, not including this one.

A list of the previous values.

By default, does nothing. If multiple values are given in the member assignment, those values become separate arguments to `__init__`;
e.g. `>>>from enum import Enum>>>class Weekday(Enum):... MONDAY=1, 'Mon' Weekday.__init__(self, 1, 'Mon')` would be called as `Weekday.__init__(self, 1, 'Mon')`

A class method that is used to further configure subsequent subclasses. By default, does nothing.

A class method for looking up values not found in `incls`. By default it does nothing, but can be overridden to implement custom search behavior: `>>>from enum import auto, StrEnum>>>class Build(StrEnum):... DEBUG=auto(...).OPTIMIZED=auto(...).@classmethod...def __missing__(cls, value):...value=value.lower()...formember in cls:...if member.value==value:...return member...return None...>>>Build.DEBUG.value'debug'>>>Build('deBUG')<Build.DEBUG: 'debug'>Added in version 3.6.`

By default, doesn't exist. If specified, either in the enum class definition or in a mixin class (such as `int`), all values given in the member assignment will be passed;

e.g. `>>>from enum import Enum>>>class MyIntEnum(int, Enum):... TWENTYSIX='1a', 16` results in the call `int('1a', 16)` and a value of 26 for the member. Note when writing a custom `__new__`, do not use `super().__new__` – call the appropriate `__new__` instead.

Returns the string used for `repr()` calls. By default, returns the `Enum` name, member name, and value, but can be overridden: `>>>from enum import auto, Enum>>>class OtherStyle(Enum):... ALTERNATE=auto()... OTHER=auto()... SOMETHING_ELSE=auto()...def __repr__(self):...cls_name=self.__class__.__name__...return f'{cls_name}.{self.name}'...>>>OtherStyle.ALTERNATE, str(OtherStyle.ALTERNATE), f'{OtherStyle.ALTERNATE}'(OtherStyle.ALTERNATE, 'OtherStyle.ALTERNATE', 'OtherStyle.ALTERNATE')`

Returns the string used for `str()` calls. By default, returns the `Enum` name and member name, but can be overridden: `>>>from enum import auto, Enum>>>class OtherStyle(Enum):... ALTERNATE=auto()... OTHER=auto()... SOMETHING_ELSE=auto()...def __str__(self):...return f'{self.name}'...>>>OtherStyle.ALTERNATE, str(OtherStyle.ALTERNATE), f'{OtherStyle.ALTERNATE}'(<OtherStyle.ALTERNATE: 1>, 'ALTERNATE', 'ALTERNATE')`

Returns the string used for `format()` and `f-string` calls. By default, returns `__str__()` return value, but can be overridden: `>>>from enum import auto, Enum>>>class OtherStyle(Enum):... ALTERNATE=auto()... OTHER=auto()... SOMETHING_ELSE=auto()...def __format__(self, spec):...return f'{self.name}'...>>>OtherStyle.ALTERNATE, str(OtherStyle.ALTERNATE), f'{OtherStyle.ALTERNATE}'(<OtherStyle.ALTERNATE: 1>, 'OtherStyle.ALTERNATE', 'ALTERNATE')`

Adds a new name as an alias to an existing member: `>>>Color.RED._add_alias_('ERROR')>>>Color.ERROR<Color.RED: 1>` Raises a `NameError` if the name is already assigned to a different member. Added in version 3.13.

Adds a new value as an alias to an existing member: `>>>Color.RED._add_value_alias_(42)>>>Color(42)<Color.RED: 1>` Raises a `ValueError` if the value is already linked with a different member. See `MultiValueEnum` for an example. Added in version 3.13.

class enum.IntEnum

`IntEnum` is the same as `Enum`, but its members are also integers and can be used anywhere that an integer can be used. If any integer operation is performed with an `IntEnum` member, the resulting value loses its enumeration status. `>>>from enum import IntEnum>>>class Number(IntEnum):... ONE=1... TWO=2... THREE=3...>>>Number.THREE<Number.THREE: 3>>>Number.ONE+Number.TWO3>>>Number.THREE+58>>>Number.THREE==3True` Note using `auto` with `IntEnum` results in integers of increasing value, starting with 1. Changed in version 3.

class enum.StrEnum

`StrEnum` is the same as `Enum`, but its members are also strings and can be used in most of the same places that a string can be used. The result of any string operation performed on or with a `StrEnum` member is not part of the enumeration. `>>>from enum import StrEnum, auto>>>class Color(StrEnum):... RED='r'... GREEN='g'... BLUE=''`

```
b'...UNKNOWN=auto()...>>>Color.RED<Color.RED: 'r'>>>Color.UNKNOWN<Color.UNKNOWN:
'unknown'>>>str(Color.UNKNOWN)'unknown'NoteThere are places in the stdlib that check for an exactstrin
```

classenum.Flag

```
__contains__(self,value)
```

```
__iter__(self):
```

```
__len__(self):
```

```
__bool__(self):
```

```
__or__(self,other)
```

```
__and__(self,other)
```

```
__xor__(self,other)
```

```
__invert__(self):
```

```
__numeric_repr__()
```

Flag is the same as Enum, but its members support the bitwise operators & (AND), | (OR), ^ (XOR), and ~ (INVERT); the results of those operations are (aliases of) members of the enumeration. `__contains__(self,value)` Returns True if value is in self: `>>>from enum import Flag, auto>>>class Color(Flag):...RED=auto()...GREEN=auto()...BLUE=auto()...>>>purple=Color.RED|Color.BLUE>>>white=Color.RED|Color.GREEN|Color.BLUE>>>Color.GREEN in purple False>>>Color.GREEN in white True>>>purple in white True>>>white in purple False` `__iter__(s`

Returns True if value is in self: `>>>from enum import Flag, auto>>>class Color(Flag):...RED=auto()...GREEN=auto()...BLUE=auto()...>>>purple=Color.RED|Color.BLUE>>>white=Color.RED|Color.GREEN|Color.BLUE>>>Color.GREEN in purple False>>>Color.GREEN in white True>>>purple in white True>>>white in purple False`

Returns all contained non-alias members: `>>>list(Color.RED)[<Color.RED: 1>]>>>list(purple)[<Color.RED: 1>, <Color.BLUE: 4>]` Added in version 3.11.

Returns number of members in flag: `>>>len(Color.GREEN)1>>>len(white)3` Added in version 3.11.

Returns True if any members in flag, False otherwise: `>>>bool(Color.GREEN)True>>>bool(white)True>>>black=Color(0)>>>bool(black)False`

Returns current flag binary or'ed with other: `>>>Color.RED|Color.GREEN<Color.RED|GREEN: 3>`

Returns current flag binary and'ed with other: `>>>purple&white<Color.RED|BLUE: 5>>>purple&Color.GREEN<Color: 0>`

Returns current flag binary xor'ed with other: `>>>purple^white<Color.GREEN: 2>>>purple^Color.GREEN<Color.RED|GREEN|BLUE: 7>`

Returns all the flags in type(self) that are not in self: `>>>~white<Color: 0>>>~purple<Color.GREEN: 2>>>~Color.RED<Color.GREEN|BLUE: 6>`

Function used to format any remaining unnamed numeric values. Default is the value's repr; common choices are `hex()` and `oct()`.

classenum.IntFlag

IntFlag is the same as Flag, but its members are also integers and can be used anywhere that an integer can be used. `>>>from enum import IntFlag, auto>>>class Color(IntFlag):...RED=auto()...GREEN=auto()...BLUE=auto()...>>>Color.RED&2<Color: 0>>>Color.RED|2<Color.RED|GREEN: 3>` If any integer operation is performed with an IntFlag member, the result is not an IntFlag: `>>>Color.RED+23` If a Flag operation is performed with an IntFlag member and the result is a valid IntFlag, an IntFlag is returned; the result is not a va

classenum.ReprEnum

ReprEnum uses the repr() of Enum, but the str() of the mixed-in data type: int.__str__() for IntEnum and IntFlag, str.__str__() for StrEnum. Inherit from ReprEnum to keep the str()/format() of the mixed-in data type instead of using the Enum-default str(). Added in version 3.11.

classenum.EnumCheck

UNIQUE

CONTINUOUS

NAMED_FLAGS

EnumCheck contains the options used by the verify() decorator to ensure various constraints; failed constraints result in a ValueError. UNIQUE Ensure that each value has only one name: >>> from enum import Enum, verify, UNIQUE >>> @verify(UNIQUE)... class Color(Enum): ... RED=1 ... GREEN=2 ... BLUE=3 ... CRIMSON=1 Traceback (most recent call last): ... ValueError: aliases found in <enum 'Color': CRIMSON -> RED CONTINUOUS Ensure that there are no missing values between the lowest-valued member and the highest-valued member: >>>

Ensure that each value has only one name: >>> from enum import Enum, verify, UNIQUE >>> @verify(UNIQUE)... class Color(Enum): ... RED=1 ... GREEN=2 ... BLUE=3 ... CRIMSON=1 Traceback (most recent call last): ... ValueError: aliases found in <enum 'Color': CRIMSON -> RED

Ensure that there are no missing values between the lowest-valued member and the highest-valued member: >>> from enum import Enum, verify, CONTINUOUS >>> @verify(CONTINUOUS)... class Color(Enum): ... RED=1 ... GREEN=2 ... BLUE=5 Traceback (most recent call last): ... ValueError: invalid enum 'Color': missing values 3, 4

Ensure that any flag groups/masks contain only named flags – useful when values are specified instead of being generated by auto(): >>> from enum import Flag, verify, NAMED_FLAGS >>> @verify(NAMED_FLAGS)... class Color(Flag): ... RED=1 ... GREEN=2 ... BLUE=4 ... WHITE=15 ... NEON=31 Traceback (most recent call last): ... ValueError: invalid Flag 'Color': aliases WHITE and NEON are missing combined values of 0x18 [use enum.show_flag_values(value) for details]

classenum.FlagBoundary

STRICT

CONFORM

EJECT

KEEP

FlagBoundary controls how out-of-range values are handled in Flag and its subclasses. STRICT Out-of-range values cause a ValueError to be raised. This is the default for Flag: >>> from enum import Flag, STRICT, auto >>> class StrictFlag(Flag, boundary=STRICT): ... RED=auto() ... GREEN=auto() ... BLUE=auto() ... >>> StrictFlag(2**2+2**4) Traceback (most recent call last): ... ValueError: <flag 'StrictFlag'> invalid value 20 given 0b010100 allowed 0b000111 CONFORM Out-of-range values have invalid values removed, leaving a valid Flag

Out-of-range values cause a ValueError to be raised. This is the default for Flag: >>> from enum import Flag, STRICT, auto >>> class StrictFlag(Flag, boundary=STRICT): ... RED=auto() ... GREEN=auto() ... BLUE=auto() ... >>> StrictFlag(2**2+2**4) Traceback (most recent call last): ... ValueError: <flag 'StrictFlag'> invalid value 20 given 0b010100 allowed 0b000111

Out-of-range values have invalid values removed, leaving a valid Flag value: >>> from enum import Flag, CONFORM, auto >>> class ConformFlag(Flag, boundary=CONFORM): ... RED=auto() ... GREEN=auto() ... BLUE=auto() ... >>> ConformFlag(2**2+2**4) <ConformFlag.BLUE: 4>

Out-of-range values lose their Flag membership and revert to int: >>> from enum import Flag, EJECT, auto >>> class EjectFlag(Flag, boundary=EJECT): ... RED=auto() ... GREEN=auto() ... BLUE=auto() ... >>> EjectFlag(2**2+2**4) 20

Out-of-range values are kept, and the `Flag` membership is kept. This is the default for `IntFlag`:

```
>>>from enum import
Flag,KEEP,auto>>>class KeepFlag(Flag,boundary=KEEP):...RED=auto()...GREEN=auto()...BLUE=auto()...>
>>KeepFlag(2**2+2**4)<KeepFlag.BLUE|16: 20>
```

class enum.EnumDict

member_names

`EnumDict` is a subclass of `dict` that is used as the namespace for defining enum classes (see [Preparing the class namespace](#)). It is exposed to allow subclasses of `EnumType` with advanced behavior like having multiple values per member. It should be called with the name of the enum class being created, otherwise private names and internal classes will not be handled correctly. Note that only the `MutableMapping` interface (`__setitem__()` and `update()`) is overridden. It may be possible to bypass the checks using `o`

A list of member names.

Supported __dunder__ names

`__members__` is a read-only ordered mapping of `member_name:member` items. It is only available on the class.

`__new__()`, if specified, must create and return the enum members; it is also a very good idea to set the member's `__value__` appropriately. Once all the members are created it is no longer used.

Supported __sunder__ names

- `__name__` – name of the member
- `__value__` – value of the member; can be set in `__new__`
- `__missing__()` – a lookup function used when a value is not found; may be overridden
- `__ignore__` – a list of names, either as a list or a str, that will not be transformed into members, and will be removed from the final class
- `__order__` – no longer used, kept for backward compatibility (class attribute, removed during class creation)
- `__generate_next_value__()` – used to get an appropriate value for an enum member; may be overridden
- `__add_alias__()` – adds a new name as an alias to an existing member.
- `__add_value_alias__()` – adds a new value as an alias to an existing member.
- While `__sunder__` names are generally reserved for the further development of the `Enum` class and can not be used, some are explicitly allowed: `__repr__` (e.g. `__repr_html__`), as used in IPython's rich display

Added in version 3.6: `__missing__`, `__order__`, `__generate_next_value__`

Added in version 3.7: `__ignore__`

Added in version 3.13: `__add_alias__`, `__add_value_alias__`, `__repr__`

Utilities and decorators

class enum.auto

`auto` can be used in place of a value. If used, the `Enum` machinery will call an `Enum`'s `__generate_next_value__()` to get an appropriate value. For `Enum` and `IntEnum` that appropriate value will be the last value plus one; for `Flag` and `IntFlag` it will be the first power-of-two greater than the highest value; for `StrEnum` it will be the lower-cased version of the member's name. Care must be taken if mixing `auto()` with manually specified values. `auto` instances are only resolved when at the top level of an assignment, either

@enum.property

A decorator similar to the built-in `property`, but specifically for enumerations. It allows member attributes to have the same names as members themselves. Note the `property` and the member must be defined in separate classes; for example, the `value` and `name` attributes are defined in the `Enum` class, and `Enum` subclasses can define members with the names `value` and `name`. Added in version 3.11.

@enum.unique

A class decorator specifically for enumerations. It searches an enumeration's `__members__`, gathering any aliases it finds; if any are found `ValueError` is raised with the details:

```
>>> from enum import Enum, unique
>>> @unique
class Mistake(Enum):
    ONE=1
    TWO=2
    THREE=3
    FOUR=3
Traceback (most recent call last):
  ...
ValueError: duplicate values found in <enum 'Mistake': FOUR -> THREE
```

@enum.verify

A class decorator specifically for enumerations. Members from `EnumCheck` are used to specify which constraints should be checked on the decorated enumeration. Added in version 3.11.

@enum.member

A decorator for use in enums: its target will become a member. Added in version 3.11.

@enum.nonmember

A decorator for use in enums: its target will not become a member. Added in version 3.11.

@enum.global_enum

A decorator to change the `str()` and `repr()` of an enum to show its members as belonging to the module instead of its class. Should only be used when the enum members are exported to the module global namespace (see `re.RegexFlag` for an example). Added in version 3.11.

enum.show_flag_values(value)

Return a list of all power-of-two integers contained in a flag value. Added in version 3.11.

enum.bin(num, max_bits=None)

Like built-in `bin()`, except negative values are represented in two's complement, and the leading bit always indicates sign (0 implies positive, 1 implies negative).

```
>>> import enum
>>> enum.bin(10)
'0b01010'
>>> enum.bin(~10)
'-11'
0b10101'
Added in version 3.11.
```

Notes

`IntEnum`, `StrEnum`, and `IntFlag`

These three enum types are designed to be drop-in replacements for existing integer- and string-based values; as such, they have extra limitations:

- `__str__` uses the value and not the name of the enum member
- `__format__`, because it uses `__str__`, will also use the value of the enum member instead of its name

If you do not need/want those limitations, you can either create your own base class by mixing in the `int` or `str` type yourself:

```
>>> from enum import Enum
>>> class MyIntEnum(int, Enum):
...     pass
```

or you can reassign the appropriate `str()`, etc., in your enum:

```
>>> from enum import Enum, IntEnum
>>> class MyIntEnum(IntEnum):
```

```
... __str__ = Enum.__str__
```

graphlib— Functionality to operate with graph-like structures

Source code: `Lib/graphlib.py`

```
class graphlib.TopologicalSorter(graph=None)
```

```
add(node,*predecessors)
```

```
prepare()
```

```
is_active()
```

```
done(*nodes)
```

```
get_ready()
```

```
static_order()
```

Provides functionality to topologically sort a graph of hashable nodes. A topological order is a linear ordering of the vertices in a graph such that for every directed edge $u \rightarrow v$ from vertex u to vertex v , vertex u comes before vertex v in the ordering. For instance, the vertices of the graph may represent tasks to be performed, and the edges may represent constraints that one task must be performed before another; in this example, a topological ordering is just a valid sequence for the tasks. A

Add a new node and its predecessors to the graph. Both the node and all elements in `predecessors` must be hashable. If called multiple times with the same node argument, the set of dependencies will be the union of all dependencies passed in. It is possible to add a node with no dependencies (`predecessors` is not provided) or to provide a dependency twice. If a node that has not been provided before is included among `predecessors` it will be automatically added to the graph with no predecessors of its own. Raises

Mark the graph as finished and check for cycles in the graph. If any cycle is detected, `CycleError` will be raised, but `get_ready()` can still be used to obtain as many nodes as possible until cycles block more progress. After a call to this function, the graph cannot be modified, and therefore no more nodes can be added using `add()`. A `ValueError` will be raised if the sort has been started by `static_order()` or `get_ready()`. Changed in version 3.14: `prepare()` can now be called more than once as long as the sort has

Returns `True` if more progress can be made and `False` otherwise. Progress can be made if cycles do not block the resolution and either there are still nodes ready that haven't yet been returned by `TopologicalSorter.get_ready()` or the number of nodes marked `TopologicalSorter.done()` is less than the number that have been returned by `TopologicalSorter.get_ready()`. The `__bool__()` method of this class defers to this function, so instead of `if ts.is_active():...` it is possible to simply do `if ts:...`. Raises `ValueError` if

Marks a set of nodes returned by `TopologicalSorter.get_ready()` as processed, unblocking any successor of each node in `nodes` for being returned in the future by a call to `TopologicalSorter.get_ready()`. Raises `ValueError` if any node in `nodes` has already been marked as processed by a previous call to this method or if a node was not added to the graph by using `TopologicalSorter.add()`, if called without calling `prepare()` or if node has not yet been returned by `get_ready()`.

Returns a tuple with all the nodes that are ready. Initially it returns all nodes with no predecessors, and once those are marked as processed by calling `TopologicalSorter.done()`, further calls will return all new nodes that have all their predecessors already processed. Once no more progress can be made, empty tuples are returned. Raises `ValueError` if called without calling `prepare()` previously.

Returns an iterator object which will iterate over nodes in a topological order. When using this method, `prepare()` and `done()` should not be called. This method is equivalent to:

```
def static_order(self):
    self.prepare()
    while self.is_active():
        node_group = self.get_ready()
        yield from node_group
```


from node_groupself.done(*node_group) The particular order that is returned may depend on the specific order in which the items were inserted in the graph. For

example: >>> ts = TopologicalSorter() >>> ts.add(3,2,1) >>> ts.add(1,0) >>> print(*ts.s

Exceptions

The graphlib module defines the following exception classes:

exception graphlib.CycleError

Subclass of ValueError raised by TopologicalSorter.prepare() if cycles exist in the working graph. If multiple cycles exist, only one undefined choice among them will be reported and included in the exception. The detected cycle can be accessed via the second element in the args attribute of the exception instance and consists in a list of nodes, such that each node is, in the graph, an immediate predecessor of the next node in the list. In the reported list, the first and the last node will be the same

Numeric and Mathematical Modules

The modules described in this chapter provide numeric and math-related functions and data types.

The `numbers` module defines an abstract hierarchy of numeric types. The `math` and `cmath` modules contain various mathematical functions for floating-point and complex numbers. The `decimal` module supports exact representations of decimal numbers, using arbitrary precision arithmetic.

The following modules are documented in this chapter:

- `numbers`— Numeric abstract base classesThe numeric towerNotes for type implementersAdding More Numeric ABCsImplementing the arithmetic operations
- `math`— Mathematical functionsNumber-theoretic functionsFloating point arithmeticFloating point manipulation functionsPower, exponential and logarithmic functionsSummation and product functionsAngular conversionTrigonometric functionsHyperbolic functionsSpecial functionsConstants
- `cmath`— Mathematical functions for complex numbersConversions to and from polar coordinatesPower and logarithmic functionsTrigonometric functionsHyperbolic functionsClassification functionsConstants
- `decimal`— Decimal fixed-point and floating-point arithmeticQuick-start tutorialDecimal objectsLogical operandsContext objectsConstantsRounding modesSignalsFloating-point notesMitigating round-off error with increased precisionSpecial valuesWorking with threadsRecipesDecimal FAQ
- `fractions`— Rational numbers
- `random`— Generate pseudo-random numbersBookkeeping functionsFunctions for bytesFunctions for integersFunctions for sequencesDiscrete distributionsReal-valued distributionsAlternative GeneratorNotes on ReproducibilityExamplesRecipesCommand-line usageCommand-line example
- `statistics`— Mathematical statistics functionsAverages and measures of central locationMeasures of spreadStatistics for relations between two inputsFunction detailsExceptionsNormalDistobjectsExamples and RecipesClassic probability problemsMonte Carlo inputs for simulationsApproximating binomial distributionsNaive bayesian classifier

numbers— Numeric abstract base classes

Source code: `Lib/numbers.py`

The `numbers` module (PEP 3141) defines a hierarchy of numeric abstract base classes which progressively define more operations. None of the types defined in this module are intended to be instantiated.

`class numbers.Number`

The root of the numeric hierarchy. If you just want to check if an argument `x` is a number, without caring what kind, use `isinstance(x, Number)`.

The numeric tower

`class numbers.Complex`

`real`

`imag`

`abstractmethod conjugate()`

Subclasses of this type describe complex numbers and include the operations that work on the built-in `complex` type. These are: conversions to `complex` and `bool`, `real`, `imag`, `+`, `-`, `*`, `/`, `**`, `abs()`, `conjugate()`, `==`, and `!=`. All except `and` `!=` are abstract. `real` Abstract. Retrieves the real component of this number. `imag` Abstract. Retrieves the imaginary component of this number. `abstractmethod conjugate()` Abstract. Returns the complex conjugate. For example, `(1+3j).conjugate() == (1-3j)`.

Abstract. Retrieves the real component of this number.

Abstract. Retrieves the imaginary component of this number.

Abstract. Returns the complex conjugate. For example, `(1+3j).conjugate() == (1-3j)`.

`class numbers.Real`

To `Complex`, `Real` adds the operations that work on real numbers. In short, those are: a conversion to `float`, `math.trunc()`, `round()`, `math.floor()`, `math.ceil()`, `divmod()`, `//`, `%`, `<`, `<=`, `>`, and `>=`. `Real` also provides defaults for `complex()`, `real`, `imag`, and `conjugate()`.

`class numbers.Rational`

`numerator`

`denominator`

Subtypes `Real` and adds `numerator` and `denominator` properties. It also provides a default for `float()`. The `numerator` and `denominator` values should be instances of `Integral` and should be in lowest terms with `denominator` positive. `numerator` Abstract. The numerator of this rational number. `denominator` Abstract. The denominator of this rational number.

Abstract. The numerator of this rational number.

Abstract. The denominator of this rational number.

`class numbers.Integral`

Subtypes `Rational` and adds a conversion to `int`. Provides defaults for `float()`, `numerator`, and `denominator`. Adds abstract methods for `pow()` with modulus and bit-string operations: `<<`, `>>`, `&`, `^`, `|`, `~`.

Notes for type implementers

Implementers should be careful to make equal numbers equal and hash them to the same values. This may be subtle if there are two different extensions of the real numbers. For

example, `Fraction` implements `hash()` as follows:

```
def __hash__(self):
    if self.denominator == 1:
        # Get integers right.
        return hash(self.numerator)
    # Expensive check, but definitely correct.
    if self == float(self):
        return hash(float(self))
    else:
        # Use tuple's hash to avoid a high collision rate on
        # simple fractions.
        return hash((self.numerator, self.denominator))
```

Adding More Numeric ABCs

There are, of course, more possible ABCs for numbers, and this would be a poor hierarchy if it precluded the possibility of adding those. You can add `MyFoo` between `Complex` and `Real` with:

```
class MyFoo(Complex): ...
MyFoo.register(Real)
```

Implementing the arithmetic operations

We want to implement the arithmetic operations so that mixed-mode operations either call an implementation whose author knew about the types of both arguments, or convert both to the nearest built in type and do the operation there. For subtypes of `Integral`, this means that `__add__()` and `__radd__()` should be defined as:

```
class MyIntegral(Integral):

    def __add__(self, other):
        if isinstance(other, MyIntegral):
            return do_my_adding_stuff(self, other)
        elif isinstance(other, OtherTypeIKnowAbout):
            return do_my_other_adding_stuff(self, other)
        else:
            return NotImplemented

    def __radd__(self, other):
        if isinstance(other, MyIntegral):
            return do_my_adding_stuff(other, self)
        elif isinstance(other, OtherTypeIKnowAbout):
            return do_my_other_adding_stuff(other, self)
        elif isinstance(other, Integral):
            return int(other) + int(self)
        elif isinstance(other, Real):
            return float(other) + float(self)
        elif isinstance(other, Complex):
            return complex(other) + complex(self)
        else:
            return NotImplemented
```

There are 5 different cases for a mixed-type operation on subclasses of `Complex`. I'll refer to all of the above code that doesn't refer to `MyIntegral` and `OtherTypeIKnowAbout` as "boilerplate". `a` will be an instance of `A`, which is a subtype of `Complex` (`a: A <: Complex`), and `b: B <: Complex`. I'll consider `a+b`:

- If `A` defines an `__add__()` which accepts `b`, all is well.
- If `A` falls back to the boilerplate code, and it were to return a value from `__add__()`, we'd miss the possibility that `B` defines a more intelligent `__radd__()`, so the boilerplate should return `NotImplemented` from `__add__()`.

(OrA may not implement `__add__()` at all.)

- Then B's `__radd__()` gets a chance. If it accepts a, all is well.
- If it falls back to the boilerplate, there are no more possible methods to try, so this is where the default implementation should live.
- If $B <: A$, Python tries `B.__radd__` before `A.__add__`. This is ok, because it was implemented with knowledge of A, so it can handle those instances before delegating to `Complex`.

If $A <: \text{Complex}$ and $B <: \text{Real}$ without sharing any other knowledge, then the appropriate shared operation is the one involving the built `incomplex`, and both `__radd__()`s land there, so $a+b == b+a$.

Because most of the operations on any given type will be very similar, it can be useful to define a helper function which generates the forward and reverse instances of any given operator. For example, `fractions.Fraction` uses:

```
def _operator_fallbacks(monomorphic_operator, fallback_operator):
    def forward(a, b):
        if isinstance(b, (int, Fraction)):
            return monomorphic_operator(a, b)
        elif isinstance(b, float):
            return fallback_operator(float(a), b)
        elif isinstance(b, complex):
            return fallback_operator(complex(a), b)
        else:
            return NotImplemented
    forward.__name__ = '__' + fallback_operator.__name__ + '__'
    forward.__doc__ = monomorphic_operator.__doc__

    def reverse(b, a):
        if isinstance(a, Rational):
            # Includes ints.
            return monomorphic_operator(a, b)
        elif isinstance(a, Real):
            return fallback_operator(float(a), float(b))
        elif isinstance(a, Complex):
            return fallback_operator(complex(a), complex(b))
        else:
            return NotImplemented
    reverse.__name__ = '__r' + fallback_operator.__name__ + '__'
    reverse.__doc__ = monomorphic_operator.__doc__

    return forward, reverse

def _add(a, b):
    """a + b"""
    return Fraction(a.numerator * b.denominator +
                    b.numerator * a.denominator,
                    a.denominator * b.denominator)

__add__, __radd__ = _operator_fallbacks(_add, operator.add)

# ...
```

math— Mathematical functions

This module provides access to common mathematical functions and constants, including those defined by the C standard.

These functions cannot be used with complex numbers; use the functions of the same name from the `cmath` module if you require support for complex numbers. The distinction between functions which support complex numbers and those which don't is made since most users do not want to learn quite as much mathematics as required to understand complex numbers. Receiving an exception instead of a complex result allows earlier detection of the unexpected complex number used as a parameter, so that the programmer can determine how and why it was generated in the first place.

The following functions are provided by this module. Except when explicitly noted otherwise, all return values are floats.

```
Number-theoretic functions | comb(n, k) | Number of ways to choose k items from n items without repetition and without order | factorial(n) | n | factorial | gcd(*integers) | Greatest common divisor of the integer arguments | isqrt(n) | Integer square root of a nonnegative integer | n | lcm(*integers) | Least common multiple of the integer arguments | perm(n, k) | Number of ways to choose k items from n items without repetition and with order | Floating point arithmetic | ceil(x) | Ceiling of x, the smallest integer greater than or equal to x | fabs(x) | Absolute value of x | floor(x) | Floor of x, the largest integer less than or equal to x | fma(x, y, z) | Fused multiply-add operation: (x * y) + z | fmod(x, y) | Remainder of division |
```

Number-theoretic functions

math.comb(n,k)

Return the number of ways to choose k items from n items without repetition and without order. Evaluates to $n!/(k!(n-k)!)$ when $k \leq n$ and evaluates to zero when $k > n$. Also called the binomial coefficient because it is equivalent to the coefficient of k -th term in polynomial expansion of $(1+x)^n$. Raises `TypeError` if either of the arguments are not integers. Raises `ValueError` if either of the arguments are negative. Added in version 3.8.

math.factorial(n)

Return factorial of the nonnegative integer n . Changed in version 3.10: Floats with integral values (like 5.0) are no longer accepted.

math.gcd(*integers)

Return the greatest common divisor of the specified integer arguments. If any of the arguments is nonzero, then the returned value is the largest positive integer that is a divisor of all arguments. If all arguments are zero, then the returned value is 0. `gcd()` without arguments returns 0. Added in version 3.5. Changed in version 3.9: Added support for an arbitrary number of arguments. Formerly, only two arguments were supported.

math.isqrt(n)

Return the integer square root of the nonnegative integer n . This is the floor of the exact square root of n , or equivalently the greatest integer a such that $a^2 \leq n$. For some applications, it may be more convenient to have the least integer a such that $n \leq a^2$, or in other words the ceiling of the exact square root of n . For positive n , this can be computed using `a=1+isqrt(n-1)`. Added in version 3.8.

math.lcm(*integers)

Return the least common multiple of the specified integer arguments. If all arguments are nonzero, then the returned value is the smallest positive integer that is a multiple of all arguments. If any of the arguments is zero, then the returned value is 0. `lcm()` without arguments returns 1. Added in version 3.9.

math.perm(n,k=None)

Return the number of ways to choose k items from n items without repetition and with order. Evaluates to $n!/(n-k)!$ when $k \leq n$ and evaluates to zero when $k > n$. If k is not specified or is `None`, then k defaults to n and the function returns $n!$. Raises `TypeError` if either of the arguments are not integers. Raises `ValueError` if either of the arguments are negative. Added in version 3.8.

Floating point arithmetic

math.ceil(x)

Return the ceiling of x , the smallest integer greater than or equal to x . If x is not a float, delegates to `x.__ceil__`, which should return an `Integral` value.

math.fabs(x)

Return the absolute value of x .

math.floor(x)

Return the floor of x , the largest integer less than or equal to x . If x is not a float, delegates to `x.__floor__`, which should return an `Integral` value.

math.fma(x,y,z)

Fused multiply-add operation. Return $(x*y)+z$, computed as though with infinite precision and range followed by a single round to the float format. This operation often provides better accuracy than the direct expression $(x*y)+z$. This function follows the specification of the fused `MultiplyAdd` operation described in the IEEE 754 standard. The standard leaves one case implementation-defined, namely the result of `fma(0,inf,nan)` and `fma(inf,0,nan)`. In these cases, `math.fma` returns a `NaN`, and does not raise any

math.fmod(x,y)

Return the floating-point remainder of x/y , as defined by the platform C library function `fmod(x,y)`. Note that the Python expression `x%y` may not return the same result. The intent of the C standard is that `fmod(x,y)` be exactly (mathematically; to infinite precision) equal to $x - n*y$ for some integer n such that the result has the same sign as x and magnitude less than $\text{abs}(y)$. Python's `x%y` returns a result with the sign of y instead, and may not be exactly computable for float arguments. For example, `fmod(-1e-100,`

math.modf(x)

Return the fractional and integer parts of x . Both results carry the sign of x and are floats. Note that `modf()` has a different call/return pattern than its C equivalents: it takes a single argument and return a pair of values, rather than returning its second return value through an 'output parameter' (there is no such thing in Python).

math.remainder(x,y)

Return the IEEE 754-style remainder of x with respect to y . For finite x and finite non-zero y , this is the difference $x - n*y$, where n is the closest integer to the exact value of the quotient x/y . If x/y is exactly halfway between two consecutive integers, the nearest even integer is used for n . The remainder `r = remainder(x,y)` thus always satisfies $\text{abs}(r) \leq 0.5 * \text{abs}(y)$. Special cases follow IEEE 754: in particular, `remainder(x,math.inf)` is `x` for any finite x , and `remainder(x,0)` and `remainder(math.inf,x)` raise `ValueError` for a

math.trunc(x)

Return x with the fractional part removed, leaving the integer part. This rounds toward 0: `trunc()` is equivalent to `floor()` for positive x , and equivalent to `ceil()` for negative x . If x is not a float, delegates to `x.__trunc__`, which should return an `Integral` value.

For the `ceil()`, `floor()`, and `modf()` functions, note that all floating-point numbers of sufficiently large magnitude are exact integers. Python floats typically carry no more than 53 bits of precision (the same as the platform C double type), in which case any float x with $\text{abs}(x) \geq 2^{52}$ necessarily has no fractional bits.

Floating point manipulation functions

`math.copysign(x,y)`

Return a float with the magnitude (absolute value) of `x` but the sign of `y`. On platforms that support signed zeros, `copysign(1.0,-0.0)` returns `-1.0`.

`math.frexp(x)`

Return the mantissa and exponent of `x` as the pair `(m,e)`. `m` is a float and `e` is an integer such that $x == m * 2^e$ exactly. If `x` is zero, returns `(0.0,0)`, otherwise $0.5 \leq \text{abs}(m) < 1$. This is used to “pick apart” the internal representation of a float in a portable way. Note that `frexp()` has a different call/return pattern than its C equivalents: it takes a single argument and return a pair of values, rather than returning its second return value through an ‘output parameter’ (there is no such thing in Python).

`math.isclose(a,b,*,rel_tol=1e-09,abs_tol=0.0)`

Return `True` if the values `a` and `b` are close to each other and `False` otherwise. Whether or not two values are considered close is determined according to given absolute and relative tolerances. If no errors occur, the result will be: $\text{abs}(a-b) \leq \max(\text{rel_tol} * \max(\text{abs}(a), \text{abs}(b)), \text{abs_tol})$. `rel_tol` is the relative tolerance – it is the maximum allowed difference between `a` and `b`, relative to the larger absolute value of `a` or `b`. For example, to set a tolerance of 5%, pass `rel_tol=0.05`. The default tolerance is `1e-09`, which

`math.isfinite(x)`

Return `True` if `x` is neither an infinity nor a NaN, and `False` otherwise. (Note that `0.0` is considered finite.) Added in version 3.2.

`math.isinf(x)`

Return `True` if `x` is a positive or negative infinity, and `False` otherwise.

`math.isnan(x)`

Return `True` if `x` is a NaN (not a number), and `False` otherwise.

`math.ldexp(x,i)`

Return $x * (2^i)$. This is essentially the inverse of function `frexp()`.

`math.nextafter(x,y,steps=1)`

Return the floating-point value `steps` steps after `x` towards `y`. If `x` is equal to `y`, return `y`, unless `steps` is zero. Examples: `math.nextafter(x,math.inf)` goes up: towards positive infinity. `math.nextafter(x,-math.inf)` goes down: towards minus infinity. `math.nextafter(x,0.0)` goes towards zero. `math.nextafter(x,math.copysign(math.inf,x))` goes away from zero. See also `math.ulp()`. Added in version 3.9. Changed in version 3.12: Added the `steps` argument.

`math.ulp(x)`

Return the value of the least significant bit of the float `x`: If `x` is a NaN (not a number), return `x`. If `x` is negative, return `ulp(-x)`. If `x` is a positive infinity, return `x`. If `x` is equal to zero, return the smallest positive denormalized representable float (smaller than the minimum positive normalized float, `sys.float_info.min`). If `x` is equal to the largest positive representable float, return the value of the least significant bit of `x`, such that the first float smaller than `x` is `x-ulp(x)`. Otherwise (`x` is a positive finite

Power, exponential and logarithmic functions

math.cbrt(x)

Return the cube root of x . Added in version 3.11.

math.exp(x)

Return e raised to the power x , where $e = 2.718281\dots$ is the base of natural logarithms. This is usually more accurate than `math.e**x` or `pow(math.e, x)`.

math.exp2(x)

Return 2 raised to the power x . Added in version 3.11.

math.expm1(x)

Return e raised to the power x , minus 1. Here e is the base of natural logarithms. For small floats x , the subtraction in `exp(x)-1` can result in a significant loss of precision; the `expm1()` function provides a way to compute this quantity to full precision:

```
>>> from math import exp, expm1
>>> exp(1e-5)-1 # gives result accurate to 11 places
1.0000050000069649e-05
>>> expm1(1e-5) # result accurate to full precision
1.0000050000166668e-05
```

 Added in version 3.2.

math.log(x[,base])

With one argument, return the natural logarithm of x (to base e). With two arguments, return the logarithm of x to the given base, calculated as `log(x)/log(base)`.

math.log1p(x)

Return the natural logarithm of $1+x$ (base e). The result is calculated in a way which is accurate for x near zero.

math.log2(x)

Return the base-2 logarithm of x . This is usually more accurate than `log(x, 2)`. Added in version 3.3. See also `int.bit_length()` returns the number of bits necessary to represent an integer in binary, excluding the sign and leading zeros.

math.log10(x)

Return the base-10 logarithm of x . This is usually more accurate than `log(x, 10)`.

math.pow(x,y)

Return x raised to the power y . Exceptional cases follow the IEEE 754 standard as far as possible. In particular, `pow(1.0, x)` and `pow(x, 0.0)` always return `1.0`, even when x is a zero or a NaN. If both x and y are finite, x is negative, and y is not an integer then `pow(x, y)` is undefined, and raises `ValueError`. Unlike the built-in `**` operator, `math.pow()` converts both its arguments to type `float`. Use `**` or the built-in `pow()` function for computing exact integer powers. Changed in version 3.11: The special cases `pow(0.0, -inf)` and `pow(0.0, inf)` return `0.0` and `inf` respectively.

math.sqrt(x)

Return the square root of x .

Summation and product functions

math.dist(p,q)

Return the Euclidean distance between two points `p` and `q`, each given as a sequence (or iterable) of coordinates. The two points must have the same dimension. Roughly equivalent to: `sqrt(sum((px-qx)**2.0 for px, qx in zip(p, q)))` Added in version 3.8.

math.fsum(iterable)

Return an accurate floating-point sum of values in the iterable. Avoids loss of precision by tracking multiple intermediate partial sums. The algorithm's accuracy depends on IEEE-754 arithmetic guarantees and the typical case where the rounding mode is half-even. On some non-Windows builds, the underlying C library uses extended precision addition and may occasionally double-round an intermediate sum causing it to be off in its least significant bit. For further discussion and two alternative ap

math.hypot(*coordinates)

Return the Euclidean norm, $\sqrt{\sum(x^2 \text{ for } x \text{ in } \text{coordinates})}$. This is the length of the vector from the origin to the point given by the coordinates. For a two dimensional point (x, y) , this is equivalent to computing the hypotenuse of a right triangle using the Pythagorean theorem, $\sqrt{x^2 + y^2}$. Changed in version 3.8: Added support for n-dimensional points. Formerly, only the two dimensional case was supported. Changed in version 3.10: Improved the algorithm's accuracy so that the maximum error is unde

math.prod(iterable, *, start=1)

Calculate the product of all the elements in the input iterable. The default start value for the product is 1. When the iterable is empty, return the start value. This function is intended specifically for use with numeric values and may reject non-numeric types. Added in version 3.8.

math.sumprod(p, q)

Return the sum of products of values from two iterables `p` and `q`. Raises `ValueError` if the inputs do not have the same length. Roughly equivalent to: `sum(map(operator.mul, p, q, strict=True))` For float and mixed int/float inputs, the intermediate products and sums are computed with extended precision. Added in version 3.12.

Angular conversion

math.degrees(x)

Convert angle `x` from radians to degrees.

math.radians(x)

Convert angle `x` from degrees to radians.

Trigonometric functions

math.acos(x)

Return the arc cosine of `x`, in radians. The result is between 0 and π .

math.asin(x)

Return the arc sine of `x`, in radians. The result is between $-\pi/2$ and $\pi/2$.

math.atan(x)

Return the arc tangent of `x`, in radians. The result is between $-\pi/2$ and $\pi/2$.

math.atan2(y, x)

Return $\text{atan}(y/x)$, in radians. The result is between $-\pi$ and π . The vector in the plane from the origin to point (x, y) makes this angle with the positive X axis. The point of `atan2()` is that the signs of both inputs are known to it, so it can compute the correct quadrant for the angle. For example, `atan(1)` and `atan2(1, 1)` are both $\pi/4$, but `atan2(-1, -1)` is $-3\pi/4$.

math.cos(x)

Return the cosine of x radians.

math.sin(x)

Return the sine of x radians.

math.tan(x)

Return the tangent of x radians.

Hyperbolic functions

Hyperbolic functions are analogs of trigonometric functions that are based on hyperbolas instead of circles.

math.acosh(x)

Return the inverse hyperbolic cosine of x .

math.asinh(x)

Return the inverse hyperbolic sine of x .

math.atanh(x)

Return the inverse hyperbolic tangent of x .

math.cosh(x)

Return the hyperbolic cosine of x .

math.sinh(x)

Return the hyperbolic sine of x .

math.tanh(x)

Return the hyperbolic tangent of x .

Special functions

math.erf(x)

Return the error function at x . The `erf()` function can be used to compute traditional statistical functions such as the cumulative standard normal distribution: `def phi(x): 'Cumulative distribution function for the standard normal distribution' return (1.0 + erf(x / sqrt(2.0))) / 2.0` Added in version 3.2.

math.erfc(x)

Return the complementary error function at x . The complementary error function is defined as $1.0 - \text{erf}(x)$. It is used for large values of x where a subtraction from one would cause a loss of significance. Added in version 3.2.

math.gamma(x)

Return the Gamma function at x . Added in version 3.2.

math.lgamma(x)

Return the natural logarithm of the absolute value of the Gamma function at x . Added in version 3.2.

Constants

math.pi

The mathematical constant $\pi = 3.141592\dots$, to available precision.

math.e

The mathematical constant $e = 2.718281\dots$, to available precision.

math.tau

The mathematical constant $\tau = 6.283185\dots$, to available precision. Tau is a circle constant equal to 2π , the ratio of a circle's circumference to its radius. To learn more about Tau, check out Vi Hart's video [Pi is \(still\) Wrong](#), and start celebrating Tau day by eating twice as much pie! Added in version 3.6.

math.inf

A floating-point positive infinity. (For negative infinity, use `-math.inf`.) Equivalent to the output of `float('inf')`. Added in version 3.5.

math.nan

A floating-point “not a number” (NaN) value. Equivalent to the output of `float('nan')`. Due to the requirements of the IEEE-754 standard, `math.nan` and `float('nan')` are not considered to equal to any other numeric value, including themselves. To check whether a number is a NaN, use the `isnan()` function to test for NaNs instead of `is or ==`. Example:

```
>>> import math
>>> math.nan == math.nan
False
>>> float('nan') == float('nan')
False
>>> math.isnan(math.nan)
True
>>> math.isnan(float('nan'))
True
```

 Added in version 3.5. Changed in

CPython implementation detail: The `math` module consists mostly of thin wrappers around the platform C math library functions. Behavior in exceptional cases follows Annex F of the C99 standard where appropriate. The current implementation will raise `ValueError` for invalid operations like `sqrt(-1.0)` or `log(0.0)` (where C99 Annex F recommends signaling invalid operation or divide-by-zero), and `OverflowError` for results that overflow (for example, `exp(1000.0)`). A NaN will not be returned from any of the functions above unless one or more of the input arguments was a NaN; in that case, most functions will return a NaN, but (again following C99 Annex F) there are some exceptions to this rule, for example `pow(float('nan'), 0.0)` or `hypot(float('nan'), float('inf'))`.

Note that Python makes no effort to distinguish signaling NaNs from quiet NaNs, and behavior for signaling NaNs remains unspecified. Typical behavior is to treat all NaNs as though they were quiet.

See also

Module `cmath`

Complex number versions of many of these functions.

cmath— Mathematical functions for complex numbers

This module provides access to mathematical functions for complex numbers. The functions in this module accept integers, floating-point numbers or complex numbers as arguments. They will also accept any Python object that has either a `__complex__()` or a `__float__()` method: these methods are used to convert the object to a complex or floating-point number, respectively, and the function is then applied to the result of the conversion.

■ Nota: NoteFor functions involving branch cuts, we have the problem of deciding how to define those functions on the cut itself. Following Kahan's "Branch cuts for complex elementary functions" paper, as well as Annex G of C99 and later C standards, we use the sign of zero to distinguish one side of the branch cut from the other: for a branch cut along (a portion of) the real axis we look at the sign of the imaginary part, while for a branch cut along the imaginary axis we look at the sign of the real part. For example, the `cmath.sqrt()` function has a branch cut along the negative real axis. An argument

```
Conversions to and from polar coordinates | phase(z) | Return the phase of | z | polar(z) |
Return the representation of | z | in polar coordinates | rect(r, | phi) | Return the complex
number | z | with polar coordinates | r | and | phi | Power and logarithmic functions | exp(z) |
Return | e | raised to the power | z | log(z[, | base]) | Return the logarithm of | z | to the
given | base | ( | e | by default) | log10(z) | Return the base-10 logarithm of | z | sqrt(z) |
Return the square root of | z | Trigonometric functions | acos(z) | Return the arc cosine of | z |
| asin(z) | Return the arc sine of | z | atan(z) | Return the arc tangent of | z | cos(z) |
Return the cosine of | z | sin(z) | Return the sine of | z | tan(z) | Return the tangent of | z |
| Hyperbolic functions | acosh(z) | Retur
```

Conversions to and from polar coordinates

A Python complex number `z` is stored internally using rectangular or Cartesian coordinates. It is completely determined by its real part `z.real` and its imaginary part `z.imag`.

Polar coordinates give an alternative way to represent a complex number. In polar coordinates, a complex number `z` is defined by the modulus `r` and the phase angle `phi`. The modulus `r` is the distance from `z` to the origin, while the phase `phi` is the counterclockwise angle, measured in radians, from the positive x-axis to the line segment that joins the origin to `z`.

The following functions can be used to convert from the native rectangular coordinates to polar coordinates and back.

`cmath.phase(z)`

Return the phase of `z` (also known as the argument of `z`), as a float. `phase(z)` is equivalent to `math.atan2(z.imag, z.real)`. The result lies in the range $[-\pi, \pi]$, and the branch cut for this operation lies along the negative real axis. The sign of the result is the same as the sign of `z.imag`, even when `z.imag` is zero: `>>> phase(-1+0j) 3.141592653589793 >>> phase(-1-0j) -3.141592653589793`

■ Nota: NoteThe modulus (absolute value) of a complex number `z` can be computed using the built-in `abs()` function. There is no separate `cmath` module function for this operation.

`cmath.polar(z)`

Return the representation of `z` in polar coordinates. Returns a pair `(r, phi)` where `r` is the modulus of `z` and `phi` is the phase of `z`. `polar(z)` is equivalent to `(abs(z), phase(z))`.

`cmath.rect(r, phi)`

Return the complex number `z` with polar coordinates `r` and `phi`. Equivalent to `complex(r * math.cos(phi), r * math.sin(phi))`.

Power and logarithmic functions

`cmath.exp(z)`

Return e^z , where e is the base of natural logarithms.

`cmath.log(z[,base])`

Return the logarithm of z to the given base. If the base is not specified, returns the natural logarithm of z . There is one branch cut, from 0 along the negative real axis to $-\infty$.

`cmath.log10(z)`

Return the base-10 logarithm of z . This has the same branch cut as `log()`.

`cmath.sqrt(z)`

Return the square root of z . This has the same branch cut as `log()`.

Trigonometric functions

`cmath.acos(z)`

Return the arc cosine of z . There are two branch cuts: One extends right from 1 along the real axis to ∞ . The other extends left from -1 along the real axis to $-\infty$.

`cmath.asin(z)`

Return the arc sine of z . This has the same branch cuts as `acos()`.

`cmath.atan(z)`

Return the arc tangent of z . There are two branch cuts: One extends from $1j$ along the imaginary axis to ∞j . The other extends from $-1j$ along the imaginary axis to $-\infty j$.

`cmath.cos(z)`

Return the cosine of z .

`cmath.sin(z)`

Return the sine of z .

`cmath.tan(z)`

Return the tangent of z .

Hyperbolic functions

`cmath.acosh(z)`

Return the inverse hyperbolic cosine of z . There is one branch cut, extending left from 1 along the real axis to $-\infty$.

`cmath.asinh(z)`

Return the inverse hyperbolic sine of z . There are two branch cuts: One extends from $1j$ along the imaginary axis to ∞j . The other extends from $-1j$ along the imaginary axis to $-\infty j$.

`cmath.atanh(z)`

Return the inverse hyperbolic tangent of z . There are two branch cuts: One extends from 1 along the real axis to ∞ . The other extends from -1 along the real axis to $-\infty$.

cmath.cosh(z)

Return the hyperbolic cosine of z .

cmath.sinh(z)

Return the hyperbolic sine of z .

cmath.tanh(z)

Return the hyperbolic tangent of z .

Classification functions

cmath.isfinite(z)

Return `True` if both the real and imaginary parts of z are finite, and `False` otherwise. Added in version 3.2.

cmath.isinf(z)

Return `True` if either the real or the imaginary part of z is an infinity, and `False` otherwise.

cmath.isnan(z)

Return `True` if either the real or the imaginary part of z is a NaN, and `False` otherwise.

cmath.isclose(a,b,*,rel_tol=1e-09,abs_tol=0.0)

Return `True` if the values a and b are close to each other and `False` otherwise. Whether or not two values are considered close is determined according to given absolute and relative tolerances. If no errors occur, the result will be: $\text{abs}(a-b) \leq \max(\text{rel_tol} * \max(\text{abs}(a), \text{abs}(b)), \text{abs_tol})$. rel_tol is the relative tolerance – it is the maximum allowed difference between a and b , relative to the larger absolute value of a or b . For example, to set a tolerance of 5%, pass $\text{rel_tol}=0.05$. The default tolerance is $1e-09$, which

Constants

cmath.pi

The mathematical constant π , as a float.

cmath.e

The mathematical constant e , as a float.

cmath.tau

The mathematical constant τ , as a float. Added in version 3.6.

cmath.inf

Floating-point positive infinity. Equivalent to `float('inf')`. Added in version 3.6.

cmath.infj

Complex number with zero real part and positive infinity imaginary part. Equivalent to `complex(0.0, float('inf'))`. Added in version 3.6.

cmath.nan

A floating-point “not a number” (NaN) value. Equivalent to `float('nan')`. See also `math.nan`. Added in version 3.6.

cmath.nanj

Complex number with zero real part and NaN imaginary part. Equivalent to `complex(0.0, float('nan'))`. Added in version 3.6.

Note that the selection of functions is similar, but not identical, to that in `module math`. The reason for having two modules is that some users aren't interested in complex numbers, and perhaps don't even know what they are. They would rather have `math.sqrt(-1)` raise an exception than return a complex number. Also note that the functions defined in `cmath` always return a complex number, even if the answer can be expressed as a real number (in which case the complex number has an imaginary part of zero).

A note on branch cuts: They are curves along which the given function fails to be continuous. They are a necessary feature of many complex functions. It is assumed that if you need to compute with complex functions, you will understand about branch cuts. Consult almost any (not too elementary) book on complex variables for enlightenment. For information of the proper choice of branch cuts for numerical purposes, a good reference should be the following:

See also

Kahan, W: Branch cuts for complex elementary functions; or, Much ado about nothing's sign bit. In Iserles, A., and Powell, M. (eds.), *The state of the art in numerical analysis*. Clarendon Press (1987) pp165–211.

decimal— Decimal fixed-point and floating-point arithmetic

Source code: `Lib/decimal.py`

The `decimal` module provides support for fast correctly rounded decimal floating-point arithmetic. It offers several advantages over the `float` datatype:

- Decimal “is based on a floating-point model which was designed with people in mind, and necessarily has a paramount guiding principle – computers must provide an arithmetic that works in the same way as the arithmetic that people learn at school.” – excerpt from the decimal arithmetic specification.
- Decimal numbers can be represented exactly. In contrast, numbers like 1.1 and 2.2 do not have exact representations in binary floating point. End users typically would not expect 1.1+2.2 to display as 3.3000000000000003 as it does with binary floating point.
- The exactness carries over into arithmetic. In decimal floating point, 0.1+0.1+0.1-0.3 is exactly equal to zero. In binary floating point, the result is 5.5511151231257827e-017. While near to zero, the differences prevent reliable equality testing and differences can accumulate. For this reason, decimal is preferred in accounting applications which have strict equality invariants.
- The decimal module incorporates a notion of significant places so that 1.30+1.20 is 2.50. The trailing zero is kept to indicate significance. This is the customary presentation for monetary applications. For multiplication, the “schoolbook” approach uses all the figures in the multiplicands. For instance, 1.3*1.2 gives 1.56 while 1.30*1.20 gives 1.5600.
- Unlike hardware based binary floating point, the decimal module has a user alterable precision (defaulting to 28 places) which can be as large as needed for a given problem:

```
>>>from decimal import *>>>getcontext().prec=6>>>Decimal(1)/Decimal(7)Decimal('0.142857')>>>getcontext().prec=28>>>Decimal(1)/Decimal(7)Decimal('0.142857142857142857142857142857142857142857142857142857142857142857')
```
- Both binary and decimal floating point are implemented in terms of published standards. While the built-in float type exposes only a modest portion of its capabilities, the decimal module exposes all required parts of the standard. When needed, the programmer has full control over rounding and signal handling. This includes an option to enforce exact arithmetic by using exceptions to block any in
- The decimal module was designed to support “without prejudice, both exact unrounded decimal arithmetic (sometimes called fixed-point arithmetic) and rounded floating-point arithmetic.” – excerpt from the decimal arithmetic specification.

The module design is centered around three concepts: the decimal number, the context for arithmetic, and signals.

A decimal number is immutable. It has a sign, coefficient digits, and an exponent. To preserve significance, the coefficient digits do not truncate trailing zeros. Decimals also include special values such as `Infinity`, `-Infinity`, and `NaN`. The standard also differentiates 0 from +0.

The context for arithmetic is an environment specifying precision, rounding rules, limits on exponents, flags indicating the results of operations, and trap enablers which determine whether signals are treated as exceptions. Rounding options include `ROUND_CEILING`, `ROUND_DOWN`, `ROUND_FLOOR`, `ROUND_HALF_DOWN`, `ROUND_HALF_EVEN`, `ROUND_HALF_UP`, `ROUND_UP`, and `ROUND_05UP`.

Signals are groups of exceptional conditions arising during the course of computation. Depending on the needs of the application, signals may be ignored, considered as informational, or treated as exceptions. The signals in the decimal module are: `Clamped`, `InvalidOperation`, `DivisionByZero`, `Inexact`, `Rounded`, `Subnormal`, `Overflow`, `Underflow` and `FloatOperation`.

For each signal there is a flag and a trap enabler. When a signal is encountered, its flag is set to one, then, if the trap enabler is set to one, an exception is raised. Flags are sticky, so the user needs to reset them before monitoring a calculation.

See also

- IBM's General Decimal Arithmetic Specification, The General Decimal Arithmetic Specification.

Quick-start tutorial

The usual start to using decimals is importing the module, viewing the current context with `getcontext()` and, if necessary, setting new values for precision, rounding, or enabled traps:

```
>>> from decimal import *
>>> getcontext()
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999,
        capitals=1, clamp=0, flags=[], traps=[Overflow, DivisionByZero,
        InvalidOperation])

>>> getcontext().prec = 7          # Set a new precision
```

Decimal instances can be constructed from integers, strings, floats, or tuples. Construction from an integer or a float performs an exact conversion of the value of that integer or float. Decimal numbers include special values such as `NaN` which stands for “Not a number”, positive and negative `Infinity`, and `-0`:

```
>>> getcontext().prec = 28
>>> Decimal(10)
Decimal('10')
>>> Decimal('3.14')
Decimal('3.14')
>>> Decimal(3.14)
Decimal('3.1400000000000000124344978758017532527446746826171875')
>>> Decimal((0, (3, 1, 4), -2))
Decimal('3.14')
>>> Decimal(str(2.0 ** 0.5))
Decimal('1.4142135623730951')
>>> Decimal(2) ** Decimal('0.5')
Decimal('1.414213562373095048801688724')
>>> Decimal('NaN')
Decimal('NaN')
>>> Decimal('-Infinity')
Decimal('-Infinity')
```

If the `FloatOperations` signal is trapped, accidental mixing of decimals and floats in constructors or ordering comparisons raises an exception:

```
>>> c = getcontext()
>>> c.traps[FloatOperation] = True
>>> Decimal(3.14)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
decimal.FloatOperation: [
>>> Decimal('3.5') < 3.7
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
decimal.FloatOperation: [
>>> Decimal('3.5') == 3.5
True
```

Added in version 3.3.

The significance of a new Decimal is determined solely by the number of digits input. Context precision and rounding only come into play during arithmetic operations.

If the internal limits of the C version are exceeded, constructing a decimal raises `InvalidOperation`:

Changed in version 3.3.

```
>>> data = list(map(Decimal, '1.34 1.87 3.45 2.35 1.00 0.03 9.25'.split()))
>>> max(data)
Decimal('9.25')
>>> min(data)
Decimal('0.03')
>>> sorted(data)
[Decimal('0.03'), Decimal('1.00'), Decimal('1.34'), Decimal('1.87'),
 Decimal('2.35'), Decimal('3.45'), Decimal('9.25')]
>>> sum(data)
Decimal('19.29')
>>> a,b,c = data[:3]
>>> str(a)
'1.34'
>>> float(a)
1.34
>>> round(a, 1)
Decimal('1.3')
>>> int(a)
1
>>> a * 5
Decimal('6.70')
>>> a * b
Decimal('2.5058')
>>> c % a
Decimal('0.77')
```

```
>>> format(Decimal('2.675'), "f")
'2.675'
>>> format(Decimal('2.675'), ".2f")
'2.68'
>>> f"{Decimal('2.675'):.2f}"
'2.68'
>>> format(Decimal('2.675'), ".2e")
'2.68e+0'
>>> with localcontext() as ctx:
...     ctx.rounding = ROUND_DOWN
...     print(format(Decimal('2.675'), ".2f"))
...
2.67
```

And some mathematical functions are also available to Decimal:

```
>>> getcontext().prec = 28
>>> Decimal(2).sqrt()
Decimal('1.414213562373095048801688724')
>>> Decimal(1).exp()
Decimal('2.718281828459045235360287471')
>>> Decimal('10').ln()
Decimal('2.302585092994045684017991455')
>>> Decimal('10').log10()
Decimal('1')
```

The `quantize()` method rounds a number to a fixed exponent. This method is useful for monetary applications that often round results to a fixed number of places:

```
>>> Decimal('7.325').quantize(Decimal('.01'), rounding=ROUND_DOWN)
Decimal('7.32')
>>> Decimal('7.325').quantize(Decimal('1.'), rounding=ROUND_UP)
Decimal('8')
```

As shown above, the `getcontext()` function accesses the current context and allows the settings to be changed. This approach meets the needs of most applications.

For more advanced work, it may be useful to create alternate contexts using the `Context()` constructor. To make an alternate active, use the `setcontext()` function.

In accordance with the standard, the `decimal` module provides two ready to use standard contexts, `BasicContext` and `ExtendedContext`. The former is especially useful for debugging because many of the traps are enabled:

```
>>> myothercontext = Context(prec=60, rounding=ROUND_HALF_DOWN)
>>> setcontext(myothercontext)
>>> Decimal(1) / Decimal(7)
Decimal('0.142857142857142857142857142857142857142857142857142857')

>>> ExtendedContext
Context(prec=9, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999,
       capitals=1, clamp=0, flags=[], traps=[])
>>> setcontext(ExtendedContext)
>>> Decimal(1) / Decimal(7)
Decimal('0.142857143')
>>> Decimal(42) / Decimal(0)
Decimal('Infinity')

>>> setcontext(BasicContext)
>>> Decimal(42) / Decimal(0)
Traceback (most recent call last):
  File "<pyshell#143>", line 1, in <module>
    Decimal(42) / Decimal(0)
DivisionByZero: x / 0
```

Contexts also have signal flags for monitoring exceptional conditions encountered during computations. The flags remain set until explicitly cleared, so it is best to clear the flags before each set of monitored computations by using the `clear_flags()` method.

```
>>> setcontext(ExtendedContext)
>>> getcontext().clear_flags()
>>> Decimal(355) / Decimal(113)
Decimal('3.14159292')
>>> getcontext()
Context(prec=9, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999,
       capitals=1, clamp=0, flags=[Inexact, Rounded], traps=[])
```

The `flags` entry shows that the rational approximation to pi was rounded (digits beyond the context precision were thrown away) and that the result is inexact (some of the discarded digits were non-zero).

Individual traps are set using the dictionary in the `traps` attribute of a context:

```
>>> setcontext(ExtendedContext)
>>> Decimal(1) / Decimal(0)
Decimal('Infinity')
>>> getcontext().traps[DivisionByZero] = 1
>>> Decimal(1) / Decimal(0)
Traceback (most recent call last):
  File "<pyshell#112>", line 1, in -toplevel-
    Decimal(1) / Decimal(0)
DivisionByZero: x / 0
```

Most programs adjust the current context only once, at the beginning of the program. And, in many applications, data is converted to `Decimal` with a single cast inside a loop. With context set and decimals created, the bulk of the program manipulates the data no differently than with other Python numeric types.

Decimal objects

`class decimal.Decimal(value='0', context=None)`

`adjusted()`

`as_integer_ratio()`

`as_tuple()`

`canonical()`

`compare(other, context=None)`

`compare_signal(other, context=None)`

`compare_total(other, context=None)`

`compare_total_mag(other, context=None)`

`conjugate()`

`copy_abs()`

`copy_negate()`

`copy_sign(other, context=None)`

`exp(context=None)`

`classmethod from_float(f, /)`

`classmethod from_number(number, /)`

`fma(other, third, context=None)`

`is_canonical()`

`is_finite()`

`is_infinite()`

`is_nan()`

`is_normal(context=None)`

`is_qnan()`

`is_signed()`

`is_snan()`

is_subnormal(context=None)
is_zero()
ln(context=None)
log10(context=None)
logb(context=None)
logical_and(other,context=None)
logical_invert(context=None)
logical_or(other,context=None)
logical_xor(other,context=None)
max(other,context=None)
max_mag(other,context=None)
min(other,context=None)
min_mag(other,context=None)
next_minus(context=None)
next_plus(context=None)
next_toward(other,context=None)
normalize(context=None)
number_class(context=None)
quantize(exp,rounding=None,context=None)
radix()
remainder_near(other,context=None)
rotate(other,context=None)
same_quantum(other,context=None)
scaleb(other,context=None)
shift(other,context=None)
sqrt(context=None)
to_eng_string(context=None)
to_integral(rounding=None,context=None)
to_integral_exact(rounding=None,context=None)
to_integral_value(rounding=None,context=None)
round(number)
round(number,ndigits)

Construct a new `Decimal` object based from `value`. `value` can be an integer, string, tuple, float, or another `Decimal` object. If no `value` is given, returns `Decimal('0')`. If `value` is a string, it should conform to the decimal numeric string syntax after leading and trailing whitespace characters, as well as underscores throughout, are removed: `sign::='+'|'-'` `digit::='0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9'` `indicator::='e'|'E'` `digits::=digit[digit]...decimal-part::=digits['[digits]']` `['.']digitsexponent-part::=indicat`

Return the adjusted exponent after shifting out the coefficient's rightmost digits until only the lead digit remains: `Decimal('321e+5').adjusted()` returns seven. Used for determining the position of the most significant digit with respect to the decimal point.

Return a pair (n, d) of integers that represent the given `Decimal` instance as a fraction, in lowest terms and with a positive denominator: `>>>Decimal('-3.14').as_integer_ratio()(-157, 50)` The conversion is exact. Raise `OverflowError` on infinities and `ValueError` on NaNs.

Return a named tuple representation of the number: `DecimalTuple(sign, digits, exponent)`.

Return the canonical encoding of the argument. Currently, the encoding of a `Decimal` instance is always canonical, so this operation returns its argument unchanged.

Compare the values of two `Decimal` instances. `compare()` returns a `Decimal` instance, and if either operand is a NaN then the result is a

NaN: `a or b is a NaN ==> Decimal('NaN') a < b ==> Decimal('-1') a == b ==> Decimal('0') a > b ==> Decimal('1')`

This operation is identical to the `compare()` method, except that all NaNs signal. That is, if neither operand is a signaling NaN then any quiet NaN operand is treated as though it were a signaling NaN.

Compare two operands using their abstract representation rather than their numerical value. Similar to the `compare()` method, but the result gives a total ordering on `Decimal` instances. Two `Decimal` instances with the same numeric value but different representations compare unequal in this ordering: `>>>Decimal('12.0').compare_total(Decimal('12'))Decimal('-1')` Quiet and signaling NaNs are also included in the total ordering. The result of this function is `Decimal('0')` if both operands have the same repres

Compare two operands using their abstract representation rather than their value as in `compare_total()`, but ignoring the sign of each operand. `x.compare_total_mag(y)` is equivalent to `x.copy_abs().compare_total(y.copy_abs())`. This operation is unaffected by context and is quiet: no flags are changed and no rounding is performed. As an exception, the C version may raise `InvalidOperation` if the second operand cannot be converted exactly.

Just returns self, this method is only to comply with the Decimal Specification.

Return the absolute value of the argument. This operation is unaffected by the context and is quiet: no flags are changed and no rounding is performed.

Return the negation of the argument. This operation is unaffected by the context and is quiet: no flags are changed and no rounding is performed.

Return a copy of the first operand with the sign set to be the same as the sign of the second operand. For example: `>>>Decimal('2.3').copy_sign(Decimal('-1.5'))Decimal('-2.3')` This operation is unaffected by context and is quiet: no flags are changed and no rounding is performed. As an exception, the C version may raise `InvalidOperation` if the second operand cannot be converted exactly.

Return the value of the (natural) exponential function e^x at the given number. The result is correctly rounded using the `ROUND_HALF_EVEN` rounding mode. `>>>Decimal(1).exp()Decimal('2.718281828459045235360287471')` `>>>Decimal(321).exp()Decimal('2.561702493119680037517373933E+139')`

Alternative constructor that only accepts instances of `float` or `int`. Note `Decimal.from_float(0.1)` is not the same as `Decimal('0.1')`. Since 0.1 is not exactly representable in binary floating point, the value is stored as the nearest representable value which is $0x1.999999999999ap-4$. That equivalent value in decimal is `0.1000000000000000055511151231257827021181583404541015625`. Note From Python 3.2 onwards, a `Decimal` instance can also be constructed directly from a `float`. `>>>Decimal.from_float(0.1)Decimal('0.10`

Alternative constructor that only accepts instances of `float`, `int` or `Decimal`, but not strings or tuples. `>>>Decimal.from_number(314)Decimal('314')` `>>>Decimal.from_number(0.1)Decimal('0.1000000000000000055511151231257827021181583404541015625')` `>>>Decimal.from_number(Decimal('3.14'))Decimal('3.14')` Added in version 3.14.

Fused multiply-add. Return $\text{self} * \text{other} + \text{third}$ with no rounding of the intermediate product $\text{self} * \text{other}$. `>>>Decimal(2).fma(3,5)Decimal('11')`

Return `True` if the argument is canonical and `False` otherwise. Currently, a `Decimal` instance is always canonical, so this operation always returns `True`.

Return `True` if the argument is a finite number, and `False` if the argument is an infinity or a NaN.

Return `True` if the argument is either positive or negative infinity and `False` otherwise.

Return `True` if the argument is a (quiet or signaling) NaN and `False` otherwise.

Return `True` if the argument is a normal finite number. Return `False` if the argument is zero, subnormal, infinite or a NaN.

Return `True` if the argument is a quiet NaN, and `False` otherwise.

Return `True` if the argument has a negative sign and `False` otherwise. Note that zeros and NaNs can both carry signs.

Return `True` if the argument is a signaling NaN and `False` otherwise.

Return `True` if the argument is subnormal, and `False` otherwise.

Return `True` if the argument is a (positive or negative) zero and `False` otherwise.

Return the natural (base e) logarithm of the operand. The result is correctly rounded using the `ROUND_HALF_EVEN` rounding mode.

Return the base ten logarithm of the operand. The result is correctly rounded using the `ROUND_HALF_EVEN` rounding mode.

For a nonzero number, return the adjusted exponent of its operand as a `Decimal` instance. If the operand is a zero then `Decimal('-Infinity')` is returned and the `DivisionByZero` flag is raised. If the operand is an infinity then `Decimal('Infinity')` is returned.

`logical_and()` is a logical operation which takes two logical operands (see `Logical operands`). The result is the digit-wise `and` of the two operands.

`logical_invert()` is a logical operation. The result is the digit-wise inversion of the operand.

`logical_or()` is a logical operation which takes two logical operands (see `Logical operands`). The result is the digit-wise `or` of the two operands.

`logical_xor()` is a logical operation which takes two logical operands (see `Logical operands`). The result is the digit-wise exclusive `or` of the two operands.

Like `max(self, other)` except that the context rounding rule is applied before returning and that NaN values are either signaled or ignored (depending on the context and whether they are signaling or quiet).

Similar to the `max()` method, but the comparison is done using the absolute values of the operands.

Like `min(self, other)` except that the context rounding rule is applied before returning and that NaN values are either signaled or ignored (depending on the context and whether they are signaling or quiet).

Similar to the `min()` method, but the comparison is done using the absolute values of the operands.

Return the largest number representable in the given context (or in the current thread's context if no context is given) that is smaller than the given operand.

Return the smallest number representable in the given context (or in the current thread's context if no context is given) that is larger than the given operand.

If the two operands are unequal, return the number closest to the first operand in the direction of the second operand. If both operands are numerically equal, return a copy of the first operand with the sign set to be the same as the sign of the second operand.

Used for producing canonical values of an equivalence class within either the current context or the specified context. This has the same semantics as the unary plus operation, except that if the final result is finite it is reduced to its simplest form, with all trailing zeros removed and its sign preserved. That is, while the coefficient is non-zero and a multiple of ten the coefficient is divided by ten and the exponent is incremented by 1. Otherwise (the coefficient is zero) the exponent is s

Return a string describing the class of the operand. The returned value is one of the following ten strings: "-Infinity", indicating that the operand is negative infinity. "-Normal", indicating that the operand is a negative normal number. "-Subnormal", indicating that the operand is negative and subnormal. "-Zero", indicating that the operand is a negative zero. "+Zero", indicating that the operand is a positive zero. "+Subnormal", indicating that the operand is positive and subnormal. "+Normal", indic

Return a value equal to the first operand after rounding and having the exponent of the second operand. >>>Decimal('1.41421356').quantize(Decimal('1.000'))Decimal('1.414') Unlike other operations, if the length of the coefficient after the quantize operation would be greater than precision, then an InvalidOperation is signaled. This guarantees that, unless there is an error condition, the quantized exponent is always equal to that of the right-hand operand. Also unlike other operations, quantize neve

ReturnDecimal(10), the radix (base) in which the Decimal class does all its arithmetic. Included for compatibility with the specification.

Return the remainder from dividing self by other. This differs from self % other in that the sign of the remainder is chosen so as to minimize its absolute value. More precisely, the return value is self - n * other where n is the integer nearest to the exact value of self / other, and if two integers are equally near then the even one is chosen. If the result is zero then its sign will be the sign of self. >>>Decimal(18).remainder_near(Decimal(10))Decimal('-2') >>>Decimal(25).remainder_near(Decimal(10))Decimal('5')

Return the result of rotating the digits of the first operand by an amount specified by the second operand. The second operand must be an integer in the range -precision through precision. The absolute value of the second operand gives the number of places to rotate. If the second operand is positive then rotation is to the left; otherwise rotation is to the right. The coefficient of the first operand is padded on the left with zeros to length precision if necessary. The sign and exponent of

Test whether self and other have the same exponent or whether both are NaN. This operation is unaffected by context and is quiet: no flags are changed and no rounding is performed. As an exception, the C version may raise InvalidOperation if the second operand cannot be converted exactly.

Return the first operand with exponent adjusted by the second. Equivalently, return the first operand multiplied by 10**other. The second operand must be an integer.

Return the result of shifting the digits of the first operand by an amount specified by the second operand. The second operand must be an integer in the range -precision through precision. The absolute value of the second operand gives the number of places to shift. If the second operand is positive then the shift is to the left; otherwise the shift is to the right. Digits shifted into the coefficient are zeros. The sign and exponent of the first operand are unchanged.

Return the square root of the argument to full precision.

Convert to a string, using engineering notation if an exponent is needed. Engineering notation has an exponent which is a multiple of 3. This can leave up to 3 digits to the left of the decimal place and may require the addition of either one or two trailing zeros. For example, this converts Decimal('123E+1') to Decimal('1.23E+3').

Identical to the to_integral_value() method. The to_integralname has been kept for compatibility with older versions.

Round to the nearest integer, signaling `Inexact` or `Rounded` as appropriate if rounding occurs. The rounding mode is determined by the `rounding` parameter if given, else by the `givencontext`. If neither parameter is given then the rounding mode of the current context is used.

Round to the nearest integer without signaling `Inexact` or `Rounded`. If given, applies `rounding`; otherwise, uses the rounding method in either the supplied `context` or the current context.

If `ndigits` is not given or `None`, returns the nearest `int` to `number`, rounding ties to even, and ignoring the rounding mode of the `Decimal` context. Raises `OverflowError` if `number` is an infinity or `ValueError` if it is a (quiet or signaling) NaN. If `ndigits` is an `int`, the context's rounding mode is respected and a `Decimal` representing `number` rounded to the nearest multiple of `Decimal('1E-ndigits')` is returned; in this case, `round(number, ndigits)` is equivalent to `self.quantize(Decimal('1E-ndigits'))`. Returns `Decimal('NaN')` if `n`

Logical operands

The `logical_and()`, `logical_invert()`, `logical_or()`, and `logical_xor()` methods expect their arguments to be logical operands. A logical operand is a `Decimal` instance whose exponent and sign are both zero, and whose digits are all either 0 or 1.

Context objects

Contexts are environments for arithmetic operations. They govern precision, set rules for rounding, determine which signals are treated as exceptions, and limit the range for exponents.

Each thread has its own current context which is accessed or changed using the `getcontext()` and `setcontext()` functions:

`decimal.getcontext()`

Return the current context for the active thread.

`decimal.setcontext(c, /)`

Set the current context for the active thread to `c`.

You can also use the `with` statement and the `localcontext()` function to temporarily change the active context.

`decimal.localcontext(ctx=None, **kwargs)`

Return a context manager that will set the current context for the active thread to a copy of `ctx` on entry to the `with`-statement and restore the previous context when exiting the `with`-statement. If no context is specified, a copy of the current context is used. The `kwargs` argument is used to set the attributes of the new context. For example, the following code sets the current decimal precision to 42 places, performs a calculation, and then automatically restores the previous context:

```
from decimal import
```

`decimal.IEEEContext(bits)`

Return a context object initialized to the proper values for one of the IEEE interchange formats. The argument must be a multiple of 32 and less than `IEEE_CONTEXT_MAX_BITS`. Added in version 3.14.

New contexts can also be created using the `Context` constructor described below. In addition, the module provides three pre-made contexts:

`decimal.BasicContext`

This is a standard context defined by the General Decimal Arithmetic Specification. Precision is set to nine. Rounding is set to `ROUND_HALF_UP`. All flags are cleared. All traps are enabled (treated as exceptions) except `Inexact`, `Rounded`, and `Subnormal`. Because many of the traps are enabled, this context is useful for debugging.

decimal.ExtendedContext

This is a standard context defined by the General Decimal Arithmetic Specification. Precision is set to nine. Rounding is set to ROUND_HALF_EVEN. All flags are cleared. No traps are enabled (so that exceptions are not raised during computations). Because the traps are disabled, this context is useful for applications that prefer to have result value of NaN or Infinity instead of raising exceptions. This allows an application to complete a run in the presence of conditions that would otherwise hal

decimal.DefaultContext

This context is used by the Context constructor as a prototype for new contexts. Changing a field (such a precision) has the effect of changing the default for new contexts created by the Context constructor. This context is most useful in multi-threaded environments. Changing one of the fields before threads are started has the effect of setting system-wide defaults. Changing the fields after threads have started is not recommended as it would require thread synchronization to prevent race condit

In addition to the three supplied contexts, new contexts can be created with the Context constructor.

class decimal.Context(prec=None, rounding=None, Emin=None, Emax=None, capitals=None, clamp=None, flags=None, traps=None)

prec

rounding

traps

flags

Emin

Emax

capitals

clamp

clear_flags()

clear_traps()

copy()

copy_decimal(num, /)

create_decimal(num='0', /)

create_decimal_from_float(f, /)

Etiny()

Etop()

abs(x, /)

add(x, y, /)

canonical(x, /)

compare(x, y, /)

compare_signal(x, y, /)

compare_total(x, y, /)

compare_total_mag(x, y, /)

copy_abs(x,l)
copy_negate(x,l)
copy_sign(x,y,l)
divide(x,y,l)
divide_int(x,y,l)
divmod(x,y,l)
exp(x,l)
fma(x,y,z,l)
is_canonical(x,l)
is_finite(x,l)
is_infinite(x,l)
is_nan(x,l)
is_normal(x,l)
is_qnan(x,l)
is_signed(x,l)
is_snan(x,l)
is_subnormal(x,l)
is_zero(x,l)
ln(x,l)
log10(x,l)
logb(x,l)
logical_and(x,y,l)
logical_invert(x,l)
logical_or(x,y,l)
logical_xor(x,y,l)
max(x,y,l)
max_mag(x,y,l)
min(x,y,l)
min_mag(x,y,l)
minus(x,l)
multiply(x,y,l)
next_minus(x,l)
next_plus(x,l)
next_toward(x,y,l)
normalize(x,l)
number_class(x,l)

plus(x,l)
power(x,y,modulo=None)
quantize(x,y,l)
radix()
remainder(x,y,l)
remainder_near(x,y,l)
rotate(x,y,l)
same_quantum(x,y,l)
scaleb(x,y,l)
shift(x,y,l)
sqrt(x,l)
subtract(x,y,l)
to_eng_string(x,l)
to_integral_exact(x,l)
to_sci_string(x,l)

Creates a new context. If a field is not specified or isNone, the default values are copied from theDefaultContext. If theflagsfield is not specified or isNone, all flags are cleared.precAn integer in the range [1,MAX_PREC] that sets the precision for arithmetic operations in the context.roundingOne of the constants listed in the sectionRounding Modes.trapsflagsLists of any signals to be set. Generally, new contexts should only set traps and leave the flags clear.EminEmaxIntegers specifying th

An integer in the range [1,MAX_PREC] that sets the precision for arithmetic operations in the context.

One of the constants listed in the sectionRounding Modes.

Lists of any signals to be set. Generally, new contexts should only set traps and leave the flags clear.

Integers specifying the outer limits allowable for exponents.Eminmust be in the range [MIN_EMIN,0],Emaxin the range [0,MAX_EMAX].

Either0or1(the default). If set to1, exponents are printed with a capitalE; otherwise, a lowercaseeis used:Decimal('6.02e+23').

Either0(the default) or1. If set to1, the exponenteof aDecimalinstance representable in this context is strictly limited to the rangeEmin-prec+1<=e<=Emax-prec+1. Ifclampis0then a weaker condition holds: the adjusted exponent of theDecimalinstance is at mostEmax. Whenclampis1, a large normal number will, where possible, have its exponent reduced and a corresponding number of zeros added to its coefficient, in order to fit the exponent constraints; this preserves the value of the number but lose

Resets all of the flags to0.

Resets all of the traps to0.Added in version 3.3.

Return a duplicate of the context.

Return a copy of the Decimal instance num.

Creates a new Decimal instance fromnumbut usingselfas context. Unlike theDecimalconstructor, the context precision, rounding method, flags, and traps are applied to the conversion.This is useful because constants are often given to a greater precision than is needed by the application. Another benefit is that rounding immediately eliminates unintended effects from digits beyond the current precision. In the following example, using

unrounded inputs means that adding zero to a sum can change the

Creates a new Decimal instance from a float but rounding using self as the context. Unlike the `Decimal.from_float()` class method, the context precision, rounding method, flags, and traps are applied to the conversion.

```
>>> context = Context(prec=5, rounding=ROUND_DOWN)
>>> context.create_decimal_from_float(math.pi)
Decimal('3.1415')
>>> context = Context(prec=5, traps=[Inexact])
>>> context.create_decimal_from_float(math.pi)
Traceback (most recent call last): ...decimal.Inexact: NoneAdded in version 3.1.
```

Returns a value equal to $E_{\min} - \text{prec} + 1$ which is the minimum exponent value for subnormal results. When underflow occurs, the exponent is set to E_{tiny} .

Returns a value equal to $E_{\max} - \text{prec} + 1$.

Returns the absolute value of x .

Return the sum of x and y .

Returns the same Decimal object x .

Compares x and y numerically.

Compares the values of the two operands numerically.

Compares two operands using their abstract representation.

Compares two operands using their abstract representation, ignoring sign.

Returns a copy of x with the sign set to 0.

Returns a copy of x with the sign inverted.

Copies the sign from y to x .

Return x divided by y .

Return x divided by y , truncated to an integer.

Divides two numbers and returns the integer part of the result.

Return $e^{**}x$.

Returns x multiplied by y , plus z .

Returns `True` if x is canonical; otherwise returns `False`.

Returns `True` if x is finite; otherwise returns `False`.

Returns `True` if x is infinite; otherwise returns `False`.

Returns `True` if x is a qNaN or sNaN; otherwise returns `False`.

Returns `True` if x is a normal number; otherwise returns `False`.

Returns `True` if x is a quiet NaN; otherwise returns `False`.

Returns `True` if x is negative; otherwise returns `False`.

Returns `True` if x is a signaling NaN; otherwise returns `False`.

Returns `True` if x is subnormal; otherwise returns `False`.

Returns `True` if x is a zero; otherwise returns `False`.

Returns the natural (base e) logarithm of x .

Returns the base 10 logarithm of x .

Returns the exponent of the magnitude of the operand's MSD.

Applies the logical operation `and` between each operand's digits.

Invert all the digits in `x`.

Applies the logical operation `or` between each operand's digits.

Applies the logical operation `xor` between each operand's digits.

Compares two values numerically and returns the maximum.

Compares the values numerically with their sign ignored.

Compares two values numerically and returns the minimum.

Compares the values numerically with their sign ignored.

Minus corresponds to the unary prefix minus operator in Python.

Return the product of `x` and `y`.

Returns the largest representable number smaller than `x`.

Returns the smallest representable number larger than `x`.

Returns the number closest to `x`, in direction towards `y`.

Reduces `x` to its simplest form.

Returns an indication of the class of `x`.

Plus corresponds to the unary prefix plus operator in Python. This operation applies the context precision and rounding, so it is not an identity operation.

Return `x` to the power of `y`, reduced modulo `modulo` if given. With two arguments, compute `x**y`. If `x` is negative then `y` must be integral. The result will be inexact unless `y` is integral and the result is finite and can be expressed exactly in 'precision' digits. The rounding mode of the context is used. Results are always correctly rounded in the Python version. `Decimal(0)**Decimal(0)` results in `InvalidOperation`, and if `InvalidOperation` is not trapped, then results in `Decimal('NaN')`. Changed in version 3.3: The C mode

Returns a value equal to `x` (rounded), having the exponent of `y`.

Just returns 10, as this is `Decimal`, :))

Returns the remainder from integer division. The sign of the result, if non-zero, is the same as that of the original dividend.

Returns `sx-y*n`, where `n` is the integer nearest the exact value of `ofx/y` (if the result is 0 then its sign will be the sign of `ofx`).

Returns a rotated copy of `ofx`, `y` times.

Returns `True` if the two operands have the same exponent.

Returns the first operand after adding the second value its exp.

Returns a shifted copy of `ofx`, `y` times.

Square root of a non-negative number to context precision.

Return the difference between `x` and `y`.

Convert to a string, using engineering notation if an exponent is needed. Engineering notation has an exponent which is a multiple of 3. This can leave up to 3 digits to the left of the decimal place and may require the addition of either one or two trailing zeros.

Rounds to an integer.

Converts a number to a string using scientific notation.

Constants

The constants in this section are only relevant for the C module. They are also included in the pure Python version for compatibility.

```
32-bit | 64-bit | decimal. | MAX_PREC | 425000000 | 9999999999999999 | decimal. | MAX_EMAX |  
425000000 | 9999999999999999 | decimal. | MIN_EMIN | -425000000 | -9999999999999999 |  
decimal. | MIN_ETINY | -849999999 | -19999999999999997 | decimal. | IEEE_CONTEXT_MAX_BITS |  
256 | 512
```

decimal.HAVE_THREADS

The value is `True`. Deprecated, because Python now always has threads. Deprecated since version 3.9.

decimal.HAVE_CONTEXTVAR

The default value is `True`. If Python is configured using the `--without-decimal-contextvar` option, the C version uses a thread-local rather than a coroutine-local context and the value is `False`. This is slightly faster in some nested context scenarios. Added in version 3.8.3.

Rounding modes

decimal.ROUND_CEILING

Round towards infinity.

decimal.ROUND_DOWN

Round towards zero.

decimal.ROUND_FLOOR

Round towards -infinity.

decimal.ROUND_HALF_DOWN

Round to nearest with ties going towards zero.

decimal.ROUND_HALF_EVEN

Round to nearest with ties going to nearest even integer.

decimal.ROUND_HALF_UP

Round to nearest with ties going away from zero.

decimal.ROUND_UP

Round away from zero.

decimal.ROUND_05UP

Round away from zero if last digit after rounding towards zero would have been 0 or 5; otherwise round towards zero.

Signals

Signals represent conditions that arise during computation. Each corresponds to one context flag and one context trap enabler.

The context flag is set whenever the condition is encountered. After the computation, flags may be checked for informational purposes (for instance, to determine whether a computation was exact). After checking the flags, be sure to clear all flags before starting the next computation.

If the context's trap enabler is set for the signal, then the condition causes a Python exception to be raised. For example, if the `DivisionByZero` trap is set, then a `DivisionByZero` exception is raised upon encountering the condition.

class decimal.Clamped

Altered an exponent to fit representation constraints. Typically, clamping occurs when an exponent falls outside the context's `Emin` and `Emax` limits. If possible, the exponent is reduced to fit by adding zeros to the coefficient.

class decimal.DecimalException

Base class for other signals and a subclass of `ArithmeticError`.

class decimal.DivisionByZero

Signals the division of a non-infinite number by zero. Can occur with division, modulo division, or when raising a number to a negative power. If this signal is not trapped, returns `Infinity` or `-Infinity` with the sign determined by the inputs to the calculation.

class decimal.Inexact

Indicates that rounding occurred and the result is not exact. Signals when non-zero digits were discarded during rounding. The rounded result is returned. The signal flag or trap is used to detect when results are inexact.

class decimal.InvalidOperation

An invalid operation was performed. Indicates that an operation was requested that does not make sense. If not trapped, returns `NaN`. Possible causes

include: `Infinity - Infinity`, `0 * Infinity`, `Infinity / Infinity`, `x % 0`, `Infinity % x`, `sqrt(-x)` and `x > 0`, `0 ** x` (non-integer), `x ** Infinity`

class decimal.Overflow

Numerical overflow. Indicates the exponent is larger than `Context.Emax` after rounding has occurred. If not trapped, the result depends on the rounding mode, either pulling inward to the largest representable finite number or rounding outward to `Infinity`. In either case, `Inexact` and `Rounded` are also signaled.

class decimal.Rounded

Rounding occurred though possibly no information was lost. Signaled whenever rounding discards digits; even if those digits are zero (such as rounding `5.00` to `5.0`). If not trapped, returns the result unchanged. This signal is used to detect loss of significant digits.

class decimal.Subnormal

Exponent was lower than `Emin` prior to rounding. Occurs when an operation result is subnormal (the exponent is too small). If not trapped, returns the result unchanged.

class decimal.Underflow

Numerical underflow with result rounded to zero. Occurs when a subnormal result is pushed to zero by rounding. `Inexact` and `Subnormal` are also signaled.

class decimal.FloatOperation

Enable stricter semantics for mixing floats and Decimals. If the signal is not trapped (default), mixing floats and Decimals is permitted in the `Decimal` constructor, `create_decimal()` and all comparison operators. Both conversion and comparisons are exact. Any occurrence of a mixed operation is silently recorded by setting `FloatOperation` in

the context flags. Explicit conversions with `from_float()` or `create_decimal_from_float()` do not set the flag. Otherwise (the signal is trapped), only equality comparisons

The following table summarizes the hierarchy of signals:

```
exceptions.ArithmeticError(exceptions.Exception)
    DecimalException
        Clamped
        DivisionByZero(DecimalException, exceptions.ZeroDivisionError)
        Inexact
            Overflow(Inexact, Rounded)
            Underflow(Inexact, Rounded, Subnormal)
        InvalidOperation
        Rounded
        Subnormal
        FloatOperation(DecimalException, exceptions.TypeError)
```

Floating-point notes

Mitigating round-off error with increased precision

The use of decimal floating point eliminates decimal representation error (making it possible to represent 0.1 exactly); however, some operations can still incur round-off error when non-zero digits exceed the fixed precision.

The effects of round-off error can be amplified by the addition or subtraction of nearly offsetting quantities resulting in loss of significance. Knuth provides two instructive examples where rounded floating-point arithmetic with insufficient precision causes the breakdown of the associative and distributive properties of addition:

```
# Examples from Seminumerical Algorithms, Section 4.2.2.
>>> from decimal import Decimal, getcontext
>>> getcontext().prec = 8

>>> u, v, w = Decimal(11111113), Decimal(-11111111), Decimal('7.51111111')
>>> (u + v) + w
Decimal('9.5111111')
>>> u + (v + w)
Decimal('10')

>>> u, v, w = Decimal(20000), Decimal(-6), Decimal('6.0000003')
>>> (u*v) + (u*w)
Decimal('0.01')
>>> u * (v+w)
Decimal('0.0060000')
```

The `decimal` module makes it possible to restore the identities by expanding the precision sufficiently to avoid loss of significance:

```
>>> getcontext().prec = 20
>>> u, v, w = Decimal(11111113), Decimal(-11111111), Decimal('7.51111111')
>>> (u + v) + w
Decimal('9.51111111')
>>> u + (v + w)
Decimal('9.51111111')
>>>
>>> u, v, w = Decimal(20000), Decimal(-6), Decimal('6.0000003')
>>> (u*v) + (u*w)
Decimal('0.0060000')
>>> u * (v+w)
Decimal('0.0060000')
```

Special values

The number system for the `decimal` module provides special values including `NaN`, `sNaN`, `-Infinity`, `Infinity`, and two zeros, `+0` and `-0`.

Infinities can be constructed directly with `Decimal('Infinity')`. Also, they can arise from dividing by zero when the `DivisionByZero` signal is not trapped. Likewise, when the `Overflow` signal is not trapped, infinity can result from rounding beyond the limits of the largest representable number.

The infinities are signed (affine) and can be used in arithmetic operations where they get treated as very large, indeterminate numbers. For instance, adding a constant to infinity gives another infinite result.

Some operations are indeterminate and return `NaN`, or if the `InvalidOperation` signal is trapped, raise an exception. For example, `0/0` returns `NaN` which means “not a number”. This variety of `NaN` is quiet and, once created, will flow through other computations always resulting in another `NaN`. This behavior can be useful for a series of computations that occasionally have missing inputs — it allows the calculation to proceed while flagging specific results as invalid.

A variant is `sNaN` which signals rather than remaining quiet after every operation. This is a useful return value when an invalid result needs to interrupt a calculation for special handling.

The behavior of Python’s comparison operators can be a little surprising where a `NaN` is involved. A test for equality where one of the operands is a quiet or signaling `NaN` always returns `False` (even when doing `Decimal('NaN')==Decimal('NaN')`), while a test for inequality always returns `True`. An attempt to compare two `Decimals` using any of the `<`, `<=`, `>` or `>=` operators will raise the `InvalidOperation` signal if either operand is a `NaN`, and return `False` if this signal is not trapped. Note that the General Decimal Arithmetic specification does not specify the behavior of direct comparisons; these rules for comparisons involving a `NaN` were taken from the IEEE 854 standard (see Table 3 in section 5.7). To ensure strict standards-compliance, use the `compare()` and `compare_signal()` methods instead.

The signed zeros can result from calculations that underflow. They keep the sign that would have resulted if the calculation had been carried out to greater precision. Since their magnitude is zero, both positive and negative zeros are treated as equal and their sign is informational.

In addition to the two signed zeros which are distinct yet equal, there are various representations of zero with differing precisions yet equivalent in value. This takes a bit of getting used to. For an eye accustomed to normalized floating-point representations, it is not immediately obvious that the following calculation returns a value equal to zero:

```
>>> 1 / Decimal('Infinity')
Decimal('0E-1000026')
```

Working with threads

The `getcontext()` function accesses a different `Context` object for each thread. Having separate thread contexts means that threads may make changes (such as `getcontext().prec=10`) without interfering with other threads.

Likewise, the `setcontext()` function automatically assigns its target to the current thread.

If `setcontext()` has not been called before `getcontext()`, then `getcontext()` will automatically create a new context for use in the current thread. New context objects have default values set from the `decimal.DefaultContext` object.

The `sys.flags.thread_inherit_context` flag affects the context for new threads. If the flag is false, new threads will start with an empty context. In this case, `getcontext()` will create a new context object when called and use the default values from `DefaultContext`. If the flag is true, new threads will start with a copy of context from the caller of `threading.Thread.start()`.

To control the defaults so that each thread will use the same values throughout the application, directly modify the `DefaultContext` object. This should be done before any threads are started so that there won’t be a race

condition between threads calling `getcontext()`. For example:

```
# Set applicationwide defaults for all threads about to be launched
DefaultContext.prec = 12
DefaultContext.rounding = ROUND_DOWN
DefaultContext.traps = ExtendedContext.traps.copy()
DefaultContext.traps[InvalidOperation] = 1
setcontext(DefaultContext)

# Afterwards, the threads can be started
t1.start()
t2.start()
t3.start()
. . .
```

Recipes

Here are a few recipes that serve as utility functions and that demonstrate ways to work with the `Decimal` class:

```
def moneyfmt(value, places=2, curr='', sep=',', dp='.',
             pos='', neg='-', trailneg=''):
    """Convert Decimal to a money formatted string.

    places:  required number of places after the decimal point
    curr:    optional currency symbol before the sign (may be blank)
    sep:     optional grouping separator (comma, period, space, or blank)
    dp:      decimal point indicator (comma or period)
             only specify as blank when places is zero
    pos:     optional sign for positive numbers: '+', space or blank
    neg:     optional sign for negative numbers: '-', '(', space or blank
    trailneg: optional trailing minus indicator: ' ', ')', space or blank

    >>> d = Decimal('-1234567.8901')
    >>> moneyfmt(d, curr='$')
    '-$1,234,567.89'
    >>> moneyfmt(d, places=0, sep='.', dp='', neg='', trailneg='-')
    '1.234.568-'
    >>> moneyfmt(d, curr='$', neg='(', trailneg=')')
    '($1,234,567.89)'
    >>> moneyfmt(Decimal(123456789), sep=' ')
    '123 456 789.00'
    >>> moneyfmt(Decimal('-0.02'), neg='<', trailneg='>')
    '<0.02>'

    """
    q = Decimal(10) ** -places        # 2 places --> '0.01'
    sign, digits, exp = value.quantize(q).as_tuple()
    result = []
    digits = list(map(str, digits))
    build, next = result.append, digits.pop
    if sign:
        build(trailneg)
    for i in range(places):
        build(next() if digits else '0')
    if places:
        build(dp)
    if not digits:
        build('0')
    i = 0
    while digits:
        build(next())
        i += 1
        if i == 3 and digits:
```

```

        i = 0
        build(sep)
    build(curr)
    build(neg if sign else pos)
    return ''.join(reversed(result))

def pi():
    """Compute Pi to the current precision.

    >>> print(pi())
    3.141592653589793238462643383

    """
    getcontext().prec += 2 # extra digits for intermediate steps
    three = Decimal(3)     # substitute "three=3.0" for regular floats
    lasts, t, s, n, na, d, da = 0, three, 3, 1, 0, 0, 24
    while s != lasts:
        lasts = s
        n, na = n+na, na+8
        d, da = d+da, da+32
        t = (t * n) / d
        s += t
    getcontext().prec -= 2
    return +s               # unary plus applies the new precision

def exp(x):
    """Return e raised to the power of x.  Result type matches input type.

    >>> print(exp(Decimal(1)))
    2.718281828459045235360287471
    >>> print(exp(Decimal(2)))
    7.389056098930650227230427461
    >>> print(exp(2.0))
    7.38905609893
    >>> print(exp(2+0j))
    (7.38905609893+0j)

```

Decimal FAQ

Q: It is cumbersome to type `decimal.Decimal('1234.5')`. Is there a way to minimize typing when using the interactive interpreter?

A: Some users abbreviate the constructor to just a single letter:

```

>>> D = decimal.Decimal
>>> D('1.23') + D('3.45')
Decimal('4.68')

```

Q: In a fixed-point application with two decimal places, some inputs have many places and need to be rounded. Others are not supposed to have excess digits and need to be validated. What methods should be used?

A: The `quantize()` method rounds to a fixed number of decimal places. If the `Inexact` trap is set, it is also useful for validation:

```

>>> TWOPLACES = Decimal(10) ** -2           # same as Decimal('0.01')

>>> # Round to two places
>>> Decimal('3.214').quantize(TWOPLACES)
Decimal('3.21')

>>> # Validate that a number does not exceed two places
>>> Decimal('3.21').quantize(TWOPLACES, context=Context(traps=[Inexact]))
Decimal('3.21')

```

```
>>> Decimal('3.214').quantize(TWOPLACES, context=Context(traps=[Inexact]))
Traceback (most recent call last):
...
Inexact: None
```

Q: Once I have valid two place inputs, how do I maintain that invariant throughout an application?

A: Some operations like addition, subtraction, and multiplication by an integer will automatically preserve fixed point. Others operations, like division and non-integer multiplication, will change the number of decimal places and need to be followed-up with `quantize()` step:

```
>>> a = Decimal('102.72')          # Initial fixed-point values
>>> b = Decimal('3.17')
>>> a + b                          # Addition preserves fixed-point
Decimal('105.89')
>>> a - b
Decimal('99.55')
>>> a * 42                         # So does integer multiplication
Decimal('4314.24')
>>> (a * b).quantize(TWOPLACES)    # Must quantize non-integer multiplication
Decimal('325.62')
>>> (b / a).quantize(TWOPLACES)    # And quantize division
Decimal('0.03')
```

In developing fixed-point applications, it is convenient to define functions to handle the `quantize()` step:

```
>>> def mul(x, y, fp=TWOPLACES):
...     return (x * y).quantize(fp)
...
>>> def div(x, y, fp=TWOPLACES):
...     return (x / y).quantize(fp)

>>> mul(a, b)                     # Automatically preserve fixed-point
Decimal('325.62')
>>> div(b, a)
Decimal('0.03')
```

Q: There are many ways to express the same value. The numbers 200, 200.000, 2E2, and .02E+4 all have the same value at various precisions. Is there a way to transform them to a single recognizable canonical value?

A: The `normalize()` method maps all equivalent values to a single representative:

```
>>> values = map(Decimal, '200 200.000 2E2 .02E+4'.split())
>>> [v.normalize() for v in values]
[Decimal('2E+2'), Decimal('2E+2'), Decimal('2E+2'), Decimal('2E+2')]
```

Q: When does rounding occur in a computation?

A: It occurs after the computation. The philosophy of the decimal specification is that numbers are considered exact and are created independent of the current context. They can even have greater precision than current context. Computations process with those exact inputs and then rounding (or other context operations) is applied to the result of the computation:

```
>>> getcontext().prec = 5
>>> pi = Decimal('3.1415926535')   # More than 5 digits
>>> pi                             # All digits are retained
Decimal('3.1415926535')
>>> pi + 0                         # Rounded after an addition
Decimal('3.1416')
>>> pi - Decimal('0.00005')        # Subtract unrounded numbers, then round
Decimal('3.1415')
>>> pi + 0 - Decimal('0.00005').    # Intermediate values are rounded
Decimal('3.1416')
```

Q: Some decimal values always print with exponential notation. Is there a way to get a non-exponential representation?

A: For some values, exponential notation is the only way to express the number of significant places in the coefficient. For example, expressing `5.0E+3` as `5000` keeps the value constant but cannot show the original's two-place significance.

If an application does not care about tracking significance, it is easy to remove the exponent and trailing zeroes, losing significance, but keeping the value unchanged:

```
>>> def remove_exponent(d):
...     return d.quantize(Decimal(1)) if d == d.to_integral() else d.normalize()

>>> remove_exponent(Decimal('5E+3'))
Decimal('5000')
```

Q: Is there a way to convert a regular float to a Decimal?

A: Yes, any binary floating-point number can be exactly expressed as a Decimal though an exact conversion may take more precision than intuition would suggest:

```
>>> Decimal(math.pi)
Decimal('3.141592653589793115997963468544185161590576171875')
```

Q: Within a complex calculation, how can I make sure that I haven't gotten a spurious result because of insufficient precision or rounding anomalies.

A: The decimal module makes it easy to test results. A best practice is to re-run calculations using greater precision and with various rounding modes. Widely differing results indicate insufficient precision, rounding mode issues, ill-conditioned inputs, or a numerically unstable algorithm.

Q: I noticed that context precision is applied to the results of operations but not to the inputs. Is there anything to watch out for when mixing values of different precisions?

A: Yes. The principle is that all values are considered to be exact and so is the arithmetic on those values. Only the results are rounded. The advantage for inputs is that "what you type is what you get". A disadvantage is that the results can look odd if you forget that the inputs haven't been rounded:

```
>>> getcontext().prec = 3
>>> Decimal('3.104') + Decimal('2.104')
Decimal('5.21')
>>> Decimal('3.104') + Decimal('0.000') + Decimal('2.104')
Decimal('5.20')
```

The solution is either to increase precision or to force rounding of inputs using the unary plus operation:

```
>>> getcontext().prec = 3
>>> +Decimal('1.23456789')      # unary plus triggers rounding
Decimal('1.23')
```

Alternatively, inputs can be rounded upon creation using the `Context.create_decimal()` method:

```
>>> Context(prec=5, rounding=ROUND_DOWN).create_decimal('1.2345678')
Decimal('1.2345')
```

Q: Is the CPython implementation fast for large numbers?

A: Yes. In the CPython and PyPy3 implementations, the C/CFFI versions of the decimal module integrate the high speed `libmpdec` library for arbitrary precision correctly rounded decimal floating-point arithmetic[1]. `libmpdec` uses Karatsuba multiplication for medium-sized numbers and the Number Theoretic Transform for very large numbers.

The context must be adapted for exact arbitrary precision arithmetic. `Emin` and `Emax` should always be set to the maximum values, `clamp` should always be 0 (the default). Setting `prec` requires some care.

The easiest approach for trying out bignum arithmetic is to use the maximum value for `prec` as well[2]:

```
>>> setcontext(Context(prec=MAX_PREC, Emax=MAX_EMAX, Emin=MIN_EMIN))
>>> x = Decimal(2) ** 256
>>> x / 128
Decimal('904625697166532776746648320380374280103671755200316906558262375061821325312')
```

For inexact results, `MAX_PREC` is far too large on 64-bit platforms and the available memory will be insufficient:

```
>>> Decimal(1) / 3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

On systems with overallocation (e.g. Linux), a more sophisticated approach is to adjust `prec` to the amount of available RAM. Suppose that you have 8GB of RAM and expect 10 simultaneous operands using a maximum of 500MB each:

```
>>> import sys
>>>
>>> # Maximum number of digits for a single operand using 500MB in 8-byte words
>>> # with 19 digits per word (4-byte and 9 digits for the 32-bit build):
>>> maxdigits = 19 * ((500 * 1024**2) // 8)
>>>
>>> # Check that this works:
>>> c = Context(prec=maxdigits, Emax=MAX_EMAX, Emin=MIN_EMIN)
>>> c.traps[Inexact] = True
>>> setcontext(c)
>>>
>>> # Fill the available precision with nines:
>>> x = Decimal(0).logical_invert() * 9
>>> sys.getsizeof(x)
524288112
>>> x + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
decimal.Inexact: [<class 'decimal.Inexact'>]
```

In general (and especially on systems without overallocation), it is recommended to estimate even tighter bounds and set `theInexacttrap` if all calculations are expected to be exact.

Added in version 3.3.

Changed in version 3.9: This approach now works for all exact results except for non-integer powers.

fractions— Rational numbers

Source code: `Lib/fractions.py`

The `fractions` module provides support for rational number arithmetic.

A `Fraction` instance can be constructed from a pair of rational numbers, from a single number, or from a string.

`class fractions.Fraction(numerator=0, denominator=1)`

`class fractions.Fraction(number)`

`class fractions.Fraction(string)`

`numerator`

`denominator`

`as_integer_ratio()`

`is_integer()`

`classmethod from_float(f)`

`classmethod from_decimal(dec)`

`classmethod from_number(number)`

`limit_denominator(max_denominator=1000000)`

`__floor__()`

`__ceil__()`

`__round__()`

`__round__(ndigits)`

`__format__(format_spec, /)`

The first version requires that `numerator` and `denominator` are instances of `numbers.Rational` and returns a new `Fraction` instance with a value equal to `numerator/denominator`. If `denominator` is zero, it raises a `ZeroDivisionError`. The second version requires that `number` is an instance of `numbers.Rational` or has the `as_integer_ratio()` method (this includes `float` and `decimal.Decimal`). It returns a `Fraction` instance with exactly the same value. Assumed, that the `as_integer_ratio()` method returns a pair of coprime integers and

Numerator of the Fraction in lowest term.

Denominator of the Fraction in lowest terms. Guaranteed to be positive.

Return a tuple of two integers, whose ratio is equal to the original Fraction. The ratio is in lowest terms and has a positive denominator. Added in version 3.8.

Return `True` if the Fraction is an integer. Added in version 3.12.

Alternative constructor which only accepts instances of `float` or `numbers.Integral`. Beware that `Fraction.from_float(0.3)` is not the same value as `Fraction(3,10)`. Note From Python 3.2 onwards, you can also construct a `Fraction` instance directly from a `float`.

Alternative constructor which only accepts instances of `decimal.Decimal` or `numbers.Integral`. Note From Python 3.2 onwards, you can also construct a `Fraction` instance directly from a `decimal.Decimal` instance.

Alternative constructor which only accepts instances of `numbers.Integral`, `numbers.Rational`, `float` or `decimal.Decimal`, and objects with the `as_integer_ratio()` method, but not strings. Added in version 3.14.

Finds and returns the closest `Fraction` to `self` that has denominator at most `max_denominator`. This method is useful for finding rational approximations to a given floating-point number: `>>>from fractions import Fraction>>>Fraction('3.1415926535897932').limit_denominator(1000)Fraction(355, 113)` or for recovering a rational number that's represented as a float: `>>>from math import pi, cos>>>Fraction(cos(pi/3))Fraction(4503599627370497, 9007199254740992)>>>Fraction(cos(pi/3)).limit_denominator()Fraction(1, 2)>>>`

Returns the greatest `int` `<= self`. This method can also be accessed through `math.floor()` function: `>>>from math import floor>>>floor(Fraction(355, 113))3`

Returns the least `int` `>= self`. This method can also be accessed through `math.ceil()` function.

The first version returns the nearest `int` to `self`, rounding half to even. The second version rounds `self` to the nearest multiple of `Fraction(1, 10**ndigits)` (logically, if `ndigits` is negative), again rounding half toward even. This method can also be accessed through the `round()` function.

Provides support for formatting of `Fraction` instances via the `str.format()` method, the `format()` built-in function, or `Formatted string literals`. If the `format_spec` specification string does not end with one of the presentation types `'e', 'E', 'f', 'F', 'g', 'G'` or `'%'` then formatting follows the general rules for fill, alignment, sign handling, minimum width, and grouping as described in the `format` specification mini-language. The “alternate form” flag `'#'` is supported: if present, it forces the output string

See also

Module numbers

The abstract base classes making up the numeric tower.

random— Generate pseudo-random numbers

Source code: `Lib/random.py`

This module implements pseudo-random number generators for various distributions.

For integers, there is uniform selection from a range. For sequences, there is uniform selection of a random element, a function to generate a random permutation of a list in-place, and a function for random sampling without replacement.

On the real line, there are functions to compute uniform, normal (Gaussian), lognormal, negative exponential, gamma, and beta distributions. For generating distributions of angles, the von Mises distribution is available.

Almost all module functions depend on the basic function `random()`, which generates a random float uniformly in the half-open range $0.0 \leq X < 1.0$. Python uses the Mersenne Twister as the core generator. It produces 53-bit precision floats and has a period of $2^{19937}-1$. The underlying implementation in C is both fast and threadsafe. The Mersenne Twister is one of the most extensively tested random number generators in existence. However, being completely deterministic, it is not suitable for all purposes, and is completely unsuitable for cryptographic purposes.

The functions supplied by this module are actually bound methods of a hidden instance of `therandom.Randomclass`. You can instantiate your own instances of `Random` to get generators that don't share state.

`ClassRandom` can also be subclassed if you want to use a different basic generator of your own devising: see the documentation on that class for more details.

The `random` module also provides the `SystemRandom` class which uses the system function `os.urandom()` to generate random numbers from sources provided by the operating system.

■■ Advertencia: Warning The pseudo-random generators of this module should not be used for security purposes. For security or cryptographic uses, see the `secrets` module.

See also

M. Matsumoto and T. Nishimura, “Mersenne Twister: A 623-dimensionally equidistributed uniform pseudorandom number generator”, *ACM Transactions on Modeling and Computer Simulation* Vol. 8, No. 1, January pp.3–30 1998.

Complementary-Multiply-with-Carry recipe for a compatible alternative random number generator with a long period and comparatively simple update operations.

■ Nota: Note The global random number generator and instances of `Random` are thread-safe. However, in the free-threaded build, concurrent calls to the global generator or to the same instance of `Random` may encounter contention and poor performance. Consider using separate instances of `Random` per thread instead.

Bookkeeping functions

`random.seed(a=None, version=2)`

Initialize the random number generator. If `a` is omitted or `None`, the current system time is used. If randomness sources are provided by the operating system, they are used instead of the system time (see the `os.urandom()` function for details on availability). If `a` is an int, its absolute value is used directly. With version 2 (the default), `a` str, bytes, or bytearray object gets converted to an int and all of its bits are used. With version 1 (provided for reproducing random sequences from older versions of Pyt

`random.getstate()`

Return an object capturing the current internal state of the generator. This object can be passed to `setstate()` to restore the state.

`random.setstate(state)`

`state` should have been obtained from a previous call to `getstate()`, and `setstate()` restores the internal state of the generator to what it was at the time `getstate()` was called.

Functions for bytes

`random.randbytes(n)`

Generate random bytes. This method should not be used for generating security tokens. Use `secrets.token_bytes()` instead. Added in version 3.9.

Functions for integers

`random.randrange(stop)`

`random.randrange(start, stop[, step])`

Return a randomly selected element from `range(start, stop, step)`. This is roughly equivalent to `choice(range(start, stop, step))` but supports arbitrarily large ranges and is optimized for common cases. The positional argument pattern matches the `range()` function. Keyword arguments should not be used because they can be interpreted in unexpected ways. For example `randrange(start=100)` is interpreted as `randrange(0, 100, 1)`. Changed in version 3.2: `randrange()` is more sophisticated about producing equally distributed

`random.randint(a, b)`

Return a random integer N such that $a \leq N \leq b$. Alias for `randrange(a, b+1)`.

`random.getrandbits(k)`

Returns a non-negative Python integer with `k` random bits. This method is supplied with the Mersenne Twister generator and some other generators may also provide it as an optional part of the API. When available, `getrandbits()` enables `randrange()` to handle arbitrarily large ranges. Changed in version 3.9: This method now accepts zero for `k`.

Functions for sequences

`random.choice(seq)`

Return a random element from the non-empty sequence `seq`. If `seq` is empty, raises `IndexError`.

`random.choices(population, weights=None, *, cum_weights=None, k=1)`

Return a `k`-sized list of elements chosen from the `population` with replacement. If the `population` is empty, raises `IndexError`. If a `weights` sequence is specified, selections are made according to the relative weights. Alternatively, if a `cum_weights` sequence is given, the selections are made according to the cumulative weights (perhaps computed using `itertools.accumulate()`). For example, the relative weights `[10, 5, 30, 5]` are equivalent to the cumulative weights `[10, 15, 45, 50]`. Internally, the relative weights are

`random.shuffle(x)`

Shuffle the sequence `x` in place. To shuffle an immutable sequence and return a new shuffled list, use `sample(x, k=len(x))` instead. Note that even for small `len(x)`, the total number of permutations of `x` can quickly

grow larger than the period of most random number generators. This implies that most permutations of a long sequence can never be generated. For example, a sequence of length 2080 is the largest that can fit within the period of the Mersenne Twister random number generator. Changed in version 3.

random.sample(population,k,*,counts=None)

Return a k-length list of unique elements chosen from the population sequence. Used for random sampling without replacement. Returns a new list containing elements from the population while leaving the original population unchanged. The resulting list is in selection order so that all sub-slices will also be valid random samples. This allows raffle winners (the sample) to be partitioned into grand prize and second place winners (the subslices). Members of the population need not be hashable or unique.

Discrete distributions

The following function generates a discrete distribution.

random.binomialvariate(n=1,p=0.5)

Binomial distribution. Return the number of successes for n independent trials with the probability of success in each trial being p. Mathematically equivalent to: $\sum(\text{random()} < p \text{ for } i \text{ in range}(n))$. The number of trials n should be a non-negative integer. The probability of success p should be between $0.0 \leq p \leq 1.0$. The result is an integer in the range $0 \leq X \leq n$. Added in version 3.12.

Real-valued distributions

The following functions generate specific real-valued distributions. Function parameters are named after the corresponding variables in the distribution's equation, as used in common mathematical practice; most of these equations can be found in any statistics text.

random.random()

Return the next random floating-point number in the range $0.0 \leq X < 1.0$.

random.uniform(a,b)

Return a random floating-point number N such that $a \leq N \leq b$ for $a \leq b$ and $b \leq N \leq a$ for $b < a$. The end-point value b may or may not be included in the range depending on floating-point rounding in the expression $a + (b - a) * \text{random}()$.

random.triangular(low,high,mode)

Return a random floating-point number N such that $\text{low} \leq N \leq \text{high}$ and with the specified mode between those bounds. The low and high bounds default to zero and one. The mode argument defaults to the midpoint between the bounds, giving a symmetric distribution.

random.betavariate(alpha,beta)

Beta distribution. Conditions on the parameters are $\alpha > 0$ and $\beta > 0$. Returned values range between 0 and 1.

random.expovariate(lambd=1.0)

Exponential distribution. lambd is 1.0 divided by the desired mean. It should be non-zero. (The parameter would be called "lambda", but that is a reserved word in Python.) Returned values range from 0 to positive infinity if lambd is positive, and from negative infinity to 0 if lambd is negative. Changed in version 3.12: Added the default value for lambd.

random.gammavariate(alpha,beta)

Gamma distribution. (Not the gamma function!) The shape and scale parameters, α and β , must have positive values. (Calling conventions vary and some sources define 'beta' as the inverse of the scale). The probability distribution function

is: $x^{(\alpha-1)} \cdot \text{math.exp}(-x/\beta) / \text{pdf}(x) = \text{math.gamma}(\alpha) \cdot \beta^{\alpha}$

random.gauss(mu=0.0,sigma=1.0)

Normal distribution, also called the Gaussian distribution. μ is the mean, and σ is the standard deviation. This is slightly faster than the `normalvariate()` function defined below. Multithreading note: When two threads call this function simultaneously, it is possible that they will receive the same return value. This can be avoided in three ways. 1) Have each thread use a different instance of the random number generator. 2) Put locks around all calls. 3) Use the slower, but thread-safe `normalv`

random.lognormvariate(mu,sigma)

Log normal distribution. If you take the natural logarithm of this distribution, you'll get a normal distribution with mean μ and standard deviation σ . μ can have any value, and σ must be greater than zero.

random.normalvariate(mu=0.0,sigma=1.0)

Normal distribution. μ is the mean, and σ is the standard deviation. Changed in version 3.11: μ and σ now have default arguments.

random.vonmisesvariate(mu,kappa)

μ is the mean angle, expressed in radians between 0 and 2π , and κ is the concentration parameter, which must be greater than or equal to zero. If κ is equal to zero, this distribution reduces to a uniform random angle over the range 0 to 2π .

random.paretovariate(alpha)

Pareto distribution. α is the shape parameter.

random.weibullvariate(alpha,beta)

Weibull distribution. α is the scale parameter and β is the shape parameter.

Alternative Generator

class random.Random([seed])

seed(a=None,version=2)

getstate()

setstate(state)

random()

getrandbits(k)

randbytes(n)

Class that implements the default pseudo-random number generator used by the `random` module. Changed in version 3.11: Formerly `seed` could be any hashable object. Now it is limited to: `None`, `int`, `float`, `str`, `bytes`, or `bytearray`. Subclasses of `Random` should override the following methods if they wish to make use of a different basic generator: `seed(a=None,version=2)` Override this method in subclasses to customise the `seed()` behaviour of `Random` instances. `getstate()` Override this method in subclasses to customise the

Override this method in subclasses to customise the `seed()` behaviour of `Random` instances.

Override this method in subclasses to customise the `getstate()` behaviour of `Random` instances.

Override this method in subclasses to customise the `setstate()` behaviour of `Random` instances.

Override this method in subclasses to customise the `random()` behaviour of `Random` instances.

Override this method in subclasses to customise the `getrandbits()` behaviour of `Random` instances.

Override this method in subclasses to customise the `randbytes()` behaviour of `Random` instances.

class random.SystemRandom([seed])

Class that uses the `os.urandom()` function for generating random numbers from sources provided by the operating system. Not available on all systems. Does not rely on software state, and sequences are not reproducible. Accordingly, the `seed()` method has no effect and is ignored. The `getstate()` and `setstate()` methods raise `NotImplementedError` if called.

Notes on Reproducibility

Sometimes it is useful to be able to reproduce the sequences given by a pseudo-random number generator. By reusing a seed value, the same sequence should be reproducible from run to run as long as multiple threads are not running.

Most of the random module's algorithms and seeding functions are subject to change across Python versions, but two aspects are guaranteed not to change:

- If a new seeding method is added, then a backward compatible seeder will be offered.
- The generator's `random()` method will continue to produce the same sequence when the compatible seeder is given the same seed.

Examples

Basic examples:

```
>>> random()                                # Random float:  0.0 <= x < 1.0
0.37444887175646646

>>> uniform(2.5, 10.0)                      # Random float:  2.5 <= x <= 10.0
3.1800146073117523

>>> expovariate(1 / 5)                      # Interval between arrivals averaging 5 seconds
5.148957571865031

>>> randrange(10)                           # Integer from 0 to 9 inclusive
7

>>> randrange(0, 101, 2)                    # Even integer from 0 to 100 inclusive
26

>>> choice(['win', 'lose', 'draw'])          # Single random element from a sequence
'draw'

>>> deck = 'ace two three four'.split()
>>> shuffle(deck)                            # Shuffle a list
>>> deck
['four', 'two', 'ace', 'three']

>>> sample([10, 20, 30, 40, 50], k=4)        # Four samples without replacement
[40, 10, 50, 30]
```

Simulations:

```
>>> # Six roulette wheel spins (weighted sampling with replacement)
>>> choices(['red', 'black', 'green'], [18, 18, 2], k=6)
['red', 'green', 'black', 'black', 'red', 'black']

>>> # Deal 20 cards without replacement from a deck
>>> # of 52 playing cards, and determine the proportion of cards
>>> # with a ten-value: ten, jack, queen, or king.
>>> deal = sample(['tens', 'low cards'], counts=[16, 36], k=20)
>>> deal.count('tens') / 20
0.15

>>> # Estimate the probability of getting 5 or more heads from 7 spins
>>> # of a biased coin that settles on heads 60% of the time.
>>> sum(binomialvariate(n=7, p=0.6) >= 5 for i in range(10_000)) / 10_000
0.4169

>>> # Probability of the median of 5 samples being in middle two quartiles
>>> def trial():
...     return 2_500 <= sorted(choices(range(10_000), k=5))[2] < 7_500
...
>>> sum(trial() for i in range(10_000)) / 10_000
0.7958
```

Example of statistical bootstrapping using resampling with replacement to estimate a confidence interval for the mean of a sample:

```
# https://www.thoughtco.com/example-of-bootstrapping-3126155
from statistics import fmean as mean
from random import choices

data = [41, 50, 29, 37, 81, 30, 73, 63, 20, 35, 68, 22, 60, 31, 95]
means = sorted(mean(choices(data, k=len(data))) for i in range(100))
print(f'The sample mean of {mean(data):.1f} has a 90% confidence '
      f'interval from {means[5]:.1f} to {means[94]:.1f}')
```

Example of a resampling permutation test to determine the statistical significance or p-value of an observed difference between the effects of a drug versus a placebo:

```
# Example from "Statistics is Easy" by Dennis Shasha and Manda Wilson
from statistics import fmean as mean
from random import shuffle

drug = [54, 73, 53, 70, 73, 68, 52, 65, 65]
placebo = [54, 51, 58, 44, 55, 52, 42, 47, 58, 46]
observed_diff = mean(drug) - mean(placebo)

n = 10_000
count = 0
combined = drug + placebo
for i in range(n):
    shuffle(combined)
    new_diff = mean(combined[:len(drug)]) - mean(combined[len(drug):])
    count += (new_diff >= observed_diff)

print(f'{n} label reshufflings produced only {count} instances with a difference')
print(f'at least as extreme as the observed difference of {observed_diff:.1f}.')
print(f'The one-sided p-value of {count / n:.4f} leads us to reject the null')
print(f'hypothesis that there is no difference between the drug and the placebo.')
```

Simulation of arrival times and service deliveries for a multiserver queue:

```
from heapq import heapify, heapreplace
from random import expovariate, gauss
from statistics import mean, quantiles

average_arrival_interval = 5.6
```



```

average_service_time = 15.0
stdev_service_time = 3.5
num_servers = 3

waits = []
arrival_time = 0.0
servers = [0.0] * num_servers # time when each server becomes available
heapify(servers)
for i in range(1_000_000):
    arrival_time += expovariate(1.0 / average_arrival_interval)
    next_server_available = servers[0]
    wait = max(0.0, next_server_available - arrival_time)
    waits.append(wait)
    service_duration = max(0.0, gauss(average_service_time, stdev_service_time))
    service_completed = arrival_time + wait + service_duration
    heapreplace(servers, service_completed)

print(f'Mean wait: {mean(waits):.1f}    Max wait: {max(waits):.1f}')
print('Quartiles:', [round(q, 1) for q in quantiles(waits)])

```

See also

Statistics for Hackers a video tutorial by Jake VanderPlas on statistical analysis using just a few fundamental concepts including simulation, sampling, shuffling, and cross-validation.

Economics Simulation a simulation of a marketplace by Peter Norvig that shows effective use of many of the tools and distributions provided by this module (gauss, uniform, sample, betavariate, choice, triangular, and randrange).

A Concrete Introduction to Probability (using Python) a tutorial by Peter Norvig covering the basics of probability theory, how to write simulations, and how to perform data analysis using Python.

Recipes

These recipes show how to efficiently make random selections from the combinatoric iterators in the `itertools` module:

```

import random

def random_product(*iterables, repeat=1):
    "Random selection from itertools.product(*iterables, repeat=repeat)"
    pools = tuple(map(tuple, iterables)) * repeat
    return tuple(map(random.choice, pools))

def random_permutation(iterable, r=None):
    "Random selection from itertools.permutations(iterable, r)"
    pool = tuple(iterable)
    r = len(pool) if r is None else r
    return tuple(random.sample(pool, r))

def random_combination(iterable, r):
    "Random selection from itertools.combinations(iterable, r)"
    pool = tuple(iterable)
    n = len(pool)
    indices = sorted(random.sample(range(n), r))
    return tuple(pool[i] for i in indices)

def random_combination_with_replacement(iterable, r):
    "Choose r elements with replacement. Order the result to match the iterable."
    # Result will be in set(itertools.combinations_with_replacement(iterable, r)).
    pool = tuple(iterable)
    n = len(pool)

```

```

indices = sorted(random.choices(range(n), k=r))
return tuple(pool[i] for i in indices)

def random_derangement(iterable):
    "Choose a permutation where no element stays in its original position."
    seq = tuple(iterable)
    if len(seq) < 2:
        if not seq:
            return ()
        raise IndexError('No derangements to choose from')
    perm = list(range(len(seq)))
    start = tuple(perm)
    while True:
        random.shuffle(perm)
        if all(p != q for p, q in zip(start, perm)):
            return tuple([seq[i] for i in perm])

```

The default `random()` returns multiples of 2^{-53} in the range $0.0 \leq x < 1.0$. All such numbers are evenly spaced and are exactly representable as Python floats. However, many other representable floats in that interval are not possible selections. For example, 0.05954861408025609 isn't an integer multiple of 2^{-53} .

The following recipe takes a different approach. All floats in the interval are possible selections. The mantissa comes from a uniform distribution of integers in the range $2^{52} \leq \text{mantissa} < 2^{53}$. The exponent comes from a geometric distribution where exponents smaller than -53 occur half as often as the next larger exponent.

```

from random import Random
from math import ldexp

class FullRandom(Random):

    def random(self):
        mantissa = 0x10_0000_0000_0000 | self.getrandbits(52)
        exponent = -53
        x = 0
        while not x:
            x = self.getrandbits(32)
            exponent += x.bit_length() - 32
        return ldexp(mantissa, exponent)

```

All real valued distributions in the class will use the new method:

```

>>> fr = FullRandom()
>>> fr.random()
0.05954861408025609
>>> fr.expovariate(0.25)
8.87925541791544

```

The recipe is conceptually equivalent to an algorithm that chooses from all the multiples of 2^{-53} in the range $0.0 \leq x < 1.0$. All such numbers are evenly spaced, but most have to be rounded down to the nearest representable Python float. (The value 2^{-53} is the smallest positive unnormalized float and is equal to `math.ulp(0.0)`.)

See also

Generating Pseudo-random Floating-Point Values a paper by Allen B. Downey describing ways to generate more fine-grained floats than normally generated by `random()`.

Command-line usage

Added in version 3.13.

The `random` module can be executed from the command line.

```
python -m random [-h] [-c CHOICE [CHOICE ...] | -i N | -f N] [input ...]
```

The following options are accepted:

-h,--help

Show the help message and exit.

-cCHOICE[CHOICE...]

--choiceCHOICE[CHOICE...]

Print a random choice, using `choice()`.

-i<N>

--integer<N>

Print a random integer between 1 and N inclusive, using `randint()`.

-f<N>

--float<N>

Print a random floating-point number between 0 and N inclusive, using `uniform()`.

If no options are given, the output depends on the input:

- String or multiple: same as `--choice`.
- Integer: same as `--integer`.
- Float: same as `--float`.

Command-line example

Here are some examples of the `random` command-line interface:

```
$ # Choose one at random
$ python -m random egg bacon sausage spam "Lobster Thermidor aux crevettes with a Mornay sauce"
Lobster Thermidor aux crevettes with a Mornay sauce

$ # Random integer
$ python -m random 6
6

$ # Random floating-point number
$ python -m random 1.8
1.7080016272295635

$ # With explicit arguments
$ python -m random --choice egg bacon sausage spam "Lobster Thermidor aux crevettes with a Mornay sauce"
egg

$ python -m random --integer 6
3

$ python -m random --float 1.8
1.5666339105010318

$ python -m random --integer 6
5

$ python -m random --float 6
3.1942323316565915
```

statistics— Mathematical statistics functions

Added in version 3.4.

Source code: `Lib/statistics.py`

This module provides functions for calculating mathematical statistics of numeric (Real-valued) data.

The module is not intended to be a competitor to third-party libraries such as NumPy, SciPy, or proprietary full-featured statistics packages aimed at professional statisticians such as Minitab, SAS and Matlab. It is aimed at the level of graphing and scientific calculators.

Unless explicitly noted, these functions support `int`, `float`, `Decimal` and `Fraction`. Behaviour with other types (whether in the numeric tower or not) is currently unsupported. Collections with a mix of types are also undefined and implementation-dependent. If your input data consists of mixed types, you may be able to use `map()` to ensure a consistent result, for example: `map(float, input_data)`.

Some datasets use NaN (not a number) values to represent missing data. Since NaNs have unusual comparison semantics, they cause surprising or undefined behaviors in the statistics functions that sort data or that count occurrences. The functions affected

`aremedian()`, `median_low()`, `median_high()`, `median_grouped()`, `mode()`, `multimode()`, and `quantiles()`.

The NaN values should be stripped before calling these functions:

```
>>> from statistics import median
>>> from math import isnan
>>> from itertools import filterfalse

>>> data = [20.7, float('NaN'), 19.2, 18.3, float('NaN'), 14.4]
>>> sorted(data) # This has surprising behavior
[20.7, nan, 14.4, 18.3, 19.2, nan]
>>> median(data) # This result is unexpected
16.35

>>> sum(map(isnan, data)) # Number of missing values
2
>>> clean = list(filterfalse(isnan, data)) # Strip NaN values
>>> clean
[20.7, 19.2, 18.3, 14.4]
>>> sorted(clean) # Sorting now works as expected
[14.4, 18.3, 19.2, 20.7]
>>> median(clean) # This result is now well defined
18.75
```

Averages and measures of central location

These functions calculate an average or typical value from a population or sample.

`mean()` | Arithmetic mean ("average") of data. | `fmean()` | Fast, floating-point arithmetic mean, with optional weighting. | `geometric_mean()` | Geometric mean of data. | `harmonic_mean()` | Harmonic mean of data. | `kde()` | Estimate the probability density distribution of the data. | `kde_random()` | Random sampling from the PDF generated by `kde()`. | `median()` | Median (middle value) of data. | `median_low()` | Low median of data. | `median_high()` | High median of data. | `median_grouped()` | Median (50th percentile) of grouped data. | `mode()` | Single mode (most common value) of discrete or nominal data. | `multimode()` | List of modes (most common values) of discrete or nominal data. | `quantiles()` | Divide data into intervals with equal probability.

Measures of spread

These functions calculate a measure of how much the population or sample tends to deviate from the typical or average values.

`pstdev()` | Population standard deviation of data. | `pvariance()` | Population variance of data. | `stdev()` | Sample standard deviation of data. | `variance()` | Sample variance of data.

Statistics for relations between two inputs

These functions calculate statistics regarding relations between two inputs.

`covariance()` | Sample covariance for two variables. | `correlation()` | Pearson and Spearman's correlation coefficients. | `linear_regression()` | Slope and intercept for simple linear regression.

Function details

Note: The functions do not require the data given to them to be sorted. However, for reading convenience, most of the examples show sorted sequences.

statistics.mean(data)

Return the sample arithmetic mean of data which can be a sequence or iterable. The arithmetic mean is the sum of the data divided by the number of data points. It is commonly called "the average", although it is only one of many different mathematical averages. It is a measure of the central location of the data. If data is empty, `StatisticsError` will be raised. Some examples of use: `>>>mean([1,2,3,4,4])`2.8 `>>>mean([-1.0,2.5,3.25,5.75])`2.625 `>>>from fractions import Fraction as F` `>>>mean([F(3,7),F(1,21),F(5,3),`

statistics.fmean(data, weights=None)

Convert data to floats and compute the arithmetic mean. This runs faster than the `mean()` function and it always returns a float. The data may be a sequence or iterable. If the input dataset is empty, raises a `StatisticsError`. `>>>fmean([3.5,4.0,5.25])`4.25 Optional weighting is supported. For example, a professor assigns a grade for a course by weighting quizzes at 20%, homework at 20%, a midterm exam at 30%, and a final exam at 30%: `>>>grades=[85,92,83,91]` `>>>weights=[0.20,0.20,0.30,0.30]` `>>>fmean(grades,we`

statistics.geometric_mean(data)

Convert data to floats and compute the geometric mean. The geometric mean indicates the central tendency or typical value of the data using the product of the values (as opposed to the arithmetic mean which uses their sum). Raises a `StatisticsError` if the input dataset is empty, if it contains a zero, or if it contains a negative value. The data may be a sequence or iterable. No special efforts are made to achieve exact results. (However, this may change in the future.) `>>>round(geometric_mean([54,24,36]),1`

statistics.harmonic_mean(data, weights=None)

Return the harmonic mean of data, a sequence or iterable of real-valued numbers. If weights is omitted or `None`, then equal weighting is assumed. The harmonic mean is the reciprocal of the arithmetic mean of the reciprocals of the data. For example, the harmonic mean of three values `a`, `b` and `c` will be equivalent to $3/(1/a+1/b+1/c)$. If one of the values is zero, the result will be zero. The harmonic mean is a type of average, a measure of the central location of the data. It is often appropriate when avera

statistics.kde(data, h, kernel='normal', *, cumulative=False)

Kernel Density Estimation (KDE): Create a continuous probability density function or cumulative distribution function from discrete samples. The basic idea is to smooth the data using a kernel function. to help draw inferences about a population from a sample. The degree of smoothing is controlled by the scaling parameter `h` which is called the bandwidth. Smaller values emphasize local features while larger values give

smoother results. The kernel determines the relative weights of the sample data point

statistics.kde_random(data, h, kernel='normal', *, seed=None)

Return a function that makes a random selection from the estimated probability density function produced by `kde(data, h, kernel)`. Providing a seed allows reproducible selections. In the future, the values may change slightly as more accurate kernel inverse CDF estimates are implemented. The seed may be an integer, float, str, or bytes. A `StatisticsError` will be raised if the data sequence is empty. Continuing the example from `kde()`, we can use `kde_random()` to generate new random selections from an estimated pr

statistics.median(data)

Return the median (middle value) of numeric data, using the common “mean of middle two” method. If data is empty, `StatisticsError` is raised. data can be a sequence or iterable. The median is a robust measure of central location and is less affected by the presence of outliers. When the number of data points is odd, the middle data point is returned: `>>> median([1, 3, 5])` 3 When the number of data points is even, the median is interpolated by taking the average of the two middle values: `>>> median([1, 3, 5, 7])` 4

statistics.median_low(data)

Return the low median of numeric data. If data is empty, `StatisticsError` is raised. data can be a sequence or iterable. The low median is always a member of the data set. When the number of data points is odd, the middle value is returned. When it is even, the smaller of the two middle values is returned. `>>> median_low([1, 3, 5])` 3 `>>> median_low([1, 3, 5, 7])` 3 Use the low median when your data are discrete and you prefer the median to be an actual data point rather than interpolated.

statistics.median_high(data)

Return the high median of data. If data is empty, `StatisticsError` is raised. data can be a sequence or iterable. The high median is always a member of the data set. When the number of data points is odd, the middle value is returned. When it is even, the larger of the two middle values is returned. `>>> median_high([1, 3, 5])` 3 `>>> median_high([1, 3, 5, 7])` 5 Use the high median when your data are discrete and you prefer the median to be an actual data point rather than interpolated.

statistics.median_grouped(data, interval=1.0)

Estimates the median for numeric data that has been grouped or binned around the midpoints of consecutive, fixed-width intervals. The data can be any iterable of numeric data with each value being exactly the midpoint of a bin. At least one value must be present. The interval is the width of each bin. For example, demographic information may have been summarized into consecutive ten-year age groups with each group being represented by the 5-year midpoints of the intervals: `>>> from collections import Counter`

statistics.mode(data)

Return the single most common data point from discrete or nominal data. The mode (when it exists) is the most typical value and serves as a measure of central location. If there are multiple modes with the same frequency, returns the first one encountered in the data. If the smallest or largest of those is desired instead, use `min(multimode(data))` or `max(multimode(data))`. If the input data is empty, `StatisticsError` is raised. mode assumes discrete data and returns a single value. This is the standard treat

statistics.multimode(data)

Return a list of the most frequently occurring values in the order they were first encountered in the data. Will return more than one result if there are multiple modes or an empty list if the data is empty: `>>> multimode('aabbccdddeeffgg')['b', 'd', 'f']` `>>> multimode('')` [] Added in version 3.8.

statistics.pstdev(data, mu=None)

Return the population standard deviation (the square root of the population variance). See `variance()` for arguments and other details. `>>> pstdev([1.5, 2.5, 2.5, 2.75, 3.25, 4.75])` 0.986893273527251

statistics.pvariance(data,mu=None)

Return the population variance of data, a non-empty sequence or iterable of real-valued numbers. Variance, or second moment about the mean, is a measure of the variability (spread or dispersion) of data. A large variance indicates that the data is spread out; a small variance indicates it is clustered closely around the mean. If the optional second argument mu is given, it should be the population mean of the data. It can also be used to compute the second moment around a point that is not the mean.

statistics.stdev(data,xbar=None)

Return the sample standard deviation (the square root of the sample variance). See variance() for arguments and other details. >>> stdev([1.5, 2.5, 2.5, 2.75, 3.25, 4.75]) 1.0810874155219827

statistics.variance(data,xbar=None)

Return the sample variance of data, an iterable of at least two real-valued numbers. Variance, or second moment about the mean, is a measure of the variability (spread or dispersion) of data. A large variance indicates that the data is spread out; a small variance indicates it is clustered closely around the mean. If the optional second argument xbar is given, it should be the sample mean of data. If it is missing or None (the default), the mean is automatically calculated. Use this function when your

statistics.quantiles(data,*,n=4,method='exclusive')

Divide data into continuous intervals with equal probability. Returns a list of n-1 cut points separating the intervals. Set n to 4 for quartiles (the default). Set n to 10 for deciles. Set n to 100 for percentiles which gives the 99 cut points that separate data into 100 equal sized groups. Raises StatisticsError if n is not at least 1. The data can be any iterable containing sample data. For meaningful results, the number of data points in data should be larger than n. Raises StatisticsError if there is not at least

statistics.covariance(x,y,l)

Return the sample covariance of two inputs x and y. Covariance is a measure of the joint variability of two inputs. Both inputs must be of the same length (no less than two), otherwise StatisticsError is raised. Examples: >>> x=[1,2,3,4,5,6,7,8,9] >>> y=[1,2,3,1,2,3,1,2,3] >>> covariance(x,y) 0.75 >>> z=[9,8,7,6,5,4,3,2,1] >>> covariance(x,z) -7.5 >>> covariance(z,x) -7.5 Added in version 3.10.

statistics.correlation(x,y,l,*,method='linear')

Return the Pearson's correlation coefficient for two inputs. Pearson's correlation coefficient takes values between -1 and +1. It measures the strength and direction of a linear relationship. If method is "ranked", computes Spearman's rank correlation coefficient for two inputs. The data is replaced by ranks. Ties are averaged so that equal values receive the same rank. The resulting coefficient measures the strength of a monotonic relationship. Spearman's correlation coefficient is appropriate for or

statistics.linear_regression(x,y,l,*,proportional=False)

Return the slope and intercept of simple linear regression parameters estimated using ordinary least squares. Simple linear regression describes the relationship between an independent variable x and a dependent variable y in terms of this linear function: $y = \text{slope} * x + \text{intercept} + \text{noise}$ where slope and intercept are the regression parameters that are estimated, and noise represents the variability of the data that was not explained by the linear regression (it is equal to the difference between predicted a

Exceptions

A single exception is defined:

exception statistics.StatisticsError

Subclass of ValueError for statistics-related exceptions.

NormalDist objects

NormalDist is a tool for creating and manipulating normal distributions of a random variable. It is a class that treats the mean and standard deviation of data measurements as a single entity.

Normal distributions arise from the Central Limit Theorem and have a wide range of applications in statistics.

class statistics.NormalDist(mu=0.0, sigma=1.0)

mean

median

mode

stdev

variance

classmethod from_samples(data)

samples(n, *, seed=None)

pdf(x)

cdf(x)

inv_cdf(p)

overlap(other)

quantiles(n=4)

zscore(x)

Returns a new NormalDist object where `mu` represents the arithmetic mean and `sigma` represents the standard deviation. If `sigma` is negative, raises `StatisticsError`. `mean` A read-only property for the arithmetic mean of a normal distribution. `median` A read-only property for the median of a normal distribution. `mode` A read-only property for the mode of a normal distribution. `stdev` A read-only property for the standard deviation of a normal distribution. `variance` A read-only property for the variance of a normal distribution. Equal to

A read-only property for the arithmetic mean of a normal distribution.

A read-only property for the median of a normal distribution.

A read-only property for the mode of a normal distribution.

A read-only property for the standard deviation of a normal distribution.

A read-only property for the variance of a normal distribution. Equal to the square of the standard deviation.

Makes a normal distribution instance with `mu` and `sigma` parameters estimated from the data using `fmean()` and `stdev()`. The data can be any iterable and should consist of values that can be converted to `float`. If data does not contain at least two elements, raises `StatisticsError` because it takes at least one point to estimate a central value and at least two points to estimate dispersion.

Generates `n` random samples for a given mean and standard deviation. Returns a list of float values. If `seed` is given, creates a new instance of the underlying random number generator. This is useful for creating reproducible results, even in a multi-threading context. Changed in version 3.13. Switched to a faster algorithm. To reproduce samples from previous versions, use `random.seed()` and `random.gauss()`.

Using a probability density function (pdf), compute the relative likelihood that a random variable `X` will be near the given value `x`. Mathematically, it is the limit of the ratio $P(x \leq X < x + dx) / dx$ as `dx` approaches zero. The relative

likelihood is computed as the probability of a sample occurring in a narrow range divided by the width of the range (hence the word “density”). Since the likelihood is relative to other points, its value can be greater than 1.0.

Using an cumulative distribution function (cdf), compute the probability that a random variable X will be less than or equal to x . Mathematically, it is written $P(X \leq x)$.

Compute the inverse cumulative distribution function, also known as the quantile function or the percent-point function. Mathematically, it is written $x: P(X \leq x) = p$. Finds the value x of the random variable X such that the probability of the variable being less than or equal to that value equals the given probability p .

Measures the agreement between two normal probability distributions. Returns a value between 0.0 and 1.0 giving the overlapping area for the two probability density functions.

Divide the normal distribution into continuous intervals with equal probability. Returns a list of $(n - 1)$ cut points separating the intervals. Set n to 4 for quartiles (the default). Set n to 10 for deciles. Set n to 100 for percentiles which gives the 99 cut points that separate the normal distribution into 100 equal sized groups.

Compute the Standard Score describing x in terms of the number of standard deviations above or below the mean of the normal distribution: $(x - \text{mean}) / \text{stdev}$. Added in version 3.9.

Examples and Recipes

Classic probability problems

NormalDist readily solves classic probability problems.

For example, given historical data for SAT exams showing that scores are normally distributed with a mean of 1060 and a standard deviation of 195, determine the percentage of students with test scores between 1100 and 1200, after rounding to the nearest whole number:

```
>>> sat = NormalDist(1060, 195)
>>> fraction = sat.cdf(1200 + 0.5) - sat.cdf(1100 - 0.5)
>>> round(fraction * 100.0, 1)
18.4
```

Find the quartiles and deciles for the SAT scores:

```
>>> list(map(round, sat.quantiles()))
[928, 1060, 1192]
>>> list(map(round, sat.quantiles(n=10)))
[810, 896, 958, 1011, 1060, 1109, 1162, 1224, 1310]
```

Monte Carlo inputs for simulations

To estimate the distribution for a model that isn't easy to solve analytically, NormalDist can generate input samples for a Monte Carlo simulation:

```
>>> def model(x, y, z):
...     return (3*x + 7*x*y - 5*y) / (11 * z)
...
>>> n = 100_000
>>> X = NormalDist(10, 2.5).samples(n, seed=3652260728)
>>> Y = NormalDist(15, 1.75).samples(n, seed=4582495471)
>>> Z = NormalDist(50, 1.25).samples(n, seed=6582483453)
>>> quantiles(map(model, X, Y, Z))
[1.4591308524824727, 1.8035946855390597, 2.175091447274739]
```

Approximating binomial distributions

Normal distributions can be used to approximate Binomial distributions when the sample size is large and when the probability of a successful trial is near 50%.

For example, an open source conference has 750 attendees and two rooms with a 500 person capacity. There is a talk about Python and another about Ruby. In previous conferences, 65% of the attendees preferred to listen to Python talks. Assuming the population preferences haven't changed, what is the probability that the Python room will stay within its capacity limits?

```
>>> n = 750          # Sample size
>>> p = 0.65         # Preference for Python
>>> q = 1.0 - p      # Preference for Ruby
>>> k = 500          # Room capacity

>>> # Approximation using the cumulative normal distribution
>>> from math import sqrt
>>> round(NormalDist(mu=n*p, sigma=sqrt(n*p*q)).cdf(k + 0.5), 4)
0.8402

>>> # Exact solution using the cumulative binomial distribution
>>> from math import comb, fsum
>>> round(fsum(comb(n, r) * p**r * q**(n-r) for r in range(k+1)), 4)
0.8402

>>> # Approximation using a simulation
>>> from random import seed, binomialvariate
>>> seed(8675309)
>>> mean(binomialvariate(n, p) <= k for i in range(10_000))
0.8406
```

Naive bayesian classifier

Normal distributions commonly arise in machine learning problems.

Wikipedia has an nice example of a Naive Bayesian Classifier. The challenge is to predict a person's gender from measurements of normally distributed features including height, weight, and foot size.

We're given a training dataset with measurements for eight people. The measurements are assumed to be normally distributed, so we summarize the data with `NormalDist`:

```
>>> height_male = NormalDist.from_samples([6, 5.92, 5.58, 5.92])
>>> height_female = NormalDist.from_samples([5, 5.5, 5.42, 5.75])
>>> weight_male = NormalDist.from_samples([180, 190, 170, 165])
>>> weight_female = NormalDist.from_samples([100, 150, 130, 150])
>>> foot_size_male = NormalDist.from_samples([12, 11, 12, 10])
>>> foot_size_female = NormalDist.from_samples([6, 8, 7, 9])
```

Next, we encounter a new person whose feature measurements are known but whose gender is unknown:

```
>>> ht = 6.0         # height
>>> wt = 130          # weight
>>> fs = 8            # foot size
```

Starting with a 50% prior probability of being male or female, we compute the posterior as the prior times the product of likelihoods for the feature measurements given the gender:

```
>>> prior_male = 0.5
>>> prior_female = 0.5
>>> posterior_male = (prior_male * height_male.pdf(ht) *
...                  weight_male.pdf(wt) * foot_size_male.pdf(fs))

>>> posterior_female = (prior_female * height_female.pdf(ht) *
...                    weight_female.pdf(wt) * foot_size_female.pdf(fs))
```

The final prediction goes to the largest posterior. This is known as the maximum a posteriori MAP:

```
>>> 'male' if posterior_male > posterior_female else 'female'
'female'
```

Functional Programming Modules

The modules described in this chapter provide functions and classes that support a functional programming style, and general operations on callables.

The following modules are documented in this chapter:

- `itertools`— Functions creating iterators for efficient looping
`itertools` Functions
`itertools` Recipes
- `functools`— Higher-order functions and operations on callable objects
`partial` Objects
- `operator`— Standard operators as functions
Mapping Operators to Functions
In-place Operators

itertools— Functions creating iterators for efficient looping

This module implements a number of iterator building blocks inspired by constructs from APL, Haskell, and SML. Each has been recast in a form suitable for Python.

The module standardizes a core set of fast, memory efficient tools that are useful by themselves or in combination. Together, they form an “iterator algebra” making it possible to construct specialized tools succinctly and efficiently in pure Python.

For instance, SML provides a tabulation tool: `tabulate(f)` which produces a sequence `f(0), f(1), ...`. The same effect can be achieved in Python by combining `map()` and `count()` to form `map(f, count())`.

General iterators:

Iterator	Arguments	Results	Example
<code>accumulate()</code>	<code>p [,func]</code>	<code>p0, p0+p1, p0+p1+p2, ...</code>	<code>accumulate([1,2,3,4,5])</code>
<code>batched()</code>	<code>p, n</code>	<code>(p0, p1, ..., p_n-1), ...</code>	<code>batched('ABCDEFG', n=3)</code>
<code>chain()</code>	<code>p, q, ...</code>	<code>p0, p1, ... plast, q0, q1, ...</code>	<code>chain('ABC', 'DEF')</code>
<code>chain.from_iterable()</code>	<code>iterable</code>	<code>p0, p1, ... plast, q0, q1, ...</code>	<code>chain.from_iterable(['ABC', 'DEF'])</code>
<code>compress()</code>	<code>data, selectors</code>	<code>(d[0] if s[0]), (d[1] if s[1]), ...</code>	<code>compress('ABCDEF', [1,0,1,0,1,1])</code>
<code>count()</code>	<code>[start[, step]]</code>	<code>start, start+step, start+2*step, ...</code>	<code>count(10)</code>
<code>cycle()</code>	<code>p</code>	<code>p0, p1, ... plast, p0, p1, ...</code>	<code>cycle('ABCD')</code>

Combinatoric iterators:

Iterator	Arguments	Results
<code>product()</code>	<code>p, q, ... [repeat=1]</code>	cartesian product, equivalent to a nested for-loop
<code>permutations()</code>	<code>p[, r]</code>	<code>r</code> -length tuples, all possible orderings, no repeated elements
<code>combinations()</code>	<code>p, r</code>	<code>r</code> -length tuples, in sorted order, no repeated elements
<code>combinations_with_replacement()</code>	<code>p, r</code>	<code>r</code> -length tuples, in sorted order, with repeated elements

Examples	Results
<code>product('ABCD', repeat=2)</code>	<code>AA AB AC AD BA BB BC BD CA CB CC CD DA DB DC DD</code>
<code>permutations('ABCD', 2)</code>	<code>AB AC AD BA BC BD CA CB CD DA DB DC</code>
<code>combinations('ABCD', 2)</code>	<code>AB AC AD BC BD CD</code>
<code>combinations_with_replacement('ABCD', 2)</code>	<code>AA AB AC AD BB BC BD CC CD DD</code>

Itertool Functions

The following functions all construct and return iterators. Some provide streams of infinite length, so they should only be accessed by functions or loops that truncate the stream.

`itertools.accumulate(iterable[,function,*,initial=None])`

Make an iterator that returns accumulated sums or accumulated results from other binary functions. The function defaults to addition. The function should accept two arguments, an accumulated total and a value from the iterable. If an initial value is provided, the accumulation will start with that value and the output will have one more element than the input iterable. Roughly equivalent

```
to: def accumulate(iterable, function=operator.add, *, initial=None):
    'Return running totals'
    # accumulate([1,2,3,4,5])
    → 1 3
```

`itertools.batched(iterable,n,*,strict=False)`

Batch data from the iterable into tuples of length `n`. The last batch may be shorter than `n`. If `strict` is true, will raise a `ValueError` if the final batch is shorter than `n`. Loops over the input iterable and accumulates data into tuples up to size `n`. The input is consumed lazily, just enough to fill a batch. The result is yielded as soon as the batch is full or when the input iterable is exhausted:

```
>>> flattened_data = ['roses', 'red', 'violets', 'blue', 'sugar', 'sweet']
>>> unflatted_data = list(batched(flattened_data, 3, strict=True))
unflatted_data
[('roses', 'red', 'violets'), ('blue', 'sugar', 'sweet')]
>>>
```

```
ned=list(batched(flattened_data,2))
```

itertools.chain(*iterables)

Make an iterator that returns elements from the first iterable until it is exhausted, then proceeds to the next iterable, until all of the iterables are exhausted. This combines multiple data sources into a single iterator. Roughly equivalent to: `def chain(*iterables): # chain('ABC', 'DEF') → A B C D E` `for iterable in iterables: yield from iterable`

itertools.chain.from_iterable(iterable)

Alternate constructor for `chain()`. Gets chained inputs from a single iterable argument that is evaluated lazily. Roughly equivalent to: `def from_iterable(iterables): # chain.from_iterable(['ABC', 'DEF']) → A B C D E` `for iterable in iterables: yield from iterable`

itertools.combinations(iterable,r)

Return `r` length subsequences of elements from the input iterable. The output is a subsequence of `product()` keeping only entries that are subsequences of the iterable. The length of the output is given by `math.comb()` which computes $n!/r!/(n-r)!$ when $0 \leq r \leq n$ or zero when $r > n$. The combination tuples are emitted in lexicographic order according to the order of the input iterable. If the input iterable is sorted, the output tuples will be produced in sorted order. Elements are treated as unique based on their position,

itertools.combinations_with_replacement(iterable,r)

Return `r` length subsequences of elements from the input iterable allowing individual elements to be repeated more than once. The output is a subsequence of `product()` that keeps only entries that are subsequences (with possible repeated elements) of the iterable. The number of subsequence returned is $(n+r-1)!/r!/(n-1)!$ when $n > 0$. The combination tuples are emitted in lexicographic order according to the order of the input iterable. If the input iterable is sorted, the output tuples will be produced in sorted order

itertools.compress(data,selectors)

Make an iterator that returns elements from `data` where the corresponding element in `selectors` is true. Stops when either the `data` or `selectors` iterables have been exhausted. Roughly equivalent to: `def compress(data,selectors): # compress('ABCDEF', [1,0,1,0,1,1]) → A C E` `Return (datum,selector) for datum,selector in zip(data,selectors) if selector` Added in version 3.1.

itertools.count(start=0,step=1)

Make an iterator that returns evenly spaced values beginning with `start`. Can be used with `map()` to generate consecutive data points or with `zip()` to add sequence numbers. Roughly equivalent to: `def count(start=0,step=1): # count(10) → 10 11 12 13 14 ... # count(2.5, 0.5) → 2.5 3.0 3.5 ...` `n=start` while True: `yield n` `n+=step` When counting with floating-point numbers, better accuracy can sometimes be achieved by substituting multiplicative code such as: `(start+step*i for i in count())`. Changed in version 3.1: Added `step`

itertools.cycle(iterable)

Make an iterator returning elements from the iterable and saving a copy of each. When the iterable is exhausted, return elements from the saved copy. Repeats indefinitely. Roughly equivalent to: `def cycle(iterable): # cycle('ABCD') → A B C D A B C D A B C D ...` `saved=[]` `for element in iterable: yield element` `saved.append(element)` `while saved: for element in saved: yield element` This `itertools` may require significant auxiliary storage (depending on the length of the iterable).

itertools.dropwhile(predicate,iterable)

Make an iterator that drops elements from the iterable while the predicate is true and afterwards returns every element. Roughly equivalent to: `def dropwhile(predicate,iterable): # dropwhile(lambda x: x<5, [1,4,6,3,8]) → 6 3 8` `iterator=iter(iterable)` `for x in iterator: if not predicate(x): yield x` `break` `for x in iterator: yield x` Note this does not produce any output until the predicate first becomes false, so this `itertools` may have a lengthy start-up time.

itertools.filterfalse(predicate,iterable)

Make an iterator that filters elements from the iterable returning only those for which the predicate returns a false value. If predicate is None, returns the items that are false. Roughly equivalent to: `def filterfalse(predicate, iterable): # filterfalse(lambda x: x < 5, [1, 4, 6, 3, 8]) → 6`
8 if predicate is None: predicate = bool
for x in iterable: if not predicate(x): yield x

itertools.groupby(iterable, key=None)

Make an iterator that returns consecutive keys and groups from the iterable. The key is a function computing a key value for each element. If not specified or is None, key defaults to an identity function and returns the element unchanged. Generally, the iterable needs to already be sorted on the same key function. The operation of `groupby()` is similar to the `uniqfilter` in Unix. It generates a break or new group every time the value of the key function changes (which is why it is usually necessary to h

itertools.islice(iterable, stop)

itertools.islice(iterable, start, stop[, step])

Make an iterator that returns selected elements from the iterable. Works like sequence slicing but does not support negative values for start, stop, or step. If start is zero or None, iteration starts at zero. Otherwise, elements from the iterable are skipped until start is reached. If stop is None, iteration continues until the input is exhausted, if at all. Otherwise, it stops at the specified position. If step is None, the step defaults to one. Elements are returned consecutively unless step is set higher th

itertools.pairwise(iterable)

Return successive overlapping pairs taken from the input iterable. The number of 2-tuples in the output iterator will be one fewer than the number of inputs. It will be empty if the input iterable has fewer than two values. Roughly equivalent to: `def pairwise(iterable): # pairwise('ABCDEFG') → AB BC CD DE EF`
FG
iterator = iter(iterable)
a = next(iterator, None)
for b in iterator: yield a, b
Added in version 3.10.

itertools.permutations(iterable, r=None)

Return successive length permutations of elements from the iterable. If r is not specified or is None, then r defaults to the length of the iterable and all possible full-length permutations are generated. The output is a subsequence of `product()` where entries with repeated elements have been filtered out. The length of the output is given by `math.perm()` which computes $n!/(n-r)!$ when $0 \leq r \leq n$ or zero when $r > n$. The permutation tuples are emitted in lexicographic order according to the order of the input iterable. If the

itertools.product(*iterables, repeat=1)

Cartesian product of the input iterables. Roughly equivalent to nested for-loops in a generator expression. For example, `product(A, B)` returns the same as `((x, y) for x in A for y in B)`. The nested loops cycle like an odometer with the rightmost element advancing on every iteration. This pattern creates a lexicographic ordering so that if the input's iterables are sorted, the product tuples are emitted in sorted order. To compute the product of an iterable with itself, specify the number of repetitions with the

itertools.repeat(object[, times])

Make an iterator that returns object over and over again. Runs indefinitely unless the times argument is specified. Roughly equivalent to: `def repeat(object, times=None): # repeat(10, 3) → 10 10`
10 if times is None: while True: yield object
else: for i in range(times): yield object
A common use for repeat is to supply a stream of constant values to `map` or `zip`: `>>> list(map(pow, range(10), repeat(2)))`
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

itertools.starmap(function, iterable)

Make an iterator that computes the function using arguments obtained from the iterable. Used instead of `map()` when argument parameters have already been “pre-zipped” into tuples. The difference between `map()` and `starmap()` parallels the distinction between `function(a, b)` and `function(*c)`. Roughly equivalent

```
to: defstarmap(function, iterable): # starmap(pow, [(2,5), (3,2), (10,3)]) → 32 9
1000 for args in iterable: yield function(*args)
```

itertools.takewhile(predicate, iterable)

Make an iterator that returns elements from the iterable as long as the predicate is true. Roughly equivalent to: `def takewhile(predicate, iterable): # takewhile(lambda x: x<5, [1,4,6,3,8]) → 1 4` for x in iterable: if not predicate(x): break yield x Note, the element that first fails the predicate condition is consumed from the input iterator and there is no way to access it. This could be an issue if an application wants to further consume the input iterator after takewhile has been run to exhaustion. To work around

itertools.tee(iterable, n=2)

Return n independent iterators from a single iterable. Roughly equivalent to: `def tee(iterable, n=2): if n<0: raise ValueError if n==0: return() iterator = _tee(iterable) result = [iterator] for _ in range(n-1): result.append(_tee(iterator)) return tuple(result)` class _tee: def __init__(self, iterable): it = iter(iterable) if isinstance(it, _tee): self.iterator = it.iterator self.link = it.link else: self.iterator = it self.link = [None, None] def __iter__(self): return self def __next__(self): link = self.link if link[1] is None: link[0] = next(self.iterator)

itertools.zip_longest(*iterables, fillvalue=None)

Make an iterator that aggregates elements from each of the iterables. If the iterables are of uneven length, missing values are filled-in with fillvalue. If not specified, fillvalue defaults to None. Iteration continues until the longest iterable is exhausted. Roughly equivalent to: `def zip_longest(*iterables, fillvalue=None): # zip_longest('ABCD', 'xy', fillvalue='-') → Ax By C- D` iterators = list(map(iter, iterables)) num_active = len(iterators) if not num_active: return while True: values = [] for i, iterator in enumerate(

Itertools Recipes

This section shows recipes for creating an extended toolset using the existing itertools as building blocks.

The primary purpose of the itertools recipes is educational. The recipes show various ways of thinking about individual tools — for example, that `chain.from_iterable` is related to the concept of flattening. The recipes also give ideas about ways that the tools can be combined — for example, how `starmap()` and `repeat()` can work together. The recipes also show patterns for using itertools with the `operator` and `collections` modules as well as with the built-in itertools such as `map()`, `filter()`, `reversed()`, and `enumerate()`.

A secondary purpose of the recipes is to serve as an incubator. The `accumulate()`, `compress()`, and `pairwise()` itertools started out as recipes. Currently, the `sliding_window()`, `derangements()`, and `sieve()` recipes are being tested to see whether they prove their worth.

Substantially all of these recipes and many, many others can be installed from the `more-itertools` project found on the Python Package Index:

```
python -m pip install more-itertools
```

Many of the recipes offer the same high performance as the underlying toolset. Superior memory performance is kept by processing elements one at a time rather than bringing the whole iterable into memory all at once. Code volume is kept small by linking the tools together in a functional style. High speed is retained by preferring “vectorized” building blocks over the use of for-loops and generators which incur interpreter overhead.

```
from itertools import (accumulate, batched, chain, combinations, compress,
count, cycle, filterfalse, groupby, islice, permutations, product,
repeat, starmap, tee, zip_longest)
from collections import Counter, deque
from contextlib import suppress
from functools import reduce
from math import comb, isqrt, prod, sumprod
from operator import getitem, is_not, itemgetter, mul, neg, truediv
```

```

# ==== Basic one liners ====

def take(n, iterable):
    "Return first n items of the iterable as a list."
    return list(islice(iterable, n))

def prepend(value, iterable):
    "Prepend a single value in front of an iterable."
    # prepend(1, [2, 3, 4]) → 1 2 3 4
    return chain([value], iterable)

def running_mean(iterable):
    "Yield the average of all values seen so far."
    # running_mean([8.5, 9.5, 7.5, 6.5]) -> 8.5 9.0 8.5 8.0
    return map(truediv, accumulate(iterable), count(1))

def repeatfunc(function, times=None, *args):
    "Repeat calls to a function with specified arguments."
    if times is None:
        return starmap(function, repeat(args))
    return starmap(function, repeat(args, times))

def flatten(list_of_lists):
    "Flatten one level of nesting."
    return chain.from_iterable(list_of_lists)

def ncycles(iterable, n):
    "Returns the sequence elements n times."
    return chain.from_iterable(repeat(tuple(iterable), n))

def loops(n):
    "Loop n times. Like range(n) but without creating integers."
    # for _ in loops(100): ...
    return repeat(None, n)

def tail(n, iterable):
    "Return an iterator over the last n items."
    # tail(3, 'ABCDEFG') → E F G
    return iter(deque(iterable, maxlen=n))

def consume(iterator, n=None):
    "Advance the iterator n-steps ahead. If n is None, consume entirely."
    # Use functions that consume iterators at C speed.
    if n is None:
        deque(iterator, maxlen=0)
    else:
        next(islice(iterator, n, n), None)

def nth(iterable, n, default=None):
    "Returns the nth item or a default value."
    return next(islice(iterable, n, None), default)

def quantify(iterable, predicate=bool):
    "Given a predicate that returns True or False, count the True results."
    return sum(map(predicate, iterable))

def first_true(iterable, default=False, predicate=None):
    "Returns the first true value or the *default* if there is no true value."
    # first_true([a, b, c], x) → a or b or c or x
    # first_true([a, b], x, f) → a if f(a) else b if f(b) else x
    return next(filter(predicate, iterable), default)

def all_equal(iterable, key=None):
    "Returns True if all the elements are equal to each other."
    # all_equal('4■■■■', key=int) → True

```



```
return len(take(2, groupby(iterable, key))) <= 1
```

```
# ==== Data pipelines ====
```